艾逗笔
(@idoubi)



ПРОТОКОЛ MCP ДЛЯ СИСТЕМ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

ДМК ПРЕСС
ИЗДАТЕЛЬСТВО

Айдоуби (@idoubi)

Протокол MCP для систем искусственного интеллекта

Книги для программистов: https://t.me/free_books_for_it

TURING

这就是 MCP



艾逗笔（@idoubi） 著

人民邮电出版社
北 京

Книги для программистов: https://t.me/free_books_for_it

Протокол MCP для систем искусственного интеллекта

Айдоуби (@idoubi)

ДМК Пресс
издательство
Москва, 2026

Книги для программистов: https://t.me/free_books_for_it

УДК 004.8
ББК 16.6
А36

Айдоуби (@idoubi)
А36 Протокол MCP для систем искусственного интеллекта / пер. с кит. В. И. Бахура. – М.: ДМК Пресс, 2026. – 256 с.: ил.

ISBN 978-5-93700-439-0

Протокол контекста модели (Model Context Protocol, MCP) служит связующим звеном между ИИ-приложениями и внешними расширениями. MCP стремительно меняет экосистему ИИ и стал общепринятым отраслевым стандартом. Основная цель этой книги – помочь читателю получить всестороннее и системное представление о MCP, включая историю его появления, принципы работы, архитектуру протокола, сценарии применения и экосистему. На примере реальных кейсов в книге рассказывается об использовании и разработке MCP-серверов и клиентов, а также о том, как создавать на основе MCP востребованные ИИ-приложения – диалоговых помощников и интеллектуальных агентов.

Издание адресовано всем, кто хочет детально изучить MCP и применять его в работе.

УДК 004.8
ББК 16.6

«这就是 MCP»（ISBN: 978-7-115-67747-1），作者：唐敏华 (艾逗笔 @idoubi) Copyrights © 2025 Posts and Telecom Press. Russian translation rights © 2026. Russian translation rights are arranged with Posts & Telecom Press Co., Ltd. through Media Solutions, Tokyo Japan (info@mediasolutions.jp). All rights reserved.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN (кит.) 978-7-115-67747-1
ISBN (рус.) 978-5-93700-439-0

Copyrights © 2025 Posts and Telecom Press
© Оформление, издание, перевод, ДМК Пресс, 2026

Книги для программистов: https://t.me/free_books_for_it

Оглавление

От издательства 9
Предисловие: почему я написал эту книгу 10
Для кого написана эта книга 11
О чем эта книга 12
Как читать эту книгу 14
Автор о себе 15
Благодарности 17

Глава 1. Что такое MCP 19
1.1. Как работает MCP 20
1.1.1. Хост–клиент–сервер 20
1.1.2. Пояснительный пример 23
1.2. Как появился MCP 25
1.2.1. В контексте больших языковых моделей 25
1.2.2. В контексте применения ИИ 32
1.2.3. В контексте технического проектирования 36
1.3. Как MCP обрел популярность 39
1.3.1. Появление из ниоткуда 39
1.3.2. Прорыв благодаря эффекту популярности 39
1.3.3. Стремительный рост за счет двойного сетевого эффекта 42
1.4. Возможности MCP 43
1.4.1. Применение в корпоративной среде 43
1.4.2. Применение в сценарии персонального ИИ-ассистента 44
1.4.3. Помощь поставщикам ресурсов в расширении возможностей 47
1.4.4 Помощь разработчикам в эффективном создании ИИ-агентов 47
1.5. Заключение 48

Глава 2. Анализ архитектуры MCP 49
2.1. Основная архитектура: хост–клиент–сервер 49
2.1.1. Введение в архитектуру 49
2.1.2. Принципы разработки 51
2.1.3. Ключевые компоненты 51
2.1.4. Механизм согласования функциональных возможностей 52
2.1.5. Безопасность коммуникаций 53
2.2. Основы коммуникации: JSON-RPC 54
2.2.1. Что такое JSON 54
2.2.2. Что такое RPC 55
2.2.3. Что такое JSON-RPC 56
2.2.4. Спецификация протокола JSON-RPC 2.0 57
2.3. Жизненный цикл соединения 59

Книги для программистов: https://t.me/free_books_for_it

2.3.1. Этап инициализации 60
2.3.2. Этап взаимодействия 63
2.3.3. Этап закрытия 63
2.3.4. Механизм тайм-аута 63
2.3.5 Обработка ошибок 64
2.4. Механизмы передачи: stdio / SSE / потоковый HTTP 64
2.4.1. Передача через stdio 65
2.4.2 Передача данных по SSE 68
2.4.3. Потоковая HTTP-передача 74
2.5. Возможности сервера: подсказки / ресурсы / инструменты 80
2.5.1. Примеры подсказок и взаимодействия с ними 81
2.5.2. Ресурсы и примеры взаимодействия 84
2.5.3. Инструменты и примеры их взаимодействия 88
2.6. Возможности клиента: root / sampling 91
2.6.1. Корень и примеры взаимодействия 91
2.6.2. Выборка и примеры взаимодействия 94
2.7. Механизм авторизации 96
2.7.1. OAuth 2.1 96
2.7.2. Основной процесс авторизации MCP 98
2.7.3. Определение метаданных сервера авторизации 99
2.7.4. Динамическая регистрация клиентов 100
2.7.5. Использование токена доступа 102
2.7.6. Процесс авторизации через третью сторону 102
2.8. Заключение 103

Глава 3. Разработка MCP-сервера 104

3.1. Процесс разработки MCP-сервера 104
3.1.1. Подготовительные работы 104
3.1.2. Основные этапы 106
3.2. Пример 1. Разработка MCP-сервера flomo для эффективного ведения заметок 108
3.2.1. Цели разработки 108
3.2.2. Подготовительные работы 109
3.2.3. Создание проекта MCP-сервера flomo 110
3.2.4. Отладка MCP-сервера flomo 111
3.2.5. Реализация бизнес-логики MCP-сервера flomo 114
3.2.6. Оптимизация MCP-сервера flomo 118
3.2.7. Тестирование MCP-сервера flomo в клиенте большой языковой модели 121
3.2.8. Публикация MCP-сервера flomo 122
3.2.9. Подведение итогов примера 1 128
3.3. Пример 2. Разработка MCP-сервера HeyBeauty для быстрой виртуальной примерки одежды 128
3.3.1. Цели разработки 128
3.3.2. Подготовительные работы и концепция разработки 130
3.3.3. Создание проекта MCP-сервера HeyBeauty 132

Книги для программистов: https://t.me/free_books_for_it

3.3.4. Реализация бизнес-логики MCP-сервера HeyBeauty 135
3.3.5. Публикация MCP-сервера HeyBeauty 159
3.3.6. Итоги примера 2 161
3.4. Заключение 162

Глава 4. Разработка MCP-клиента 163
4.1. Процесс разработки MCP-клиента 163
4.1.1. Ключевые шаги и подходы к реализации 164
4.2. Пример 1. Разработка диалогового ИИ-ассистента 165
4.2.1. Цели разработки 165
4.2.2. Предварительная подготовка 168
4.2.3. Чтение списка MCP-серверов, настроенных пользователем 171
4.2.4. Получение списка инструментов с MCP-сервера 172
4.2.5. Запрос на выбор инструментов большой языковой моделью 174
4.2.6. Анализ информации об инструментах в ответе большой языковой модели 178
4.2.7. Вызов инструмента 180
4.2.8. Оптимизация логики взаимодействия 182
4.2.9. Подведение итогов примера 1 187
4.3. Пример 2. Разработка поискового ИИ-агента 188
4.3.1. Цели разработки 188
4.3.2. Подготовительные работы 190
4.3.3. Определение конфигурации MCP-сервера 195
4.3.4. Фильтрация списка инструментов MCP-сервера 196
4.3.5. Организация рабочего процесса с помощью подсказок 197
4.3.6. Создание кода для взаимодействия с большой языковой моделью 199
4.3.7. Отладка логики интерфейса 201
4.3.8. Тестирование в Cursor 202
4.3.9. Подведение итогов примера 2 204
4.4. Заключение 204

Глава 5. Классические сценарии применения MCP 206
5.1. Использование MCP-сервера с популярными клиентами 206
5.1.1. Использование MCP-сервера с Cline 206
5.1.2. Использование MCP-сервера с ChatWise 207
5.1.3. Использование MCP-сервера с Cherry Studio 209
5.1.4. Использование MCP-сервера с DeepChat 210
5.1.5. Использование MCP-сервера в веб-версии ChatMCP 213
5.1.6. Подведение итогов 214
5.2. Пример 1. Реализация генератора ИИ-подкастов на основе MCP 215
5.2.1. Цели реализации 216
5.2.2. Подготовка MCP-серверов 217
5.2.3. Генерация тематических подкастов 221
5.2.4. Создание подкаста по URL-адресу веб-страницы 226
5.2.5. Итоги примера 1 228

Книги для программистов: https://t.me/free_books_for_it

5.3. Пример 2. Разработка ИИ-генератора веб-страниц на основе MCP 229
5.3.1. Цели реализации 230
5.3.2. Подготовка MCP-сервера 230
5.3.3. Создание веб-страниц с помощью подсказок 234
5.3.4. Генерация веб-страницы на основе макета Figma 239
5.3.5. Итоги примера 2 241
5.4. Заключение 242

Глава 6 . Экосистема MCP 243

6.1. Набор инструментов MCP 243
6.2. Платформа и сервисы MCP 244
6.3 Взаимодействие MCP с выше- и нижестоящими звеньями экосистемы. Цепочка интеграции 248
6.4. Сообщество MCP и его ресурсы 249
6.5. Ограничения MCP 249
6.6. MCP и A2A 250
6.7. Заключение 252
Предметный указатель 253

Книги для программистов: https://t.me/free_books_for_it

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте *www.dmkpress.com*, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу *dmkpress@gmail.com*; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу *http://dmkpress.com/authors/publish_book/* или напишите в издательство по адресу *dmkpress@gmail.com*.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в основном тексте или программном коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу *dmkpress@gmail.com*, и мы исправим это в следующих тиражах.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу *dmkpress@gmail.com*.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Книги для программистов: https://t.me/free_books_for_it

Предисловие: почему я написал эту книгу

За последние три месяца популярность MCP резко возросла, и он фактически стал коммуникационным стандартом в индустрии искусственного интеллекта (ИИ).

Являясь связующим звеном между приложениями ИИ и внешними расширениями, технология MCP стремительно меняет всю экосистему. Несмотря на то что с момента официального выпуска прошло всего полгода, MCP быстро стал общепринятым отраслевым стандартом – практически все, от клиентов основных крупных моделей и других приложений ИИ до сервис-провайдеров и платформ данных, стремящихся открыть свои возможности через API, активно переходят на MCP.

Можно сказать, что за последние три года бурного развития ИИ технология MCP стала первой и наиболее значимой платформенной технологией с неограниченным потенциалом развития.

И когда в конце марта 2025 года редактор Иньцзы из издательства «Тьюринг» (People's Posts and Telecommunications Press) обратилась ко мне с предложением написать книгу об MCP, я без колебаний согласился. Если рынок нуждается в книге с систематическим объяснением возможностей MCP, я, пожалуй, являюсь одним из наиболее подходящих авторов, и на это есть три основные причины.

- С самого начала, когда MCP только появился, я твердо верил в его успех. Я внимательно изучил проектную документацию по MCP и опубликовал в социальных сетях множество статей с анализом MCP и прогнозами его будущего развития. С этой точки зрения я являюсь активным популяризатором MCP.
- Я разрабатывал на базе MCP серверы, клиентские приложения и интеллектуальные агенты, проектировал механизмы передачи данных MCP, создал магазин приложений MCP и занимался развертыванием MCP в облаке. С этой точки зрения я являюсь разработчиком MCP с богатым практическим опытом.
- Созданный мной веб-сайт MCP.so является крупнейшим на сегодняшний день магазином приложений MCP, где представлено наибольшее количество MCP-серверов. Он неоднократно выходил на первое место в результатах поиска Google по ключевому слову **MCP**, его посещаемость превышает миллион просмотров в месяц, и он упоминался в рыночном отчете a16z. С этой точки зрения я являюсь MCP-предпринимателем, который смог воспользоваться преимуществами раннего стартапа.

Я верю, что MCP обязательно будет играть роль ключевой инфраструктуры в экосистеме ИИ-приложений. Я надеюсь, что с помощью этой книги смогу систематизировать свои познания, опыт и идеи, чтобы поделиться ими с читателями.

Книги для программистов: https://t.me/free_books_for_it

Для кого написана эта книга

Приступая к написанию этой книги, я поставил перед собой довольно амбициозную цель. Эта книга должна стать:

- самым доступным и понятным введением в принципы работы MCP;
- самым доступным и понятным изложением основ архитектуры MCP;
- самым содержательным сборником практических примеров разработки с использованием MCP;
- сборником наиболее часто встречающихся и практичных примеров использования MCP;
- самым актуальным руководством по запуску стартапа с использованием MCP.

Однако в процессе работы над книгой мне пришлось столкнуться с серьезными трудностями – чем больше я писал, тем больше понимал, насколько сложна эта задача. В итоге мне оставалось лишь постараться сделать все, что в моих силах, а вот будут ли достигнуты изначально поставленные цели – это покажет время и отзывы читателей.

В силу моих ограниченных познаний в книге неизбежно присутствуют пробелы и неточности. Именно по этой причине меня особенно интересуют отзывы читателей: как одобрение, так и критика – все это будет стимулировать меня двигаться вперед и непрерывно совершенствоваться.

Книги для программистов: https://t.me/free_books_for_it

О чем эта книга

Основная цель этой книги – помочь читателю получить всестороннее и системное представление об MCP, включая историю его появления, принципы работы, архитектуру протокола, сценарии применения и экосистему. На примере реальных кейсов в книге рассказывается об использовании и разработке сервера MCP, а также о том, как на его основе создавать наиболее востребованные ИИ-приложения (диалоговые помощники, интеллектуальные агенты).

Книга состоит из 6 глав, каждая из которых посвящена следующим темам.

Глава 1 «Что такое MCP». Эта глава начинается с классической проблемы в сфере разработки ПО, где рассматриваются сложности интеграции ИИ-приложений с внешними расширениями и способы их решения. Здесь читатель получит системное представление о предпосылках создания MCP, принципах его работы и о том, как MCP помогает решать отраслевые проблемы. Далее в главе прослеживается путь MCP от момента выпуска до стремительного роста популярности и объясняются причины быстрого формирования отраслевого консенсуса. В заключение представлены типичные сценарии применения MCP, что поможет читателю лучше понять его практическую ценность.

Глава 2 «Анализ архитектуры MCP». В этой главе проводится системный анализ MCP на уровне протоколов. Читатель познакомится с основной архитектурой MCP «хост–клиент–сервер», принципами взаимодействия JSON-RPC, жизненным циклом сеанса соединения между клиентом и сервером, а также с философией проектирования MCP.

В этой главе особое внимание уделяется трем механизмам передачи данных MCP, а через анализ исходного кода читатель сможет понять принципы их реализации и соответствующие сценарии применения. Кроме того, в этой главе представлены обширные возможности, поддерживаемые MCP-сервером и MCP-клиентом, а также примеры их взаимодействия. Читатель познакомится с расширенными функциями ИИ-приложений на базе MCP, которые можно создавать так же просто, как собирать конструктор.



Основной текст и иллюстрации главы 2 взяты из официальной документации MCP (опубликована в открытом доступе по лицензии MIT; авторские права: Copyright © 2024–2025 Anthropic, PBC и авторы). В духе открытого исходного кода содержание этой главы предоставляется в виде бесплатного PDF-файла, который можно скачать и прочитать на странице книги в сообществе Turing (*http://ituring.cn/book/3508*).

Глава 3 «Разработка MCP-сервера». В этой главе на примере двух реальных кейсов представлен полный процесс разработки MCP-сервера, включая создание проекта, разработку функционала, отладку и публикацию. Читатели

Книги для программистов: https://t.me/free_books_for_it

с определенным опытом разработки смогут быстро овладеть навыками создания MCP-сервера.

Первый пример посвящен этапам разработки MCP-сервера, приемам отладки и процессу развертывания, а второй пример содержит комплексное объяснение логики реализации трех основных возможностей MCP-сервера: подсказок, ресурсов и инструментов.

Глава 4 «Разработка MCP-клиента». В этой главе на примере двух реальных кейсов представлен основной процесс разработки MCP-клиента, включая использование клиентского SDK от команды официальных разработчиков MCP, получение списка инструментов MCP-сервера, разработку подсказок для запроса инструментов выбора больших языковых моделей, а также логику вызова инструментов.

В этой главе рассматривается разработка логики интерфейсов для двух типов ИИ-приложений на базе MCP: диалоговых помощников и интеллектуальных агентов. Цель главы состоит в том, чтобы читатель научился интегрировать разнообразные инструменты через MCP-сервер, а также выполнять задачи с помощью внешних инструментов посредством планирования работы больших языковых моделей, тем самым ускоряя разработку ИИ-приложений.

Глава 5 «Классические сценарии применения MCP». В этой главе преимущественно рассматривается использование MCP-сервера с распространенными клиентами для больших языковых моделей, а также на примере двух реальных сценариев демонстрируется возможность объединения нескольких MCP-серверов для выполнения типичных задач. Цель этой главы заключается в том, чтобы порекомендовать читателю качественные серверы и инструменты MCP для классических сценариев, а также познакомить его с приемами комбинированного использования MCP-серверов.

В первом примере рассказывается о создании генератора подкастов на базе ИИ с использованием инструментов MCP-сервера, таких как сетевой поиск, чтение веб-контента и преобразование текста в аудио. Во втором примере рассказывается о создании генератора веб-страниц на базе ИИ с использованием инструментов MCP-сервера, таких как сетевой поиск, развертывание кода и получение макетов.

Глава 6 «Экосистема MCP». В этой главе представлена общая концепция экосистемы MCP, включая официальные источники, ресурсы сообщества, инструменты разработки и т. д. Цель главы состоит в том, чтобы ознакомить читателя с актуальным состоянием экосистемы MCP и рассказать о возможностях участия в ее будущем развитии.

Кроме того, в этой главе рассматриваются возможные пути развития MCP, анализируются перспективные возможности и преимущества экосистемы, а также предлагаются идеи и рекомендации для разработчиков, желающих углубить свои знания в этой области.

Книги для программистов: https://t.me/free_books_for_it

Как читать эту книгу

Эта книга представляет собой всестороннее и систематическое введение в MCP, охватывающее такие аспекты, как объяснение принципов работы MCP, анализ архитектуры, практическое использование, разработка и экосистема. Книга рассчитана на широкий круг читателей, и она может быть интересна всем, кто интересуется MCP. Для того чтобы читатели с различными интересами смогли лучше понять содержание книги, ниже приведены рекомендации по чтению в соответствии с содержанием каждой главы.

Глава 1 представляет собой вводную часть по MCP, которая подходит для всех читателей, поэтому рекомендуется прочитать ее в первую очередь. Эта глава откроет для вас мир MCP, а также позволит понять процесс развития и эволюции ИИ-отрасли.

Главы 2, **3** и **4** посвящены анализу архитектуры, разработке MCP-сервера и MCP-клиента соответственно. Для глубокого понимания этого материала требуются определенные базовые знания в области информатики и навыки программирования. Читателям с опытом разработки рекомендуется читать главы по порядку, остальным читателям – по своему усмотрению.

Глава 5 посвящена основным сценариям использования MCP-сервера для достижения конкретных целей, включая настройку, компоновку, разработку подсказок и отладку MCP-сервера. Эта глава не требует от читателя знаний в области программирования и подходит для всех, кто интересуется ИИ, занимается исследованием продуктов и инструментов ИИ, например для менеджеров по продуктам, дизайнеров и т. д.

Глава 6 посвящена структуре экосистемы MCP и ценности ее отдельных подсистем. Она сопровождается обзорной схемой MCP. Читатель сможет лучше понять свое место в экосистеме и определить направления для дальнейшей работы. Рекомендуем прочитать эту главу тем читателям, кто считает MCP перспективным направлением и хочет присоединиться к совместному развитию этой экосистемы, например предпринимателям, инвесторам и т. д.

Книги для программистов: https://t.me/free_books_for_it

Автор о себе

В 2015 году я окончил Уханьский университет (Wuhan University, WHU) по специальности «ядерная инженерия». На втором курсе проявился мой глубокий интерес к компьютерам: я самостоятельно освоил программирование, переквалифицировался в ИТ-специалиста и после окончания университета начал работать в интернет-индустрии. Будучи большим поклонником программных продуктов компании Adobe, я выбрал себе ник **Айдоуби** (艾逗笔, @idoubi) в знак уважения к этой компании, и этот ник я использую по сей день.

В 2018 году устроился в компанию Tencent на должность инженера по разработке бэкенда. За пять лет работы в Tencent первые два с половиной года занимался разработкой системы электронных платежей в подразделении TEG (Техническая инженерная группа), ежедневно обслуживающей миллионы активных пользователей. В последующие два с половиной года работал в подразделении WXG (группа WeChat), где отвечал за проектирование архитектуры системы приема международных платежей и создал с нуля систему облачной разработки.

Опыт работы в Tencent значительно расширил мой технический кругозор и позволил мне превратиться из программиста без специального образования в зрелого разработчика ПО. За это время я трижды подряд получал награду «Лучший сотрудник Tencent», дважды подряд продвигался по службе до старшего инженера (T10) и в качестве интервьюера участвовал в наборе выпускников и сотрудников из числа опытных специалистов.

В ноябре 2023 года я уволился из Tencent, стал фрилансером и официально пополнил ряды независимых разработчиков. За полтора года работы в качестве независимого разработчика я добился определенных успехов и разработал несколько продуктов:

- Zhiliao Reading, инструмент для создания резюме на основе ИИ, стал победителем второго хакатона Alibaba Cloud;
- ThinkAny, поисковая система на базе ИИ, привлекла сотни тысяч пользователей за рубежом;
- ShipAny, фреймворк для разработки ИИ-приложений, ставший предпочтительным выбором для многих независимых разработчиков при выходе на международный рынок;
- MCP.so, магазин приложений MCP, привлекающий внимание разработчиков по всему миру, с ежемесячной посещаемостью более миллиона пользователей.

Кроме того, я создал более десяти ИИ-продуктов, охватывающих такие направления, как обзор чатов, виртуальная примерка одежды, музыкальные плееры, генераторы подкастов и вспомогательные инструменты для программирования, что свидетельствует о широком спектре моих интересов.

Книги для программистов: https://t.me/free_books_for_it

Перейдя с должности бэкенд-инженера на должность разработчика полного стека, я накопил богатый опыт независимой разработки, постоянно экспериментируя с новыми продуктами. Развитие ИИ-технологий значительно снизило порог входа в разработку, и с помощью различных ИИ-инструментов независимые программисты могут создавать все более разнообразные продукты. Это благоприятная эпоха для разработчиков-фрилансеров, и я продолжаю свои исследования, стремясь создать еще больше полезных и интересных продуктов.

Книги для программистов: https://t.me/free_books_for_it

Благодарности

Когда в начале года я писал итоговый отчет за 2024 год, одной из моих целей на 2025 год стала публикация своей первой книги. Я очень рад, что эта цель вот-вот осуществится – моя первая книга скоро выйдет в свет! Мои эмоции трудно описать словами, и я хочу воспользоваться этой возможностью, чтобы поблагодарить всех, кто поддерживал и сопровождал меня на этом пути.

Спасибо моему редактору, госпоже Инцзы. Как упоминалось ранее, в конце марта мы с госпожой Инцзы договорились о написании этой книги. Поскольку времени было мало, после начала работы над книгой я приостановил некоторые текущие разработки и посвятил большую часть времени писательскому труду. Тем не менее, поскольку я писал книгу впервые, мне не удавалось правильно распределить темп и контролировать объем: первоначальные черновики были либо слишком поверхностными и невыразительными, либо слишком длинными и многословными. Несколько раз меня одолевали глубокие сомнения в собственных силах, и я даже подумывал о том, чтобы бросить эту затею. Пару раз работа над книгой на время замирала, и я возвращался в свою зону комфорта – писать код, что приводило к отставанию от графика сдачи рукописи.

Благодарю госпожу Инцзы за веру в мой профессионализм, за то, что она подбадривала меня, побуждала не сдаваться и следила за ходом работы. Она также постоянно напоминала, что мы – одна команда, которая должна сражаться бок о бок, «делая все, что в наших силах, и доверяя остальное судьбе». После сдачи чернового варианта госпожа Инцзы предложила некоторые корректировки и дополнения к содержанию. На их основе мы снова проработали текст, стремясь сделать его структуру и изложение более четкими и логичными. Благодаря нашим совместным усилиям окончательный вариант книги значительно превысил первоначально задуманные 30–40 тысяч слов, а по богатству и профессионализму содержания мы достигли максимально возможного результата в рамках наших возможностей.

Благодарю мою супругу, госпожу Че, за огромную поддержку во время написания книги. Каждый день она посещала со мной различные кафе, помогая мне сосредоточиться на творчестве в комфортной обстановке, а в моменты тревоги и растерянности оказывала мне огромную поддержку и утешение.

Спасибо нашим двум маленьким сокровищам, Цзаоцай и Тяотяо (две пятилетние оранжевые кошки, родные сестры), которые часто были рядом со мной, когда я писал книгу, и каждый день были такими милыми и очаровательными, что заставляли меня чувствовать, что жизнь прекрасна, а мир достоин того, чтобы в нем жить.

Книги для программистов: https://t.me/free_books_for_it

Наконец, спасибо этому великому времени. После того как я выбрал фриланс, как раз начался бум ИИ, и мне посчастливилось окунуться в водоворот этого времени, уловить один тренд за другим, создать несколько интересных продуктов и завоевать признание многих людей – какое счастье!

Эту книгу я посвящаю своей семье, друзьям и всем коллегам, которые с энтузиазмом принимают искусственный интеллект.

Айдоуби, 19 июня 2025 года, Наньша, Гуанчжоу

Книги для программистов: https://t.me/free_books_for_it

Глава 1

Что такое MCP

В сфере разработки ПО существует сложная задача под названием **проблема *M*×*N***: когда несколько приложений (*M*) необходимо интегрировать с несколькими внешними расширениями (*N*), но при этом отсутствует единый стандарт и каждая комбинация требует отдельной адаптации. Это означает, что с добавлением каждого нового приложения или расширения общий объем работ по интеграции увеличивается по формуле *M*×*N*. Так, например, при увеличении числа приложений с 3 до 4 и расширений с 4 до 5 объем работ по адаптации увеличивается с 12 до 20 подключений.

В сфере разработки приложений искусственного интеллекта (ИИ) эта проблема стоит особенно остро. ИИ-приложениям для расширения своих возможностей необходимо вызывать различные внешние инструменты, например, для получения информации в реальном времени, доступа к базам данных, обработки локальных файлов и т. д. Однако зачастую ИИ-приложения и инструменты разрабатываются разными командами, не имеющими единых стандартов взаимодействия. Это приводит к тому, что разработчикам приходится создавать отдельную логику адаптации для каждой комбинации «ИИ-приложение–инструмент», а это означает многократное дублирование работы, высокие затраты на взаимодействие и растущую сложность обслуживания.

Решение проблемы интеграции ИИ-приложений с внешними инструментами с помощью универсального протокола стало отправной точкой для разработки MCP. На рис. 1.1 представлен пример использования MCP для снижения сложности проблемы *M*×*N* (где *M* и *N* равны 4).

25 ноября 2024 года компания Anthropic опубликовала в открытом доступе стандарт MCP под лицензией MIT, что стало важной вехой в истории развития ИИ.

Аббревиатура MCP расшифровывается как **Model Context Protocol** (*протокол контекста модели*). С помощью MCP определяется способ стандартизированной интеграции внешних инструментов в приложения ИИ с целью дополнения контекста (Context) для больших языковых моделей (Large Language Model), и, по сути, он представляет собой протокол прикладного уровня (Protocol).

Протокол – это стандарт. Разработка стандартов направлена на формирование отраслевого консенсуса. Как только консенсус сформирован, это позволяет снизить издержки повторной разработки и способствует упорядоченному и быстрому развитию всей экосистемы.

Книги для программистов: https://t.me/free_books_for_it

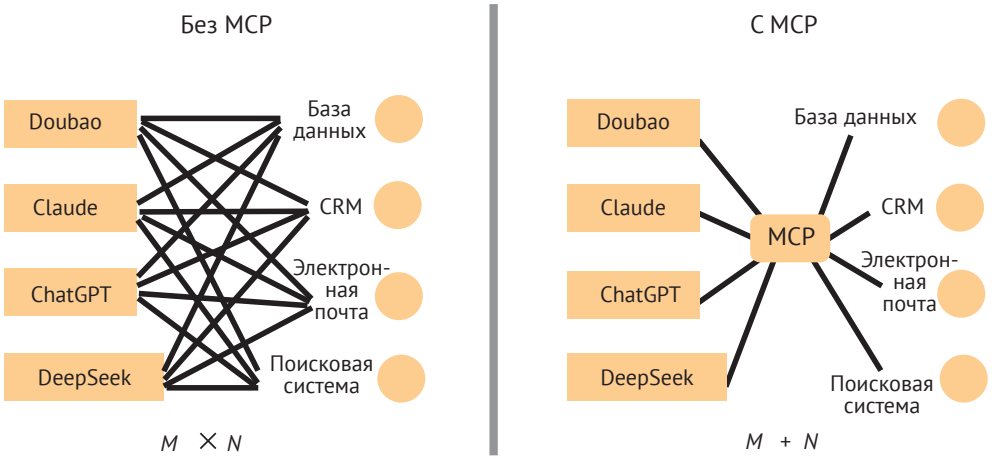


Рис. 1.1. Снижение сложности задачи *M*×*N* с помощью MCP

Для простой иллюстрации значимости появления MCP можно привести пример объединения шести государств первым императором Цинь. В период Воюющих царств каждое из семи государств управлялось независимо друг от друга, при этом использовались различные письменности, валюты, системы мер и весов, что затрудняло взаимодействие между ними и замедляло развитие экономики, культуры и науки. После объединения шести государств первым императором Цинь для удобства управления были унифицированы письменность, валюта и система мер и весов. С тех пор взаимодействие между странами стало более удобным, а экономика, культура и наука получили значительный импульс к развитию.

> «Единые законы, меры и веса, единые единицы длины и объема. Единые колеи для колесниц, единая письменность».
>
> — *История Китая: Хроники Первого императора Цинь*

1.1. Как работает MCP

Принцип работы MCP несложен: суть протокола заключается в определении трех ролей и установлении моделей взаимодействия между ними.

1.1.1. Хост–клиент–сервер

1. MCP-хост

MCP-хост (иногда сокращенно называемый просто хостом[1]) является центральным узлом MCP. Он выступает как терминал для прямого взаимодействия пользователя с большой языковой моделью и как центр координации и распределения внешних ресурсов.

[1] В данной книге в зависимости от контекста термин MCP добавляется или опускается перед словами «хост», «клиент» и «сервер».

Книги для программистов: https://t.me/free_books_for_it

Хост обладает двойными свойствами клиента.

- В качестве клиента для взаимодействия с большой языковой моделью: принимает ввод пользователя и запрашивает у большой языковой модели сгенерированный ответ. Все ИИ-приложения, поддерживающие MCP (в основном это ИИ-помощники для диалога, интеллектуальные агенты и ИИ-редакторы), можно рассматривать в качестве хостов.
- В качестве MCP-клиента для координации сервисов: хост создает и управляет несколькими экземплярами клиентов (каждый из которых соответствует одному серверу). В процессе запроса ответа у большой языковой модели хост может через экземпляры клиентов запрашивать данные у внешних серверов или выполнять определенные задачи. Хост предоставляет данные, полученные от внешних серверов, или результаты выполненных задач в качестве контекста для большой языковой модели, чтобы она могла сгенерировать ответ, более соответствующий потребностям пользователя.

2. MCP-сервер

MCP-сервер (иногда сокращенно называемый просто сервером) является ключевым компонентом MCP, в основном отвечающим за интеграцию с внешними данными или сервисами и отправку ответов клиентам MCP в стандартном формате.

Сервер MCP можно рассматривать как прокси-шлюз для внешних данных или сервисов, через который проходит весь обмен данными.

Например, карта Amap (в качестве поставщика данных или услуг) открыла свой API для планирования маршрутов: до появления MCP приложениям ИИ приходилось самостоятельно подключаться к API Amap с помощью программного кода. С появлением MCP карта Amap может использовать MCP-сервер для предоставления своих возможностей, а приложения ИИ, поддерживающие MCP, могут напрямую интегрироваться с этим MCP-сервером без дополнительных операций по подключению.

3. MCP-клиент

MCP-клиент (иногда сокращенно называемый клиентом) является мостом между хостом MCP и сервером MCP. Клиент «живет» на хосте, создается и управляется хостом; он устанавливает соединение с сервером, обеспечивая взаимодействие с внешними данными или сервисами.



MCP разработан на основе классической клиент-серверной (C/S) архитектуры. MCP-клиент – это концепция, противопоставленная MCP-серверу. Обратите внимание: в этом разделе сначала описывается MCP-сервер, и только после него следует MCP-клиент. Это сделано для наглядности: хост создает экземпляр клиента только тогда, когда возникает потребность в подключении к серверу. Однако из-за сложившихся терминологических традиций мы по-прежнему называем эту архитектуру **хост–клиент–сервер**. Чтобы MCP-хост мог вызвать внешний MCP-сервер, необходимо создать MCP-клиента, и только после этого можно установить соединение между ними. MCP-клиент и MCP-сервер однозначно соответствуют один одному.

Книги для программистов: https://t.me/free_books_for_it

В свою очередь, MCP-хост сам по себе является клиентом, использующим большую языковую модель в качестве объекта взаимодействия. Обычно его называют клиентом большой языковой модели. Хотя в некоторых сценариях легко перепутать «клиент большой языковой модели» и «MCP-клиент», на уровне системной архитектуры между ними необходимо проводить четкое различие: клиент большой языковой модели – это ориентированное на пользователя (видимое пользователю) ИИ-приложение[1], а MCP-клиент – это процесс, созданный MCP-хостом.

Официальная архитектура MCP представлена на рис. 1.2 (подробнее об архитектуре MCP будет рассказано в главе 2).

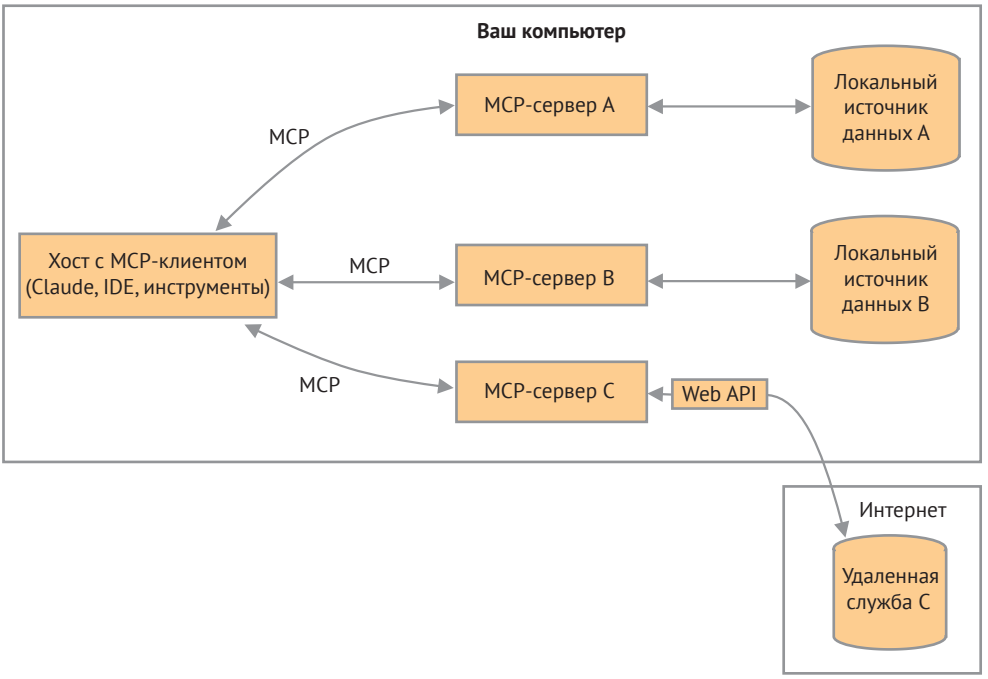


Рис. 1.2. Схема архитектуры MCP

[1] В этой книге под клиентом больших языковых моделей понимается ИИ-приложение (иногда сокращенно называемое «приложением»), и в зависимости от контекста мы используем оба термина.

Книги для программистов: https://t.me/free_books_for_it

1.1.2. Пояснительный пример

Рассмотрим механизм работы MCP на конкретном примере. Предположим, что пользователь собирается использовать MCP-сервер карты AutoNavi в приложении Claude[1] для прокладки маршрута. Процесс прокладки маршрута выглядит следующим образом.

1. Карта AutoNavi предоставляет свои возможности по прокладке маршрутов в виде API; одновременно с этим она предоставляет MCP-сервер, интегрированный с собственным API, для использования всеми клиентами.
2. Пользователь открывает Claude, настраивает способ вызова MCP-сервера AutoNavi и вводит ключ.
3. Пользователь отправляет в Claude следующий запрос: «Я хочу на следующей неделе поехать на машине из Пекина в Шанхай. Составь для меня маршрут, который займет меньше всего времени».
4. Claude обращается к большой языковой модели и сообщает ей, что доступен MCP-сервер AutoNavi с инструментом планирования маршрута.
5. Большая языковая модель отвечает Claude, что необходимо запросить у MCP-сервера AutoNavi вызов инструмента планирования маршрута с параметрами: «Место отправления = Пекин, место назначения = Шанхай».
6. Claude создает внутреннюю программу – MCP-клиента.
7. Внутренняя MCP-клиентская программа Claude отправляет запрос на MCP-сервер AutoNavi.
8. MCP-сервер AutoNavi возвращает информацию о маршруте поездки MCP-клиентской программе.
9. Claude отправляет найденный маршрут поездки вместе с первоначальным запросом пользователя в большую языковую модель, призывая ее ответить на запрос пользователя.
10. Имея маршрут поездки в качестве контекстной информации, большая языковая модель отвечает на запрос пользователя, возвращая наиболее точный ответ.
11. Claude отображает ответ пользователю.

Этот процесс можно описать с помощью диаграммы, показанной на рис. 1.3[2].

---

[1] В этой книге под «Claude» подразумевается версия Claude для настольных компьютеров.

[2] В этой книге содержится большое количество блок-схем для облегчения понимания. Эти иллюстрации предназначены для интуитивного и структурированного представления взаимодействия между различными модулями системы. Для повышения читабельности и визуальной наглядности шаги или пояснения в блок-схемах в определенной степени упрощены и не всегда полностью коррелируют с текстом.

Книги для программистов: https://t.me/free_books_for_it

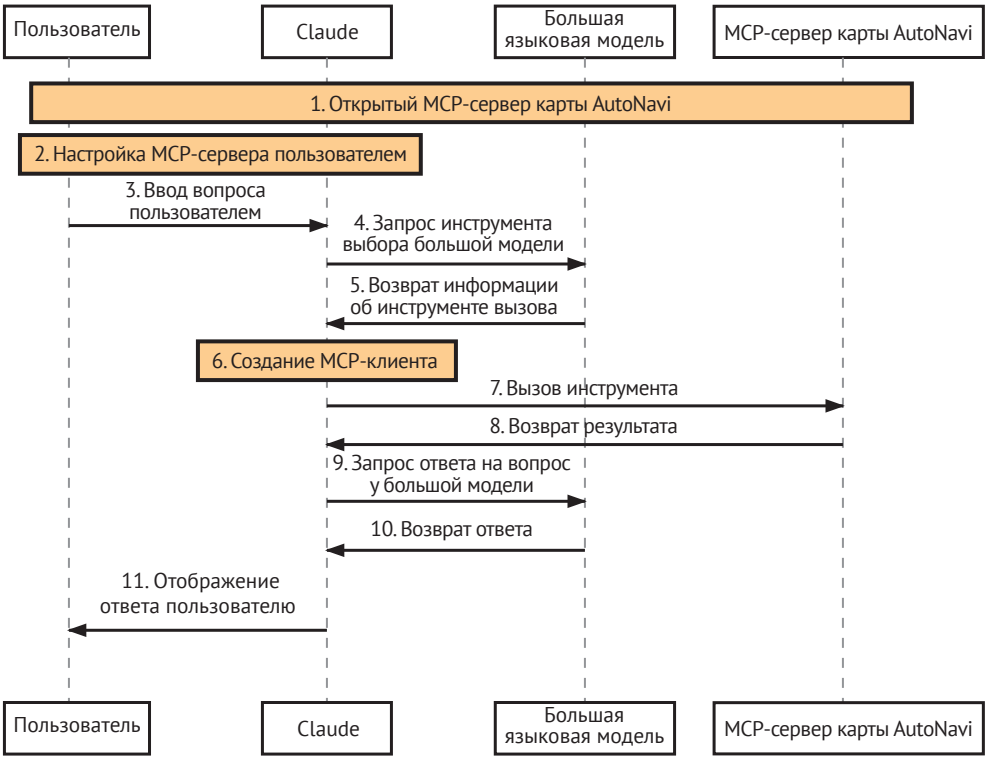


Рис. 1.3. Алгоритм взаимодействия с MCP-сервером Amap в Claude



В этом примере Claude выступает в роли хоста, который через подключенный MCP-сервер карты AutoNavi предоставляет большой языковой модели возможность планирования маршрута.

По тому же принципу любой сторонний поставщик данных или услуг может предоставлять доступ через MCP-сервер к своим данным или сервисам, в том числе к различным интернет-сервисам, файлам и базам данных на локальном компьютере и т. д.

Любое ИИ-приложение может использовать MCP, обмениваться данными с различными MCP-серверами и предоставлять полученные данные в качестве контекста для большой языковой модели, чтобы сделать ее ответы более точными.

В данном случае уместно сравнение MCP с док-станцией для ИИ-приложений, как показано на рис. 1.4.

ИИ-приложение похоже на ноутбук с ограниченным набором интерфейсов, который не может напрямую обращаться к внешним данным или сервисам. В этом случае MCP играет роль «док-станции», предоставляя ИИ-приложению возможность подключаться к различным внешним системам через стандартизированные интерфейсы на основе протоколов.

Книги для программистов: https://t.me/free_books_for_it

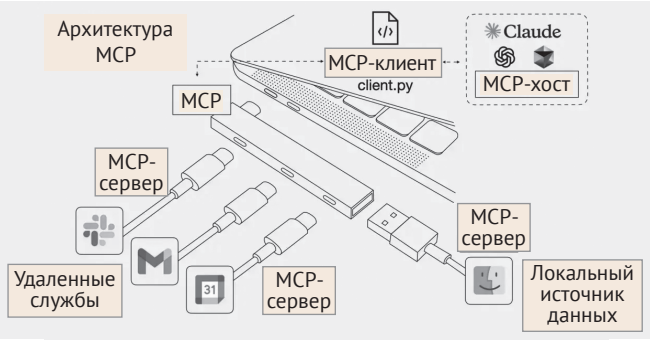


Рис. 1.4. MCP и док-станция

Благодаря MCP ИИ-приложения больше не ограничены возможностями больших языковых моделей и могут, как ноутбук с подключенной док-станцией, гибко взаимодействовать с различными внешними системами для расширения своих функциональных возможностей.

1.2. Как появился MCP

1.2.1. В контексте больших языковых моделей

30 ноября 2022 года компания OpenAI анонсировала ChatGPT – диалоговый продукт на основе большой языковой модели.

ChatGPT быстро завоевал популярность благодаря впечатляющему взаимодействию с пользователем (при помощи естественного языка можно выполнять такие сложные задачи, как программирование, написание текстов и логические выводы). Таким образом, термин **большая языковая модель** (Large Language Model, LLM) впервые вышел за пределы технического сообщества и стал известен широкой публике. В отрасли быстро сложился консенсус: большие языковые модели станут основным средством взаимодействия человека с компьютером следующего поколения.

В последующий период технологии больших языковых моделей стремительно развивались, и общение с ними через диалоговых ИИ-ассистентов для интернет-пользователей постепенно стало основным способом получения информации.

Однако большие языковые модели обучаются на исходных данных из прошлого, которые фиксируются на момент обучения такой нейросети. После завершения обучения знания модели замораживаются в состоянии на момент обучения, и она не может автоматически получать новые сведения, появившиеся позднее. Иными словами, между статическими «знаниями» большой языковой модели и «потребностями в реальном времени» динамичного мира существует фундаментальное противоречие. Чтобы преодолеть эту проблему и решить вопрос доступа большой языковой модели к информации, появившейся после обучения, отрасль разработала два основных решения.

Первый тип: микронастройка модели. Основной принцип микронастройки модели заключается в дополнительном обучении на основе предвари-

Книги для программистов: https://t.me/free_books_for_it

тельно обученной большой модели с использованием нового набора данных. Обычно базовая архитектура модели остается неизменной, а корректируются только некоторые или все параметры, что позволяет модели усваивать новые знания или адаптироваться к конкретной задаче.

По сравнению с обучением с нуля, микронастройка моделей требует меньше вычислительных ресурсов и позволяет более эффективно реализовать настройку модели под конкретные нужды. Основным ограничением микронастройки является ее привязка к принципу «сначала обучение, потом развертывание»: хотя она и позволяет дополнить модель новыми знаниями для решения задач в определенных профессиональных областях, она не подходит для сценариев, требующих высокой оперативности, таких как получение новостей в режиме реального времени, информации о погоде и т. д.

Второй тип: дополнение контекста. Дополнение контекста – это техническое решение, предоставляющее большой языковой модели внешнюю базу знаний или источники информации. Его цель – помочь модели преодолеть ограничения актуальности знаний и получить доступ к информации из конкретных профессиональных областей, что повышает точность, скорость и надежность ответов.

В отличие от микронастройки модели дополнение контекста не требует изменения параметров модели и ее повторного обучения. Оно заключается в предоставлении соответствующей информации в режиме реального времени в качестве входного контекста при генерации ответа моделью, чтобы она могла строить логические выводы на основе этой самой свежей информации. Опираясь на дополнительную контекстную информацию, большая языковая модель может отвечать на вопросы о событиях, произошедших после обучения, тем самым преодолевая разрыв между статическими знаниями и динамичным миром.

Ключевым элементом MCP является именно контекст (Context), и с этой точки зрения первоначальная цель создания MCP состояла в том, чтобы дополнять большие языковые модели необходимым контекстом.

До появления MCP в отрасли существовали следующие основные подходы к предоставлению контекста для больших языковых моделей.

1. Хранение памяти

Функция хранения памяти обычно реализуется клиентом большой языковой модели. Клиент записывает содержание каждого раунда диалога между пользователем и большой языковой моделью и устанавливает определенный объем памяти. Когда пользователь задает новый вопрос, система извлекает из хранилища памяти данные по актуальному диалогу и предоставляет их большой языковой модели в качестве дополнительного контекста, что делает ответы модели более связными, как будто модель обрела способность «запоминать».

Однако у такого механизма памяти есть определенные ограничения: способность модели «запоминать» в основном зависит от длины контекста, который поддерживает большая языковая модель. Если каждый раз передавать в модель всю историю сеанса, то из-за ограничений окна контекста информация может оказаться усеченной. Если выбирать для передачи только историю, относящуюся к актуальному вопросу, то к возможностям клиентского прило-

Книги для программистов: https://t.me/free_books_for_it

жения предъявляются более высокие требования: для отбора наиболее релевантной информации необходимо использовать такие технологии, как сопоставление по степени схожести.

2. RAG

Технология RAG (Retrieval-Augmented Generation, генерация с расширением поиска) позволяет большим языковым моделям получать доступ к актуальным данным в режиме реального времени. Принцип ее работы заключается в следующем: когда пользователь задает вопрос, ИИ-приложение с помощью векторного поиска, сопоставления ключевых слов и других методов извлекает релевантные сведения из внешних баз знаний или источников данных, а затем предоставляет найденную информацию в качестве контекста большой языковой модели, которая на ее основе формирует ответ.

Процесс взаимодействия RAG показан на рис. 1.5.

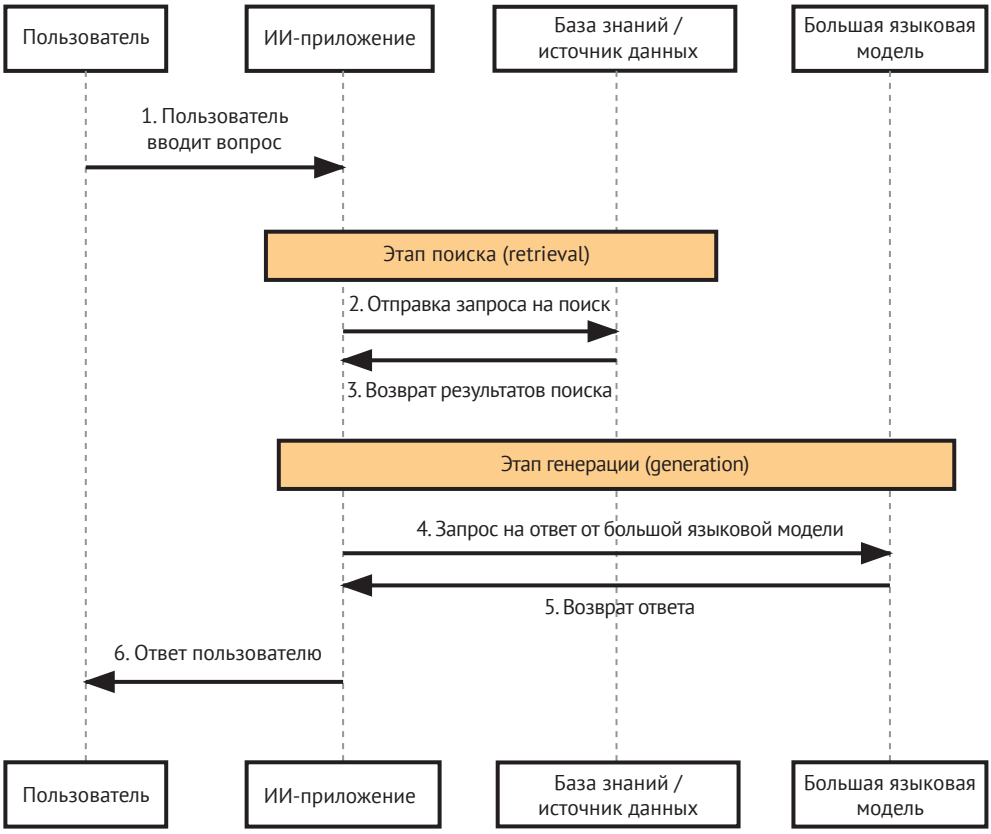


Рис. 1.5. Процесс взаимодействия RAG

Основное преимущество технологии RAG заключается в том, что переобучение модели не требуется (низкая стоимость) и есть возможность гибко обновлять базу знаний (с сохранением актуальности информации). Таким образом, технология RAG может использоваться в таких сценариях, как по-

Книги для программистов: https://t.me/free_books_for_it

иск информации в реальном времени через интернет и диалог с локальными документами (базой знаний).

Ограничения технологии RAG заключаются в том, что качество конечного ответа большой языковой модели зависит от качества найденной информации. Поэтому этап поиска является крайне важным и требует использования эффективных и точных алгоритмов поиска, а также надежных источников данных. Однако в реальных сценариях применения эти два условия редко удается выполнить одновременно.

3. Вызов функций

Вызов функций (Function Calling) – это механизм, позволяющий большой языковой модели выполнять определенные задачи. Он позволяет большой языковой модели преобразовывать запросы на естественном языке в конкретные вызовы функций, чтобы ИИ-приложение могло вызывать внешние инструменты и передавать результаты пользователю.

Принцип работы вызова функций заключается в следующем: когда пользователь задает вопрос, ИИ-приложение в качестве контекста отправляет большой языковой модели список интегрированных функций. Большая языковая модель на основе пользовательского ввода определяет конкретную вызываемую функцию и генерирует соответствующие параметры вызова. Затем ИИ-приложение выполняет эту функцию и передает результаты большой языковой модели в качестве дополнительной информации для генерации окончательного вывода или ответа.

Процесс взаимодействия при вызове функций показан на рис. 1.6.

Вызов функций расширяет границы возможностей больших языковых моделей: благодаря этому они могут подключать данные различных типов и выполнять разнообразные операции.



Вызов функций в сравнении с RAG

Основное отличие между вызовом функций и RAG в плане подключения данных заключается в следующем. RAG – это процесс, при котором ИИ-приложение на основе пользовательского запроса напрямую обращается к фиксированным источникам информации для поиска релевантного контента, а затем предоставляет его в качестве дополнительного контекста большой языковой модели для ответа на вопрос. При вызове же функций ИИ-приложение заранее интегрирует несколько источников данных с помощью функциональных инструментов, которые координируются большой языковой моделью, после чего ИИ-приложение динамически считывает эти данные и наконец предоставляет их большой языковой модели в качестве дополнительного контекста для ответа на вопрос.

С тех пор как в июне 2023 года OpenAI впервые реализовала механизм вызова функций в своих моделях серии GPT, производители крупных языковых моделей последовали этому примеру, и теперь вызов функций стал стандартом для подключения внешних данных к крупным языковым моделям.

Однако спустя 17 месяцев появился MCP.

Книги для программистов: https://t.me/free_books_for_it

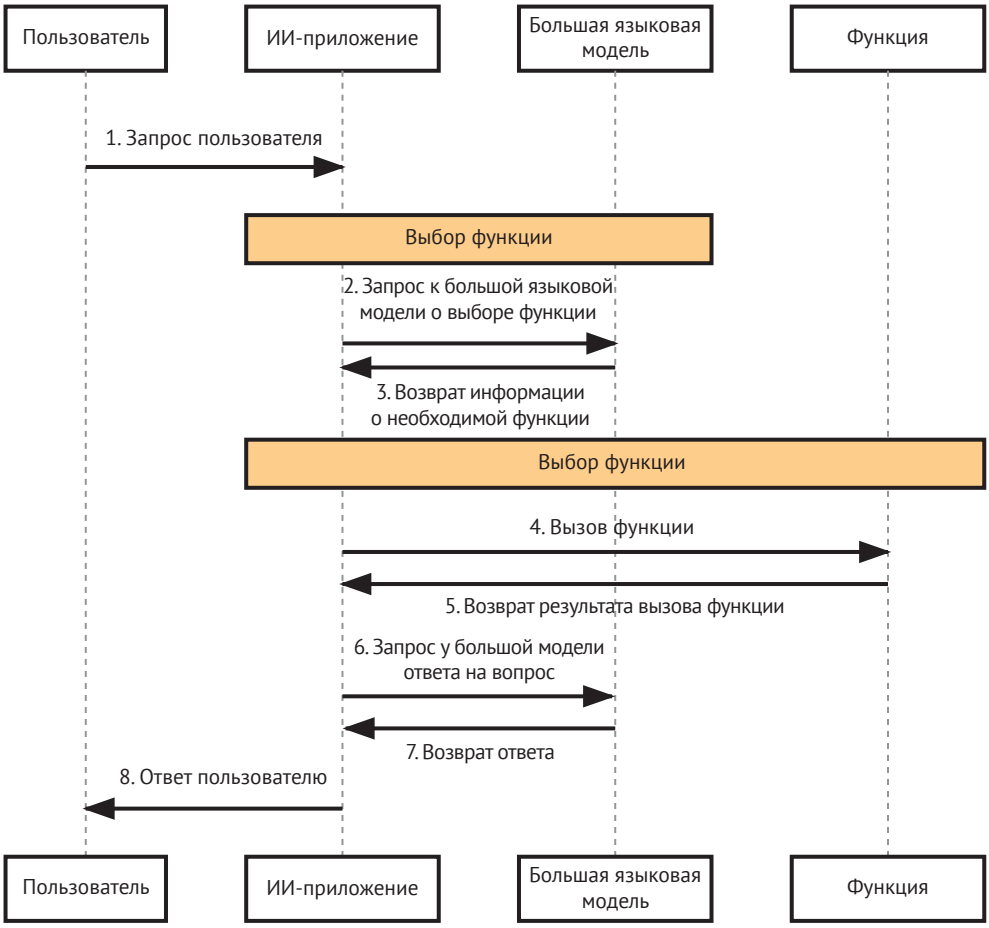


Рис. 1.6. Процесс взаимодействия при вызове функций

4. MCP

Можно считать, что MCP представляет собой дальнейшее усовершенствование и абстрагирование функций вызова, цель которого – сделать подключение ИИ-приложений к внешним ресурсам более простым, эффективным и безопасным, а также обеспечить более качественное дополнение контекстной информации для больших языковых моделей. Поэтому далее рассмотрим MCP в контексте сравнения с вызовом функций.

Вызов функций – это парадигма взаимодействия, по сути, представляющая собой шаблон проектирования, который определяет стандарты вызова внешних функций ИИ-приложениями.

С точки зрения пользователя, процесс взаимодействия при вызове функций включает трех участников: ИИ-приложение, функцию и большую языковую модель. Основной процесс взаимодействия состоит из двух ключевых этапов: выбора функции и вызова функции.

Книги для программистов: https://t.me/free_books_for_it

- **Выбор функции**: ИИ-приложение отправляет список функций и запрос пользователя в большую языковую модель, которая распознает намерение пользователя и выбирает наиболее подходящую функцию для решения его задачи.
- **Вызов функции**: приложение вызывает функцию для получения внешних данных с помощью имени функции и параметров, возвращенных большой языковой моделью.

На рис. 1.6 был представлен процесс взаимодействия при вызове функции, поэтому здесь нет необходимости повторяться.

MCP – это набор протоколов коммуникации, суть которого заключается в стандартизированных спецификациях интерфейсов, форматах данных и соглашениях о взаимодействии, разработанных для обмена данными между ИИ-приложениями и внешними ресурсами.

MCP определяет три роли: MCP-хост, MCP-клиент и MCP-сервер. По сравнению с вызовом функций, MCP представляет собой своего рода «черный ящик», в котором используется связка «MCP-клиент – MCP-сервер».

С точки зрения пользователя, процесс взаимодействия MCP также включает три роли: ИИ-приложение, «черный ящик» (MCP-клиент и MCP-сервер) и большую языковую модель. Процесс взаимодействия MCP (на примере вызова инструмента) представлен ниже (обратите внимание, что упомянуты только основные шаги):

- ИИ-приложение отправляет запрос пользователя MCP-клиенту в «черном ящике»;
- клиент в «черном ящике» запрашивает MCP-сервер в «черном ящике», чтобы получить список инструментов, определенный MCP-сервером;
- ИИ-приложение отправляет список инструментов и запрос пользователя в большую языковую модель, которая выбирает инструмент;
- ИИ-приложение формирует параметры вызова инструмента на основе данных, возвращенных большой языковой моделью, и через MCP-клиент в «черном ящике» отправляет запрос на вызов инструмента MCP-серверу в «черном ящике»;
- MCP-сервер в «черном ящике» запрашивает внешние данные или услуги и возвращает результаты MCP-клиенту;
- ИИ-приложение получает внешние данные, возвращенные «черным ящиком», и отправляет их в качестве контекстной информации в большую языковую модель для генерации ответа.

Процесс взаимодействия MCP можно обобщить с помощью диаграммы, представленной на рис. 1.7.

Книги для программистов: https://t.me/free_books_for_it

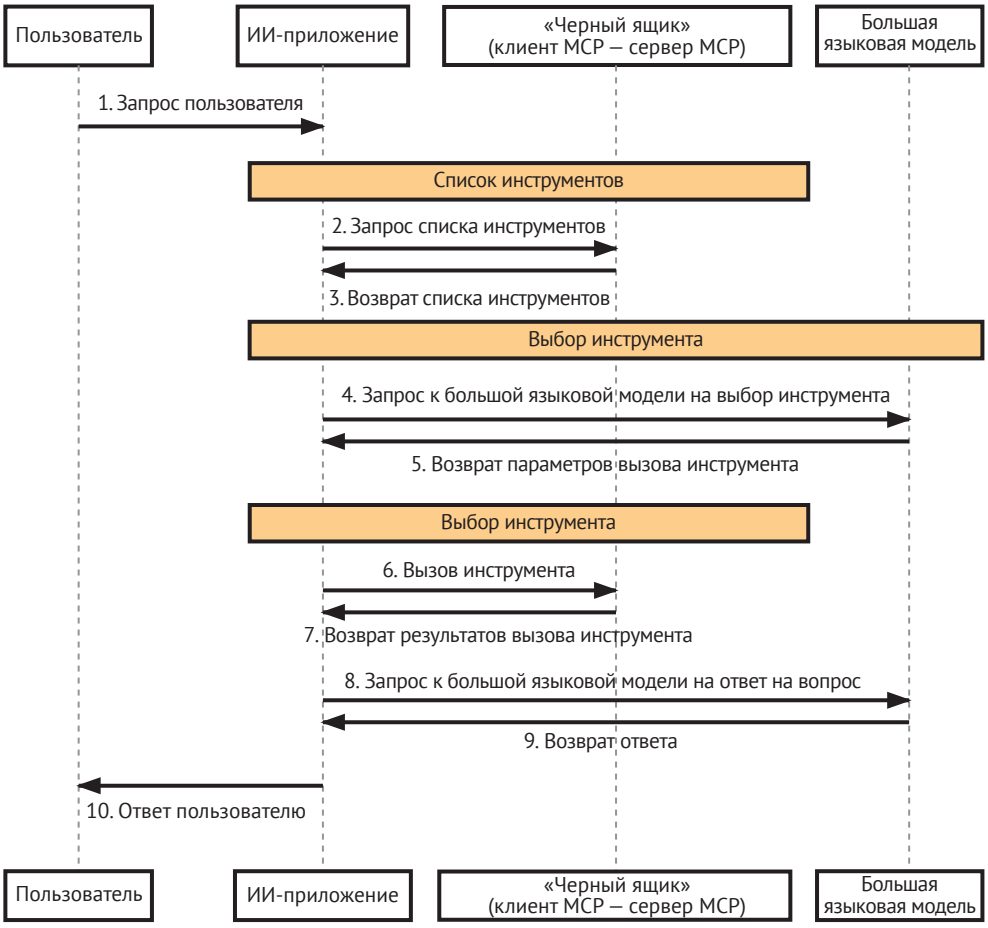


Рис. 1.7. Процесс взаимодействия MCP



Вызов функций в сравнении с MCP

Механизм вызова функций предполагает, что функции определяются непосредственно внутри ИИ-приложения, а взаимодействие с внешними данными или сервисами осуществляется непосредственно самим ИИ-приложением.

В MCP же инструменты определяются на внешнем MCP-сервере, и после добавления MCP-сервера в ИИ-приложение оно получает доступ к списку определенных на нем инструментов. ИИ-приложение не взаимодействует напрямую с внешними данными или сервисами, а осуществляет взаимодействие с внешними ресурсами через сервер MCP.

Считается, что MCP снижает нагрузку в части реализации на стороне ИИ-приложения за счет «аутсорсинга бизнес-функций».

Книги для программистов: https://t.me/free_books_for_it

Ключевым аспектом механизма вызова функций является выбор функций, который определяется способностью большой языковой модели осуществлять вывод. Когда мы говорим, что какая-либо большая языковая модель поддерживает возможность вызова функций, это означает, что данная модель прошла специальную оптимизацию обучения для задач выбора инструментов (включая встраивание описаний функций, обучение специальным форматам и настройку инструкций). Модели с поддержкой вызова функций при получении списка инструментов от ИИ-приложения могут выбрать из него наиболее подходящий инструмент в соответствии с запросом пользователя и сгенерировать структурированные параметры вызова. Благодаря специальной оптимизации модели в плане интерпретации описаний функций и заполнения параметров точность сопоставления обычно превосходит модели, не прошедшие соответствующую подготовку. Модели, не поддерживающие вызов функций, например gpt-3.5-turbo, также могут осуществлять выбор инструментов путем настройки системных подсказок, однако точность сопоставления у них ниже, чем у моделей с поддержкой вызова функций.

MCP – это протокол прикладного уровня на основе механизма вызова функций, который, по сути, предоставляет контекстные услуги для больших моделей в рамках ИИ-приложений. Поэтому можно утверждать, что та или иная большая языковая модель поддерживает или не поддерживает вызов функций, но не следует говорить, что она поддерживает или не поддерживает MCP.

1.2.2. В контексте применения ИИ

1. Появление интеллектуальных агентов

7 ноября 2023 года компания OpenAI на своей первой конференции разработчиков представила семейство нейросетевых языковых моделей GPT (Generative Pre-Trained Transformer, генеративный предварительно обученный трансформер). Это дало пользователям возможность создавать собственных ИИ-помощников с простым использованием естественного языка. Можно рассматривать GPT в качестве ранней формы интеллектуальных агентов.

Выпуск GPT стал важной вехой в развитии интеллектуальных агентов и способствовал их массовому распространению. Вслед за этим крупные производители моделей последовали этому примеру и запустили свои собственные платформы интеллектуальных агентов.

Принцип работы агентов от различных производителей моделей в основном совпадает с принципом работы GPT: пользователь создает их самостоятельно в диалоговых продуктах производителя (таких как Zhipu Qingyan, Doubao и т. д.), где может настраивать системные подсказки, загружать файлы базы знаний, создавать инструментальные функции для интеграции с внешними API и т. д.

Интерфейс создания приложения GPT на платформе ChatGPT представлен на рис. 1.8.

Книги для программистов: https://t.me/free_books_for_it

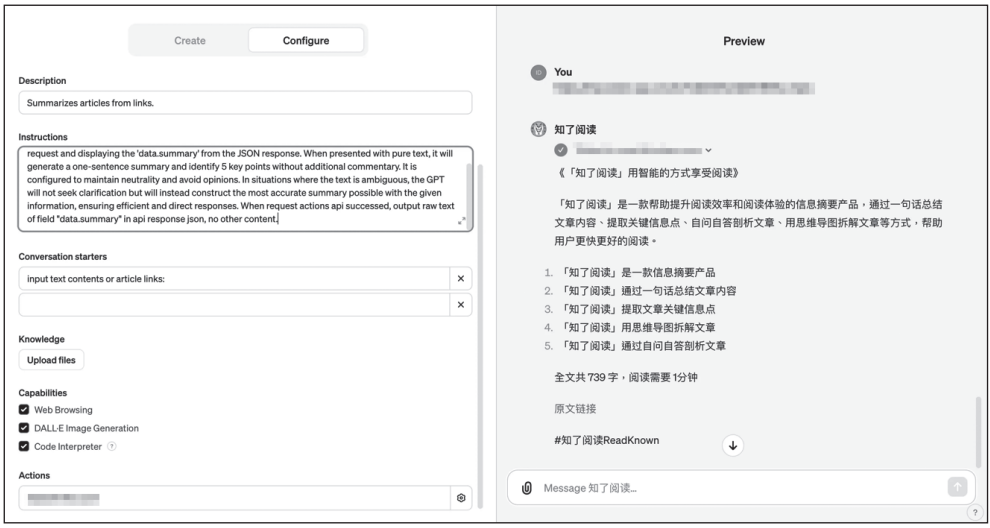


Рис. 1.8. Создание приложения GPT на платформе ChatGPT

Из этого несложно сделать вывод, что суть реализации подобных продуктов заключается в дальнейшей инкапсуляции механизма вызова функций: в дополнение к способности больших языковых моделей к автоматическому созданию контента добавляется возможность получения и обработки бизнес-данных. Однако вследствие резкого роста количества интеллектуальных агентов процветание этой отрасли выявило ряд проблем, требующих неотложного решения.



Некоторое время OpenAI продвигала механизм плагинов (plugins), позволяющий клиентам ChatGPT подключаться к различным внешним данным или сервисам с их помощью, но из-за слишком высоких затрат на разработку это решение в конечном итоге было свернуто. Впоследствии OpenAI представила более легкий вариант GPT, позволяющий пользователям создавать собственные интеллектуальные агенты с помощью простой настройки. Этот механизм на время привел к взрывному росту количества GPT: за короткий промежуток времени появились миллионы сторонних моделей. Тем не менее, поскольку большинство GPT представляли собой лишь простую упаковку подсказок, имели низкий порог входа и не обладали высокой практической ценностью, а также могли использоваться только на платформе ChatGPT, экосистема оказалась слишком закрытой, и со временем интерес к ним пошел на убыль. ИИ продолжает бурное развитие, появляются все новые и новые типы интеллектуальных агентов. Реализация их функций зависит от возможностей больших языковых моделей, но также требует внешних инструментов, в том числе для получения данных в реальном времени. Вопрос о том, как лучше решить проблему интеграции больших языковых моделей с внешними данными или сервисами для упрощения разработки интеллектуальных агентов, стал предметом всеобщего отраслевого интереса.

Книги для программистов: https://t.me/free_books_for_it

2. Проблемы экосистемы ИИ-приложений и их решение с помощью MCP



Искусственный интеллект – это действительно горячая тема! Независимый разработчик Сяоду с нетерпением готов приступить к делу. Он хочет создать «помощника для создания аннотаций веб-страниц»: пользователю достаточно будет нажать кнопку в браузере, чтобы запустить большую языковую модель для составления резюме текста. Такие приложения кажутся простыми, но в процессе работы Сяоду столкнулся с множеством затруднений.

Сначала на платформе ChatGPT он создал модель GPT, настройка которой его вполне устроила. Но когда он решил интегрировать функцию создания резюме в плагин для своего браузера или развернуть ее на других платформах (например, Zhipu Qingyan), то обнаружил, что GPT использовать повторно не получается, и пришлось создавать все заново. Если же экспортировать API через сторонние платформы, такие как Coze или Dify, и подключить его к своему приложению, то возникает проблема несогласованности форматов интерфейсов, что требует огромных затрат на адаптацию.

В то же время некоторым пользователям нужен помощник по созданию резюме, способный анализировать локальные файлы с клиентской перепиской для выявления потенциальных заказчиков. Но поскольку по умолчанию модели GPT обрабатывают данные только в облаке, то при необходимости загрузки файлов пользователи начинают беспокоиться об утечке личных данных. К тому же при каждом обновлении файлов приходится загружать их заново, что значительно снижает удобство использования.

Еще более сложной проблемой оказалось то, что, когда Сяоду захотел улучшить эффективность составления резюме и подключить возможности сетевого поиска Perplexity, ему пришлось писать код для интеграции с API, поскольку с помощью GPT не удалось наладить связь между сервисами. Поставщики услуг заинтересованы в широкомасштабном повторном использовании своих сервисов, клиентские платформы же теряются в многообразии API, а разработчики, оказавшись зажатыми между ними, сталкиваются с чрезвычайно высокими затратами на интеграцию и взаимодействие.

Простая задача по разработке интеллектуального агента выявила три системные проблемы экосистемы ИИ-приложений:

- фрагментацию интерфейсов, приводящую к низкой кросс-платформенной совместимости;
- сложность обеспечения баланса между обработкой данных и конфиденциальностью, ограничивающую раскрытие интеллектуального потенциала;
- низкую эффективность интеграции и расширения сервисов, а также высокие издержки построения экосистемы.

Поговорим о том, как MCP позволяет решить эти три проблемы.

Во-первых, решение проблемы фрагментации интерфейсов: унификация стандартов протоколов, снижение затрат на разработку.

Книги для программистов: https://t.me/free_books_for_it

- **Возможность повторного использования MCP-сервера**. MCP-сервер достаточно разработать один раз, чтобы использовать его в любом ИИ-приложении с поддержкой MCP. Если пользователю требуется использовать какой-либо MCP-сервер, достаточно просто добавить соответствующую настройку в ИИ-приложение. По сравнению с прежними GPT или интеллектуальными агентами затраты на повторное использование MCP-сервера значительно ниже.
- **Унифицированный способ интеграции упрощает разработку MCP-клиента**. При разработке ИИ-приложений, если требуется подключение к внешним ресурсам, разработчикам обычно приходится поочередно интегрироваться с различными внешними API. Различные API имеют разные адреса запросов, форматы параметров и способы аутентификации, а разнообразие способов интеграции приводит к увеличению объема разработки.

При интеграции функций в ИИ-приложения на основе MCP, независимо от количества подключенных MCP-серверов, формат данных запросов и ответов остается единым. ИИ-приложению достаточно создать один MCP-клиент и передавать параметры в соответствии с согласованными MCP правилами, после чего MCP-сервер вернет результаты в едином формате. Благодаря стандартизированному формату обмена данными MCP значительно снижает затраты на разработку и интеграцию для обеих сторон.

Во-вторых, баланс между обработкой данных и конфиденциальностью: безопасное использование, минимальное раскрытие.

- **Безопасное использование конфиденциальных данных**. MCP-сервер может работать в виде локального процесса с доступом к частным данным на компьютере пользователя и предоставлять их в качестве контекста для большой языковой модели. Поскольку весь процесс обработки данных происходит локально, нет необходимости загружать исходные файлы в облако, что эффективно снижает риск утечки конфиденциальной информации.
- **Извлечение по запросу, минимальное раскрытие**. В отличие от традиционных внешних баз знаний, где загружается весь массив данных за один раз, MCP-сервер использует механизм обработки по запросу. Большая языковая модель получает только минимально необходимую информацию, относящуюся к текущему запросу, что снижает вероятность раскрытия конфиденциальных данных.

В-третьих, решение проблемы низкой эффективности подключения сервисов: «функция-как-сервис» для быстрого построения экосистемы.

- **Простая интеграция MCP-сервера**. MCP-сервер может работать не только как внешний плагин, добавляемый пользователем в ИИ-приложение, но и как отдельный сервис, интегрируемый в ИИ-приложение. По сравнению с подключением к функциональным API интеграция ИИ-приложений с сервером MCP проще и гибче. MCP-сервер можно рассматривать как строительный блок: каждый MCP-сервер интегрирует одну или несколько атомарных функций, благодаря чему разработка ИИ-приложений становится похожей на конструирование из кубиков: быстрая сборка значительно повышает эффективность разработки.

Книги для программистов: https://t.me/free_books_for_it

- **Контроль подключения поставщиками данных или услуг**. MCP-сервер отличается возможностью повторного использования, и это является большим преимуществом для поставщиков данных или услуг.

  С помощью механизма «настройка = подключение» MCP позволяет системно решить проблему разрозненности инициатив и неконтролируемости подключений со стороны поставщиков услуг. Поставщикам данных или услуг достаточно инкапсулировать свои возможности в MCP-сервер и опубликовать его конфигурационный адрес, чтобы реализовать принцип «единая разработка, использование на многих платформах».

- **ИИ-приложениям не требуется повторная интеграция**. Благодаря унифицированным стандартам интерфейсов и механизму пользовательской настройки MCP решает проблемы ИИ-приложений, связанные со сложностью отбора, интеграции и подключения.

  ИИ-приложению достаточно одного подключения к MCP для получения возможности обмена данными со всеми MCP-серверами. Обработка на стороне клиента может быть использована повторно, что позволяет избежать значительных издержек на разработку, связанных с написанием логики подключения для каждого сервиса в отдельности.

  Что еще более важно, интеграция сервисов превращается из «разработки подключения» в «настройку пользователем». Для добавления нового сервиса больше не требуется обновлять код MCP-клиента или выпускать новую версию – достаточно, чтобы пользователь вставил адрес конфигурации, и можно сразу же воспользоваться возможностями, предоставляемыми этим MCP-сервером. Такое изменение значительно снижает дополнительные расходы на интеграцию сервисов, благодаря чему ИИ-приложениям становится проще создавать маркетплейсы плагинов или платформы сервисов.

1.2.3. В контексте технического проектирования

1. От LSP к MCP

Из интервью-подкаста с разработчиками MCP Дэвидом Сориа Парра (David Soria Parra) и Джастином Спар-Саммерсом (Justin Spahr-Summers) можно узнать, что дизайн MCP получил свое происхождение от LSP.

LSP (Language Server Protocol, протокол языкового сервера) – это коммуникационный протокол, предложенный Microsoft в 2016 году с целью стандартизации взаимодействия между редакторами / IDE и сервисами языковых сред. Он позволяет отделить от эмуляторов такие функции, как автодополнение кода, проверка ошибок и интеллектуальный переход, и передать их на обслуживание независимым языковым серверам.

LSP основан на JSON-RPC и обеспечивает эффективную коммуникацию между редакторами / IDE и языковыми серверами благодаря унифицированному формату запросов и ответов. Процесс обмена данными с помощью LSP показан на рис. 1.9.

Книги для программистов: https://t.me/free_books_for_it

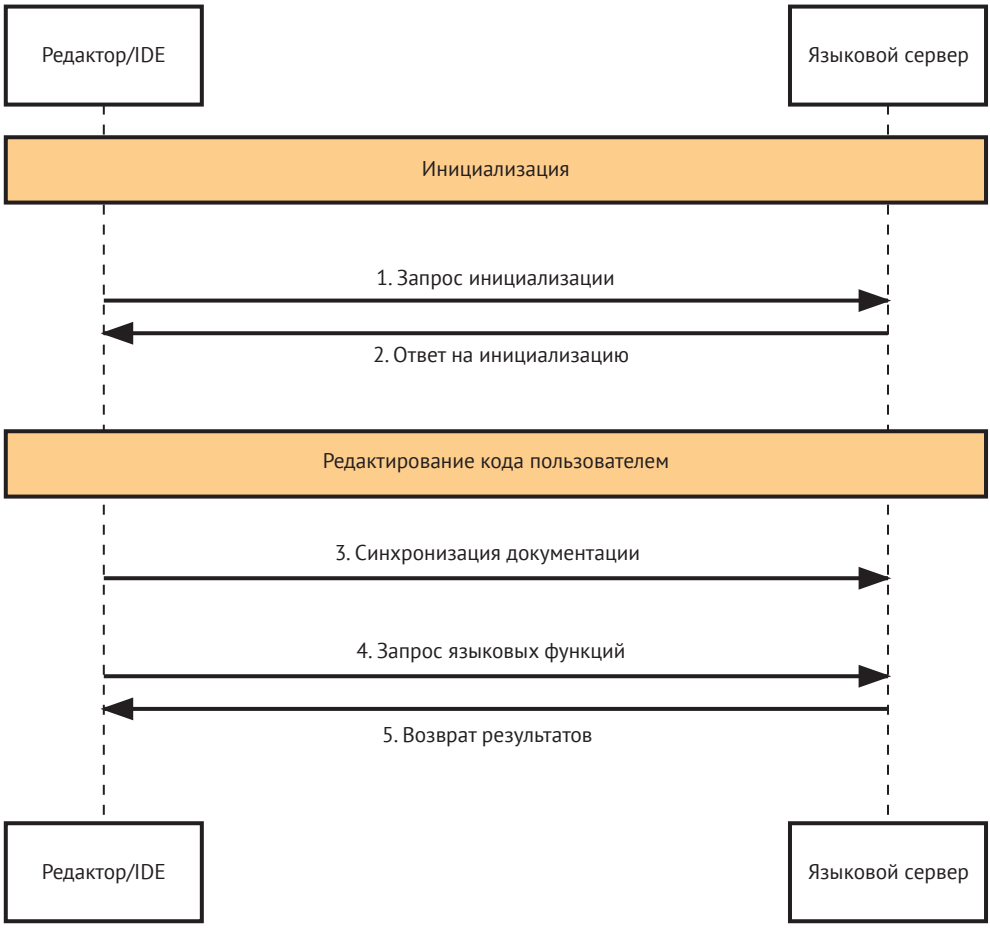


Рис. 1.9. Процесс взаимодействия с помощью LSP

Ключевые преимущества LSP включают:

- **независимость от эмулятора**: разработчики могут пользоваться унифицированными интеллектуальными языковыми сервисами в любом эмуляторе, поддерживающем LSP;
- **исключение повторного программирования**: для каждого языка достаточно реализовать языковой сервер один раз, чтобы он был доступен для вызова из всех редакторов;
- **хорошую масштабируемость**: для добавления поддержки нового языка не требуется модифицировать редактор, достаточно добавить языковой сервер.

В MCP использована архитектура LSP, при этом на прикладном уровне определены три типа ролей: хост, клиент и сервер. Хост – это ИИ-приложение (например, Cursor), в котором можно создавать MCP-клиентов. Комбинация хоста и клиента в MCP аналогична редактору/IDE в LSP. MCP-сервер, в свою очередь, предоставляет конкретные данные или услуги, что аналогично

Книги для программистов: https://t.me/free_books_for_it

языковому серверу в LSP. Кроме того, MCP заимствует из LSP еще несколько других передовых концепций проектирования:

- разделение ресурсов и приложений, подключение к данным или сервисам через внешний сервер;
- поддержку локального запуска – сервер может работать как локальный процесс, обмениваясь данными с клиентом через stdio;
- унифицированный протокол связи, использование JSON-RPC для стандартизированного обмена данными;
- «единожды создано, везде используется»: клиент и сервер необходимо разработать только один раз, после чего они могут использоваться в любой среде, поддерживающей MCP, что позволяет избежать дублирования трудозатрат.

LSP соответствует принципу разделения интересов в программной инженерии и воплощает инженерную концепцию «один раз реализовано, везде используется», успешно способствуя стандартизации и расширяемости экосистемы редакторов. MCP же продолжает и расширяет концепцию управления протоколами. По сравнению с ограниченным количеством языковых серверов LSP масштаб серверов MCP гораздо больше, что позволяет создать более богатую и открытую экосистему сервисов для ИИ-приложений.

2. От HTTP к MCP

После анонса MCP многие посчитали, что наступила «эра HTTP с использованием искусственного интеллекта» и что его ценность невозможно переоценить.

HTTP (HyperText Transfer Protocol, протокол передачи гипертекста) – это базовый протокол Всемирной паутины (WWW), определяющий порядок обмена данными между клиентом и сервером. С тех пор как доктор Тим Бернерс-Ли (Tim Berners-Lee) и его команда в 1989 году выдвинули концепцию Всемирной паутины, а в 1990–1991 годах разработали и реализовали первоначальные версии HTTP и веб-системы, HTTP эволюционировал от протокола, который изначально поддерживал только передачу текста, до современного базового интернет-протокола, поддерживающего веб-страницы, изображения, аудио и видео, 3D-контент и многое другое. Каждое его обновление (например, HTTP/1.1, HTTP/2, HTTP/3) взаимосвязано с прогрессом веб-технологий, и в совокупности они сформировали сегодняшнюю обширную веб-экосистему.

Архитектура MCP аналогична HTTP и также использует архитектуру «клиент–сервер», но вводит новую ключевую роль – хост , создавая свою основную архитектуру «хост–клиент–сервер». Комбинацию хоста и клиента в MCP можно сравнить с браузером в HTTP, а сервер в MCP – с веб-сервером в HTTP.

Для более четкого понимания позиционирования MCP в табл. 1.1 приведено сравнение MCP и HTTP.

Подобно тому, как HTTP более 30 лет назад открыл эру интернета, благодаря чему впоследствии Всемирная паутина действительно вошла в каждый дом и изменила способы получения информации и общения людей, можно предвидеть, что появление MCP знаменует формирование совершенно новой экосистемы нативных ИИ-приложений, а вызванные этим информационные преобразования будут еще более глубокими, чем те, которые принесла с собой эра интернета.

Книги для программистов: https://t.me/free_books_for_it

Таблица 1.1. MCP в сравнении с HTTP

Параметр	HTTP	MCP
Уровень протокола	Прикладной уровень на основе TCP/IP	Прикладной уровень, может работать поверх HTTP
Режим взаимодействия	Пользователь вводит URL-адрес через браузер, браузер взаимодействует с веб-сервером	Пользователь вводит запрос через ИИ-приложение, которое создает MCP. Клиент взаимодействует с MCP-сервером
Модель архитектуры	Клиент–сервер	Хост–клиент–сервер
Назначение	Создание экосистемы веб-приложений	Создание экосистемы ИИ-приложений

1.3. Как MCP обрел популярность

1.3.1. Появление из ниоткуда

Представленный 25 ноября 2024 года MCP, можно сказать, стал настоящим откровением. В день его анонса СМИ по всему миру поспешили осветить это событие, описывая появление MCP словами «шокирующий» и «прорывной». Издания заявляли, что «в сфере ИИ грядут радикальные перемены».

Я сразу же ознакомился с документацией по MCP и очень оптимистично оценил его значимость и перспективы. Тогда я сформулировал следующие три вывода.

- MCP представляет собой усовершенствование механизма вызова функций. Он устанавливает стандарт интеграции внешних ресурсов в крупные языковые модели и позволяет эффективно расширить границы их возможностей.
- Как в случае с вызовом функций, так и в случае с MCP главная сложность заключается в распознавании большими языковыми моделями пользовательских намерений. Ключевым моментом всего механизма является определение наиболее подходящего инструмента для вызова из списка передаваемых клиентом инструментов и возвращение точных параметров вызова.
- Станет ли MCP универсальным стандартом в индустрии ИИ, зависит от создания экосистемы и формирования консенсуса. Здесь присутствует классический «парадокс яйца», то есть вопрос о том, что было первым: курица или яйцо. Если достаточное количество поставщиков данных или услуг откроет свои возможности на основе MCP, другие ИИ-приложения предпочтут последовать их примеру. Если же основные ИИ-приложения будут поддерживать добавление расширений через MCP, сторонние поставщики данных или услуг также будут готовы открыть свои возможности.

1.3.2. Прорыв благодаря эффекту популярности

В начале запуска MCP официальные разработчики выпустили более десяти MCP-серверов для классических приложений, включая такие категории, как сбор веб-контента, обработка временных данных и запросы данных, а также добавили поддержку подключения к MCP-серверам в Claude.

Книги для программистов: https://t.me/free_books_for_it

В течение двух месяцев после запуска MCP пребывал в относительном затишье: время от времени появлялись новые MCP-серверы и ИИ-приложения объявляли о поддержке MCP, но в целом обсуждение темы не вызывало особого ажиотажа.

Начиная с конца января 2025 года MCP наконец-то начал набирать популярность, что связано с обновлениями функций нескольких важных ИИ-приложений, таких как Cursor и Windsurf.

Поддержка MCP в Cursor. 23 января 2025 года известный ИИ-редактор Cursor был обновлен до версии 0.45.x и впервые получил поддержку MCP. Пользователи могут добавлять MCP-серверы, считывать локальные данные или файлы, что еще больше расширяет возможности программирования с помощью Cursor.

Поддержка MCP в Windsurf. 13 февраля 2025 года еще один известный редактор с ИИ, Windsurf, обновился до версии Wave 3 и также получил поддержку MCP. Пользователи могут добавлять в Windsurf различные MCP-серверы для подключения различных функций.



Наряду с поддержкой MCP со стороны известных клиентских продуктов некоторые известные поставщики данных или услуг, в том числе Perplexity, Figma и др., также решили открыть доступ к некоторым своим ресурсам в формате MCP-серверов. Это еще больше способствовало росту интереса к MCP.

В марте 2025 года на фоне выпуска нескольких крупных продуктов и ряда других важных событий в ИИ-отрасли MCP наконец-то достиг настоящего всплеска популярности.

В качестве характерных примеров можно привести следующее.

Выпуск Manus. 6 марта 2025 года группа разработчиков Monica AI выпустила Manus – продукт в виде универсального интеллектуального агента. Manus способен имитировать действия пользователя в браузере и выполнять различные автоматизированные задачи с помощью ИИ.

Сразу после своего появления Manus быстро набрал популярность по всей сети, став еще одним сенсационным ИИ-продуктом вслед за DeepSeek.

После успеха Manus появились первые попытки проанализировать принципы его работы, и были высказаны предположения, что в процессе реализации использовалось большое количество MCP-серверов, например для сетевого поиска, работы с браузером, создания файлов, вызова командной строки и т. д. Ведь ранее MCP-серверы уже обеспечивали аналогичные функции.

Для такого суперинтеллектуального агента использование MCP-серверов для реализации различных функций, а также для дополнения больших языковых моделей контекстными знаниями для лучшего соответствия потребностям пользователей является вполне разумным шагом и соответствует одной из первоначальных установок MCP.

Насколько мне известно, в первоначальной реализации Manus MCP-серверы не использовались, однако популярность Manus привела к стремительному росту интереса к MCP и стимулировала развитие экосистемы MCP.

Фонд a16z опубликовал аналитический отчет рынка. 20 марта 2025 года всемирно известный венчурный фонд Андрессен-Хоровиц (Andreessen

Книги для программистов: https://t.me/free_books_for_it

Horowitz, сокращенно a16z) опубликовал отчет с анализом рынка под названием *A Deep Dive into MCP and the Future of AI Tooling* («Подробный анализ MCP и будущего ИИ-инструментария»), а также обнародовал карту рынка MCP, как показано на рис. 1.10.

В отчете представлена позитивная оценка MCP и стратегический прогноз с точки зрения финансового рынка, что еще больше повысило интерес и популярность MCP среди глобальных инвесторов и представителей технологического сообщества.

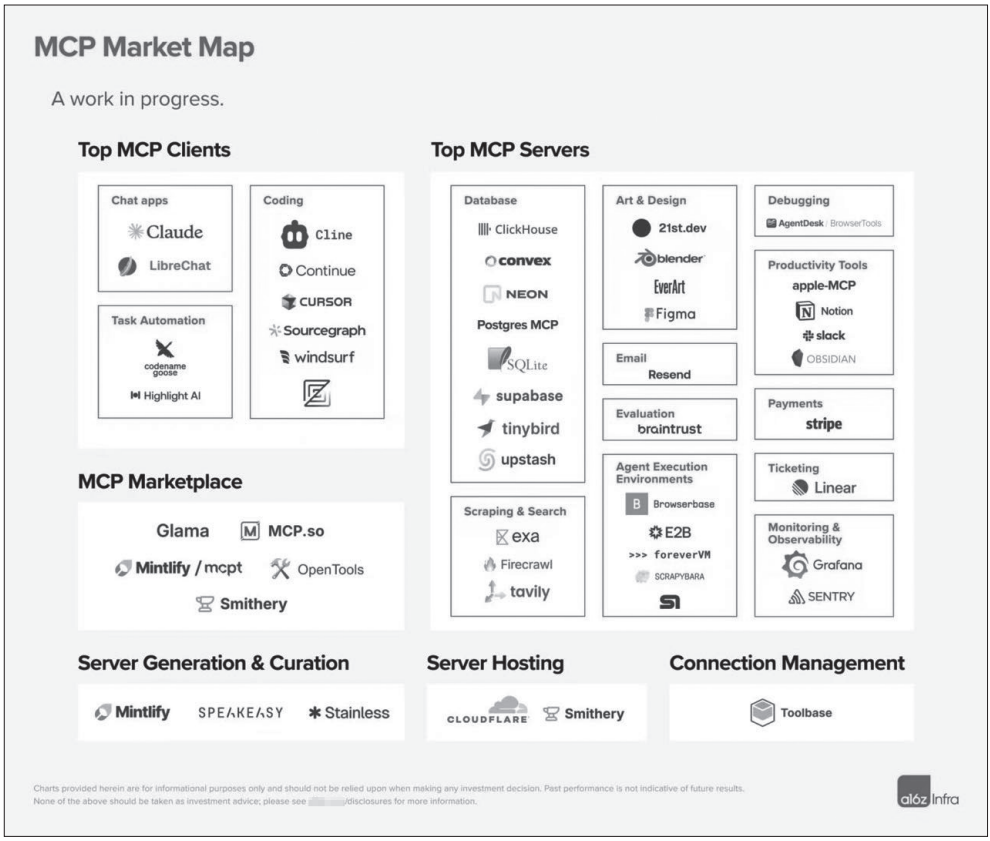


Рис. 1.10. Карта рынка MCP

OpenAI объявила о поддержке MCP. 27 марта 2025 года генеральный директор OpenAI Сэм Альтман (Sam Altman) опубликовал в своем аккаунте соцсети X сообщение с заявлением о том, что, поскольку MCP всем очень понравился, OpenAI решила добавить поддержку MCP в свои продукты. Первым эту функцию получит Agents SDK, а впоследствии команда постепенно внедрит поддержку MCP в настольную версию ChatGPT и в API.

Эта новость привела к новому всплеску интереса к MCP. Ранее многие в отрасли полагали, что OpenAI, будучи главным конкурентом Anthropic, скорее всего, выпустит собственный новый протокол, а не подключится к MCP от своего конкурента.

Книги для программистов: https://t.me/free_books_for_it

Заявление Сэма Альтмана успокоило всех, кто до этого занимал выжидательную позицию по отношению к MCP. Впоследствии различные поставщики данных и услуг стали активно подключаться к MCP, а крупные производители моделей и разработчики ИИ-приложений последовали их примеру.

1.3.3. Стремительный рост за счет двойного сетевого эффекта

Помимо запуска Manus, аналитического отчета a16z и заявления OpenAI, развитие экосистемы MCP ускорили и несколько других важных событий, произошедших в ИИ-сообществе в марте и апреле 2025 года.

- 21, 23 и 31 марта сервисы Baidu Maps, AutoNavi Maps и Tencent Maps поочередно запустили свои MCP-серверы.
- 24 марта компания Zapier представила комплексное решение для MCP, с помощью которого разработчики могут подключаться к тысячам API-сервисов Zapier через MCP-серверы.
- 25 марта Cloudflare представила решение для облачного хостинга MCP, с помощью которого разработчики могут развертывать удаленные MCP-серверы на базе Cloudflare.
- 9 апреля на платформе Alibaba Cloud Bailian появился маркетплейс приложений MCP.
- 10 апреля компания Google объявила о добавлении поддержки MCP в Gemini и SDK.
- 22 апреля компания Docker представила каталог и набор инструментов MCP.
- 23 апреля компания 360 Nano AI выпустила набор инструментов MCP.
- 25 апреля на конференции 2025 AI Developer Conference компания Baidu объявила о полной поддержке MCP.

Волна интереса к MCP продолжает нарастать: различные сервисные платформы и ИИ-приложения активно подключаются к MCP, а экосистема MCP постоянно расширяется.

Процесс развития MCP от момента выпуска до широкого распространения идеально иллюстрирует законы развития двусторонних рынков: на начальном этапе возникает типичный «парадокс яйца и курицы» – отсутствие поддержки со стороны приложений приводит к недостаточному интересу со стороны разработчиков серверных решений, а дефицит серверных решений, в свою очередь, снижает мотивацию для подключения приложений. Однако показательное участие ведущих команд разработчиков вызвало «эффект суперзвезды», который разбил эту тупиковую ситуацию и запустил «двусторонний эффект», благодаря чему стороны разработчиков приложений и серверов начали стимулировать друг друга. По мере постоянного увеличения числа участников экосистемы стал проявляться «сетевой эффект». Каждый новый участник приносит дополнительную ценность всей сети. В конечном итоге это формирует положительный цикл, который способствует взрывному росту экосистемы MCP.

Книги для программистов: https://t.me/free_books_for_it

1.4. Возможности MCP

После неожиданного появления MCP многим он показался похожим на протокол HTTP во времена развития веб-технологий, и это вызвало высокие ожидания относительно его потенциала и перспектив. Однако для большинства рядовых разработчиков, скорее всего, потребуется более глубокое понимание конкретных преимуществ MCP.

В этом разделе мы сосредоточимся на изучении практической ценности MCP, или, другими словами, на том, что именно можно сделать с помощью MCP.

1.4.1. Применение в корпоративной среде

Представьте, что вы руководите компанией, где используется множество систем автоматизации делопроизводства (Office Automation, OA).

Ваши сотрудники должны оформлять отпуска в системе учета рабочего времени, работники отдела закупок должны регистрировать заявки в системе планирования ресурсов предприятия (Enterprise Resource Planning, ERP), менеджеры по продажам должны вводить данные о потенциальных клиентах в систему управления взаимоотношениями с клиентами (Customer Relationship Management, CRM), а сотрудники отдела кадров должны просматривать резюме кандидатов в системе подбора персонала и т. д.

Если сотрудник отвечает сразу за несколько направлений, ему неизбежно приходится переключаться между различными системами и работать с разными типами корпоративных систем или внутренних данных. А вам, как руководителю, также необходимо ежедневно входить в различные внутренние системы, просматривать разнообразную информацию и утверждать различные запросы.

Быстрое развитие ИИ открыло для вас возможности повышения эффективности работы внутри компании с помощью искусственного интеллекта. Вы хотели бы иметь диалоговый ИИ-продукт, подобный ChatGPT, который позволил бы вашим сотрудникам быстро обрабатывать различную информацию посредством диалога?

- Сотрудник A: «Я хочу взять три дня отпуска по личным обстоятельствам перед праздником Цинмин, чтобы поехать на родину и посетить могилу предков». ИИ автоматически генерирует запись об отпуске и вносит ее в систему учета рабочего времени.
- Сотрудник отдела закупок B: «Сегодня было закуплено 20 единиц Mac mini M4 для выдачи новым сотрудникам, прошу зарегистрировать». ИИ автоматически генерирует запись о закупке и вносит ее в ERP-систему.
- Сотрудник отдела продаж C: «Сегодня я обсуждал с генеральным директором компании X Tech наш продукт Y. Он проявил большой интерес и назначил встречу на следующей неделе для более подробного обсуждения». ИИ автоматически генерирует запись о коммуникации с клиентом и вносит ее в CRM-систему.
- Специалист по кадрам D: «Нужно просмотреть пять резюме фронтенд-инженеров и подготовить резюме преимуществ всех пяти кандидатов». ИИ автоматически извлекает резюме из базы данных и выдает итоги HR-специалисту.
- ...

Книги для программистов: https://t.me/free_books_for_it

Вы сообщили отделу разработки о потребностях в развитии такой системы и выдвинули следующие требования:

- система должна интегрироваться с различными внутренними бизнес-ресурсами компании и управляться посредством диалога;
- система должна быть развернута во внутренней сети, а внутренние данные не должны отправляться в публичную сеть;
- цикл разработки должен быть коротким, а запуск в эксплуатацию должны быть быстрым.

Отдел разработки состоит из нескольких команд разработчиков, каждая из которых отвечает за разработку и поддержку одной из внутренних систем. Разработка этого диалогового продукта возложена на новую команду.

До появления MCP разработка нового продукта предполагала привлечение группы разработчиков внутренних систем для экспорта ключевых функций этих систем в виде API, а затем поочередную интеграцию API внутренних систем в новый продукт. Этот процесс был сопряжен с очень высокими расходами на взаимодействие и координацию.

Благодаря MCP разработка подобных систем становится проще и эффективнее:

- имеющиеся внутренние системы (например, ERP, CRM) не должны открывать API, достаточно просто создать MCP-сервер и открыть доступ к ключевым ресурсам или функциональным модулям;
- для разработки нового диалогового продукта не нужно поочередно подключать его к внутренним системам, достаточно реализовать функции, необходимые для MCP-клиента (подключение к MCP-серверу, отправка запросов на получение ресурсов / подсказок / инструментов, вызов инструментов и т. д.) в соответствии с требованиями MCP;
- на этапе окончательного развертывания и запуска достаточно разместить диалоговый продукт и MCP-серверы, соответствующие различным внутренним системам, в одном месте, обеспечив их взаимодействие через локальную сеть.

Этот пример демонстрирует ценность MCP для внутренних корпоративных приложений:

- благодаря согласованным правилам на уровне протоколов открытость и подключение систем становятся стандартизированными, что снижает издержки на взаимодействие и координацию;
- благодаря локальной связи между процессами обеспечивается безопасность взаимодействия внутренних систем, что отвечает требованиям предприятий, для которых информационная безопасность носит критически важный характер.

1.4.2. Применение в сценарии персонального ИИ-ассистента

Пользователи, привыкшие работать на персональном компьютере, наверняка знакомы с приложениями-лаунчерами. Будь то macOS, Windows или Linux, такие приложения помогают быстро находить файлы, запускать программы и т. д., тем самым повышая эффективность работы. Встроенный в macOS лаун-

Книги для программистов: https://t.me/free_books_for_it

чер называется Spotlight, а для повседневной работы автор использует стороннее приложение uTools, как показано на рис. 1.11.



Рис. 1.11. Стороннее приложение-лаунчер uTools в macOS

При каждом вызове uTools с помощью сочетания клавиш **Command**+**Space** и ввода ключевого слова открывается рабочий интерфейс встроенных приложений лаунчера, где можно выполнять такие операции, как перевод, форматирование, поиск файлов, поиск в сети и т. д.

В uTools встроено более сотни приложений, охватывающих повседневные офисные задачи. Эти приложения могут быть либо официально встроены в uTools, либо предоставлены сторонними разработчиками. Каждое приложение соответствует плагину, реализованному с помощью кода, и становится доступным сразу после установки лаунчера uTools.

Можно ли использовать концепцию дизайна приложения-лаунчера в качестве основы для разработки личного ИИ-ассистента на базе MCP?

Представьте, что с помощью сочетания клавиш вы открываете окно ввода и можете задать следующие вопросы:

- загляни в группу обмена информацией об ИИ-продуктах в WeChat и расскажи об обсуждаемых темах;
- проверь системный календарь и покажи расписание моих встреч на эту неделю;
- открой браузер и посмотри, какие новости входят в тройку лидеров популярных поисковых запросов в Weibo;
- помоги мне упорядочить файлы в каталоге *~/Documents*, сгруппировав их по месяцам.

Разве вам не хотелось бы иметь на своем компьютере постоянного ИИ-ассистента, который выполнял бы подобные задачи по вашей команде?

У вас может возникнуть вопрос: какое это имеет отношение к MCP? Разве ChatGPT и Doubao уже не делают это?

Книги для программистов: https://t.me/free_books_for_it

Действительно команды таких продуктов, как ChatGPT и Doubao, также продвигают свои настольные версии и предоставляют аналогичные функции, как у приложений-лаунчеров. С точки зрения производственной стратегии каждый хочет занять лидирующие позиции в сегменте интернет-доступа на компьютерах пользователей.

Однако у подобных продуктов в настоящее время есть следующие проблемы.

- **Данные хранятся в облаке**. Данные продуктов ChatGPT, Doubao и других версий для настольных компьютеров хранятся в облаке. Это означает, что содержание ваших диалогов в клиентском приложении отправляется на их облачные серверы. Например, когда вы просите Doubao проанализировать какой-либо файл чата, его сначала необходимо загрузить в облако, чтобы он был обработан. Этот процесс может вызвать у вас опасения по поводу утечки конфиденциальной информации.
- **Отсутствие прямого доступа к локальным ресурсам**. Такие настольные приложения, как правило, не могут напрямую считывать ваши локальные ресурсы. Для работы с ними необходимо вручную загрузить файлы, что существенно усложняет процесс их использования.
- **Ограниченные возможности расширения**. Настольные приложения, такие как ChatGPT и Doubao, по сути, являются сервисами, закрытыми и поддерживаемыми только разработчиками, и у пользователей нет возможности самостоятельно расширять их возможности. Например, если пользователь хочет автоматически сохранять содержание своих диалогов с Doubao в локальном приложении для заметок, он не имеет других автоматизированных способов для этого, кроме копирования и вставки.

Реализация персонального ИИ-ассистента на базе MCP обеспечивает больше преимуществ и более высокое качество обслуживания.

- **Ограниченный доступ к личным данным, безопасность и конфиденциальность**. Серверы MCP поддерживают чтение личных данных пользователя в локальном ограниченном диапазоне, без необходимости загрузки пользователем всего файла с личными данными в персональный ИИ-ассистент, что повышает уровень доверия со стороны пользователей.
- **Модульная архитектура, высокая гибкость**. Нет необходимости встраивать в персонального ИИ-помощника множество функций; вместо этого используется модульная архитектура, позволяющая пользователю самостоятельно выбирать, какие серверы MCP использовать, что обеспечивает более высокую гибкость персонального ИИ-помощника.
- **Неограниченные возможности расширения**. Пользователи могут самостоятельно разрабатывать или устанавливать готовые MCP-серверы, чтобы управлять различными локальными ресурсами. Использование MCP-серверов в персональном ИИ-помощнике дополняет контекст для больших моделей, позволяя им лучше понимать и реализовывать потребности пользователей.

Книги для программистов: https://t.me/free_books_for_it

1.4.3. Помощь поставщикам ресурсов в расширении возможностей

Провайдеры данных или услуг, такие как Perplexity, открывают часть своих функциональных возможностей посредством открытых API, позволяя сторонним приложениям подсоединяться к ним и использовать их, что, по сути, призвано повысить доходы в сегменте ToB (то есть B2B – Business-to-Business, корпоративный сектор).

Однако подключение к API таких провайдеров требует от разработчиков приложений дополнительных затрат на доработку и настройку в соответствии с документацией по API.

Предположим, что какое-либо ИИ-приложение для реализации функции сетевого поиска должно самостоятельно подключиться к поисковому сервису. Если и Perplexity, и Tavily предоставляют функцию сетевого поиска, то при подключении к одному из них вероятность подключения к другому будет низкой. Это связано с тем, что каждое подключение требует повторной настройки нового интерфейса API, что влечет за собой дополнительные временные затраты.

С появлением MCP компании Perplexity и Tavily реализовали на его основе собственные MCP-серверы, через которые открыли доступ к своим инструментам сетевого поиска.

Для разработчиков приложений затраты на интеграцию становятся ниже: достаточно реализовать один MCP-клиент, чтобы подключиться ко всем подобным сервисам одновременно. Несомненно, это стимулирует разработчиков к более широкой интеграции.

Даже если само приложение не подключается к MCP-серверам сетевого поиска, но позволяет пользователям самостоятельно добавлять внешние MCP-серверы, то MCP-серверы Perplexity и Tavily также могут быть добавлены пользователями в их ИИ-приложения.

Благодаря MCP поставщики ресурсов могут реализовать принцип «один раз открыть, везде использовать». Это приводит к увеличению количества B2B-вызовов у поставщиков ресурсов и росту доходов от B2B-бизнеса. С ростом числа MCP-клиентов эта тенденция будет только усиливаться.

1.4.4 Помощь разработчикам в эффективном создании ИИ-агентов

В разделе 1.3 мы упомянули, что взлет популярности MCP в определенной степени был вызван появлением Manus.

Согласно опубликованному в интернете анализу технологий Manus стало известно, что в процессе его создания было интегрировано 29 инструментов для вызова больших моделей:

- 12 инструментов для работы с браузером (поддерживают взаимодействие ИИ-приложений с браузером и веб-сайтами);
- 5 инструментов для работы с оболочкой (позволяют пользователю взаимодействовать с системой через интерфейс командной строки);
- 5 инструментов для работы с файлами (поддерживают открытие, чтение, запись, создание файлов и т. д.);

Книги для программистов: https://t.me/free_books_for_it

- 3 инструмента для развертывания (развертывают сгенерированный код в контейнере, получая общедоступный URL-адрес);
- 2 инструмента для обмена сообщениями (отправка, приостановка, удаление, потоковый ответ на сообщения и т. д.);
- 1 инструмент для поиска информации (поиск в сети);
- 1 инструмент IDLE (Integrated Development and Learning Environment, интегрированная среда разработки и обучения).

Многие разработчики, вдохновленные возможностями Manus, решили повторить его успех и создать своих собственных ИИ-агентов.

Если разработчику придется самостоятельно реализовывать все 29 вышеупомянутых инструментов, это, без сомнения, потребует огромных временных затрат на их создание и отладку. Однако благодаря MCP и его экосистеме разработчики могут выбирать подходящие инструменты из числа MCP-серверов, опубликованных другими разработчиками со всего мира (без необходимости повторной разработки), а затем собирать и интегрировать их по принципу конструктора, чтобы создавать собственные продукты на базе ИИ-агентов. Каждый инструмент на каждом MCP-сервере может выступать в качестве отдельной функциональной единицы и помогать разработчикам в более быстром и качественном построении таких интеллектуальных систем.

Благодаря объединению разработчиков со всего мира MCP значительно снижает технический порог для построения интеллектуальных систем, что свидетельствует о высокой ценности этой экосистемы.

1.5. Заключение

В этой главе было дано системное представление о принципах работы MCP и история его появления. Сначала на примере серверов Amap объясняется принцип работы MCP. Далее с точки зрения больших моделей, приложений ИИ и технического проектирования были рассмотрены решаемые MCP задачи и его особые преимущества. Затем был проанализирован процесс роста популярности MCP, а в заключение было рассказано о возможностях MCP. Изучив содержание этой главы, читатель, пожалуй, уже понимает, как развивалась платформа MCP и какие у нее конкретные сценарии применения. Далее приступим к детальному разбору архитектуры протокола, чтобы полностью разобраться в принципах работы MCP.

Книги для программистов: https://t.me/free_books_for_it

Глава 2

Анализ архитектуры MCP

После ознакомления с принципами работы MCP, историей его появления и этапами развития мы получили общее представление об этом протоколе и узнали о том, что его можно использовать для интеграции с внешними данными или сервисами, тем самым предоставляя контекст большим языковым моделям.

В этой главе основной фокус уделяется архитектуре MCP, подробному анализу ее основной структуры, принципам коммуникации, жизненному циклу соединений, механизмам передачи данных, возможностям и способам аутентификации и авторизации. Это позволит читателям глубже вникнуть в суть концепции MCP.

2.1. Основная архитектура: хост–клиент–сервер

2.1.1. Введение в архитектуру

В MCP традиционная архитектура «клиент–сервер» (C/S, client/server) была дополнена ролью хоста (host), что позволило создать базовую архитектуру «хост–клиент–сервер», как показано на рис. 2.1.

В разделе 1.1 были представлены основные определения хоста, клиента и сервера в MCP, а также на примере было продемонстрировано их взаимодействие. Далее мы подробно рассмотрим компоненты архитектуры MCP с точки зрения их задач и функций.

1. Хост

В архитектуре MCP хост выполняет роль координатора. Он отвечает за создание и управление несколькими клиентскими процессами, а также за контроль прав доступа и жизненного цикла этих клиентов. Одновременно хост должен обеспечивать безопасное соединение между клиентами и сервером, а при необходимости запрашивать авторизацию у пользователя. Помимо этого, хост выполняет задачи интеграции и выборки данных из больших языковых моделей, а также отвечает за управление агрегированием контекста между клиентами. Таким образом обеспечивается эффективное взаимодействие и безопасная работа всей системы.

Книги для программистов: https://t.me/free_books_for_it

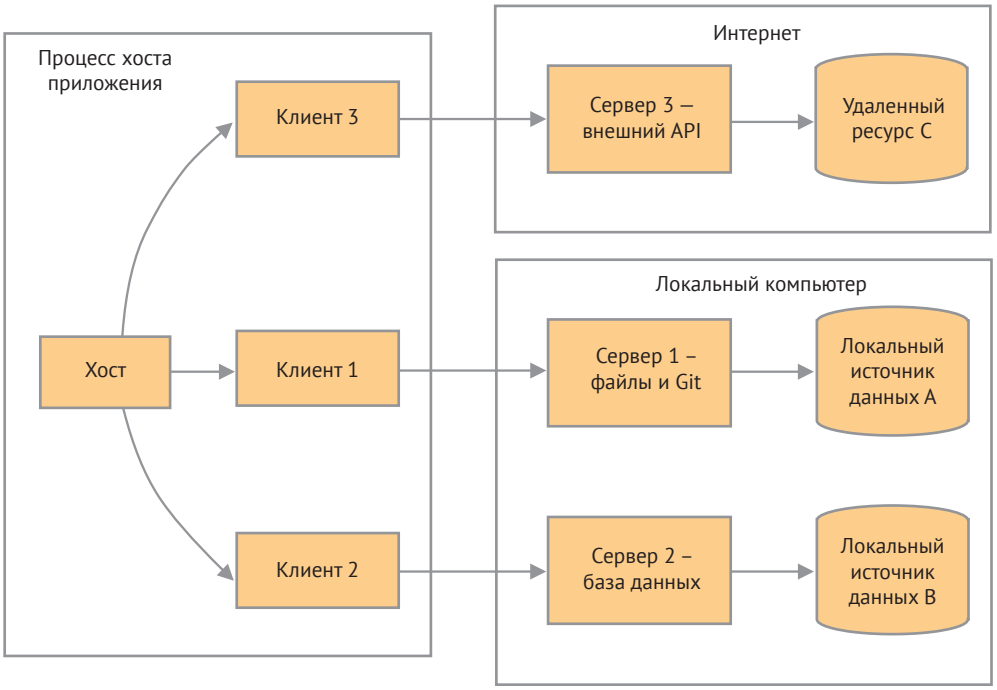


Рис. 2.1. Архитектура «хост–клиент–сервер» MCP

2. Клиент

В архитектуре MCP клиент можно рассматривать как бизнес-процесс внутри хоста, который может устанавливать соединение с серверным процессом, осуществлять обмен данными и тем самым помогать хосту получать необходимые внешние ресурсы.

Основные задачи клиентского процесса включают в себя установку и сопровождение состояния сеансов для каждого сервера, согласование версий протоколов и возможностей для беспрепятственной коммуникации между сторонами, а также двунаправленную маршрутизацию протокольных сообщений для своевременной передачи информации. Кроме того, клиент отвечает за управление подписками и уведомлениями.



Что такое процесс?

Процесс – это динамическое выполнение программы, исполняющей определенные независимые операции с конкретным набором данных. Он является базовой единицей распределения ресурсов и планирования операционной системы, а также основным механизмом работы приложений.

Один хост может создавать и администрировать несколько клиентских процессов, каждый из которых поддерживает независимое соединение 1:1 с конкретным серверным процессом.

Книги для программистов: https://t.me/free_books_for_it

3. Сервер

В архитектуре MCP сервер – это программа, работающая вне хоста и предназначенная специально для предоставления системе определенных ресурсов и функциональных возможностей. Серверы работают независимо и через MCP предоставляют хосту и клиентам подсказки, ресурсы и инструменты. Серверы могут инициировать запросы на выборку данных через клиентский интерфейс, при этом они должны строго соблюдать все ограничения безопасности. Будучи локальным процессом или удаленной службой, сервер обеспечивает гибкое расширение функциональности системы.

2.1.2. Принципы разработки

При разработке решений на основе MCP соблюдается ряд ключевых принципов, лежащих в основе как архитектуры, так и ее реализации.

- **Принцип разделения**. В MCP уделяется особое внимание упрощению создания серверов. Хост отвечает за выполнение сложных задач оркестрации, а серверы фокусируются на реализации своих специфических функций. Это позволяет снизить сложность разработки за счет использования лаконичных интерфейсов и повысить удобство обслуживания кода благодаря принципу разделения.
- **Модульная архитектура**. В MCP приветствуется высокая степень модульности серверов. Каждый сервер специализируется на отдельном функциональном модуле, несколько серверов способны бесшовно взаимодействовать благодаря общему протоколу, а модульный дизайн обеспечивает возможность непрерывного масштабирования системы.
- **Минимальное раскрытие информации**. В целях обеспечения безопасности и изоляции MCP предписывает, чтобы серверы имели доступ только к необходимой контекстной информации, а полная история диалогов находилась под единым управлением хоста. Подключения между серверами изолированы друг от друга, а взаимодействие между ними координируется хостом. Процесс хоста одновременно определяет границы безопасности, предотвращая утечку информации и несанкционированный доступ.
- **Постепенное расширение**. Основной протокол предоставляет только самые базовые функции. Сервер и клиент могут согласовывать и добавлять дополнительные возможности в соответствии с фактическими потребностями, при этом обе стороны могут развиваться независимо друг от друга. Сам протокол имеет расширяемую архитектуру, что позволяет в будущем поддерживать больше новых функций, сохраняя при этом совместимость со старыми версиями.

2.1.3. Ключевые компоненты

MCP состоит из нескольких ключевых компонентов, работающих координированно. В целом система имеет модульную архитектуру с четким разделением сфер ответственности, что повышает удобство ее обслуживания и создает основу для гибкого взаимодействия между клиентом и сервером.

Книги для программистов: https://t.me/free_books_for_it

Базовый протокол. MCP четко определяет способ кодирования при обмене данными между клиентом и сервером (JSON-RPC), полный жизненный цикл соединения, механизм передачи сообщений, а также процесс согласования доступных возможностей между сторонами.

Модули функций. Сервер и клиент могут реализовывать широкие возможности расширения. Например, сервер может предоставлять клиенту различные ресурсы, подсказки и инструменты для вызова пользователями или большими языковыми моделями. В свою очередь, клиент может предоставлять серверу такие возможности, как выборка данных, а также ограничивать доступ к рабочим каталогам сервера.

Дополнительные инструменты. MCP поддерживает такие функции, как управление конфигурацией, отслеживание статуса выполнения, прерывание связи, отчеты об ошибках и ведение логов, что дополнительно расширяет возможности взаимодействия между клиентом и сервером.

Схема авторизации. MCP предоставляет схему авторизации на основе HTTP, которая подходит для сценариев передачи данных по HTTP. Для передачи данных через `stdio` учетные данные извлекаются из переменных среды. Кроме того, клиент и сервер могут согласовывать пользовательские политики аутентификации и авторизации в соответствии с фактическими потребностями.

2.1.4. Механизм согласования функциональных возможностей

MCP использует механизм согласования функциональных возможностей. На этапе инициализации клиент и сервер объявляют о поддерживаемых ими возможностях через поле `capabilities`. Эти заявления определяют способ взаимодействия и границы возможностей обеих сторон на протяжении всего сеанса.

Так, например, сервер может заявить о поддержке таких возможностей, как подсказки, ресурсы, инструменты и т. д.; клиент, в свою очередь, может заявить о поддержке таких возможностей, как корень (узел), выборка и т. д. На протяжении всего сеанса обе стороны при исполнении логики должны строго следовать заявленным возможностям. Сам протокол также поддерживает согласование добавления дополнительных расширенных возможностей в целях поддержки поэтапной эволюции.

Каждое заявление о возможностях должно соответствовать фактической реализации. Например:

- возможности должны быть объявлены явным образом, если сервер реализует какую-либо возможность, это должно быть явно указано в поле `capabilities`;
- права ограничиваются возможностями, сервер, не объявивший возможность подписки, не сможет получать уведомления о подписке на ресурсы;
- вызов инструментов также должен быть объявлен: если сервер поддерживает возможность использования инструментов, это должно быть отмечено в списке возможностей.

Ниже приведен пример объявления клиентом возможности выборки и корня в запросе инициализации:

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {}
    },
    "clientInfo": {
      "name": "ExampleClient",
      "version": "1.0.0"
    }
  }
}
```

Если клиент поддерживает функцию выборки, это необходимо явно указать в его декларации возможностей. Такой механизм согласования возможностей обеспечивает обеим сторонам коммуникации четкое понимание границ своих возможностей, а также создает механизм гарантии расширяемости протокола.

2.1.5. Безопасность коммуникаций

MCP предоставляет системе широкие возможности доступа к данным и выполнения кода, однако это также создает множество проблем в области безопасности и надежности. Поэтому все разработчики должны уделять пристальное внимание соответствующим рискам и обеспечить использование протокола в контролируемых и надежных пределах.

1. Основные принципы безопасности

Осведомленность и автономия пользователя. Пользователь должен иметь полное право на информацию и право принятия решений в отношении всех операций доступа к данным и их обработки, а также возможность самостоятельно выбирать, какие данные могут быть переданы и какие операции могут быть выполнены. Разработчики протокола должны предоставить пользователю понятный и интуитивный интерфейс, облегчающий проверку и авторизацию различных действий.

Защита конфиденциальности данных. Перед раскрытием пользовательских данных серверу хост-устройство должно получить явное согласие пользователя. Без разрешения хост-устройство не имеет права произвольно передавать данные ресурсов в другие места. Все пользовательские данные подлежат строгому контролю доступа и защите.

Безопасность вызова инструментов. Поскольку вызов инструментов, по сути, предполагает выполнение произвольного кода, здесь необходимо проявлять особую осторожность. Описания поведения инструментов (например, комментарии) не должны приниматься на веру, если они не поступают с надежного сервера. Перед вызовом любого инструмента хост обязательно должен получить явное разрешение пользователя и удостовериться, что пользователь

Книги для программистов: https://t.me/free_books_for_it

до предоставления разрешения был полностью проинформирован о конкретных возможностях каждого инструмента.

Управление правами на выборку из больших языковых моделей. Все запросы, связанные с выборкой из больших языковых моделей, требуют явного одобрения пользователя. Пользователь должен принимать самостоятельное решение о том, разрешать ли выборку, отправлять ли подсказки, а также определять объем результатов выборки, доступных для просмотра сервером. Сам протокол также должен ограничивать доступность содержания подсказок для сервера, чтобы снизить потенциальные риски.

2. Рекомендации по внедрению

Несмотря на то что MCP не может обеспечить соблюдение всех принципов безопасности на уровне протокола, разработчики сами должны взять на себя ответственность за безопасность и принять следующие меры:

- внедрить в приложениях надежные механизмы получения согласия и авторизации;
- предоставить четкую документацию с описанием рисков безопасности;
- внедрить строгие меры контроля доступа и защиты данных;
- следовать лучшим отраслевым практикам и осуществлять интеграцию в соответствии со стандартами;
- на этапе проектирования функциональности тщательно оценивать риски для соблюдения конфиденциальности пользовательских данных.

2.2. Основы коммуникации: JSON-RPC

В MCP используется JSON-RPC в качестве основы для взаимодействия между клиентом и сервером. Прежде чем перейти к описанию JSON-RPC, давайте вкратце вспомним, что такое JSON и RPC.

2.2.1. Что такое JSON

JSON (JavaScript Object Notation, нотация объектов JavaScript) – это облегченный формат обмена данными, удобный для чтения и написания людьми. Он также удобен для машинного анализа и генерации. JSON основан на подмножестве языка программирования JavaScript, но является независимым от этого языка.

Основные особенности JSON:

- использование простого текстового формата, удобного для передачи и хранения;
- поддержка вложенных структур данных, позволяющая представлять сложные типы данных;
- типы данных включают объекты, массивы, строки, числа, логические значения и `null`;
- является стандартным форматом для многих веб-API и файлов конфигурации.

Книги для программистов: https://t.me/free_books_for_it

Синтаксис JSON лаконичен и понятен, данные организуются с помощью пар «ключ–значение», что делает его отличным универсальным форматом для обмена данными между различными системами. Пример структуры данных с использованием JSON приведен ниже.

```
{
 "name": "Чжан Сань",
 "age": 30,
 "isStudent": false,
 "hobbies": ["чтение ", "путешествия", "программирование"],
 "address": {
 "city": "Пекин",
 "postcode": "100000"
 },
 "phoneNumbers": [
 {
 "type": "домашний",
 "number": "010-12345678"
 },
 {
 "type": "рабочий",
 "number": "138XXXXXXXX"
 }
 ],
 "spouse": null
}
```

2.2.2. Что такое RPC

RPC (Remote Procedure Call, удаленный вызов процедур) – это протокол связи между компьютерами, который позволяет программам вызывать подпрограммы на другом компьютере так же, как и локальные функции, без необходимости написания низкоуровневого кода для взаимодействия по сети. К распространенным RPC-фреймворкам относятся gRPC, Thrift, Dubbo и др.

Основные особенности RPC:

- служит базовым механизмом для организации обмена данными в распределенных системах;
- позволяет выполнять удаленные вызовы на уровне кода так же, как и локальные;
- скрывает сложность низкоуровневой сетевой коммуникации;
- поддерживает взаимодействие между различными языками программирования и платформами.

Типичная система RPC состоит из двух ключевых компонентов: клиента и сервера. Клиент инициирует запрос, вызывая программу на удаленном сервере, а сервер выполняет запрошенную операцию и возвращает результат. Процесс коммуникации по протоколу RPC показан на рис. 2.2.

Книги для программистов: https://t.me/free_books_for_it

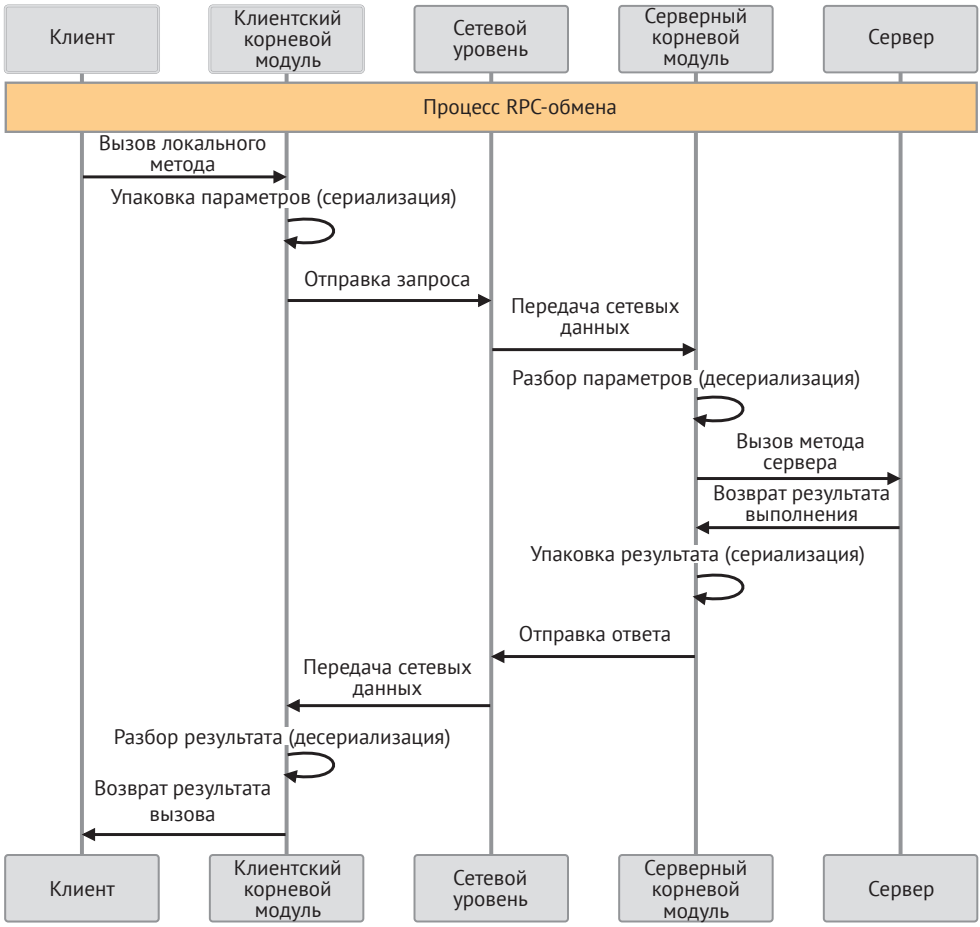


Рис. 2.2. Процесс коммуникации через RPC

Как показано на рисунке, коммуникация через RPC в основном выполняет две задачи:

- выбор формата сообщения для сериализации и десериализации параметров запроса и ответа;
- выбор протокола передачи данных для отправки и получения данных.

2.2.3. Что такое JSON-RPC

JSON-RPC – это облегченный протокол RPC на основе JSON. Он определяет унифицированный формат данных и правила обработки, используемые для удаленного вызова методов между различными системами.

Основные особенности JSON-RPC включают:

- использование JSON в качестве формата сообщений;
- поддержку стандартной модели обмена запрос–ответ;
- простоту и удобство использования, а также независимость от языка программирования и простоту кросс-платформенной реализации;

Книги для программистов: https://t.me/free_books_for_it

- возможность передачи данных по HTTP/HTTPS или другим транспортным протоколам;
- стандартную структуру запроса JSON-RPC, которая включает в себя имя метода, параметры и ID запроса. Ответ содержит результат или сообщение об ошибке, а также ID, соответствующий запросу.

На момент написания книги последней версией JSON-RPC была 2.0. MCP использует JSON-RPC 2.0 в качестве основы для связи между клиентом и сервером.

2.2.4. Спецификация протокола JSON-RPC 2.0

1. Объект запроса

Объект запроса JSON-RPC 2.0 содержит следующие поля:

- `jsonrpc`: строка с указанием версии протокола JSON-RPC, которая должна быть 2.0;
- `method`: строка, обозначающая имя вызываемого метода;
- `params` (опционально): объект или массив, содержащий параметры, передаваемые методу;
- `id`: строка, число или `null`, используется для сопоставления запроса с соответствующим ответом.

Существует два основных типа объектов запроса.

Уведомляющий запрос. Запрашивающая сторона инициирует уведомляющий запрос без параметра id, не интересуясь содержанием ответа. Он используется только для односторонней передачи сообщений, без необходимости ожидания и понимания содержания ответа запрашиваемой стороны. Пример уведомляющего запроса приведен ниже.

```
{
  "method": "notifications/initialized",
  "jsonrpc": "2.0"
}
```

Запрос с ожиданием ответа. Запрашивающая сторона отправляет запрос с ожиданием ответа, в котором необходимо указать параметр id. Отвечающая сторона должна вернуть ответ, соответствующий id запроса. Пример запроса с ожиданием ответа приведен ниже.

```
{
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05",
    "capabilities": {},
    "clientInfo": {
      "name": "mcp-client",
      "version": "1.0.0"
    }
  },
  "jsonrpc": "2.0",
  "id": 0
}
```

Книги для программистов: https://t.me/free_books_for_it

2. Объект ответа

Объект ответа JSON-RPC 2.0 содержит следующие поля:

- `jsonrpc`: строка с указанием версии протокола JSON-RPC, должна быть точно равна `2.0`;
- `result`: этот элемент должен присутствовать при успешном ответе, он включает в себя результат вызова метода;
- `error`: этот элемент должен присутствовать при неудачном ответе, он включает в себя сообщение об ошибке;
- `id`: этот элемент должен присутствовать и должен совпадать со значением `id` в объекте запроса.

Элементы `result` и `error` являются взаимоисключающими: в ответе должен присутствовать только один из них.

Успешный ответ. После успешной обработки запроса вызываемый объект возвращает успешный ответ, при этом `id` ответа совпадает с `id` запроса. Данные результата возвращаются через поле `result`. Пример успешного ответа приведен ниже.

```
{
 "jsonrpc": "2.0",
 "id": 0,
 "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "tools": {}
    },
    "serverInfo": {
      "name": "mcp-server",
      "version": "0.0.3"
    }
  }
}
```

Ошибка ответа. В случае неудачной обработки запроса сервер возвращает объект ошибки через поле `error`, при этом возвращаемый `id` должен совпадать с `id` запроса. Пример ответа с ошибкой приведен ниже.

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32602,
    "message": "Invalid params"
  },
  "id": 0
}
```

3. Объект ошибки

Если при обработке запроса у получателя возникает ошибка, возвращаемый объект ответа должен содержать поле error, значением которого является объект со следующими полями:

- `code`: код ошибки типа `integer`, используемый для обозначения типа ошибки;

Книги для программистов: https://t.me/free_books_for_it

- `message`: краткое описание ошибки типа `string`;
- `data` (опционально): дополнительная информация об ошибке.

JSON-RPC 2.0 включает предопределенный набор кодов ошибок, как показано в табл. 2.1.

Таблица 2.1. Предопределенные коды ошибок JSON-RPC 2.0

Код ошибки	Сообщение	Значение
-32700	Parse error	Получен неверный JSON
-32600	Invalid Request	Отправленный JSON не является действительным объектом запроса
-32601	Method not found	Метод не существует или недоступен
-32602	Invalid params	Недопустимые параметры запроса
-32603	Internal error	Внутренняя ошибка JSON-RPC
-32000 до -32099	Server error	Диапазон кодов ошибок, настраиваемых сервером

4. Пакетная обработка

Запрашивающая сторона может отправить несколько объектов запроса одновременно, поместив их в массив. Отвечающая сторона должна вернуть массив, содержащий соответствующие объекты ответа, однако для запросов типа уведомления ответ не возвращается.

Пример пакетного запроса приведен ниже.

```
[
  { "jsonrpc": "2.0", "method": "sum", "params": [1, 2, 4], "id": "1" },
  { "jsonrpc": "2.0", "method": "notify_hello", "params": [7] },
  { "jsonrpc": "2.0", "method": "subtract", "params": [42, 23], "id": "2" }
]
```

Пример пакетного ответа:

```
[
  { "jsonrpc": "2.0", "result": 7, "id": "1" },
  { "jsonrpc": "2.0", "result": 19, "id": "2" }
]
```

2.3. Жизненный цикл соединения

MCP определяет строгий жизненный цикл (Lifecycle) для соединения клиент–сервер, что обеспечивает обоим участникам обмена данными возможность надлежащего управления состоянием и согласования возможностей.

Жизненный цикл соединения MCP в основном делится на три этапа:

- **этап инициализации**, когда клиент и сервер согласовывают версию протокола и возможности;
- **этап выполнения операции**, когда клиент и сервер осуществляют нормальный обмен сообщениями в соответствии с протоколом;
- **этап закрытия**, когда клиент и сервер корректно завершают соединение.

Книги для программистов: https://t.me/free_books_for_it



Жизненный цикл соединения MCP спроектирован таким образом, чтобы обеспечить максимальную надежность коммуникации. Перед началом обмена данными обе стороны должны установить соединение для получения информации об идентификации друг друга, поддерживаемых версиях протоколов и реализованных возможностях. На основе этого стороны осуществляют обмен данными в пределах согласованных возможностей, что снижает вероятность возникновения сбоев. По окончании обмена данными соединение своевременно закрывается для предотвращения утечки ресурсов.

Пример взаимодействия клиента и сервера в течение жизненного цикла соединения MCP показан на рис. 2.3.

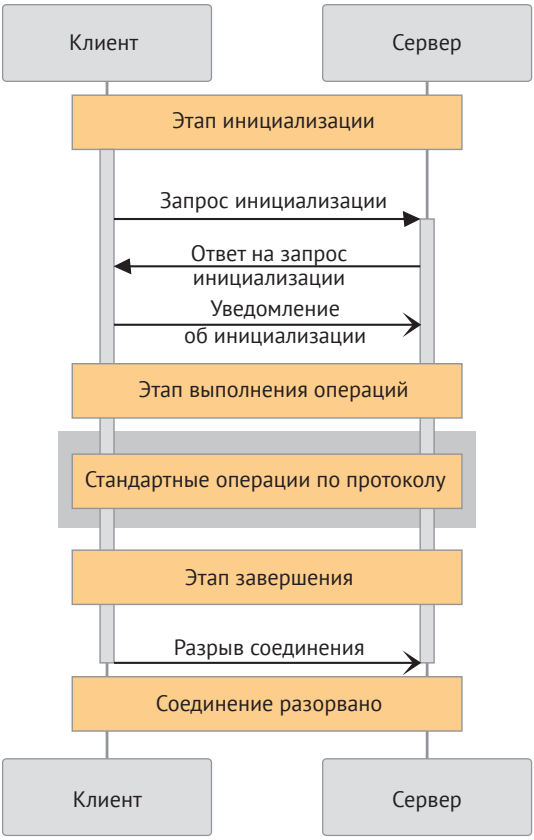


Рис. 2.3. Пример взаимодействия клиента и сервера на этапах жизненного цикла MCP

2.3.1. Этап инициализации

1. Установка соединения

Этап инициализации жизненного цикла соединения MCP представляет собой первое взаимодействие между клиентом и сервером. В ходе этого процесса

Книги для программистов: https://t.me/free_books_for_it

стороны устанавливают соединение, согласовывают версию протокола, используемого для связи, и объявляют о поддерживаемых ими возможностях.

Пример сообщения с запросом на инициализацию, отправляемого клиентом, приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {}
    },
    "clientInfo": {
      "name": "ExampleClient",
      "version": "1.0.0"
    }
  }
}
```

После получения запроса на инициализацию от клиента сервер должен откликнуться, указав поддерживаемые им возможности и основную информацию о себе. Пример сообщения ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "logging": {},
      "prompts": {
        "listChanged": true
      },
      "resources": {
        "subscribe": true,
        "listChanged": true
      },
      "tools": {
        "listChanged": true
      }
    },
    "serverInfo": {
      "name": "ExampleServer",
      "version": "1.0.0"
    },
    "instructions": "Optional instructions for the client"
  }
}
```

После получения ответа от сервера клиент должен отправить уведомление об инициализации, указывающее, что он готов к нормальной работе. Пример сообщения с уведомлением об инициализации приведен ниже.

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

Из описанных выше шагов следует, что процесс установления соединения между клиентом и сервером на этапе инициализации аналогичен процессу трехстороннего TCP-рукопожатия (Three-Way Handshake), как показано на рис. 2.4.

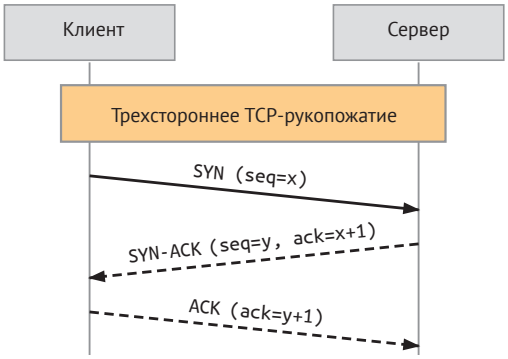


Рис. 2.4. Трехстороннее TCP-рукопожатие

2. Согласование версии

В ходе инициализации запроса клиент должен отправить самую новую из поддерживаемых им версий протокола.

- Если сервер поддерживает запрошенную версию протокола, он должен ответить с указанием той же версии. В противном случае сервер должен ответить, указав самую новую из поддерживаемых им версий.
- Если клиент не поддерживает версию, указанную в ответе сервера, он должен разорвать соединение.

3. Согласование возможностей

На этапе инициализации клиент и сервер через согласование возможностей определяют те из них, которые будут использоваться во время сеанса. Поддерживаемые клиентом и сервером возможности приведены в табл. 2.2.

Таблица 2.2. Возможности, поддерживаемые клиентом и сервером

Категория	Возможность	Описание
Клиент	roots	Предоставляет список корней файловой системы
Клиент	sampling	Поддерживает выборку из больших языковых моделей
Клиент	experimental	Поддерживает нестандартные экспериментальные возможности
Сервер	prompts	Предоставляет подсказки
Сервер	resources	Предоставляет читаемые ресурсы
Сервер	tools	Поддерживает вызов инструментов
Сервер	logging	Поддерживает структурированные логи
Сервер	experimental	Поддерживает нестандартные экспериментальные возможности

Книги для программистов: https://t.me/free_books_for_it

Декларация возможностей может содержать дополнительные поля, например:

- `listChanged`: поддержка уведомлений об изменениях в списках (для подсказок, ресурсов, инструментов, корня и т. д.);
- `subscribe`: поддержка подписки на уведомления об изменениях конкретного контента (только для ресурсов).

2.3.2. Этап взаимодействия

После установления соединения на этапе инициализации клиент и сервер переходят к этапу взаимодействия и начинают полноценную коммуникацию.

На этапе взаимодействия обе стороны должны соблюдать следующие принципы:

- придерживаться согласованной версии протокола;
- использовать только согласованные возможности.

2.3.3. Этап закрытия

На этапе закрытия клиент и сервер разрывают соединение. Одна сторона (обычно клиент) инициирует разрыв соединения, а другая сторона (обычно сервер) для завершения соединения обязана использовать механизмы низкоуровневой передачи данных.

stdio. Для сторон, осуществляющих обмен данными на основе стандарта `stdio`, процесс закрытия должен инициироваться следующим образом:

1) клиент закрывает входной поток дочернего процесса (stdin сервера);
2) ожидание завершения работы сервера или отправка сигнала `SIGTERM`, если сервер не завершил работу в установленный срок;
3) если после отправки сигнала `SIGTERM` серверу он по-прежнему не завершил работу в установленный срок, следует продолжить отправку сигнала `SIGKILL`;
4) сервер завершает процесс закрытия, закрыв выходной поток (stdout) для клиента.

HTTP. Для сторон, осуществляющих обмен данными на основе протокола HTTP, достаточно закрыть HTTP-соединение.

2.3.4. Механизм тайм-аута

Стороны, осуществляющие MCP-обмен, должны устанавливать тайм-аут для всех отправляемых запросов во избежание зависания соединения и перерасхода ресурсов. Если в течение тайм-аута не получен успешный ответ или получен ответ с ошибкой, отправитель должен направить уведомление об отмене данного запроса и прекратить ожидание ответа.

SDK и другое промежуточное программное обеспечение должны поддерживать настройку тайм-аута для запросов.

Стороны взаимодействия MCP также могут выбрать сброс тайм-аута связанного с запросом таймера при получении уведомления о ходе выполнения, что указывает на фактическое выполнение операции. Однако независимо от получения уведомления о ходе выполнения система всегда должна применять ограничение максимального тайм-аута на тот случай, если одна из сторон, не соблюдающая протокол, мешает работе системы.

Книги для программистов: https://t.me/free_books_for_it

В случае тайм-аута запроса уведомление об отмене запроса, отправляемое запрашивающей стороной, выглядит следующим образом:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/cancelled",
  "params": {
    "requestId": 2,
    "reason": "Error: MCP error -32001: Request timed out"
  }
}
```

2.3.5 Обработка ошибок

Участники MCP-соединения должны быть готовы к обработке следующих ошибок:

- несовпадение версий протоколов;
- невозможность согласования требуемых возможностей;
- тайм-аут запроса.

Пример сообщения об ошибке в ответе ответчика приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Unsupported protocol version",
    "data": {
      "supported": ["2024-11-05"],
      "requested": "1.0.0"
    }
  }
}
```

2.4. Механизмы передачи: stdio / SSE / потоковый HTTP

Механизмы передачи MCP служат мостом для обмена данными между клиентом и сервером, определяют детали этого обмена и помогают клиенту и серверу обмениваться сообщениями.

В настоящее время MCP определяет три механизма передачи для связи между клиентом и сервером:

- **stdio**: обмен данными через стандартный ввод и стандартный вывод;
- **SSE**: обмен данными посредством событий, отправляемых сервером (поддерживается с версии протокола от 05.11.2024, упразднен с 26.03.2025);
- **потоковый HTTP** (Streamable HTTP): обмен данными через HTTP с поддержкой потоковой передачи (поддерживается с версии протокола от 26.03.2025, используется в качестве замены SSE).

По требованиям MCP клиент должен по возможности поддерживать stdio.

Книги для программистов: https://t.me/free_books_for_it

Механизм передачи данных MCP является подключаемым, то есть клиент и сервер не ограничиваются несколькими механизмами передачи, определенными MCP, а также могут осуществлять связь с помощью пользовательских механизмов передачи.

2.4.1. Передача через stdio

Стандарт **stdio** (standard input & output, стандартный ввод/вывод) является одним из рекомендуемых MCP механизмов передачи, который в основном используется для связи между локальными процессами.

1. Процесс связи при передаче через stdio

Процесс связи на основе передачи через stdio показан на рис. 2.5.

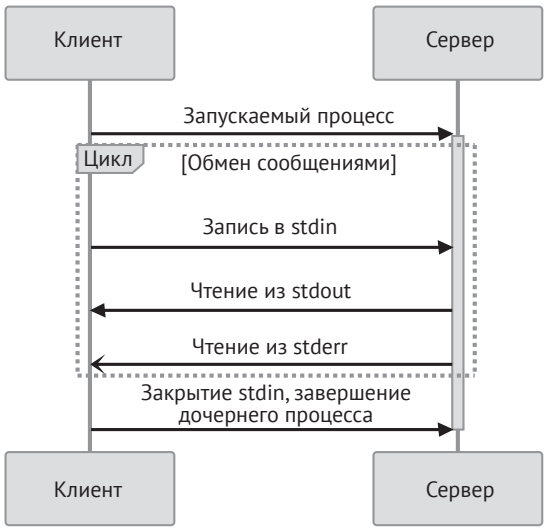


Рис. 2.5. Процесс связи при передаче через stdio

Процесс связи выглядит следующим образом:

1) клиент запускает сервер в виде дочернего процесса;
2) клиент записывает сообщение в stdin сервера;
3) сервер считывает сообщение из своего stdin;
4) сервер записывает сообщение в свой stdout;
5) клиент считывает сообщение из stdout сервера;
6) клиент завершает дочерний процесс и закрывает stdin сервера;
7) сервер закрывает свой stdout.

2. Реализация передачи данных через stdio

Рассмотрим реализацию механизма передачи данных через stdio на примере официального Typescript-SDK от MCP.

Запуск сервера. Запустим сервер локально из командной строки:

```
npx -y mcp-server-time
```

Книги для программистов: https://t.me/free_books_for_it

Создание канала связи stdio. При запуске сервера создается канал связи stdio (pipeline), используемый для двунаправленной связи с клиентом. Этот канал связи будет сохраняться до тех пор, пока клиент не отправит уведомление о закрытии или сервер не завершит работу из-за сбоя.

Класс передачи stdio `StdioServerTransport` реализует интерфейс Transport MCP, логика реализации приведена ниже.

```
export class StdioServerTransport implements Transport {
  private _readBuffer: ReadBuffer = new ReadBuffer();
  private _started = false;

  constructor(
    private _stdin: Readable = process.stdin,
    private _stdout: Writable = process.stdout
  ) {}

 onclose?: () => void;
 onerror?: (error: Error) => void;
 onmessage?: (message: JSONRPCMessage) => void;
}
```

Чтение запроса из стандартного ввода stdin. Клиент отправляет сообщение в канал связи, а сервер считывает отправленное клиентом сообщение через стандартный ввод stdin с использованием символа новой строки `\n` в качестве индикатора завершения чтения.

Логика чтения сообщения сервером реализована следующим образом (прочитанное сообщение в конечном итоге возвращается в виде структуры, закодированной в формате JSON-RPC).

```
readMessage(): JSONRPCMessage | null {
    if (!this._buffer) {
      return null;
    }

    const index = this._buffer.indexOf("\n");
    if (index === -1) {
      return null;
    }

    const line = this._buffer.toString("utf8", 0, index).replace(/\r$/, '');
    this._buffer = this._buffer.subarray(index + 1);
    return deserializeMessage(line);
}
```

Запись ответа в stdout. После выполнения внутренней логики серверу необходимо отправить ответное сообщение клиенту. Конкретный процесс реализации выглядит следующим образом: сначала сервер кодирует сообщение в формате JSON-RPC, а затем записывает его в стандартный вывод stdout. Логика реализации представлена ниже.

```
send(message: JSONRPCMessage): Promise<void> {
    return new Promise((resolve) => {
      const json = serializeMessage(message);
      if (this._stdout.write(json)) {
```

Книги для программистов: https://t.me/free_books_for_it

```
          resolve();
        } else {
          this._stdout.once("drain", resolve);
        }
      });
    }
  }
```



Два приведенных выше действия демонстрируют процесс односторонней связи, когда клиент запрашивает сервер: клиент записывает сообщение в stdin, а сервер считывает его из stdin; сервер записывает сообщение в stdout, клиент считывает сообщение из stdout. Таким же образом в случае односторонней связи, когда сервер отправляет уведомление клиенту, эти шаги следует выполнить в обратном порядке: сервер записывает сообщение в stdin, клиент считывает сообщение из stdin; клиент записывает сообщение в stdout, сервер считывает сообщение из stdout.

Закрытие канала связи stdio. Когда клиент завершает работу, он отправляет серверу уведомление о закрытии. После получения уведомления о закрытии, отправленного клиентом через канал связи stdio, или в случае внутренней ошибки выполнения сервер инициирует закрытие канала связи stdio.

Как только канал связи закрыт, клиент и сервер больше не могут обмениваться сообщениями, пока канал связи stdio не будет установлен заново. Логика закрытия канала связи представлена ниже.

```
async close(): Promise<void> {
    // Сначала удаляем обработчики событий data
    this._stdin.off("data", this._ondata);
    this._stdin.off("error", this._onerror);

    // Проверяем количество оставшихся обработчиков событий data
    const remainingDataListeners = this._stdin.listenerCount('data');
    if (remainingDataListeners === 0) {
      // Временная остановка stdin только при отсутствии дополнительных обработчиков
      // событий data
      // Во избежание конфликтов с другими программами, которые могут использовать stdin
      this._stdin.pause();
    }

    // Очистка буфера чтения и запуск обратного вызова при закрытии
    this._readBuffer.clear();
    this.onclose?.();
}
```

3. Преимущества и недостатки использования stdio. Области применения

Механизм передачи данных через stdio основан на локальной коммуникации между процессами. Его основные преимущества:

- отсутствие внешних зависимостей, простота реализации;
- отсутствие передачи по сети, высокая скорость обмена данными;
- локальная связь, высокий уровень безопасности, отсутствие риска сетевых атак.

Книги для программистов: https://t.me/free_books_for_it

Однако существуют и некоторые ограничения:

- обмен данными между одиночными процессами, невозможность параллельной обработки запросов от нескольких клиентов;
- высокие затраты ресурсов на связь между процессами, сложность запуска нескольких серверов на локальном компьютере.

Передача данных через stdio подходит для работы с локальными ресурсами, не требующими доступа извне. Например, можно использовать ИИ-ассистента для обобщения сообщений WeChat, если файлы с этими сообщениями хранятся на локальном компьютере и не могут, да и не должны быть доступны извне.

Если требуется доступ к ресурсам на удаленном сервере, по-прежнему можно использовать передачу через stdio, но реализация будет более сложной и обычно требует выполнения следующих двух шагов:

- развертывание API-сервиса на удаленном сервере для работы с ресурсами и обеспечения доступа из публичной сети;
- создание MCP-сервера для взаимодействия с этим API, а затем передача данных через stdio для локальной связи с клиентом.

Поскольку использование передачи stdio для доступа к удаленным ресурсам является столь трудоемким, существуют ли более подходящие механизмы передачи?

Ответ положительный: можно использовать передачу SSE.

2.4.2 Передача данных по SSE

Для решения проблемы доступа к удаленным ресурсам MCP использует передачу данных по SSE (Server-Sent Events, события, отправляемые сервером). На низком уровне используется HTTP-соединение, которое позволяет клиенту напрямую обращаться к удаленным ресурсам, как при использовании API, без необходимости промежуточной передачи через stdio.

При передаче данных по SSE сервер работает как отдельный процесс и может обрабатывать соединения от нескольких клиентов. Сервер должен предоставлять два конечных пункта:

- конечную точку SSE, предназначенную для установления двунаправленного соединения между клиентом и сервером (запрос GET);
- конечную точку сообщений, предназначенную для отправки сообщений с клиента на сервер (запрос POST).

1. Процесс передачи данных по SSE

Процесс передачи данных по SSE показан на рис. 2.6.

Книги для программистов: https://t.me/free_books_for_it

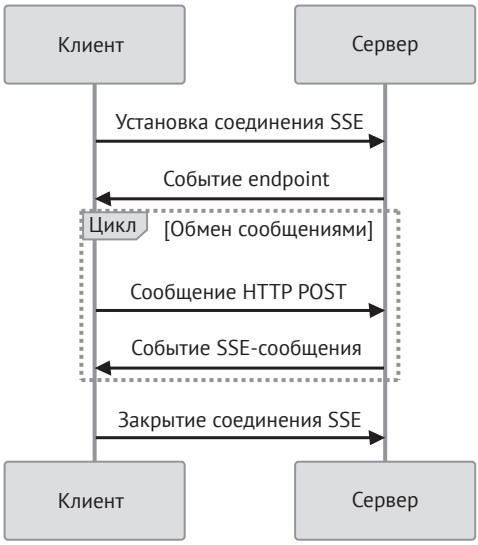


Рис. 2.6. Процесс передачи данных по SSE

Этапы передачи данных выглядят следующим образом.

1. Клиент отправляет запрос (обычно GET-запрос) на конечную точку / `sse` сервера, устанавливая SSE-соединение.
2. Сервер отправляет через это соединение сообщение-событие, сообщая клиенту адрес конечной точки для отправки сообщения.
3. Клиент отправляет запрос на отправку сообщения на сервер по указанному адресу.
4. Сервер возвращает код состояния, подтверждая получение сообщения.
5. Сервер отправляет клиенту через SSE-соединение результаты обработки или другие сообщения-события.
6. Клиент постоянно получает отправляемые сервером сообщения о событиях из потока SSE.
7. По окончании связи клиент закрывает соединение SSE.

2. Реализация передачи SSE

Рассмотрим реализацию механизма передачи SSE на примере официального Typescript-SDK от MCP.

Запуск сервера. Обычно сервер запускается из командной строки (как правило, он развертывается на удаленном сервере). В действительности запускается HTTP-сервис, который прослушивает указанный порт и предоставляет HTTP-интерфейс для внешнего доступа. Основная логика определения и запуска сервера выглядит следующим образом.

```
const server = new McpServer({
  name: "example-server",
  version: "1.0.0",
});
```

Книги для программистов: https://t.me/free_books_for_it

```
const app = express();

app.get("/sse", async (_: Request, res: Response) => {
  const transport = new SSEServerTransport("/messages", res);

  await server.connect(transport);
});

app.post("/messages", async (req: Request, res: Response) => {
  await transport.handlePostMessage(req, res);
});

app.listen(3001);
```

В этом примере с помощью фреймворка express запущен HTTP-сервис, который прослушивает порт 3001 и предоставляет два внешних интерфейса:

- `/sse`: запрос GET, используемый для установления соединения SSE;
- `/messages`: запрос POST, используемый для приема отправленных клиентом сообщений.

После запуска сервера его необходимо привязать к общедоступному доменному имени, например *example.com*, с помощью службы DNS.

- **Установка SSE-соединения**. Клиент запрашивает адрес SSE-конечной точки сервера *https://example.com:3001/sse* и устанавливает двунаправленное соединение с сервером. После установки соединения сервер отправляет клиенту сообщение о событии с адресом конечной точки для обмена сообщениями. Логика реализации представлена ниже.

```
res.writeHead(200, {
  "Content-Type": "text/event-stream",
  "Cache-Control": "no-cache, no-transform",
  Connection: "keep-alive",
});

const messagesUrl = "https://example.com:3001/messages?sessionId=xxx";

res.write(`event: endpoint\ndata: ${messagesUrl}\n\n`);
```

Клиент считывает адрес конечной точки для обмена сообщениями из возвращенного сервером события endpoint и успешно устанавливает SSE-соединение с сервером.

- **Обмен сообщениями**. После установления SSE-соединения с сервером клиент начинает отправлять сообщения на адрес конечной точки сообщений. Передача SSE в MCP представляет собой механизм двуканального ответа: после получения запроса от клиента сервер должен не только отправить ответное сообщение на текущий запрос, но и отправить сообщение-событие по ранее установленному SSE-соединению (за исключением сообщений типа уведомления).

Например, после установления SSE-соединения между клиентом и сервером первое сообщение, отправленное серверу, используется для согласования возможностей на этапе инициализации. Пример запроса, отправленного клиентом на конечную точку сообщений, приведен ниже.

Книги для программистов: https://t.me/free_books_for_it

```
curl -X POST https://example.com/messages?sessionId=xxx \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "initialize",
  "params": {
    "protocolVersion": "1.0",
    "capabilities": {},
    "clientInfo": {
      "name": "mcp-client",
      "version": "1.0.0"
    }
  }
}'
```

Сервер считывает отправленное клиентом сообщение из тела HTTP-запроса и после выполнения внутренней логики отвечает клиенту кодом состояния 202 (без тела ответа), что означает получение запроса. Логика реализации представлена ниже.

```
async handlePostMessage(
  req: IncomingMessage,
  res: ServerResponse,
  parsedBody?: unknown,
): Promise<void> {
  if (!this._sseResponse) {
    const message = "SSE connection not established";
    res.writeHead(500).end(message);
    throw new Error(message);
  }

  let body: string | unknown;
  try {
    const ct = contentType.parse(req.headers["content-type"] ?? "");
    if (ct.type !== "application/json") {
      throw new Error(`Unsupported content-type: ${ct}`);
    }

    body = parsedBody ?? await getRawBody(req, {
      limit: MAXIMUM_MESSAGE_SIZE,
      encoding: ct.parameters.charset ?? "utf-8",
    });
  } catch (error) {
    res.writeHead(400).end(String(error));
    this.onerror?.(error as Error);
    return;
  }

  try {
    await this.handleMessage(typeof body === 'string' ? JSON.parse(body) : body);
  } catch {
    res.writeHead(400).end(`Invalid message: ${body}`);
    return;
  }

  res.writeHead(202).end("Accepted");
}
```

Книги для программистов: https://t.me/free_books_for_it

Затем сервер отправляет клиенту содержимое ответа в виде события через ранее установленное SSE-соединение. Содержимое сообщения имеет формат JSON-RPC, а реализация логики выглядит следующим образом.

```
async send(message: JSONRPCMessage): Promise<void> {
  if (!this._sseResponse) {
    throw new Error("Not connected");
  }

  this._sseResponse.write(
    `event: message\ndata: ${JSON.stringify(message)}\n\n`,
  );
}
```

Клиент, основываясь на коде состояния 202 в ответе сервера, определяет, что сервер получил запрос, и начинает считывать содержимое полученного от сервера ответа из ранее установленного SSE-соединения.



SSE-соединения между клиентом и сервером должны поддерживать однозначное соответствие, чтобы избежать смешивания данных. На этапе установки соединения сервер должен присвоить каждому SSE-соединению уникальный идентификатор `sessionId` и добавить этот идентификатор в адрес конечной точки сообщений, например `/messages?sessionId=xxx`. В ходе последующего обмена сообщениями сервер может точно сопоставить соответствующее SSE-соединение по `sessionId` в параметрах запроса и отправлять сообщения о событиях только по этому соединению, обеспечивая изолированность и согласованность данных.

Разрыв SSE-соединения. Любая из сторон (сервер или клиент) может инициировать разрыв SSE-соединения. Чтобы предотвратить непреднамеренный разрыв соединения без освобождения ресурсов, сторона, сохраняющая соединение, должна реализовать механизм проверки соединения и закрытия по тайм-ауту. Например, можно периодически отправлять циклические «пульсовые» (heartbeat) сообщения через канал передачи SSE для проверки состояния другой стороны. Если в течение нескольких попыток ответ не получен, можно считать, что собеседник разорвал соединение, и в этом случае можно инициировать закрытие SSE-соединения во избежание расходования ресурсов. Пример реализации проверки связи и закрытия по тайм-ауту на языке Go приведен ниже.

```
// Настройка таймера пульсового интервала
heartbeatInterval := 30 * time.Second
heartbeatTicker := time.NewTicker(heartbeatInterval)
defer heartbeatTicker.Stop()

// Настройка тайм-аута бездействия
idleTimeout := 5 * time.Minute
idleTimer := time.NewTimer(idleTimeout)
defer idleTimer.Stop()
```

Книги для программистов: https://t.me/free_books_for_it

```
go func() {
  for {
    select {
    case <-session.Done():
      return
    case <-heartbeatTicker.C:
      // Отправка пульса
      if err := writer.SendHeartbeat(); err != nil {
        // Нет ответа на пульс, закрыть соединение
        session.Close()
        return
      }
    case <-idleTimer.C:
      // Тайм-аут бездействия, закрыть соединение
      session.Close()
      return
    }
  }
}()
```

3. Преимущества и недостатки передачи данных через SSE. Сценарии применения

Передача данных через SSE подходит для коммуникации между клиентом и сервером, находящимися в разных сетях. Например, если необходимо запросить данные из базы данных на собственном облачном сервере через ИИ-ассистента, можно развернуть на этом сервере MCP-сервер, который будет считывать содержимое базы данных, а затем настроить и подключить его в ИИ-ассистенте с помощью URL-адреса SSE.

Сервер для SSE-обмена теоретически может реализовать ту же функциональность с помощью локальной передачи stdio и API-транзита. Основные отличия заключаются в следующем:

- при использовании SSE-обмена клиент может напрямую взаимодействовать с сервером, минуя транзит через API с помощью локальной передачи stdio;
- для подключения SSE достаточно настроить один URL, нет требований к локальной среде, не требуется запускать сервер локально, что значительно снижает порог входа для пользователей.

Основные преимущества передачи SSE включают:

- поддержку доступа к удаленным ресурсам, что решает проблему ограниченности передачи stdio локальными ресурсами;
- реализацию на основе HTTP, обеспечивающую хорошую совместимость и удобство интеграции с существующей веб-инфраструктурой;
- возможность запуска сервера в качестве отдельного процесса, поддерживающего обработку подключений от нескольких клиентов;
- более простую реализацию по сравнению с WebSocket, поскольку SSE является расширением HTTP и не требует обновления протокола.

Книги для программистов: https://t.me/free_books_for_it

Основные недостатки передачи SSE следующие:

- сложность механизма ответной связи по двум каналам: передача SSE требует, чтобы сервер после получения сообщения от клиента не только ответил на текущий запрос, но и отправил сообщение о событии по ранее установленному SSE-соединению, что усложняет реализацию;
- нестабильность соединения: в среде без серверов SSE-соединения могут произвольно и часто обрываться, что делает их ненадежными;
- неудобство для постоянных соединений: бессерверная архитектура, как правило, имеет автоматическое масштабирование и не подходит для длительных соединений, в то время как SSE нуждается в поддержке постоянных соединений;
- необходимость в интенсивном управлении сессиями: передача SSE требует присвоения уникального идентификатора (`sessionId`) каждому SSE-соединению для предотвращения смешивания данных. Это означает, что разработчику приходится брать на себя дополнительную работу по управлению сессиями, что увеличивает сложность реализации системы;
- дополнительные затраты на реализацию: требуется реализация механизмов проверки пульса и тайм-аута для предотвращения утечки ресурсов.

Именно из-за этих проблем MCP в новом протоколе (версия от 26.03.2025) ввела механизм потоковой HTTP-передачи в качестве замены SSE-передачи и упразднила SSE-передачу. Новый механизм передачи сохраняет основу HTTP, но поддерживает более гибкие способы подключения, что соответствует требованиям современных облачных архитектур и бессерверных сред.

2.4.3. Потоковая HTTP-передача

Процесс связи на основе потоковой HTTP-передачи показан на рис. 2.7.

1. Процесс передачи данных в потоковом HTTP

При потоковой передаче данных по HTTP сервер обязательно должен предоставлять конечную точку HTTP, поддерживающую как запросы POST, так и запросы GET. Основные этапы процесса передачи данных в потоковом HTTP приведены ниже.

1. Клиент отправляет сообщение на конечную точку сервера.
2. Сервер возвращает ответ.
3. Клиент продолжает отправлять серверу следующее сообщение.
4. Сервер продолжает отвечать на сообщения клиента.

Книги для программистов: https://t.me/free_books_for_it

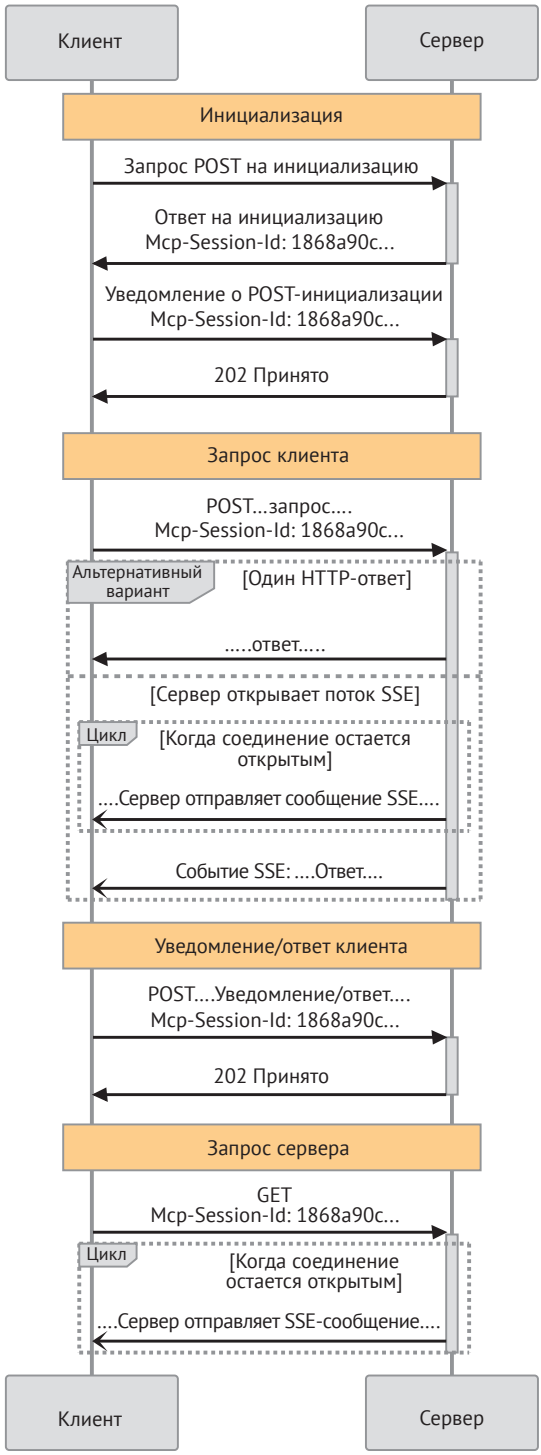


Рис. 2.7. Процесс связи при потоковой HTTP-передаче

Книги для программистов: https://t.me/free_books_for_it

В отличие от механизма двуканального ответа при передаче SSE, при потоковой HTTP-передаче обмен сообщениями между клиентом и сервером осуществляется в основном по принципу «один вход – один выход» (одноканальный ответ), то есть каждый запрос может соответствовать только одному ответу. Однако этот ответ может возвращаться синхронно или асинхронно с помощью SSE. Кроме того, потоковая HTTP-передача поддерживает два типа взаимодействия: с состоянием и без состояния. Взаимодействие с состоянием требует поддержки состояния сеанса через `Mcp-Session-Id`, тогда как взаимодействие без состояния означает, что каждый запрос является отдельным и не зависит от других.

2. Реализация поточной HTTP-передачи

Рассмотрим пример реализации механизма потоковой HTTP-передачи на основе официального Typescript-SDK для MCP.

Запуск сервера. Как и механизм передачи SSE, потоковая HTTP-передача, по сути, также основана на протоколе HTTP, поэтому для запуска HTTP-сервиса необходимо сначала запустить сервер из командной строки. Основная логика определения и запуска сервера представлена ниже.

```
const server = new McpServer({
  name: "example-server",
  version: "1.0.0",
});

const app = express();

app.all("/mcp", async (req: Request, res: Response) => {
  const transport = new StreamableHTTPServerTransport();

  await server.connect(transport);

  await transport.handleMessage(req, res);
});
app.listen(3002);
```

В этом примере с помощью фреймворка Express запущен HTTP-сервис, прослушивающий порт 3002 и предоставляющий доступ к одной конечной точке `/mcp`, которая принимает запросы от клиентов на установку соединения и обмен сообщениями.

После запуска сервера необходимо через DNS-разрешение привязать его к общедоступному доменному имени, например *example.com*.



В отличие от передачи SSE, где необходимо открыть две конечные точки (конечную точку SSE и конечную точку сообщений), для потоковой передачи HTTP достаточно открыть только одну конечную точку для приема различных типов запросов от клиентов (GET / POST / DELETE).

Обмен сообщениями. После успешного запуска сервера клиент может напрямую отправлять сообщения на адрес конечной точки сервера без необходимости предварительного установления соединения.

Книги для программистов: https://t.me/free_books_for_it

Клиент может отправлять сообщения на сервер с помощью методов GET или POST. В каждом запросе в заголовке запроса необходимо установить параметр `Accept`, передавая два следующих значения:

- `application/json`: используется для получения сообщений в кодировке JSON-RPC в ответе сервера;
- `text/event-stream`: используется для открытия потокового канала сервера, через который клиент непрерывно получает событийные сообщения.

Пример клиентского запроса приведен ниже.

```
curl -X POST https://example.com/mcp \
-H "Content-Type: application/json" \
-H "Accept: application/json, text/event-stream" \
-d '{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "initialize",
  "params": {
    "protocolVersion": "1.0",
    "capabilities": {},
    "clientInfo": {
      "name": "mcp-client",
      "version": "1.0.0"
    }
  }
}
```

При использовании механизма потоковой HTTP-передачи существуют следующие ключевые моменты взаимодействия между клиентом и сервером.

- Клиент может отправлять серверу GET-запросы без тела запроса для установления соединения SSE; сервер может инициировать отправку сообщений клиенту.
- В случае отправки клиентом сообщения JSON-RPC на сервер необходимо использовать запрос POST и установить в заголовке запроса `Accept` значение `application/json`, `text/event-stream`.
- При получении GET-запроса от клиента сервер либо возвращает `Content-Type: text/event-stream` для установления SSE- соединения, либо возвращает статус-код HTTP 405, означающий, что соединение SSE не поддерживается.
- При получении сервером POST-запроса от клиента из тела запроса считывается сообщение JSON-RPC. Если это уведомляющее сообщение, то в ответ возвращается статус HTTP 202, означающий, что сообщение получено. Если это не уведомляющее сообщение, сервер может либо вернуть `Content-Type: text/event-stream` для открытия SSE-передачи, либо вернуть `Content-Type: application/json` для синхронного ответа одним сообщением JSON-RPC.

Сохранение сеанса. Потоковая HTTP-передача поддерживает как взаимодействие без состояния (каждый запрос является независимым и не требует записи состояния), так и взаимодействие с состоянием (новый запрос может зависеть от предыдущего запроса или информации ответа в качестве контекста; такой механизм называется сохранением сеанса).

Книги для программистов: https://t.me/free_books_for_it

В случае сохранения сеанса взаимодействие между сервером и клиентом должно соблюдать следующие принципы:

- сервер, использующий потоковую HTTP-передачу, может при инициализации вернуть идентификатор сеанса через заголовок ответа `Mcp-Session-Id`;
- если сервер вернул `Mcp-Session-Id` при инициализации, клиент должен указывать идентификатор сеанса в заголовке запроса `Mcp-Session-Id` во всех последующих запросах;
- сервер может в любой момент завершить сессию, после чего он должен отвечать на запросы, содержащие этот идентификатор сессии, ответом HTTP 404 Not Found;
- когда клиент получает ответ HTTP 404 на запрос, содержащий `Mcp-Session-Id`, он должен инициировать новую сессию, отправив новый запрос `InitializeRequest` без идентификатора сессии (опционально);
- клиент, которому больше не нужна конкретная сессия, должен отправить на сервер запрос HTTP DELETE с заголовком `Mcp-Session-Id`, чтобы явно завершить сессию (необязательно, но рекомендуется).

Логика реализации проверки сессии на сервере представлена ниже.

```
/**
 * Проверка идентификатора сеанса для неинициализированного запроса
 * Если сеанс действителен, возвращается true; в противном случае возвращается false
 */
private validateSession(req: IncomingMessage, res: ServerResponse): boolean {
  if (!this._initialized) {
    // Если сервер еще не инициализирован, отклонить все запросы
    res.writeHead(400).end(JSON.stringify({
      jsonrpc: "2.0",
      error: {
        code: -32000,
        message: "Неверный запрос: сервер не инициализирован"
      },
      id: null
    }));
    return false;
  }
  if (this.sessionId === undefined) {
    //Если sessionId не установлен, это означает, что управление сессиями не включено,
    //и в этом случае проверка ID сессии не требуется
    return true;
  }
  const sessionId = req.headers["mcp-session-id"];

  if (!sessionId) {
    // Запросы, не являющиеся начальными, должны содержать идентификатор сеанса,
    // в противном случае возвращается ошибка 400
    res.writeHead(400).end(JSON.stringify({
      jsonrpc: "2.0",
      error: {
        code: -32000,
        message: "Неверный запрос: требуется заголовок Mcp-Session-Id "
      },
```

Книги для программистов: https://t.me/free_books_for_it

```
        id: null
      }));
      return false;
    } else if (Array.isArray(sessionId)) {
      res.writeHead(400).end(JSON.stringify({
        jsonrpc: "2.0",
        error: {
          code: -32000,
          message: "Неверный запрос: заголовок Mcp-Session-Id должен содержать одно значение"
        },
        id: null
      }));
      return false;
    }
    else if (sessionId !== this.sessionId) {
      // Если идентификаторы сеансов не совпадают, возвращается код 404, означающий,
      // что сеанс не найден
      res.writeHead(404).end(JSON.stringify({
        jsonrpc: "2.0",
        error: {
          code: -32001,
          message: "Сессия не обнаружена"
        },
        id: null
      }));
      return false;
    }
    return true;
  }
```

Разрыв и восстановление соединения. При потоковой HTTP-передаче, если клиент и сервер используют для связи SSE-соединение, способ разрыва соединения соответствует способу разрыва соединения при SSE-передаче: любая из сторон соединения может инициировать разрыв, а сторона, у которой соединение осталось открытым, должна реализовать механизмы проверки пульса и тайм-аута для своевременного закрытия соединения и предотвращения расхода системных ресурсов.

В отличие от SSE-передачи потоковая HTTP-передача вводит механизм восстановления соединения и поддерживает повторную отправку сообщений, которые могли быть утеряны после разрыва соединения.

Сервер может добавлять в SSE-события поле ID, которое должно быть глобально уникальным во всех потоках событий. Если после разрыва соединения клиент захочет его восстановить, он должен повторно отправить HTTP-запрос GET и передать ID последнего успешно полученного события через заголовок `Last-Event-ID`.

В соответствии с этим сервер может воспроизвести события, начиная с этого ID, что позволяет восстановить поток событий.

3. Преимущества и недостатки потоковой HTTP-передачи. Сценарии использования

Механизм потоковой HTTP-передачи сочетает в себе возможности удаленного доступа SSE-передачи и гибкость независящего от состояния HTTP, одновременно решая многие проблемы SSE-передачи.

Книги для программистов: https://t.me/free_books_for_it

Его основные преимущества следующие:

- совместимость с бессерверной средой, возможность работы в режиме коротких соединений;
- гибкий режим соединения, поддерживающий как простые запросы–ответы, так и потоковую передачу;
- более стандартизированное и понятное управление сессиями;
- поддержка восстановления после разрыва соединения и повторной отправки сообщений, что делает его более надежным, чем SSE-передача;
- сохранение возможностей потоковой SSE-передачи при одновременном решении проблем со стабильностью.

Его основные недостатки заключаются в следующем:

- чрезмерное количество проверок состояния, высокая сложность реализации;
- сложная логика обработки разрыва и восстановления соединения, высокая стоимость реализации;
- управление сессиями требует внедрения дополнительных компонентов на сервере (например, использования Redis для хранения сессий).

Потоковая HTTP-передача подходит для следующих ситуаций:

- сценарии с необходимостью удаленного доступа к сервисам, особенно в облачных и бессерверных архитектурах;
- ИИ-сервисы, требующие поддержки потокового вывода;
- сценарии, требующие активной отправки сообщений сервером клиенту;
- сервисы, ориентированные на масштабное развертывание и требующие высокой доступности и масштабируемости;
- сценарии, требующие поддержания надежной связи в нестабильной сетевой среде.

По сравнению с SSE-передачей потоковая HTTP-передача представляет собой более универсальное и гибкое решение, а также является основным механизмом передачи, рекомендуемым MCP для сценариев доступа к удаленным ресурсам.

2.5. Возможности сервера: подсказки / ресурсы / инструменты

MCP предоставляет три основные возможности для реализации сервера: подсказки (Prompts), ресурсы (Resources) и инструменты (Tools), как показано в табл. 2.3.

Можно представить три основные возможности сервера в виде конструктора: хотя каждая возможность (деталь) сама по себе проста, их совместное использование позволяет создавать сложные функции (подобно тому, как из деталей конструктора строят целые замки). Каждая деталь имеет свою конкретную функцию и не может быть сведена к более простым операциям.

Книги для программистов: https://t.me/free_books_for_it

Таблица 2.3. Три основные возможности: подсказки, ресурсы, инструменты

Характеристика	Способ управления	Описание	Пример
Подсказки (промпты)	Контроль со стороны пользователя	Подсказки, вызываемые по выбору пользователя	Команды с косой чертой, пункты меню
Ресурсы	Контроль со стороны приложения	Контекстные данные, добавляемые и управляемые клиентом	Содержимое файлов, структура таблиц базы данных
Инструменты	Контроль со стороны модели	Функции, предоставляемые большой языковой модели для выполнения операций	API-запросы, запись в файлы

2.5.1. Примеры подсказок и взаимодействия с ними

MCP предоставляет стандартный способ предоставления подсказок (промптов) от сервера клиенту. Подсказки дают серверу возможность предоставлять структурированные сообщения и инструкции для взаимодействия с большой языковой моделью. Клиент может обнаруживать доступные подсказки, просматривать их содержание и предоставлять пользовательские параметры для их использования.

Подсказки разработаны таким образом, чтобы ими управлял пользователь. Это означает, что после их предоставления клиенту со стороны сервера именно пользователь выбирает их и использует. Обработка подсказок обычно происходит в пользовательском интерфейсе клиента, где пользователь вводит команды для вызова доступных подсказок, что позволяет естественным образом обнаруживать и вызывать доступные подсказки. Например, для вызова доступных подсказок пользователь может ввести команду со слешем, как показано на рис. 2.8.



Рис. 2.8. Вызов подсказок пользователем с помощью команды со слешем

Примеры взаимодействия с подсказками. Объявление поддержки подсказок сервером: сервер, поддерживающий подсказки, должен объявить эту возможность при определении сервера.

```
{
  "capabilities": {
    "prompts": {
      "listChanged": true
    }
  }
}
```

Книги для программистов: https://t.me/free_books_for_it

`"listChanged": true` означает, что сервер поддерживает отправку уведомлений при изменении списка подсказок.

Клиент получает список подсказок. Клиент отправляет на сервер запрос `prompts/list` для получения списка определенных сервером подсказок. Пример запроса приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "prompts/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

После получения запроса сервер возвращает список подсказок. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "prompts": [
      {
        "name": "code_review",
        "description": "Запрос к LLM на анализ качества кода и предложение улучшений",
        "arguments": [
          {
            "name": "code",
            "description": "Код на анализ",
            "required": true
          }
        ]
      }
    ],
    "nextCursor": "next-page-cursor"
  }
}
```

Если на сервере определено слишком много подсказок и пользователь хочет, чтобы клиент запрашивал их партиями с помощью параметра разбиения на страницы, можно управлять разбиением на страницы с помощью поля `result.nextCursor` при отправке списка подсказок в ответе.

В следующем запросе клиент должен указать значение `result.nextCursor`, возвращенное сервером в предыдущем ответе, в качестве значения `params.cursor` нового запроса, чтобы продолжить получение следующей партии списка подсказок с сервера.

Получение клиентом одиночной подсказки. Клиент отправляет серверу запрос `prompts/get` для получения содержимого конкретной подсказки. Пример запроса приведен ниже:

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "prompts/get",
  "params": {
    "name": "code_review",
    "arguments": {
      "code": "def hello():\n print('world')"
    }
  }
}
```

После получения запроса сервер возвращает содержимое подсказки в соответствии с параметрами запроса. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "description": "Code review prompt",
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "Please review this Python code:\ndef hello():\n
print('world')"
        }
      }
    ]
  }
}
```

Отправка сервером уведомления об изменении списка подсказок. При изменении списка подсказок сервер отправляет клиенту уведомление об изменении списка подсказок. Пример уведомления приведен ниже.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/prompts/list_changed"
}
```

После получения уведомления клиент должен отправить повторный запрос на сервер для получения списка подсказок.

Пример взаимодействия с подсказками. Пример взаимодействия клиента и сервера в отношении подсказок показан на рис. 2.9.

Книги для программистов: https://t.me/free_books_for_it

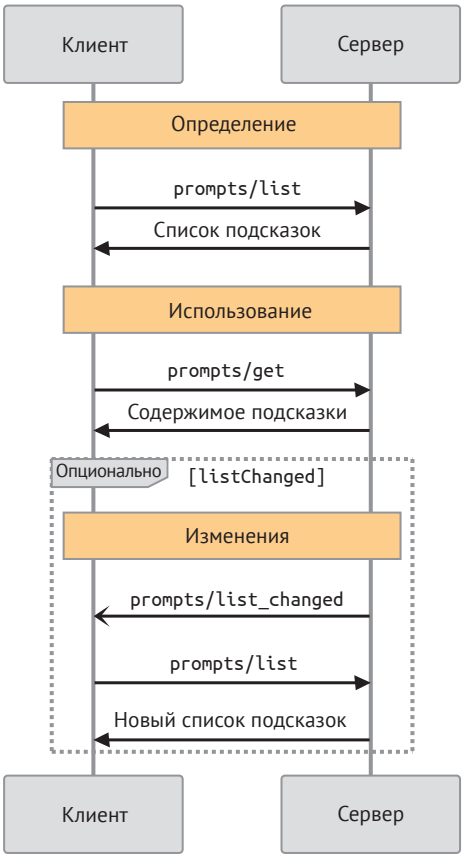


Рис. 2.9. Пример взаимодействия клиента и сервера по подсказкам

2.5.2. Ресурсы и примеры взаимодействия

MCP предоставляет стандартный способ предоставления ресурсов с сервера клиенту. Ресурсы дают серверу возможность предоставлять данные контекста для больших языковых моделей, например файлы, структуры баз данных или информацию о конкретных приложениях. Каждый ресурс однозначно идентифицируется с помощью URI (Uniform Resource Identifier, унифицированный идентификатор ресурса).

Ресурсы в MCP рассчитаны на управление со стороны приложений. Это означает, что хост использует ресурсы сервера в соответствии со своими собственными целями. Например, хост может реализовать следующее:

- отображать доступные ресурсы в виде списка или дерева каталогов;
- разрешать пользователям искать и фильтровать доступные ресурсы;
- интегрировать ресурсы в контекст.

Как показано на рис. 2.10, пользователь выбирает в Claude предоставляемые сервером ресурсы.

Книги для программистов: https://t.me/free_books_for_it

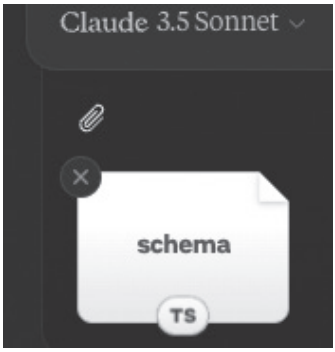


Рис. 2.10. Выбор ресурсов пользователем

Пример взаимодействия с ресурсами. Объявление сервером возможности предоставления ресурсов: сервер с функцией предоставления ресурсов должен объявить эту возможность при своем определении.

```
{
  "capabilities": {
    "resources": {
      "subscribe": true,
      "listChanged": true
    }
  }
}
```

Эта возможность поддерживает два дополнительных поля:

- `subscribe`: разрешает ли клиенту подписываться на уведомления об изменениях отдельного ресурса и получать их;
- `listChanged`: разрешает ли серверу уведомлять клиента о изменениях в списке ресурсов.

Поля `subscribe` и `listChanged` являются опциональными; сервер может поддерживать любое из них или оба.

Клиент получает список ресурсов. Клиент отправляет серверу запрос `resources/list` для получения списка ресурсов, определенных сервером. Пример запроса приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "resources/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

После получения запроса сервер возвращает список ресурсов. Пример ответа приведен ниже.

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "resources": [
      {
        "uri": "file:///project/src/main.rs",
        "name": "main.rs",
        "description": "Primary application entry point",
        "mimeType": "text/x-rust"
      }
    ],
    "nextCursor": "next-page-cursor"
  }
}
```

В соответствии с логикой получения списка подсказок получение списка ресурсов также поддерживает управление страницами.

Клиент считывает отдельный ресурс. Клиент отправляет серверу запрос `resources/read` для получения содержимого конкретного ресурса. Пример запроса приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "resources/read",
  "params": {
    "uri": "file:///project/src/main.rs"
  }
}
```

После получения запроса сервер возвращает соответствующее содержимое ресурса на основе идентификатора ресурса. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "contents": [
      {
        "uri": "file:///project/src/main.rs",
        "mimeType": "text/x-rust",
        "text": "fn main() {\n println!(\"Hello world!\");\n}"
      }
    ]
  }
}
```

Клиент получает список шаблонов ресурсов. MCP позволяет клиенту динамически получать ресурсы сервера с помощью шаблонов ресурсов.

Клиент отправляет серверу запрос `resources/templates/list` для получения списка шаблонов ресурсов, определенных сервером. Пример запроса приведен ниже.

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "resources/templates/list"
}
```

После получения запроса сервер возвращает список шаблонов ресурсов. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "resourceTemplates": [
      {
        "uriTemplate": "file:///{path}",
        "name": "Project Files",
        "description": "Access files in the project directory",
        "mimeType": "application/octet-stream"
      }
    ]
  }
}
```

Клиент может в соответствии с потребностями компоновать параметры `uriTemplate` для динамического получения ресурсов сервера.

Сервер отправляет уведомление об изменении списка ресурсов. Когда список ресурсов сервера изменяется, сервер, объявивший `listChanged`, должен отправить клиенту уведомление. Пример уведомления приведен ниже.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/list_changed"
}
```

Получив от сервера уведомление об изменении списка ресурсов, клиент должен повторно запросить у сервера список ресурсов.

Клиент подписывается на уведомления об обновлении ресурсов. Сервер, объявивший `subscribe`, позволяет клиенту подписаться на уведомления об изменениях определенного ресурса. Пример запроса клиента на подписку на уведомления об изменениях ресурсов приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "resources/subscribe",
  "params": {
    "uri": "file:///project/src/main.rs"
  }
}
```

Получив от клиента запрос на подписку на уведомления об изменениях определенного ресурса, сервер должен сохранить в кеше URI этого ресурса и отправить клиенту сообщение об успешной подписке. Пример ответа приведен ниже.

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {}
}
```

Отправка уведомлений об обновлении ресурсов сервером. Когда содержимое ресурса, на который подписан клиент, изменяется, сервер должен отправить клиенту уведомление об изменении ресурса. Пример уведомления приведен ниже.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": {
    "uri": "file:///project/src/main.rs"
  }
}
```

При получении от сервера уведомления об изменении какого-либо ресурса клиент должен использовать URI этого ресурса и повторно запросить у сервера последнюю версию его содержимого.

Пример взаимодействия с ресурсами. Пример взаимодействия клиента и сервера в отношении ресурсов показан на рис. 2.11.

2.5.3. Инструменты и примеры их взаимодействия

MCP позволяет серверу предоставлять различные инструменты, доступные для вызова крупными языковыми моделями, и поддерживает взаимодействие с внешними системами, например запросы к базам данных, вызовы API, обработку данных и т. д. Каждый инструмент должен иметь уникальный идентификатор (`name`) и описание его функциональности (`description`).

Инструменты в MCP спроектированы таким образом, чтобы ими управляли модели. Это означает, что крупные языковые модели могут автоматически находить и выбирать инструменты на основе своего восприятия контекста и введенных пользователем данных.

Из соображений конфиденциальности и безопасности вызов инструментов не должен оставаться незаметным для пользователя, и у него должно быть право отказаться от такого вызова. Приложение, реализующее вызов инструментов, должно:

- предоставлять понятный интерфейс, четко отображающий все инструменты, доступные для большой языковой модели;
- отображать четкие визуальные подсказки при вызове инструментов;
- настраивать всплывающее окно, используемое для подтверждения пользователем операции вызова инструмента.

Книги для программистов: https://t.me/free_books_for_it

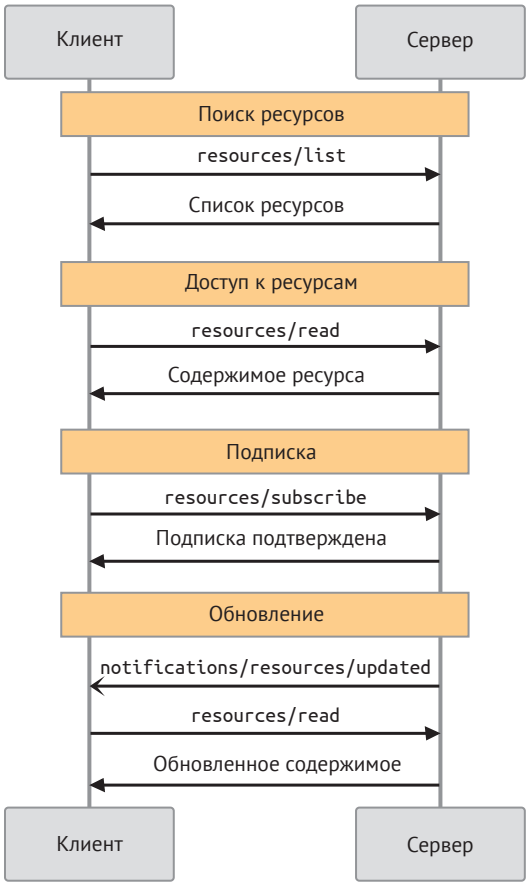


Рис. 2.11. Пример взаимодействия клиента и сервера в отношении ресурсов

Пример взаимодействия с инструментами. Сервер объявляет возможности инструментов. Сервер, поддерживающий возможности инструментов, должен объявить эту возможность при определении сервера.

```
{
  "capabilities": {
    "tools": {
      "listChanged": true
    }
  }
}
```

`"listChanged": true` указывает, что сервер поддерживает отправку уведомлений при изменении списка инструментов.

Клиент получает список инструментов. Клиент отправляет серверу запрос `tools/list` для получения списка определенных сервером инструментов. Пример запроса приведен ниже.

Книги для программистов: https://t.me/free_books_for_it

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

После получения запроса сервер возвращает список инструментов. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "description": "Получение информации о погоде по локации",
        "inputSchema": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "Название города или почтовый индекс"
            }
          },
          "required": ["location"]
        }
      }
    ],
    "nextCursor": "next-page-cursor"
  }
}
```

Получение списка инструментов, как и получение списка подсказок, поддерживает управление страницами.

Вызов инструмента клиентом. Клиент отправляет серверу запрос `tools/call` для вызова определенного инструмента. Пример запроса приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    }
  }
}
```

Книги для программистов: https://t.me/free_books_for_it

После получения запроса сервер вызывает инструмент по его названию и параметрам, а затем возвращает результат вызова. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
          "text": "Погода в Нью Йорке: сейчас \nTemperature: 72°F\nConditions:
частично облачно"
      }
    ],
    "isError": false
  }
}
```

Сервер отправляет уведомление об изменении списка инструментов. Когда список доступных на сервере инструментов изменяется, сервер должен отправить клиенту уведомление об изменении списка инструментов. Пример уведомления приведен ниже.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/tools/list_changed"
}
```

Получив от сервера уведомление об изменении списка инструментов, клиент должен повторно запросить у сервера список инструментов.

Пример взаимодействия с инструментами. Пример взаимодействия клиента и сервера в отношении инструментов показан на рис. 2.12.

2.6. Возможности клиента: root / sampling

MCP предоставляет две возможности для реализации клиента:

- корень (root);
- выборку (sampling).

На основе этих двух возможностей клиент может предоставлять серверу услуги по контролю доступа к файловой системе и обслуживанию больших языковых моделей.

2.6.1. Корень и примеры взаимодействия

MCP предоставляет клиенту стандартный способ раскрытия корня файловой системы для сервера. Корень (root) определяет границы, в пределах которых сервер может выполнять операции в файловой системе. Это дает ему возможность определить, к каким каталогам и файлам у него есть доступ. Сервер может запрашивать список корней у поддерживаемых клиентов и получать уведомления об изменениях в этом списке.

Книги для программистов: https://t.me/free_books_for_it

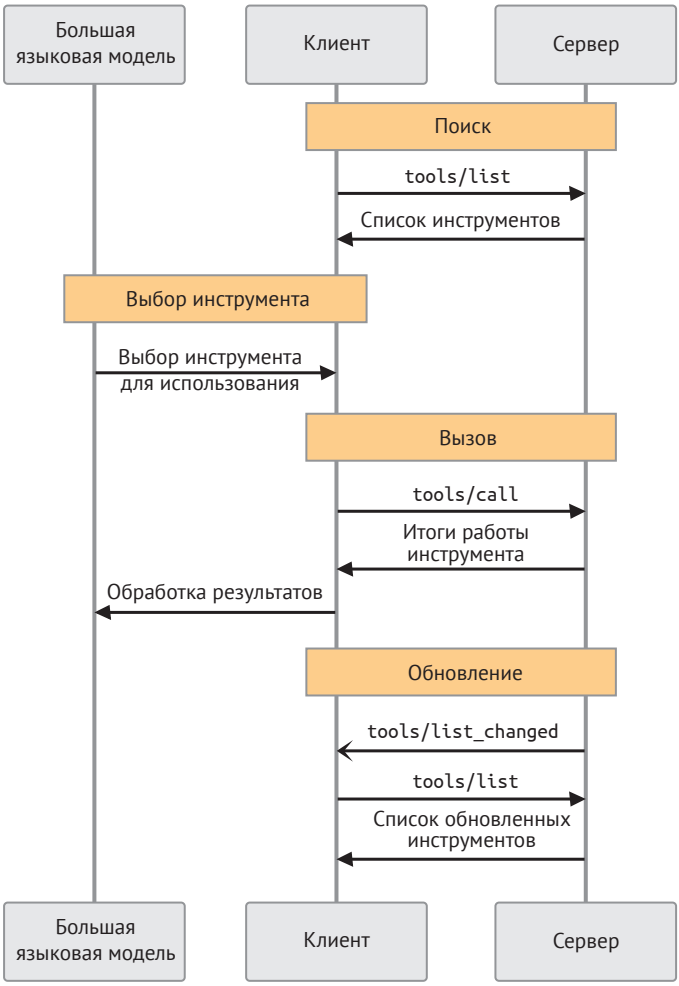


Рис. 2.12. Пример взаимодействия клиента и сервера по инструментам

В MCP корневые каталоги обычно отображаются через интерфейс настройки рабочей области или проекта. Например, хост может предоставить селектор рабочей области/проекта, который дает пользователю возможность выбирать каталоги и файлы, к которым сервер имеет право доступа, а также может автоматически определять рабочую область через систему контроля версий или открытые файлы.

По своей сути доступ к корневому каталогу – это механизм контроля доступа к файловой системе. При разработке возможностей корневого доступа для клиентов в основном учитывались следующие аспекты.

- **Контроль границ безопасности**: для повышения безопасности необходимо ограничить доступ сервера только к указанным каталогам и файлам, чтобы избежать доступа к конфиденциальному или неавторизованному контенту.

Книги для программистов: https://t.me/free_books_for_it

- **Принцип минимальных прав**: сервер запрашивает список корневых каталогов у клиента только тогда, когда ему необходимо работать с файловой системой, что снижает риски безопасности, связанные с чрезмерными правами.
- **Контроль со стороны пользователя и прозрачность**: при работе с файловой системой сервер должен запрашивать у клиента список прав root, а пользователь решает, согласиться или отказать. Этот процесс полностью прозрачен для пользователя.
- **Изоляция проектов**: при работе с разными проектами сервер должен запрашивать разные списки прав root, что обеспечивает изоляцию проектов.

Пример взаимодействия с правами root. Объявление прав root на стороне клиента. Клиенты, поддерживающие права root, должны объявить эту возможность при определении клиента.

```
{
  "capabilities": {
    "roots": {
      "listChanged": true
    }
  }
}
```

`"listChanged": true` означает, что клиент поддерживает отправку уведомлений при изменении корневого каталога.

Сервер получает список корневых каталогов. Когда серверу требуется выполнить операцию с файловой системой, он запрашивает у клиента список корневых каталогов. Пример запроса приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "roots/list"
}
```

Получив запрос, клиент возвращает список корневых каталогов. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "roots": [
      {
        "uri": "file:///home/user/projects/myproject",
        "name": "My Project"
      }
    ]
  }
}
```

Книги для программистов: https://t.me/free_books_for_it

Клиент отправляет уведомление об изменении корневого каталога. При изменении корневого каталога клиент отправляет серверу уведомление. Пример уведомления приведен ниже.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/roots/list_changed"
}
```

После получения уведомления сервер должен повторно запросить у клиента список корней.

Пример взаимодействия с корнями. Пример взаимодействия сервера и клиента в отношении корней показан на рис. 2.13.

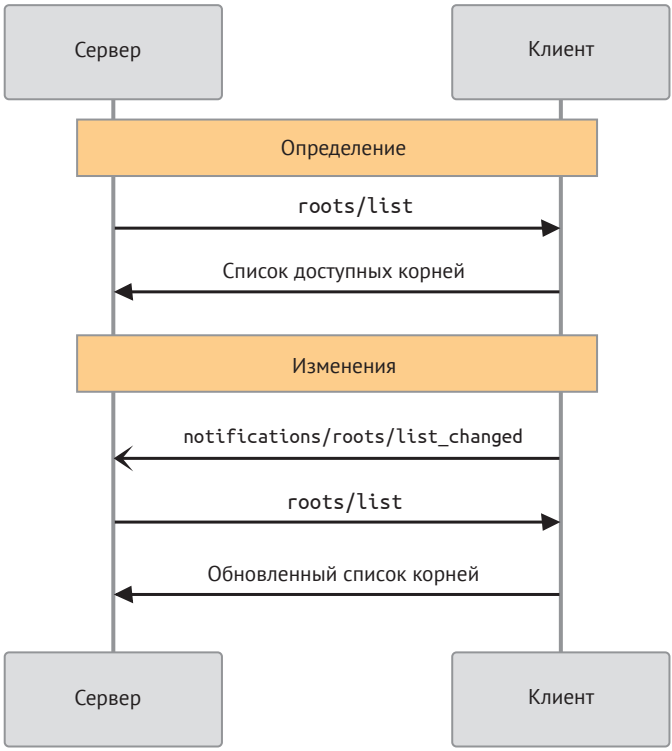


Рис. 2.13. Пример взаимодействия сервера и клиента с корнями

2.6.2. Выборка и примеры взаимодействия

MCP предоставляет серверу стандартный способ запроса у клиента выборки (sampling) для дополнительного доступа к данным или генерации контента. Этот процесс позволяет клиенту контролировать доступ к модели, возможность выбора и прав доступа, а также дает серверу возможность использовать функции большой языковой модели без необходимости настройки API-ключа модели на стороне сервера.

Книги для программистов: https://t.me/free_books_for_it

Суть выборки заключается в том, что клиент выступает в качестве посредника при вызове большой языковой модели сервером. При разработке механизма выборки на стороне клиента учитывались следующие аспекты.

- **Разделение прав управления и безопасность**. Клиент контролирует доступ, выбор и права доступа к большой модели, а сервер сосредоточен только на реализации функциональности, не обращая внимания на вопросы подключения большой модели. API-ключи большой модели централизованно управляются клиентом для повышения безопасности.
- **Реализация интеллектуального прокси-поведения**. При обработке сложных задач сервер может динамически вызывать возможности большой модели через клиентский прокси.
- **Гибкий механизм выбора модели**. Выборка дает серверу возможность предлагать конкретные модели, а окончательное решение принимает клиент.
- **Поддержка мультимодальности**. Выборка поддерживает взаимодействие с текстом, аудио и изображениями, создавая основу для сложных сценариев использования ИИ.

Пример работы выборки. Клиент определяет возможности выборки. Клиент, поддерживающий выборку, должен объявить эту возможность при определении клиента.

```
{
  "capabilities": {
    "sampling": {}
  }
}
```

Сервер получает результаты выборки. В процессе работы, если серверу требуются возможности большой языковой модели, он отправляет клиенту запрос на выборку. Пример запроса приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "Как называется столица Франции?"
        }
      }
    ],
    "modelPreferences": {
      "hints": [
        {
          "name": "claude-3-sonnet"
        }
      ],
```

Книги для программистов: https://t.me/free_books_for_it

```
        "intelligencePriority": 0.8,
        "speedPriority": 0.5
      },
      "systemPrompt": "Ты полезный помощник.",
      "maxTokens": 100
    }
}
```

После получения запроса на выборку клиент на основании содержания сообщения запрашивает у большой языковой модели ответ и отправляет серверу результат выборки. Пример ответа приведен ниже.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "role": "assistant",
    "content": {
      "type": "text",
      "text": "Столица Франции - Париж."
    },
    "model": "claude-3-sonnet-20240307",
    "stopReason": "endTurn"
  }
}
```

Пример взаимодействия при выборке. Пример взаимодействия между сервером и клиентом при выборке показан на рис. 2.14.

2.7. Механизм авторизации

С помощью механизма авторизации MCP предоставляет стандартный способ безопасного доступа клиента к защищенным ресурсам сервера. Основной принцип заключается в том, что владелец ресурса дает клиенту разрешение запрашивать защищенные ресурсы у сервера от его имени.

В соответствии с требованиями MCP сервер, реализованный на основе передачи stdio, должен считывать учетные данные авторизации из системных переменных. Сервер, реализованный на основе передачи HTTP, должен использовать механизм авторизации на основе OAuth 2.1.

2.7.1. OAuth 2.1

Прежде чем перейти к описанию OAuth 2.1, следует кратко ознакомиться с OAuth 2.0.

OAuth 2.0 – это открытая стандартная архитектура авторизации, позволяющая сторонним приложениям получать доступ к защищенным ресурсам пользователя без получения его пароля. OAuth 2.0 предоставляет стандартизированное решение для авторизации современных интернет-приложений, обеспечивая баланс между безопасностью и удобством использования, и в настоящее время является наиболее распространенной архитектурой авторизации. OAuth 2.0 широко используется для входа через учетные записи социальных сетей (Google, GitHub, WeChat и т. д.), контроля доступа к API и интеграции сторонних приложений.

Книги для программистов: https://t.me/free_books_for_it

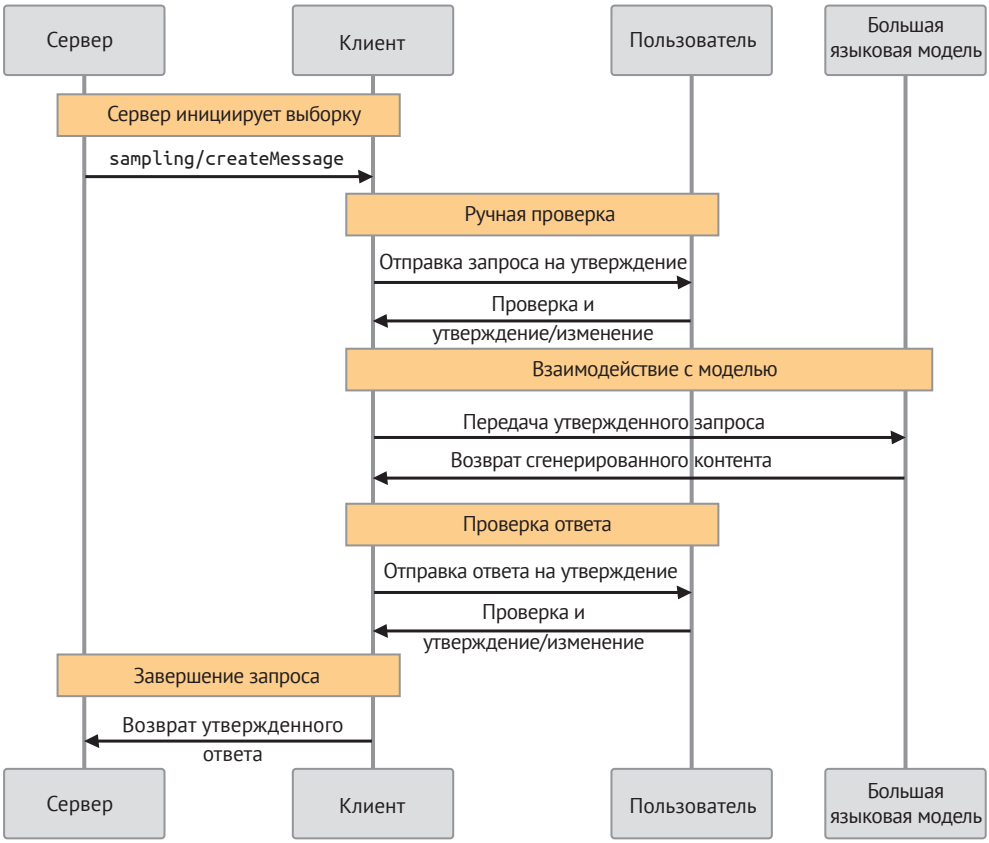


Рис. 2.14. Пример взаимодействия между сервером и клиентом при выборке

OAuth 2.1 – это новейшая версия архитектуры авторизации OAuth 2.0, которая еще больше упрощает процесс авторизации и повышает безопасность.

Основные принципы работы OAuth 2.1 и его участники:

- **владелец ресурса** (resource owner): как правило, это пользователь;
- **клиент** (client): приложение, запрашивающее доступ к ресурсу;
- **сервер ресурсов** (resource server): сервер, на котором хранятся защищенные ресурсы;
- **сервер авторизации** (authorization server): сервер, который проверяет идентификацию и выдает токен доступа.

OAuth 2.1 поддерживает следующие типы токенов:

- **токен доступа** (access token): используется для доступа к защищенным ресурсам;
- **токен обновления** (refresh token): используется для получения нового токена доступа;
- **код авторизации** (authorization code): используется для получения токена доступа.

Книги для программистов: https://t.me/free_books_for_it

Основным и наиболее рекомендуемым способом авторизации в OAuth 2.1 является процесс с кодом авторизации + PKCE (Proof Key for Code Exchange, ключ подтверждения для обмена кодами), который включает следующие шаги.

1. Клиент генерирует верификатор кода (`code_verifier`) и запрос кода (`code_challenge`).
2. Пользователь перенаправляется на сервер авторизации.
3. После авторизации пользователь получает код авторизации.
4. Клиент использует код авторизации и верификатор кода для обмена на токен доступа.

2.7.2. Основной процесс авторизации MCP

Основной процесс авторизации MCP показан на рис. 2.15.

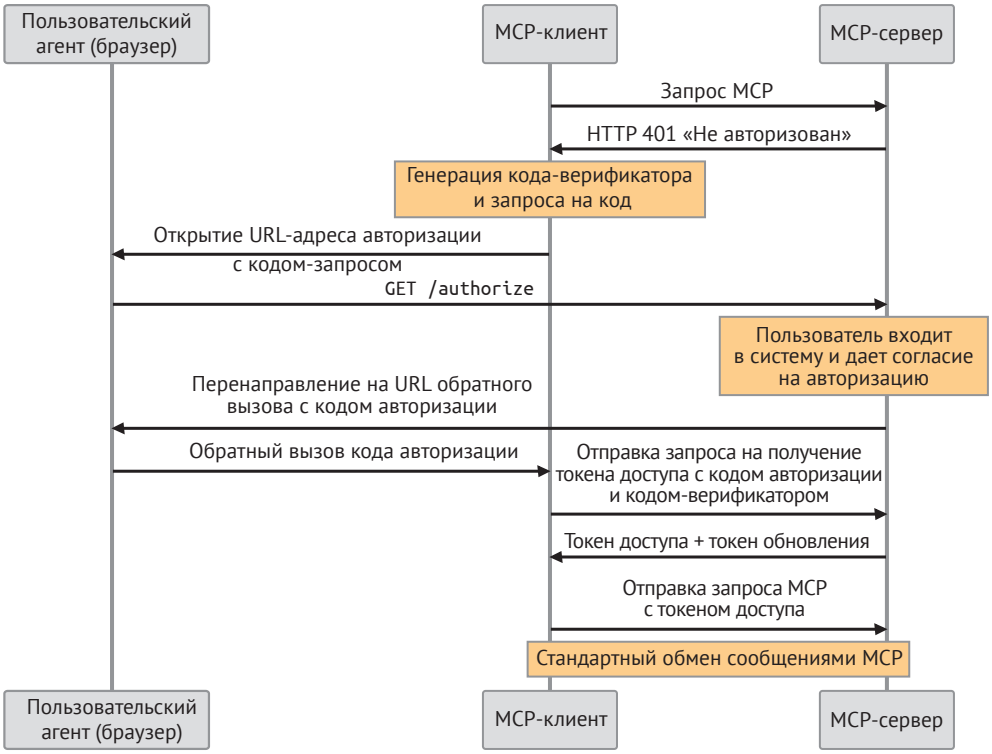


Рис. 2.15. Основной процесс авторизации MCP

Основные этапы этого процесса можно кратко описать следующим образом.

1. **Этап первоначального запроса**: MCP-клиент отправляет запрос на MCP-сервер и получает ответ 401 «Не авторизован».
2. **Подготовка PKCE**: MCP-клиент генерирует код-верификатор и код-запрос.
3. **Авторизация пользователя**: процесс авторизации пользователя осуществляется через браузер.

Книги для программистов: https://t.me/free_books_for_it

4. **Обмен токенами**: код авторизации обменивается на токен доступа.
5. **Стандартная коммуникация**: осуществляется стандартный обмен сообщениями MCP с использованием токена доступа.

Конечной целью авторизации MCP является получение токена, необходимого для доступа к ресурсам MCP-сервера. В процессе авторизации несколько этапов зависят от перенаправления по URL для передачи учетных данных, поэтому необходимо уделять особое внимание формированию URL-параметров, чтобы избежать потери важной информации об учетных данных во время перенаправления.

2.7.3. Определение метаданных сервера авторизации

В описанном выше базовом сценарии авторизации MCP по умолчанию MCP-сервер рассматривается как сервер авторизации, и процесс авторизации завершается с использованием конечной точки сервера по умолчанию.

Однако в реальных условиях сервер авторизации может существовать независимо от MCP-сервера, и конечные точки авторизации могут различаться. Поэтому MCP-клиент должен реализовать процесс определения метаданных сервера авторизации, предусмотренный OAuth 2.0, и завершить процесс авторизации с использованием определенной в метаданных конечной точки авторизации.

Процесс определения метаданных сервера авторизации показан на рис. 2.16.

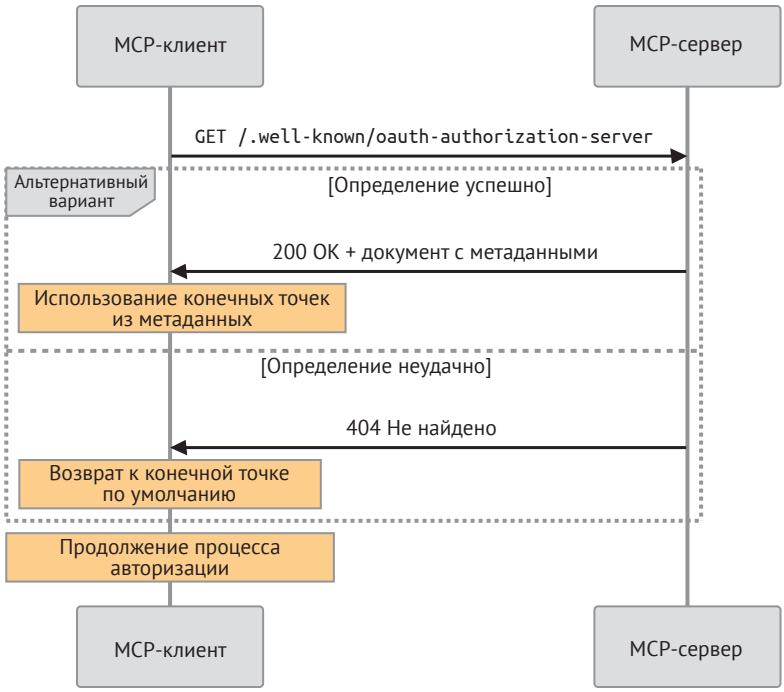


Рис. 2.16. Процесс определения метаданных сервера авторизации

Книги для программистов: https://t.me/free_books_for_it

Основные этапы этого процесса кратко описаны ниже.

1. MCP-клиент отправляет запрос на MCP-сервер для получения метаданных сервера авторизации.
2. Если сервер MCP возвращает ответ 200, это означает, что он поддерживает определение метаданных сервера авторизации, и MCP-клиент может продолжить процедуру авторизации с использованием конечной точки авторизации, указанной в метаданных.
3. Если MCP-сервер возвращает ответ 404, это означает, что он не поддерживает определение метаданных сервера авторизации, и в этом случае MCP-клиент завершает процедуру авторизации с использованием конечной точки авторизации по умолчанию.

Для MCP-серверов без функции обнаружения метаданных сервера авторизации MCP-клиент должен использовать пути к конечным точкам по умолчанию, как указано в табл. 2.4, относительно базового URL-адреса сервера авторизации.

Таблица 2.4. Пути к конечным точкам по умолчанию

Конечная точка	Путь по умолчанию	Описание
Конечная точка авторизации	*/authorize*	Используется для запросов авторизации
Конечная точка токена	*/token*	Используется для обмена и обновления токенов
Конечная точка регистрации	*/register*	Используется для динамической регистрации клиентов

2.7.4. Динамическая регистрация клиентов

В описанном выше процессе обнаружения метаданных сервера авторизации MCP-клиент получает из метаданных сервера MCP конечную точку авторизации, например `/authorize`.

В соответствии с базовым процессом авторизации, описанным в разделе 2.7.2, клиент должен перенаправить пользователя на конечную точку авторизации с передачей параметров клиента. В типичных сценариях авторизации OAuth для веб-приложений параметры клиента необходимо получить путем предварительной регистрации на сервере (например, после создания приложения на GitHub получаются `client_id` и `client_secret`).

Однако количество MCP-серверов огромно, и, если MCP-клиенту потребуется заранее запрашивать параметры авторизации для каждого MCP-сервера, затраты будут очень высокими. Поэтому MCP-клиент и MCP-сервер должны поддерживать процесс динамической регистрации клиентов, предусмотренный OAuth 2.0. Основные этапы этого процесса можно кратко описать таким образом.

1. Для начала MCP-клиент получает конечную точку регистрации посредством обнаружения метаданных, например `/register`.
2. MCP-клиент отправляет запрос на конечную точку регистрации, предоставляя информацию о клиенте (имя, версию, адрес обратного вызова и т. д.).

Книги для программистов: https://t.me/free_books_for_it

3. Сервер MCP завершает регистрацию клиента и возвращает параметры авторизации (`client_id`, `client_secret` и т. д.).



В сценарии динамической регистрации клиентов MCP-сервер должен строго проверять информацию в отправленных клиентом метаданных, включая URI перенаправления, тип авторизации, область действия и т. д. Одновременно следует усилить защиту конечной точки регистрации для предотвращения произвольной регистрации злонамеренных клиентов. К распространенным мерам защиты относятся ограничение частоты запросов, механизм белых списков и т. д.

После добавления процессов обнаружения метаданных сервера авторизации и динамической регистрации клиентов полный процесс авторизации MCP выглядит как показано на рис. 2.17.

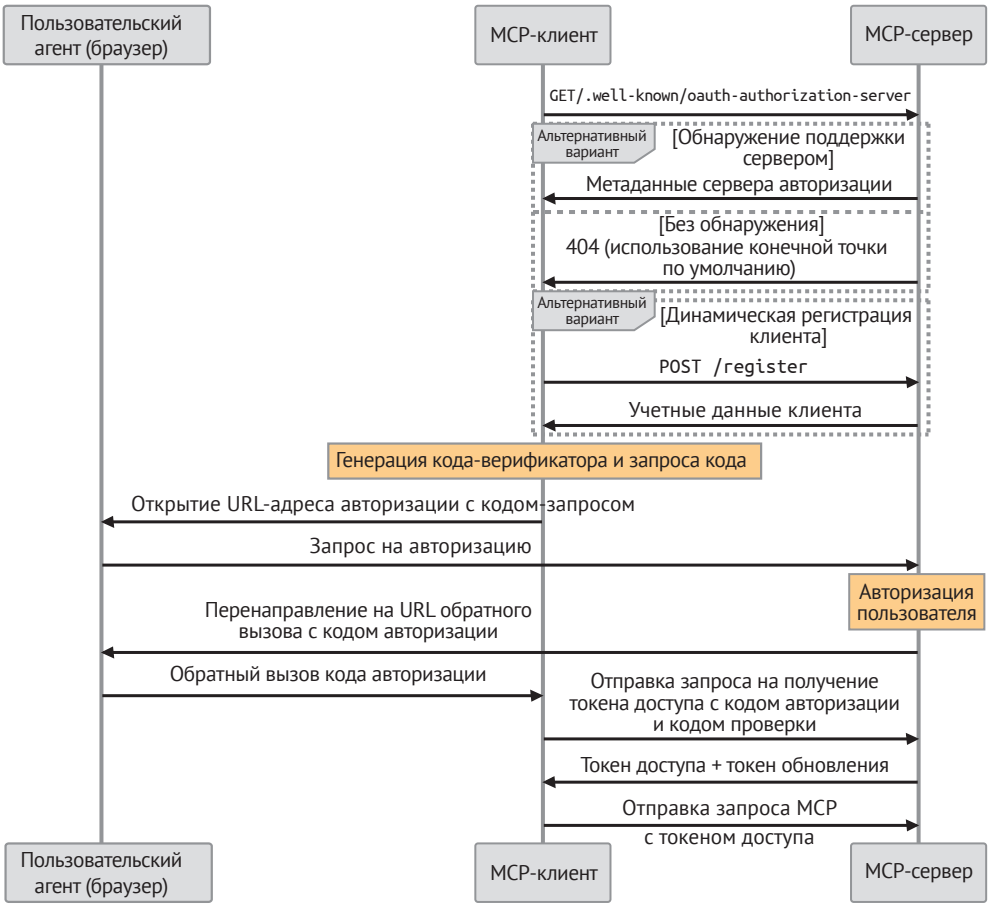


Рис. 2.17. Полный процесс авторизации MCP

Книги для программистов: https://t.me/free_books_for_it

2.7.5. Использование токена доступа

После получения токена доступа в результате прохождения процедуры авторизации MCP-клиент должен передавать этот токен при доступе к защищенным ресурсам MCP-сервера. Пример отправляемого HTTP-запроса приведен ниже.

```
GET /v1/contexts HTTP/1.1
Host: mcp.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

MCP-сервер должен проверить легитимность токена доступа MCP-клиента и на основании его прав принять решение о предоставлении доступа к защищенным ресурсам. Если запрошенный MCP-клиентом токен недействителен или просрочен, MCP-сервер должен вернуть ответ 401 «Не авторизовано».

2.7.6. Процесс авторизации через третью сторону

MCP-сервер также может поддерживать делегированную авторизацию через сервер авторизации третьей стороны. В этом случае MCP-сервер одновременно выступает в роли OAuth-клиента (по отношению к серверу авторизации третьей стороны) и OAuth-сервера авторизации (по отношению к MCP-клиенту).

Процесс авторизации через третью сторону показан на рис. 2.18.

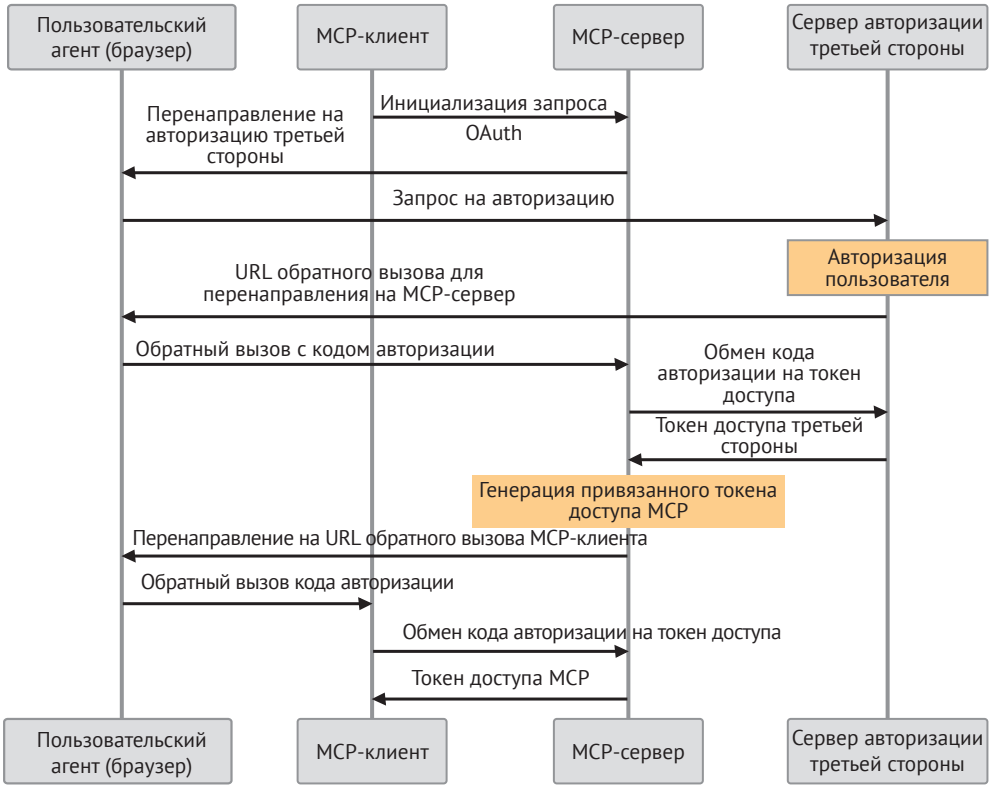


Рис. 2.18. Процесс авторизации через третью сторону

Книги для программистов: https://t.me/free_books_for_it

Основные этапы этого процесса кратко описаны ниже.

1. MCP-клиент и MCP-сервер запускают стандартный процесс OAuth.
2. MCP-сервер перенаправляет пользователя на сервер авторизации третьей стороны.
3. Пользователь проходит авторизацию через сервер третьей стороны.
4. Сервер третьей стороны перенаправляет пользователя на MCP-сервер с кодом авторизации.
5. MCP-сервер обменивает код авторизации на токен доступа третьей стороны.
6. MCP-сервер генерирует токен доступа, привязанный к сеансу третьей стороны.
7. MCP-сервер и MCP-клиент завершают процесс авторизации.

2.8. Заключение

В этой главе были представлены: архитектура MCP, основы обмена данными, жизненный цикл, механизмы передачи и механизмы авторизации. Цель этой главы заключается в предоставлении читателю системного понимания содержания протокола MCP и принципов его разработки.

Эта глава служит теоретической базой для последующих глав. В результате изучения этой главы читатель, как ожидается, получил представление об основополагающих принципах работы MCP и подготовился к разработке MCP-сервера и MCP-клиента.

Книги для программистов: https://t.me/free_books_for_it

Глава 3

Разработка MCP-сервера

После ознакомления с принципами работы MCP и архитектурой протокола вам, скорее всего, уже не терпится приступить к разработке MCP-сервера?

В этой главе планируется продемонстрировать полный процесс разработки MCP-сервера на двух практических примерах. В первом примере будет разработан MCP-сервер для ведения заметок, поддерживающий создание записей в диалоговом режиме. Во втором примере будет разработан MCP-сервер для виртуальной примерки одежды, поддерживающий быструю примерку с помощью клиента на базе большой языкового модели.

3.1. Процесс разработки MCP-сервера

Прежде чем приступить к рассмотрению примеров, давайте подготовим среду, необходимую для разработки MCP-сервера, и клиент с большой языковой моделью для отладки, а также кратко рассмотрим основные этапы разработки.

3.1.1. Подготовительные работы

1. Выбор языка разработки

MCP поддерживает разработку MCP-серверов на любом языке программирования. В настоящее время официальный сайт MCP предоставляет SDK (Software Development Kit, набор средств разработки программного обеспечения) для различных языков программирования, в том числе:

- Python SDK,
- TypeScript SDK,
- Java SDK,
- Kotlin SDK,
- C# SDK,
- Rust SDK.

Мы можем выбрать один из этих SDK и использовать соответствующий язык программирования для разработки MCP-сервера. Помимо этого, MCP также поддерживает использование других языков программирования для разработки MCP-сервера. Ниже приведены некоторые SDK от сторонних разработчиков.

- *mark3labs/mcp-go*: SDK для разработки MCP-сервера на языке Go.
- *goplus/mcp*: SDK для разработки MCP-сервера на языке Go+.
- *logiscape/mcp-sdk-php*: SDK для разработки сервера MCP на языке PHP.

Книги для программистов: https://t.me/free_books_for_it

После выпуска MCP разработчики из сообщества открытого исходного кода активно предоставляют различные инструменты разработки, что упрощает создание MCP-серверов. Нам достаточно выбрать язык программирования, которым мы владеем, и соответствующий SDK, чтобы быстро приступить к разработке MCP-сервера.

В этой главе в качестве языка разработки выбран TypeScript.

2. Настройка среды разработки

В зависимости от выбранного языка разработки необходимо настроить соответствующую среду на своем компьютере. В данном случае выбран TypeScript, поэтому перед началом разработки сервера MCP необходимо установить Node.js и инструмент управления пакетами npm.

Установка fnm. fnm – это инструмент управления версиями Node.js. Перейдите в официальный репозиторий fnm на GitHub, следуйте инструкциям в документации fnm, выберите способ установки в соответствии с типом операционной системы и выполните команду в терминале.

После завершения установки fnm выполните команду в терминале для проверки версии fnm и подтверждения успешности установки.

```
# Проверка пути установки fnm
which fnm
# Проверка версии fnm
fnm --version
# Просмотр справки по fnm
fnm --help
```

Установка Node.js. Выполните команду в терминале для установки указанной версии Node.js с помощью fnm.

```
# Просмотр всех доступных для установки версий Node.js
fnm ls-remote
# Выбор версии для установки
fnm install v22.2.0
```

В соответствии с официальными требованиями MCP версия Node.js должна быть не ниже 16; например, можно установить версию v22.2.0.

После завершения установки Node.js выполните в терминале команды для включения указанной версии Node.js и проверки версии Node.js, чтобы убедиться в успешности установки.

```
# Просмотр всех установленных версий Node.js
fnm list
# Установка версии Node.js по умолчанию
fnm use v22.2.0
# Проверка пути установки Node.js
which node
# Проверка версии Node.js
node --version
```

Установка npm. npm – это официальный инструмент управления пакетами Node.js, используемый для установки сторонних библиотек Node.js. npx – это также официальный инструмент Node.js, позволяющий напрямую запускать

Книги для программистов: https://t.me/free_books_for_it

команды из пакетов npm без необходимости глобальной или локальной установки пакетов npm.

Выполните команды в терминале, чтобы проверить, доступны ли npm и npx.

```
# Проверка пути установки npm
which npm
# Проверка версии npm
npm --version
# Проверка пути установки npx
which npx
# Проверка версии npx
npx --version
```

После успешной установки npm и npx можно приступать к разработке MCP-сервера с использованием TypeScript.

3. Установка клиентских приложений для больших языковых моделей

Для отладки функций MCP-сервера в диалоговом режиме в процессе разработки необходимо установить несколько (как минимум одно) клиентских приложений для больших языковых моделей, которые должны поддерживать MCP, то есть обеспечивать настройку MCP-сервера.

Среди рекомендуемых мной клиентских приложений для больших языковых моделей можно выделить следующие:

- официальный клиент MCP для Claude (настольный клиент);
- ИИ-редакторы, такие как VS Code, Cursor, Windsurf и др.;
- диалоговые клиенты, такие как ChatMCP, ChatWise, Cherry Studio, DeepChat и др.

В двух примерах разработки сервера MCP, представленных в этой главе, в качестве клиентов больших языковых моделей для отладки функций MCP-сервера используются Cursor и Claude. Читатели могут самостоятельно установить эти приложения с официальных сайтов Cursor и Claude.

3.1.2. Основные этапы

Процесс разработки MCP-сервера включает в себя следующие основные этапы.

1. Создание проекта MCP-сервера

Официальный сайт MCP предоставляет инструмент командной строки под названием `create-server`. Выполнив эту команду, можно быстро создать проект MCP-сервера с автоматически сгенерированной и хорошо организованной структурой кода. После этого можно сразу приступить к разработке MCP-сервера. Допустим, вам нужно разработать MCP-сервер для проверки погоды под названием `weather-mcp`. Для этого в терминале выполните следующую команду.

```
npx @modelcontextprotocol/create-server weather-mcp
```

2. Реализация бизнес-логики MCP-сервера

При создании проекта MCP-сервера с помощью официального инструмента `create-server` автоматически устанавливается TypeScript SDK: `@modelcontextpro-`

Книги для программистов: https://t.me/free_books_for_it

tocol/sdk. Основная логика проекта находится в файле src/index.ts, и для реализации конкретных бизнес-функций нам достаточно изменить этот файл.

3. Отладка MCP-сервера

Официально для MCP предоставляется инструмент отладки под названием MCP Inspector, который помогает разработчикам отлаживать функции внутренней реализации MCP-сервера. Для быстрого запуска MCP Inspector выполните следующую команду в корневом каталоге сервера MCP.

```
npx @modelcontextprotocol/inspector node build/index.js
```

В терминале отобразится адрес локального запуска. Нажмите на этот адрес, чтобы перейти в MCP Inspector. После успешного подключения к MCP-серверу можно отлаживать функции, реализованные внутри MCP-сервера, в панели инструментов.



Обратите внимание, что в процессе разработки MCP-сервера разработчики обычно отлаживают код одновременно с разработкой. Поэтому MCP Inspector обычно запускают сразу после успешного создания проекта MCP-сервера, а не ждут, пока будет завершена реализация внутренних функций MCP-сервера. В примерах, приведенных далее в этой главе, будет представлена подробная демонстрация этого процесса. Кроме того, в клиентском приложении для больших языковых моделей будет настроена разрабатываемая версия MCP-сервера для тестирования его внутренних функций.

4. Выпуск MCP-сервера

Публичный выпуск готового MCP-сервера позволит большему количеству пользователей использовать его в различных сценариях и принесет разработчикам чувство удовлетворения.

Обычно в первую очередь рекомендуется загрузить исходный код MCP-сервера на GitHub и выпустить его в формате открытого исходного кода. Кроме того, его можно опубликовать на платформе npm, чтобы пользователи могли быстро настроить и запустить его с помощью команд npx. Также можно разместить его на сторонних маркетплейсах приложений MCP (таких как MCP.so), чтобы его могли найти и использовать как можно больше пользователей. В последующих примерах эти способы публикации будут рассмотрены подробнее.

MCP.so является всемирно известным маркетплейсом сторонних приложений MCP, в котором собраны лучшие приложения MCP (включая MCP-серверы и MCP-клиенты), созданные разработчиками со всего мира. Главная страница сайта MCP.so показана на рис. 3.1.

На сайте MCP.so пользователи могут вводить ключевые слова, искать нужный MCP-сервер и просматривать список инструментов, входящих в состав MCP-сервера. Пользователи также могут щелкнуть по какому-либо инструменту для перехода в панель отладки и настроить функции этого инструмента.

Книги для программистов: https://t.me/free_books_for_it

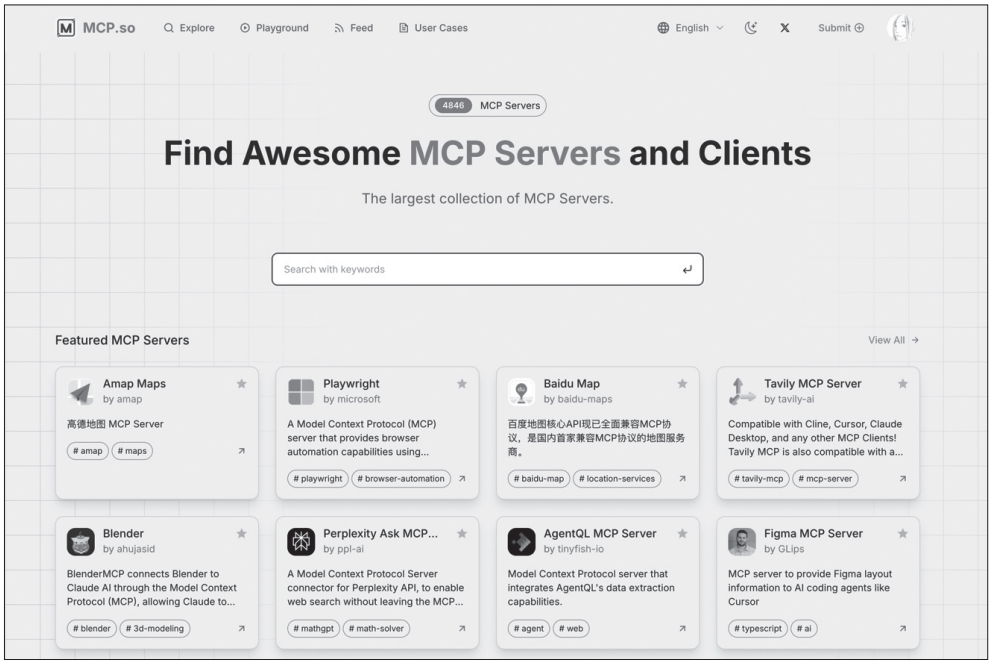


Рис. 3.1. Главная страница сайта MCP.so

После подготовки среды разработки MCP-сервера и ознакомления с основными этапами его разработки перейдем к изучению полного цикла разработки MCP-сервера на конкретных примерах.

3.2. Пример 1. Разработка MCP-сервера flomo для эффективного ведения заметок

3.2.1. Цели разработки

В этом разделе планируется разработать MCP-сервер под названием *flomo-mcp*, который будет взаимодействовать с API приложения flomo (сокращенно flomo) и позволит записывать содержание заметок в flomo в форме диалога из любого клиента с поддержкой больших языковых моделей, что значительно повысит эффективность управления фрагментарными знаниями.

flomo – это легкий инструмент для записи разрозненных данных, разработанный китайской командой. В его дизайне отказались от сложных функций традиционных программ для ведения заметок, сделав акцент на максимально простом способе фиксации вдохновения, идей и повседневных размышлений. Официальный сайт продукта flomo показан на рис. 3.2.

Книги для программистов: https://t.me/free_books_for_it



Рис. 3.2. Официальный сайт продукта flomo



Целью этого примера является объяснение принципов разработки MCP-сервера, основанного на интеграции с API. Аналогичным образом, если вы привыкли использовать такие приложения для ведения заметок, как Notion или Obsidian, вы также можете воспользоваться этим примером для реализации MCP-сервера, поддерживающего ведение заметок в диалоговом режиме.

3.2.2. Подготовительные работы

Для стабильной интеграции MCP-сервера с API flomo необходимо заранее настроить права доступа к учетной записи и проверить работоспособность API. Выполните следующие действия в указанном порядке.

1. В левом верхнем углу личной страницы flomo нажмите **Центр расширений и API** (扩展中心 & API) в раскрывающемся списке личной учетной записи, чтобы перейти в консоль **API записей**, как показано на рис. 3.3.



Рис. 3.3. Консоль API записей flomo

Книги для программистов: https://t.me/free_books_for_it

2. API записей flomo является функцией, доступной только для пользователей, имеющих статус PRO (после регистрации вы получаете 14 дней бесплатного доступа).
3. После перехода на статус PRO скопируйте свою уникальную ссылку на API записей.
4. Откройте консоль и отправьте запрос к API с помощью команды `curl`:

```
curl -X POST https://flomoapp.com/iwh/xxx/xxxxxxxxxxxxxxxx/ \
-H 'Content-Type: application/json' \
-d '{"content": "С этого момента я буду изучать разработку MCP Server"}'
```

Вернитесь в список заметок flomo (личная страница), где вы увидите заметку, только что записанную с помощью API через консоль, как показано на рис. 3.4.



Рис. 3.4. Просмотр заметки, записанной с помощью API через терминал

Далее для разработки flomo-mcp будет использоваться Node.js.

3.2.3. Создание проекта MCP-сервера flomo

С помощью официального инструмента командной строки MCP `create-server` создайте проект с помощью следующей команды:

```
npx @modelcontextprotocol/create-server flomo-mcp
```

Затем в командной строке появится запрос на ввод названия и описания MCP-сервера, а также будет задан вопрос, следует ли установить этот сервер для Claude.app. После подтверждения информации система автоматически завершит инициализацию проекта и предоставит инструкции по дальнейшим действиям, как показано на рис. 3.5.



Несмотря на то что в официальной документации MCP есть примеры ручного создания проектов серверов, для начинающих разработчиков MCP-серверов я рекомендую использовать официальный SDK для быстрого создания шаблона проекта и разработки на его основе: это значительно повысит эффективность процесса.

Книги для программистов: https://t.me/free_books_for_it

```
~/code/all-in-aigc/chatmcp (30.793s)
npx @modelcontextprotocol/create-server flomo-mcp

? What is the name of your MCP server? flomo-mcp
? What is the description of your server? write notes to flomo
? Would you like to install this server for Claude.app? No
✔ MCP server created successfully!

Next steps:
  cd flomo-mcp
  npm install
  npm run build  # or: npm run watch
  npm link       # optional, to make available globally
```

Рис. 3.5. Создание проекта MCP-сервера flomo

После завершения создания проекта перейдите в каталог flomo-mcp в соответствии с подсказками и установите зависимости проекта:

```
cd flomo-mcp
npm install
```

Откройте созданный flomo-mcp в редакторе кода, где можно увидеть структуру кода проекта, сгенерированную по умолчанию, как показано на рис. 3.6.

- `src/index.ts`: главный файл исходного кода flomo-mcp, реализующий конкретную бизнес-логику flomo-mcp.
- `build/index.js`: исполняемый файл, скомпилированный из исходного кода flomo-mcp, который используется как при отладке, так и при развертывании.

3.2.4. Отладка MCP-сервера flomo

В каталоге проекта flomo-mcp для отладки используйте следующие две команды.

- **npm run watch**. Команда `npm run watch` запускает службу прослушивания, которая отслеживает изменения в исходном файле *src/index.ts* и в режиме реального времени компилирует его в исполняемый файл *build/index.js*, который используется для последующего запуска и отладки.
- **npm run inspector**. При выполнении `npm run inspector` фактически запускается следующая команда:

```
npx @modelcontextprotocol/inspector build/index.js
```

Эта команда вызывает официальный инструмент отладки MCP Inspector, как показано на рис. 3.7.

Книги для программистов: https://t.me/free_books_for_it

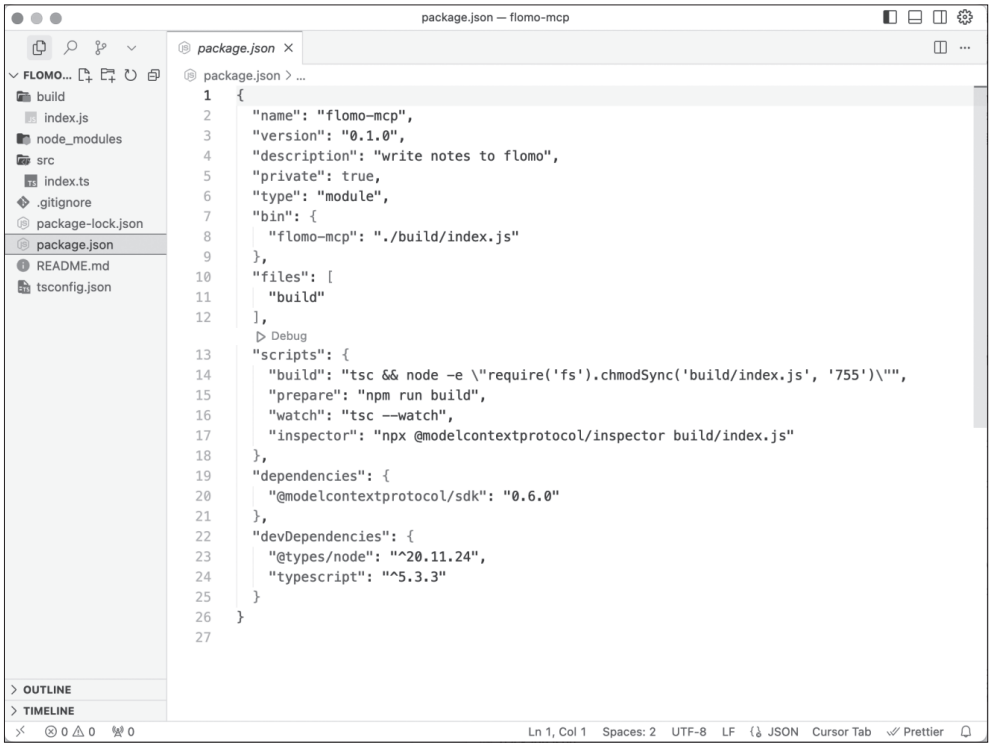


Рис. 3.6. Структура кода проекта MCP-сервера flomo-mcp

```
~/code/all-in-aigc/chatmcp/flomo-mcp
npm run inspector

> flomo-mcp@0.1.0 inspector
> npx @modelcontextprotocol/inspector build/index.js

Starting MCP inspector...
⚙️ Proxy server listening on port 6277
🔍 MCP Inspector is up and running at http://127.0.0.1:6274 🚀
```

Рис. 3.7. Запуск и подключение к MCP Inspector

После запуска команды `npm run inspector` в терминале отобразится адрес локального запуска (см. последнюю строку на рис. 3.7). Нажмите на этот адрес для перехода в MCP Inspector.

В MCP Inspector соответствующие параметры flomo-mcp уже загружены по умолчанию в левой панели меню. Нажмите на **Connect** для подключения к flomo-mcp с целью отладки. Появление надписи *Connected* означает, что подключение прошло успешно, как показано на рис. 3.8.

Книги для программистов: https://t.me/free_books_for_it

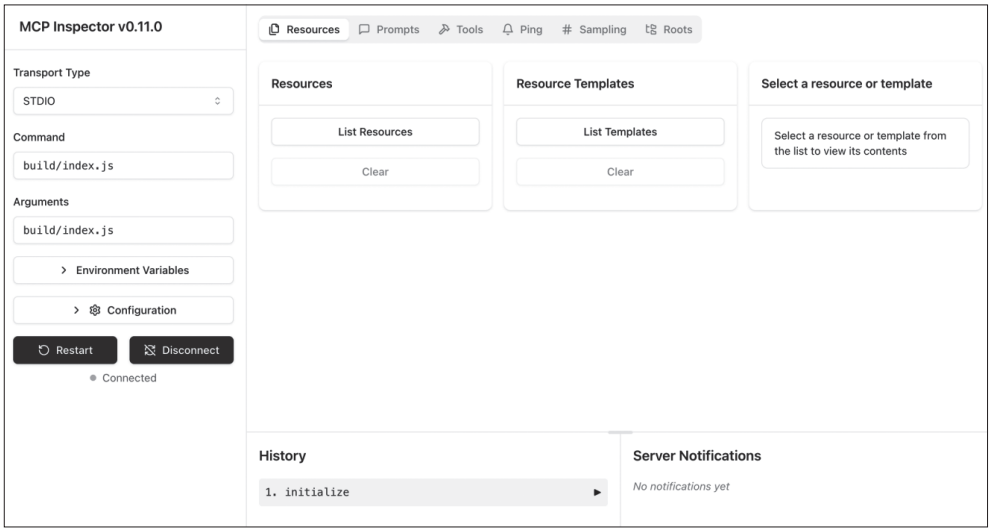


Рис. 3.8. Отладка MCP-сервера в MCP Inspector

В MCP Inspector можно настроить **параметры запуска** MCP-сервера (Arguments) и **переменные среды** (Environment Variables). Значения этих параметров можно считывать в логике реализации MCP-сервера.

В правой области MCP Inspector можно отлаживать три основных компонента внутренней реализации MCP-сервера.

- **Resources** (ресурсы): ресурсы, определенные внутри MCP-сервера.
- **Prompts** (подсказки): подсказки, определенные внутри MCP-сервера.
- **Tools** (инструменты): внутренние инструменты, определенные на MCP-сервере.

Чтобы просмотреть все инструменты, определенные в MCP-сервере, можно нажать на опцию **List Tools** (показать инструменты) на расположенной справа вкладки **Tools** (инструменты). Обратите внимание, что при создании MCP-сервера с помощью официального инструмента командной строки по умолчанию генерируется инструмент *create_note*.

Выберите какой-либо инструмент в **Tools** (например, `create_note`), заполните параметры запроса, а затем нажмите **Run Tool** (запуск инструмента) для отправки запроса этому инструменту и получения данных ответа, как показано на рис. 3.9.

Книги для программистов: https://t.me/free_books_for_it

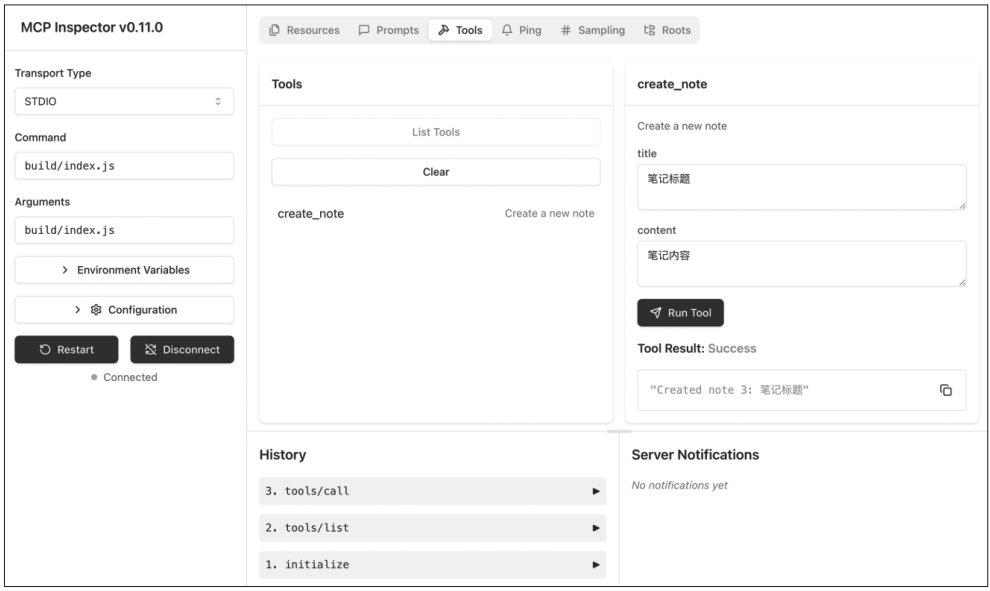


Рис. 3.9. Вызов инструмента в MCP Inspector

3.2.5. Реализация бизнес-логики MCP-сервера flomo

В разделе 3.2.2 был подготовлен специальный URL-адрес API записи flomo, а также выполнена отладка с помощью вызова API из командной строки, что позволило успешно пройти процесс создания заметки. Далее нам нужно создать в flomo-mcp инструмент для вызова API записи flomo с целью создания заметок.

1. Определение информации о flomo-mcp

Откройте файл *src/index.ts*. Вы увидите, что сгенерированный по умолчанию код создал сервер с помощью `new Server`.

```
const server = new Server(
  {
    name: "flomo-mcp",
    version: "0.1.0",
  },
  {
    capabilities: {
      resources: {},
      tools: {},
      prompts: {},
    },
  }
);
```

С учетом того, что flomo-mcp нужно реализовать только инструмент для создания заметок, а не другие функции, можно изменить код и определить, что flomo-mcp обладает только средствами Tools. Также можно настроить номер версии, например начиная с `0.0.1`.

Книги для программистов: https://t.me/free_books_for_it

```
const server = new Server(
  {
    name: "flomo-mcp",
    version: "0.0.1",
  },
  {
    capabilities: {
      tools: {},
    },
  }
);
```

2. Определение списка инструментов flomo-mcp

Измените логику получения **списка инструментов** (`ListTools`), сгенерированную по умолчанию в файле *src/index.ts*, и определите метод `write_note`. Описание функции должно быть написано на английском языке с указанием основного назначения этого инструмента: записи заметок в flomo.

Метод `write_note` принимает только один параметр: `content` (строка, обязательный). Это значит, что текст заметки должен быть в формате Markdown.

Определение `ListTools` выглядит следующим образом.

```
server.setRequestHandler(ListToolsRequestSchema, async () => {
  return {
    tools: [
      {
        name: "write_note",
        description: "Write note to flomo",
        inputSchema: {
          type: "object",
          properties: {
            content: {
              type: "string",
              description: " Текстовое содержание заметки в формате Markdown",
            },
          },
          required: ["content"],
        },
      },
    ],
  };
});
```

3. Реализация логики инструментов flomo-mcp

Сначала создайте новый файл *src/flomo.ts* и определите класс `FlomoClient` для взаимодействия с API flomo. Этот класс должен принимать передаваемый извне параметр `apiUrl` через конструктор и предоставлять метод `writeNote`, используемый для записи указанного содержимого в flomo в качестве заметки.

Книги для программистов: https://t.me/free_books_for_it

```
/**
 * Класс клиента для подключения к API flomo
 */
export class FlomoClient {
  private readonly apiUrl: string;

  /**
   * Конструктор
   * @param apiUrl - API для собственных записей
   */
  constructor({ apiUrl }: { apiUrl: string }) {
  this.apiUrl = apiUrl;
  }

  /**
   * Метод записи заметок
   * @param content - текст заметки
   * @returns Содержимое ответа API
   */
  async writeNote({ content }: { content: string }) {
    try {
      if (!content) {
        throw new Error("invalid content");
      }

      const req = {
        content,
      };

      const resp = await fetch(this.apiUrl, {
         method: "POST",
         headers: {
           "Content-Type": "application/json",
         },
         body: JSON.stringify(req),
       });

       if (!resp.ok) {
         throw new Error(`request failed with status ${resp.statusText}`);
       }

       return resp.json();
    } catch (e) {
      throw e;
    }
  }
}
```

Книги для программистов: https://t.me/free_books_for_it

Затем мы изменяем логику CallTool: при получении запроса на вызов инструмента write_note сначала извлекаем переданный параметр content, а затем вызываем метод writeNote класса FlomoClient для записи заметки в flomo.

```
import { FlomoClient } from "./flomo.js";

server.setRequestHandler(CallToolRequestSchema, async (request) => {
  switch (request.params.name) {
    case "write_note": {
      const content = String(request.params.arguments?.content);
      if (!content) {
        throw new Error("Content is required");
      }

      const apiUrl =
        "https://flomoapp.com/iwh/xxx/xxxxxxxxxxxxxxxx/";

      const flomo = new FlomoClient({ apiUrl });
      const result = await flomo.writeNote({ content });

      return {
        content: [
          {
            type: "text",
            text: `Написать заметку про успех flomo: ${JSON.stringify(result)}`,
          },
        ],
      };
    }

    default:
      throw new Error("Неизвестный инструмент");
  }
})
```

4. Отладка инструментов flomo-mcp

В MCP Inspector откройте вкладку **Tools** для flomo-mcp, выберите инструмент write_note, введите параметр content, нажмите **Run Tool** и просмотрите результат вызова инструмента, как показано на рис. 3.10.

На рисунке видно, что API flomo вернул code: 0, message: “Записано”. Это означает, что заметка была успешно записана в flomo.

Войдя в административную панель flomo, можно убедиться, что заметка действительно была записана, как показано на рис. 3.11.

Книги для программистов: https://t.me/free_books_for_it

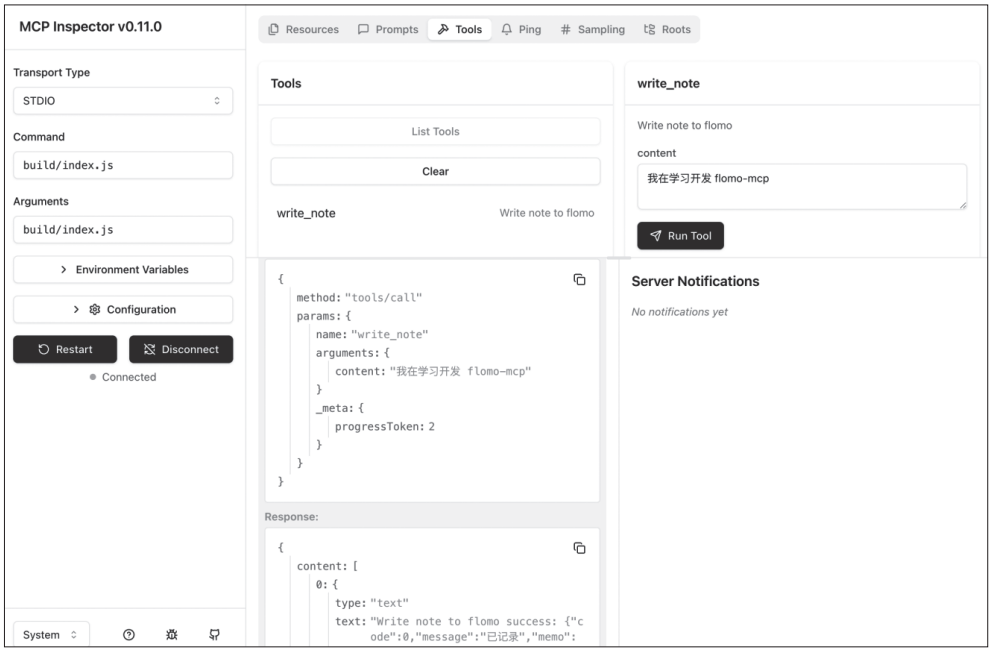


Рис. 3.10. Отладка инструмента write_note в flomo-mcp



Рис. 3.11. Просмотр заметки в административной панели flomo

3.2.6. Оптимизация MCP-сервера flomo

В ходе предыдущих шагов нам удалось реализовать базовую бизнес-логику в flomo-mcp. Перед запуском в производственную среду можно также провести дополнительную оптимизацию по некоторым второстепенным вопросам.

1. Удаление избыточного кода

Поскольку основная логика flomo-mcp представлена только инструментом `write_note`, мы можем удалить из файла *src/index.ts* весь код, сгенерированный по умолчанию и не имеющий отношения к инструменту, включая логику следующих частей:

Книги для программистов: https://t.me/free_books_for_it

- ListResources,
- ReadResource,
- ListPrompts,
- GetPrompt.

2. Изменение содержимого ответа

В разделе 3.2.4 после успешной записи заметки с помощью вызова `create_note` API-ответ кодируется в JSON и возвращается напрямую. Такой формат ответа не очень удобен для вызывающего фронтенда. Можно изменить это таким образом, чтобы возвращать ссылку на страницу с подробностями заметки flomo. Формат этой ссылки будет выглядеть примерно следующим образом.

```
https://v.flomoapp.com/mine/?memo_id=xxx
```

Для этого необходимо изменить

```
server.setRequestHandler(CallToolRequestSchema, async(request) => {
  // …
})
```

в логике обработки данных ответа.

```
const flomo = new FlomoClient({ apiUrl });
const result = await flomo.writeNote({ content });

if (!result.memo || !result.memo.slug) {
  throw new Error(
    `Не удалось записать заметку в flomo: ${result?.message || "неизвестная ошибка"}`
  );
}

const flomoUrl = `https://v.flomoapp.com/mine/?memo_id=${result.memo.slug}`;

return {
  content: [
    {
      type: "text",
     text: `Написать заметку об успехе flomo, смотреть ее по адресу: ${flomoUrl}`,
    },
  ],
};
```

3. Изменение параметров аутентификации

В предыдущей реализации `CallTool` параметр `apiUrl`, передаваемый при создании `FlomoClient`, был жестко запрограммирован. Поскольку разные пользователи при использовании flomo-mcp нуждаются в собственном API для записей flomo, настройку `apiUrl` необходимо изменить на динамическую передачу параметров.

Существует два способа реализации динамической передачи параметров.

- **Передача параметров через командную строку запуска MCP-сервера**.

  Реализуем функцию `parseArgs`, которая при запуске flomo-mcp считывает `flomo_api_url` из параметров командной строки.

Книги для программистов: https://t.me/free_books_for_it

```
/**
 * Анализ параметров командной строки
 * Пример: node index.js --flomo_api_url=https://flomoapp.com/iwh/xxx/xxx/
 */
function parseArgs() {
  const args: Record<string, string> = {};
  process.argv.slice(2).forEach((arg) => {
    if (arg.startsWith("--")) {
      const [key, value] = arg.slice(2).split("=");
      args[key] = value;
    }
  });
  return args;
}

const args = parseArgs();
const apiUrl = args.flomo_api_url || "";
```

Отредактируйте внутреннюю логику реализации `CallTool` для проверки передачи `apiUrl` и динамической передачи параметра `apiUrl` для создания заметок.

```
if (!apiUrl) {
  throw new Error("URL API flomo не указан");
}
const content = String(request.params.arguments?.content);
if (!content) {
  throw new Error("Требуется ввести контент");
}

const flomo = new FlomoClient({ apiUrl });
const result = await flomo.writeNote({ content });
```

В левой области MCP Inspector установите в поле ввода `Arguments` параметр: `--flomo_api_url=https://flomoapp.com/iwh/xxx/xxx/`. Затем, следуя описанному выше методу, отправьте запрос к инструменту `write_note`, и запись будет успешно сохранена, как показано на рис. 3.12.

Соответствующая команда запуска сервера MCP выглядит следующим образом.

```
node build/index.js --flomo_api_url=https://flomoapp.com/iwh/xxx/xxx/
```

- **Передача параметров через переменные среды**.

  В качестве дополнения к чтению параметров из командной строки можно также настроить динамические параметры через переменные среды. Измените логику чтения динамических параметров, отдавая приоритет чтению параметров из командной строки. Если параметры командной строки не переданы, считываются параметры из переменных среды.

```
const args = parseArgs();
const apiUrl = args.flomo_api_url || process.env.FLOMO_API_URL || “”;
```

Книги для программистов: https://t.me/free_books_for_it

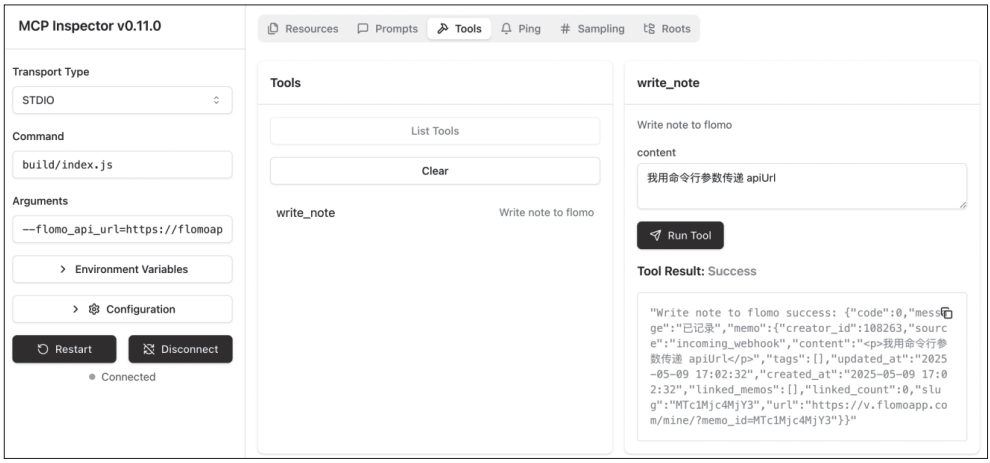


Рис. 3.12. Передача параметров через команду запуска MCP-сервера

В MCP Inspector удалите `Arguments`, добавьте переменную среды `FLOMO_API_URL` и задайте соответствующее значение. Выполните запрос к инструменту `write_note`, чтобы успешно записать заметку, как показано на рис. 3.13.

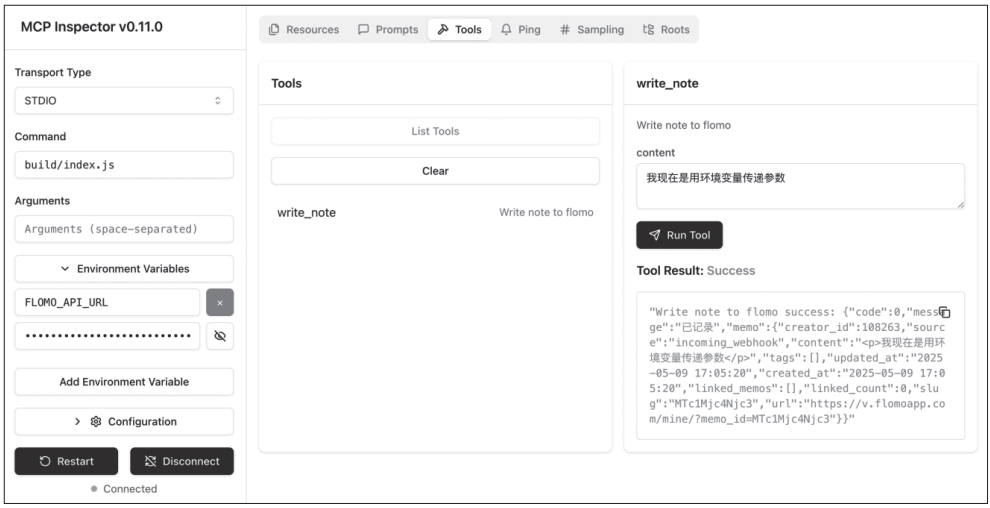


Рис. 3.13. Передача параметров через переменные среды

Соответствующая команда запуска MCP-сервера выглядит следующим образом.

```
FLOMO_API_URL=https://flomoapp.com/iwh/xxx/xxx/ node build/index.js
```

3.2.7. Тестирование MCP-сервера flomo в клиенте большой языковой модели

Для тестирования разработанного нами flomo-mcp в качестве клиента большой языковой модели был выбран Cursor.

Книги для программистов: https://t.me/free_books_for_it

Сначала откройте файл конфигурации MCP-сервера Cursor (обычно находится в *~/.cursor/mcp.json*) и добавьте конфигурацию flomo-mcp.

```
{
  "mcpServers": {
    "flomo-mcp": {
      "command": "node",
      "args": [
        "/Users/idoubi/code/all-in-aigc/chatmcp/flomo-mcp/build/index.js"
      ],
      "env": {
        "FLOMO_API_URL": "https://flomoapp.com/iwh/xxx/xxx/"
      }
    }
  }
}
```

Передайте `FLOMO_API_URL` через переменную среды, а в поле `args` укажите абсолютный путь к скомпилированному файлу flomo-mcp. На панели настроек MCP в Cursor можно увидеть, что flomo-mcp успешно запущен и получил доступ к имеющимся инструментам.

Откройте панель диалога **Cursor AI**, выберите режим **Agent** и введите следующий или похожий текст:

```
Сегодня я разработал сервер MCP, и это меня очень радует. Запиши это для меня.
```

Cursor загрузит все доступные инструменты из настроенных серверов MCP и запросит у большой языковой модели определение намерения. Затем на основе параметров, возвращенных большой языковой моделью, вызовет соответствующий инструмент. В данном случае вызван инструмент `write_note` из flomo-mcp, который записывает заметку в flomo, как показано на рис. 3.14.

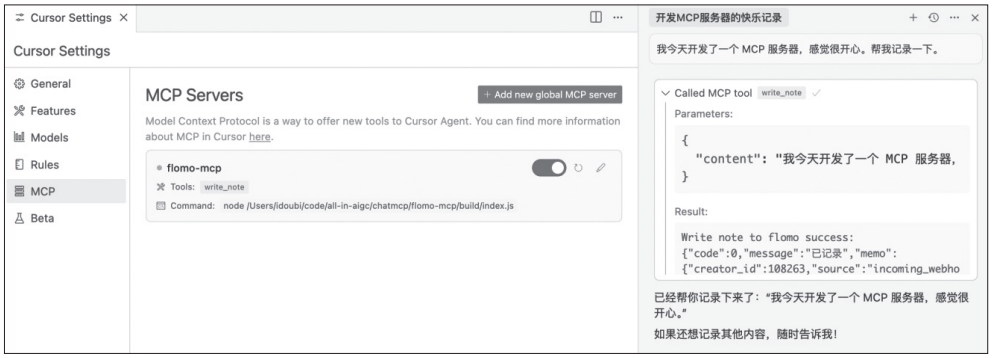


Рис. 3.14. Вызов flomo-mcp в Cursor

3.2.8. Публикация MCP-сервера flomo

Успешное тестирование в Cursor подтвердило, что разработанный нами flomo-mcp работает нормально, и теперь его можно опубликовать для доступа другим пользователям.

Книги для программистов: https://t.me/free_books_for_it

1. Публикация на платформе GitHub

Можно опубликовать код недавно разработанного flomo-mcp на GitHub, сделав его открытым исходным кодом для всех пользователей.

Сначала обновите файл *README.md*, добавив примеры конфигурации flomo-mcp и инструкции по использованию. Создайте репозиторий кода на GitHub и отправьте в него свой код.

```
cd flomo-mcp
git init
git remote add origin git@github.com:chatmcp/flomo-mcp.git
git add .
git commit -m "first version"
git push origin main
```

Репозиторий кода flomo-mcp на GitHub показан на рис. 3.15.

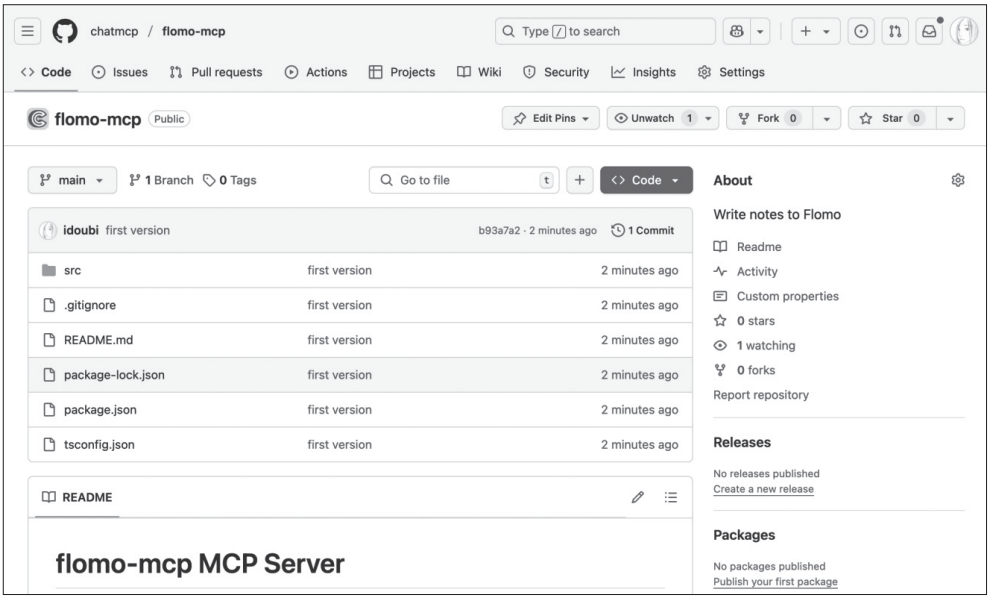


Рис. 3.15. Репозиторий кода flomo-mcp

2. Публикация на платформе npm

Чтобы другие пользователи могли использовать flomo-mcp после простой настройки, не загружая исходный код, можно опубликовать flomo-mcp в качестве пакета npm в магазине приложений **npm**.

Сначала измените файл *package.json* проекта flomo-mcp, чтобы опубликовать его как пакет npm открытого типа в заданной группе, например в @chatmcp (это созданная мной группа; вы можете создать свою собственную группу в панели управления npm).

Книги для программистов: https://t.me/free_books_for_it

```
{
  "name": "@chatmcp/flomo-mcp",
  "version": "0.0.1",
  "description": "write notes to flomo.",
  "private": false,
  "type": "module",
  "homepage": "https://github.com/chatmcp/flomo-mcp",
  "bin": {
    "flomo-mcp": "./build/index.js"
  },
  "files": ["build"],
  "scripts": {
   "build": "tsc && node -e \"require('fs').chmodSync('build/index.js', '755')\"",
    "prepare": "npm run build",
    "watch": "tsc --watch",
    "inspector": "npx @modelcontextprotocol/inspector build/index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "0.6.0"
  },
  "devDependencies": {
    "@types/node": "^20.11.24",
    "typescript": "^5.3.3"
  }
}
```

Затем выполните команды для публикации пакета npm:

```
cd flomo-mcp
npm install
npm run build
npm login # Откройте страницу входа и авторизуйтесь на платформе npm в браузере
npm publish --access public # Опубликуйте пакет в открытом доступе на npm
```

После успешной публикации откройте адрес доступа к пакету npm, где вы увидите, что @chatmcp/flomo-mcp уже доступен, как показано на рис. 3.16.

Измените настройки Cursor, запустите @chatmcp/flomo-mcp с помощью команды npx и передайте настройки переменных среды.

```
{
  “mcpServers”: {
    “flomo-mcp”: {
      “command”: “npx”,
      “args”: [“-y”, “@chatmcp/flomo-mcp”],
      “env”: {
        “FLOMO_API_URL”: “https://flomoapp.com/iwh/xxx/xxx/”
      }
    }
  }
}
```

Продолжая использовать flomo-mcp в Cursor, вы увидите, что этот MCP-сервер успешно работает в соответствии с новыми настройками. Теперь можно быстро записывать в flomo проблемы, возникающие в повседневной разработке, как показано на рис. 3.17.

Книги для программистов: https://t.me/free_books_for_it

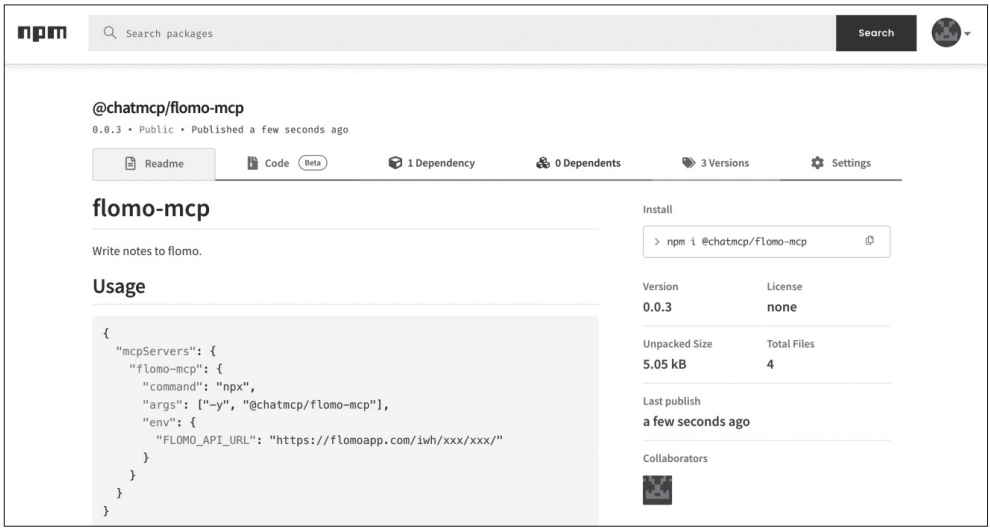


Рис. 3.16 Страница с информацией о пакете npm flomo-mcp

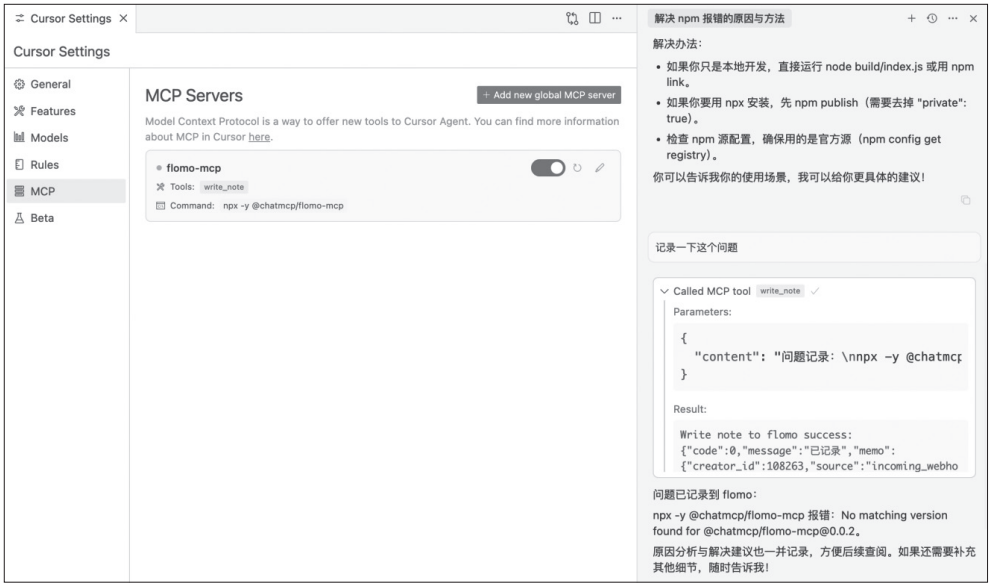


Рис. 3.17. Запись проблем разработки в Cursor



Cursor, как клиент большой языковой модели, может автоматически сопоставлять контекст, вызывать большую языковую модель для систематизации содержания заметок и осуществлять запись заметок через flomo-mcp. С помощью этого MCP-сервера можно задавать вопросы и одновременно записывать информацию в процессе программирования, что повышает общую эффективность работы.

Книги для программистов: https://t.me/free_books_for_it

В результате мы можем искать и просматривать в flomo некоторые из ежедневно записываемых вопросов, как показано на рис. 3.18.

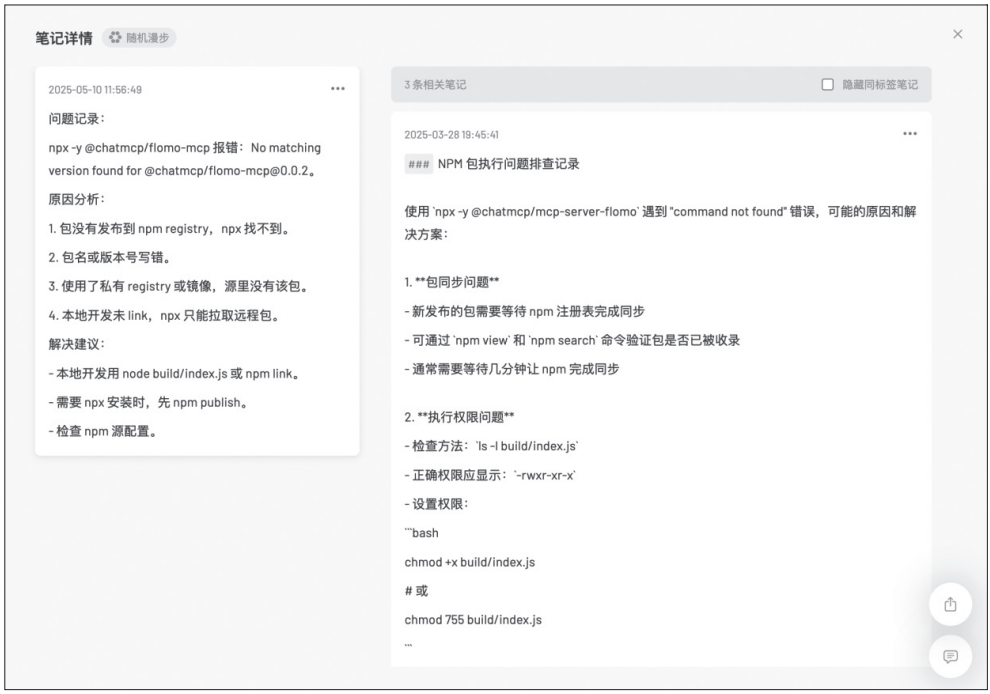


Рис. 3.18. Просмотр заметок в flomo



Из этого примера видно, что подключение MCP-сервера к клиенту большой языковой модели позволяет легко интегрироваться с внешними инструментами, такими как flomo, а также своевременно записывать и структурировать вопросы, размышления и результаты повседневной работы. Таким образом, возможности клиента большой языковой модели в области накопления знаний и анализа информации расширяются, а эффективность работы отдельных сотрудников или всей команды значительно повышается.

3. Публикация в стороннем магазине MCP-приложений

Мы можем опубликовать flomo-mcp в стороннем магазине MCP-приложений, например в MCP.so. Перейдите на страницу **Submit** (отправка), заполните информацию о MCP-сервере и отправьте flomo-mcp в магазин MCP.so, как показано на рис. 3.19.

Книги для программистов: https://t.me/free_books_for_it

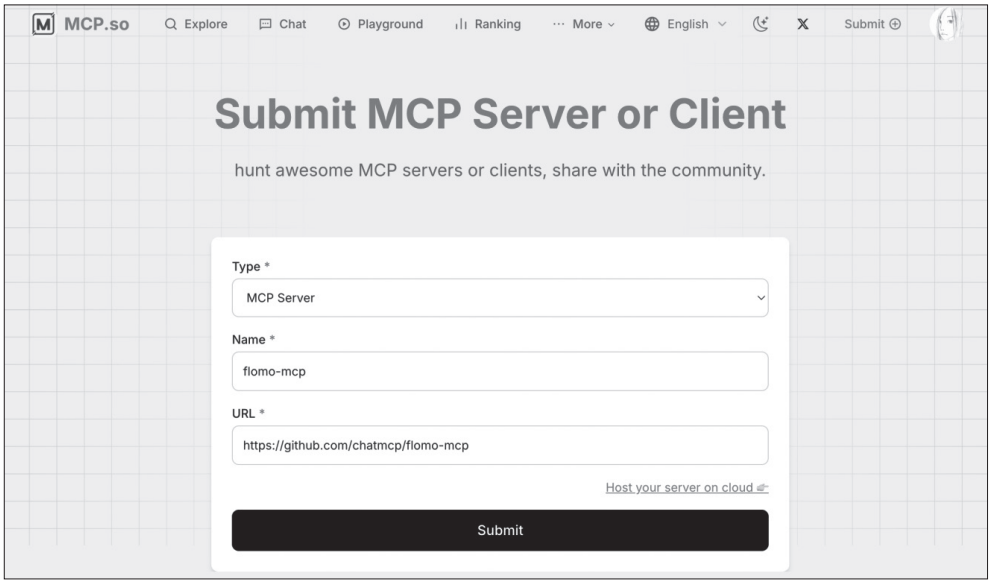


Рис. 3.19. Публикация flomo-mcp в магазине приложений MCP.so

После успешной отправки ваш MCP-сервер будет отображаться в магазине приложений MCP.so. На странице сведений о приложении пользователи могут просматривать информацию о конфигурации этого MCP-сервера и инструментах его внутренней реализации, как показано на рис. 3.20.

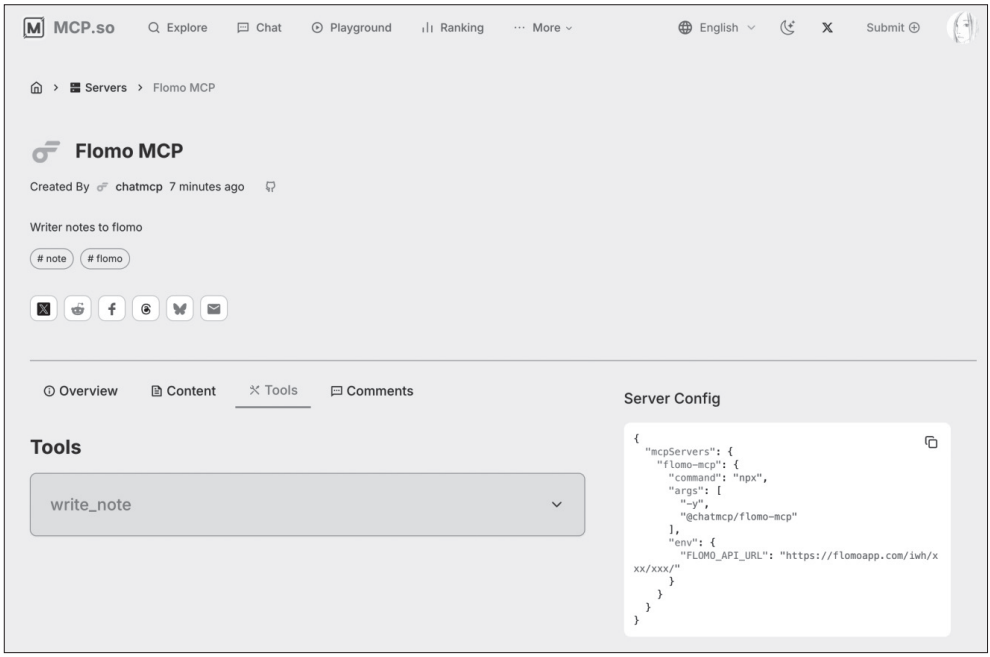


Рис. 3.20. Просмотр MCP-сервера flomo в магазине приложений

Книги для программистов: https://t.me/free_books_for_it

3.2.9. Подведение итогов примера 1

В этом разделе был разработан MCP-сервер flomo-mcp, способный интегрироваться с API записи flomo. Он предоставляет возможность эффективного ведения заметок в клиенте большой языковой модели.

В процессе создания MCP-сервера были подробно изучены этапы разработки, отладки и публикации, а также продемонстрированы процедуры определения и реализации инструментов в этом сервере.

Этот пример показывает, что можно разработать больше MCP-серверов, которые будут взаимодействовать с различными данными или сервисами через API.

3.3. Пример 2. Разработка MCP-сервера HeyBeauty для быстрой виртуальной примерки одежды

В предыдущем примере разработки MCP-сервер был в основном предназначен для приема текстового ввода и возврата текстового контента клиенту для отображения с помощью вызова инструмента.

В этом разделе нам предстоит разработать MCP-сервер для обработки изображений и на примере виртуальной примерки показать процесс приема отправленных клиентом изображений, их обработки и возврата результатов обратно клиенту. Кроме того, в предыдущем примере в основном рассматривалось определение и реализация инструментальных возможностей на MCP-сервере. В этом примере будет продемонстрирована полная реализация трех основных возможностей MCP-сервера: **инструментов** (Tools), **ресурсов** (Resources) и **подсказок** (Prompts). Большинство MCP-серверов, представленных на рынке в настоящее время, в основном поддерживают инструментальные возможности. Они обычно управляются большими языковыми моделями и ориентированы на извлечение динамического контента или интеграцию с внешними сервисами. Ресурсы и подсказки, напротив, чаще выбираются оператором или пользователем и обычно используются для предоставления статического контента. Реализация всех трех возможностей в рамках одного примера поможет вам понять их различия в реализации и взаимодействии, чтобы в дальнейшем вы смогли создать более эффективный MCP-сервер.

3.3.1. Цели разработки

В этом разделе планируется разработка MCP-сервера под названием heybeauty-mcp, который будет подключен к API платформы виртуальной примерки HeyBeauty. Пользователи смогут испытать функцию виртуальной примерки в диалоговом режиме на любом клиенте, поддерживающем MCP-сервер. Это позволит снизить порог доступа к примерке и откроет новый канал взаимодействия для персонализированного обслуживания в сфере моды с помощью ИИ.

> В апреле 2024 года я разработал продукт виртуальной примерки под названием **HeyBeauty** (*heybeauty.ai*), основная функция которого заключается в загрузке пользователем на веб-сайт своей фотографии и изображения одежды, которую он хочет примерить. С помощью собственной модели примерки HeyBeauty генерирует изображение результата примерки, как показано на рис. 3.21.

Книги для программистов: https://t.me/free_books_for_it

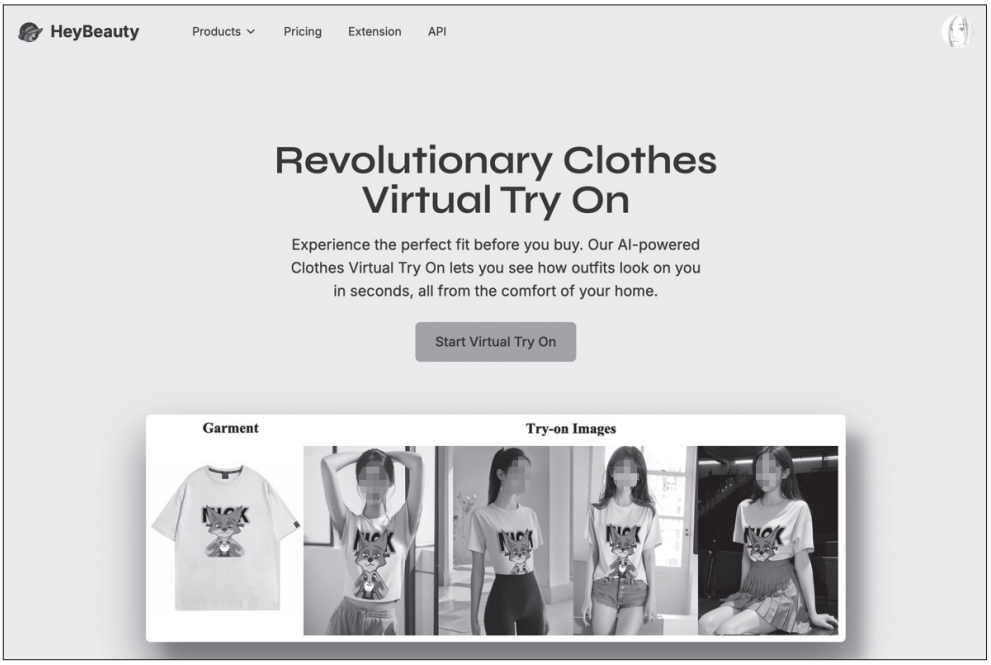


Рис. 3.21. Официальный сайт продукта HeyBeauty

Процесс примерки одежды пользователем выглядит следующим образом.

1. Пользователь открывает веб-страницу и переходит в консоль примерки HeyBeauty, как показано на рис. 3.22.
2. Пользователь загружает свою **фотографию** (User Image) и **изображение интересующей одежды** (Cloth Image).
3. Пользователь **отправляет** (Submit) запрос на виртуальную примерку, HeyBeauty начинает распознавать изображения и запускает программу виртуальной примерки.
4. Пользователь просматривает **результаты примерки** (Query Task).

Чтобы как можно больше пользователей смогли воспользоваться функцией примерки HeyBeauty, мы открыли API, позволяющий сторонним приложениям подключаться к нему и помогать потребителям в проведении виртуальной примерки с помощью различных пользовательских устройств.

За последний год HeyBeauty подключился к таким приложениям, как мини-программы WeChat, мобильные приложения, плагины Shopify, аппаратное обеспечение для зеркал-примерочных и т. д., предоставив пользователям возможность виртуальной примерки одежды с помощью различных сценариев.

Книги для программистов: https://t.me/free_books_for_it

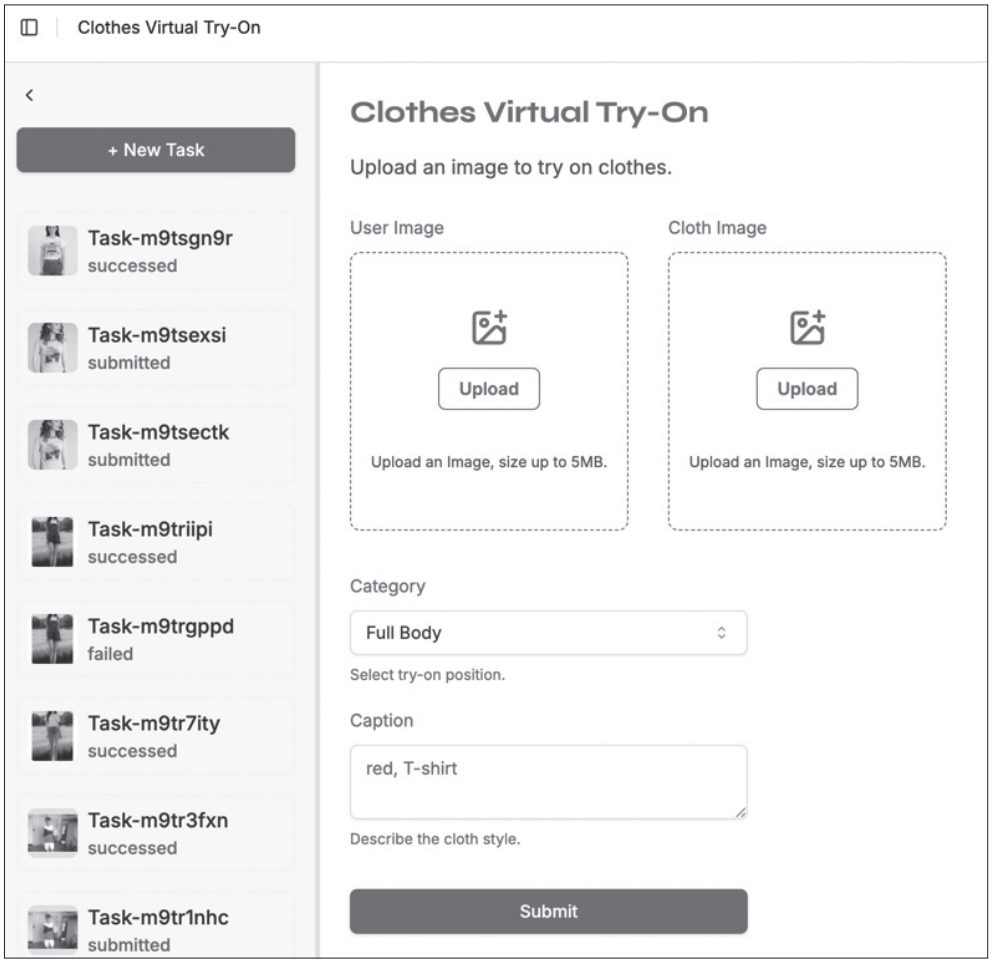


Рис. 3.22. Консоль виртуальной примерки HeyBeauty

3.3.2. Подготовительные работы и концепция разработки

Перед началом разработки MCP-сервера HeyBeauty давайте вкратце ознакомимся с открытыми интерфейсами HeyBeauty. Как видно из документации по интерфейсам HeyBeauty (*doc.heybeauty.ai*), платформа предоставляет **интерфейс виртуальной примерки** (Try-On API), который используется следующим образом.

1. **Оформление заявки на получение API-ключа** на платформе HeyBeauty.
2. **Отправка запроса на создание задачи примерки** (Create Try-On Task).
3. **Загрузка изображений**, необходимых для задачи примерки (Upload Images).
4. **Запрос результатов примерки** (Query Try-On Result).

Документация по интерфейсу примерки HeyBeauty представлена на рис. 3.23.

Книги для программистов: https://t.me/free_books_for_it

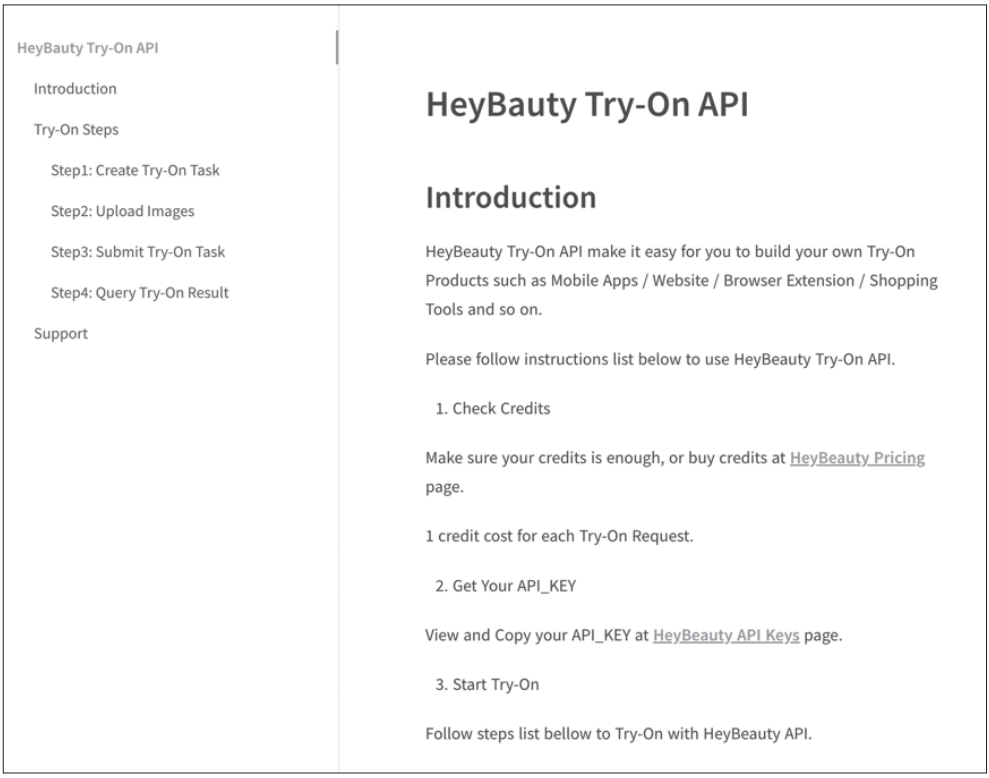


Рис. 3.23. Документация по интерфейсу виртуальной примерки HeyBeauty

Процесс подачи заявки на получение API-ключа подробно описывать не будем, пользователи могут самостоятельно подать заявку на официальном сайте HeyBeauty.

В соответствии с процессом виртуальной примерки, описанным в документации по интерфейсу HeyBeauty, давайте для начала рассмотрим структуру разрабатываемого MCP-сервера HeyBeauty. Основной процесс взаимодействия со стороны пользователя выглядит следующим образом.

1. При запуске клиент большой языковой модели отправляет запрос на MCP-сервер HeyBeauty, получает пакет изображений одежды и предлагает их пользователю на выбор.
2. Пользователь загружает свою фотографию и выбирает одно из изображений одежды для примерки.
3. Клиент большой языковой модели загружает фотографию пользователя и получает URL-адрес соответствующего изображения (`user_image_url`).
4. Клиент большой языковой модели отправляет ей запрос о выборе инструмента для виртуальной примерки одежды.
5. Получив ответ с инструментом и параметрами, клиент большой языковой модели отправляет запрос на MCP-сервер HeyBeauty с двумя адресами изображений.

Книги для программистов: https://t.me/free_books_for_it

6. MCP-сервер HeyBeauty отправляет запрос на интерфейс HeyBeauty и передает задание на виртуальную примерку.
7. Клиент большой языковой модели запрашивает результаты виртуальной примерки у MCP-сервера HeyBeauty.

На этапе 2 пользователь может не выбирать изображение одежды, возвращаемое MCP-сервером, а самостоятельно отправить собственное изображение одежды. В этом случае на этапе 3 клиент большой языковой модели должен одновременно загрузить фотографию пользователя и изображение одежды, чтобы получить адреса двух изображений (`user_image_url` и `cloth_image_url`).

На протяжении всего процесса взаимодействия клиент большой языковой модели должен реализовать логику преобразования изображений. Другими словами, загруженные пользователем изображения не могут быть отправлены в виде исходных файлов (двоичного содержимого) непосредственно в большую языковую модель или на MCP-сервер, они должны сначала быть загружены на сторонний хостинг изображений для получения URL-адреса изображения.



Несмотря на то что в соответствии с протоколом MCP клиент может передавать на MCP-сервер содержимое изображения в кодировке base64, а также принимать от MCP-сервера данные изображения в кодировке base64, на практике возникают следующие проблемы. Выбор MCP-сервера, на который клиент отправляет запрос, а также передаваемые параметры определяются большой языковой моделью. Это означает, что сначала изображение необходимо передать большой языковой модели, а затем она сообщает клиенту, какой именно MCP-сервер следует вызвать и какое именно изображение необходимо запросить. Если в этом процессе все изображения передаются в формате base64, это может привести к значительному расходу токенов и дополнительным затратам, а также к сбоям при разборе изображений или ошибкам при передаче из-за слишком большого объема данных или некорректного кодирования. По этой причине рекомендуется загружать изображения на файловый хостинг и передавать только URL-адрес изображения.

Исходя из этой логики, при разработке MCP-сервера HeyBeauty мы больше не будем заниматься преобразованием форматов изображений внутри сервера. Во всех ситуациях, связанных с обработкой изображений, мы будем использовать URL-адреса изображений, а не их содержание в кодировке base64.

Далее перейдем к реализации MCP-сервера HeyBeauty.

3.3.3. Создание проекта MCP-сервера HeyBeauty

1. Создание проекта

Как и в предыдущем примере, сначала создадим проект MCP-сервера с помощью официального инструмента командной строки MCP и назовем его ***heybeauty-mcp***.

```
npx @modelcontextprotocol/create-server heybeauty-mcp
```

Книги для программистов: https://t.me/free_books_for_it

В корневом каталоге проекта установим зависимости и запустим службу мониторинга:

```
npm install
npm run watch
```

Таким образом, при изменении кода проекта программа мониторинга автоматически выполнит перекомпиляцию и повторный запуск, что упростит отладку и разработку в режиме реального времени.

Процесс создания MCP-сервера HeyBeauty и запуска службы мониторинга показан на рис. 3.24.

```
~/code/all-in-aigc/chatmcp (2m 11.38s)
npx @modelcontextprotocol/create-server heybeauty-mcp

? What is the name of your MCP server? heybeauty-mcp
? What is the description of your server? HeyBeauty Virtual TryOn
? Would you like to install this server for Claude.app? No
✔ MCP server created successfully!

Next steps:
  cd heybeauty-mcp
  npm install
  npm run build  # or: npm run watch
  npm link       # optional, to make available globally


~/code/all-in-aigc/chatmcp (0.036s)
cd heybeauty-mcp

~/code/all-in-aigc/chatmcp/heybeauty-mcp (6.546s)
npm install

> heybeauty-mcp@0.1.0 prepare
> npm run build


> heybeauty-mcp@0.1.0 build
> tsc && node -e "require('fs').chmodSync('build/index.js', '755')"


added 17 packages, and audited 18 packages in 6s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities

~/code/all-in-aigc/chatmcp/heybeauty-mcp
npm run watch
[11:25:34 PM] Starting compilation in watch mode...

[11:25:35 PM] Found 0 errors. Watching for file changes.
```

Рис. 3.24. Создание MCP-сервера и запуск службы мониторинга из командной строки

Книги для программистов: https://t.me/free_books_for_it

2. Запуск MCP Inspector

Для быстрого запуска MCP Inspector выполните следующую команду в корневом каталоге MCP-сервера HeyBeauty.

```
npx @modelcontextprotocol/inspector build/index.js
```

Перейдите по локальному адресу, выведенному после успешного выполнения приведенной выше команды, откройте MCP Inspector, а затем подключитесь к MCP-серверу HeyBeauty, как показано на рис. 3.25.

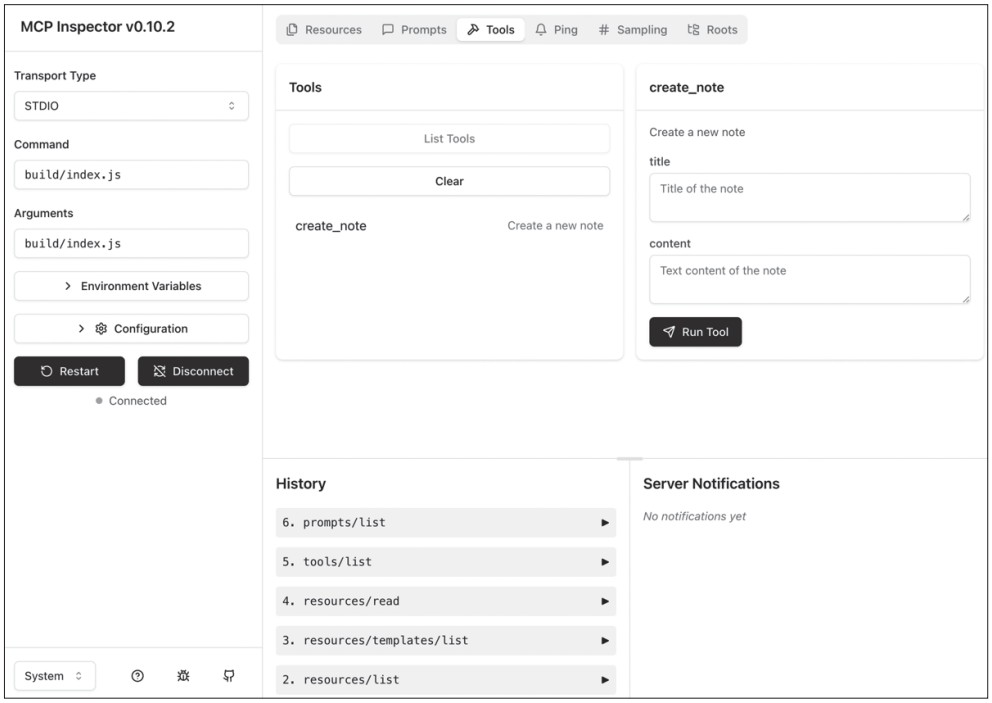


Рис. 3.25. Отладка сервера HeyBeauty MCP в MCP Inspector

Как видно на рис. 3.25, MCP-сервер HeyBeauty, созданный с помощью инструмента командной строки, по умолчанию поддерживает три основных типа функций: **Resources** (ресурсы), **Prompts** (подсказки) и **Tools** (инструменты).



MCP Inspector – это рабочая среда для модульной, систематизированной и визуальной отладки MCP-сервера, ориентированная на три основных функциональных модуля: ресурсы, подсказки и инструменты. Она поддерживает локальную отладку, быстрое подключение к MCP-серверу, а также визуальную настройку переменных среды и параметров запуска, что значительно повышает эффективность отладки.

Далее мы перейдем к реализации функций, связанных с виртуальной примеркой, внутри HeyBeauty MCP-сервера.

Книги для программистов: https://t.me/free_books_for_it

3.3.4. Реализация бизнес-логики MCP-сервера HeyBeauty

1. Создание клиентского класса для взаимодействия с API HeyBeauty

Реализация клиентского класса для вызова API HeyBeauty. Прежде чем приступить к разработке MCP-сервера HeyBeauty, необходимо разработать специальный клиентский класс для взаимодействия с API HeyBeauty. Этот класс инкапсулирует детали взаимодействия с API HeyBeauty и предоставляет лаконичный и удобный интерфейс для других функциональных модулей MCP-сервера. Мы назовем его `HeyBeautyClient` и создадим для его реализации новый файл *src/heybeauty.ts*. Его реализация представлена ниже.

```
/**
 * Класс клиента HeyBeauty API
 * Используется для запросов к интерфейсам, предоставляемым heybeauty.ai
 */
export class HeyBeautyClient {
  private readonly apiBaseUrl: string = "https://heybeauty.ai/api";
  private readonly apiKey: string;

  /**
   * Конструктор
   * @param apiKey - API-ключ, полученный на heybeauty.ai
   * @param apiBaseUrl - базовый адрес API (опционально)
   */
  constructor({ apiKey, apiBaseUrl }: { apiKey: string; apiBaseUrl?: string }) {
    this.apiKey = apiKey;
    this.apiBaseUrl = apiBaseUrl || this.apiBaseUrl;
  }

  /**
   * Получение списка одежды, доступной для примерки
   * @returns Список одежды */
   */
  async getClothes() {
    try {
      const req = {
        page: 1,
        limit: 10,
    };
    const resp = await fetch(`${this.apiBaseUrl}/get-clothes`, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${this.apiKey}`,
      },
      body: JSON.stringify(req),
    });

    if (!resp.ok) {
      throw new Error("request failed with status " + resp.status);
    }
```

Книги для программистов: https://t.me/free_books_for_it

```
      const { code, message, data } = await resp.json();
      if (code !== 0) {
        throw new Error(message);
      }

      return data;
    } catch (error) {
      throw error;
    }
  }
}
```

Отладка клиентского класса, вызов метода `getClothes`. После создания класса `HeyBeautyClient` необходимо провести его отладку, чтобы убедиться, что он корректно взаимодействует с API HeyBeauty и возвращает необходимые данные. Для этого можно создать экземпляр клиента во встроенной функции стартового файла *src/index.ts* и вызвать его методы для простого тестирования. Конкретная реализация представлена ниже.

```
/**
 * Запуск сервера с использованием stdio
 * С помощью стандартного ввода/вывода
 */
async function main() {
  const apiKey = "xxxxxx";
  const client = new HeyBeautyClient({ apiKey });
  const clothes = await client.getClothes();
  console.log(clothes);

  const transport = new StdioServerTransport();
  await server.connect(transport);
}

main().catch((error) => {
  console.error("Ошибка сервера:", error);
  process.exit(1);
});
```

Откройте терминал и выполните команду `node build/index.js`. Вы увидите, что функция `main` вызывает клиент для получения списка одежды HeyBeauty. Нас интересуют следующие 4 поля данных:

- `title`: название одежды;
- `description`: описание одежды;
- `cloth_id`: идентификатор одежды;
- `cloth_img_url`: URL-адрес изображения одежды.



При разработке класса клиента `HeyBeautyClient` в основном учитывались следующие ключевые моменты:

- **принцип единства ответственности**: класс предназначен исключительно для инкапсуляции логики запросов к API HeyBeauty;

Книги для программистов: https://t.me/free_books_for_it

- **широкие возможности настройки**: поддержка передачи пользовательского адреса API с помощью конструктора;
- **механизм обработки ошибок**: проверка и вывод ошибок как на уровне HTTP, так и на уровне бизнес-логики;
- **высокая степень повторного использования**: возможность совместного использования вызовов в нескольких модулях MCP-инструмента;
- **простота расширения**: в дальнейшем можно легко добавлять новые методы интерфейса для решения бизнес-задач.

2. Реализация функций работы с ресурсами (Resources)

Реализация логики получения списка ресурсов. Измените логику получения списка ресурсов в файле *src/index.ts*: с помощью `apiKey` создайте объект `HeyBeautyClient`, а список одежды, полученный при вызове `getClothes`, верните в качестве списка ресурсов MCP-сервера.

```
/**
 * Возврат списка одежды в виде ресурсов
 * Каждая вещь представляет собой отдельный ресурс
 * - URI cloth:// используется в качестве уникального идентификатора ресурса
 * - Ресурс имеет тип "чистый текст"
 * - Ресурс содержит поля name и description
 */
server.setRequestHandler(ListResourcesRequestSchema, async () => {
  try {
    const apiKey = "xxxxxx";
    const client = new HeyBeautyClient({ apiKey });

    const clothes = await client.getClothes();

    return {
      resources: clothes.map((clothe: any) => ({
        uri: `cloth:///${clothe.cloth_id}`,
        mimeType: "text/plain",
        name: clothe.title,
        description: `${clothe.description}`,
      })),
    };
  } catch (error: any) {
    throw new Error("ошибка получения ресурсов: " + error.message);
  }
});
```

В MCP Inspector вызовите List Resources для получения списка возвращаемых сервером ресурсов, как показано на рис. 3.26.

Книги для программистов: https://t.me/free_books_for_it

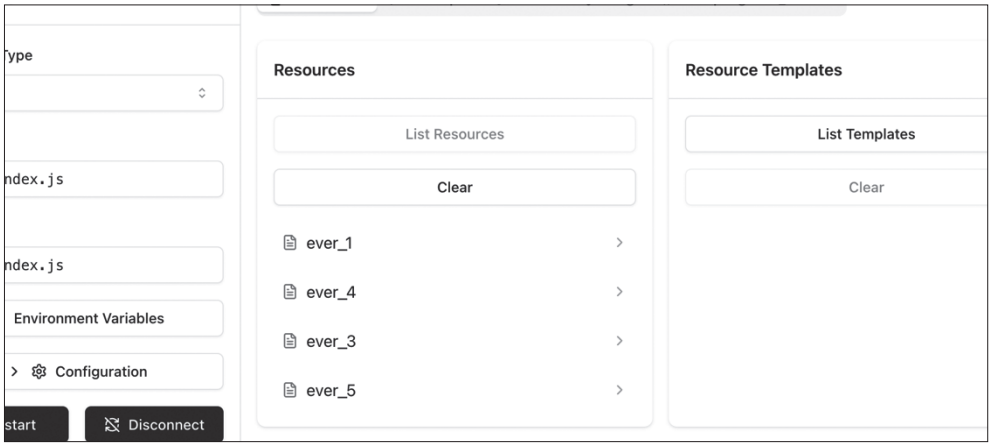


Рис. 3.26. Вызов List Resources для получения списка ресурсов

Реализация логики чтения ресурсов. Измените логику чтения содержимого ресурсов в *src/index.ts*, передайте `apiKey` для создания `HeyBeautyClient`, вызовите `getClothes` для получения списка одежды, а затем на основе запрошенного ID одежды получите адрес изображения соответствующей одежды.

```
/**
 * Чтение содержимого ресурсов
 * Получение URI cloth:// и возврат ресурса (адреса изображения одежды)
 *-в виде простого текста
 */
server.setRequestHandler(ReadResourceRequestSchema, async (request) => {
  try {
    const url = new URL(request.params.uri);
    const id = url.pathname.replace(/^\//, "");

    const apiKey = "xxxxxx";
    const client = new HeyBeautyClient({ apiKey });

    const clothes = await client.getClothes();

    const cloth = clothes.find((clothe: any) => clothe.cloth_id == id);

    if (!cloth) {
      throw new Error(`Cloth ${id} не обнаружено`);
   }

  return {
    contents: [
      {
        uri: request.params.uri,
        mimeType: "text/plain",
          text: cloth.cloth_img_url,
        },
      ],
    };
```

Книги для программистов: https://t.me/free_books_for_it

```
  } catch (error: any) {
    throw new Error("ошибка чтения ресурсов: " + error.message);
  }
});
```

Выберите в MCP Inspector один из ресурсов, возвращенных List Resources, чтобы инициировать запрос на чтение ресурса. Вы увидите возвращаемый сервером контент ресурса, как показано на рис. 3.27.

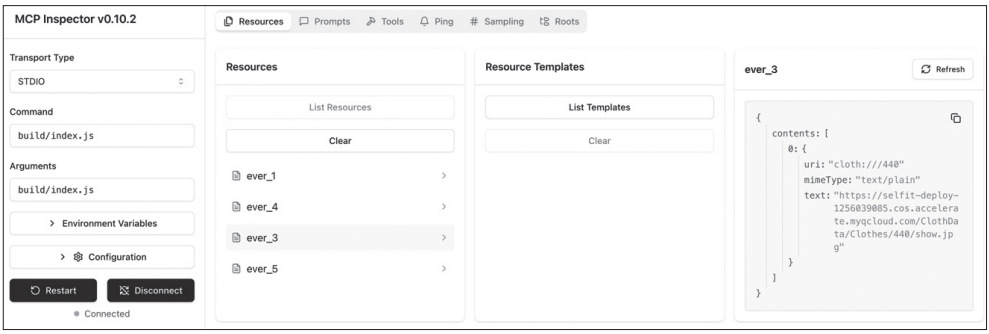


Рис. 3.27. Чтение ресурса в MCP Inspector

Тестирование ресурса в клиенте большой языковой модели. Ранее мы уже убедились, что получение списка ресурсов и чтение ресурса через MCP Inspector проходят без проблем. Теперь продолжим тестирование в клиенте большой языковой модели и посмотрим, как использовать предоставленные сервером ресурсы в этом клиенте.

Настройте MCP-сервер HeyBeauty в Claude следующим образом.

```
{
  "mcpServers": {
    "heybeauty-mcp": {
      "command": "node",
      "args": [
        "/Users/idoubi/code/all-in-aigc/chatmcp/heybeauty-mcp/build/index.js"
      ],
      "env": {
        "HEYBEAUTY_API_KEY": "xxxxxx"
      }
    }
  }
}
```

Запустите следующую команду для просмотра выходного лога Claude.

```
tail -f ~/Library/Logs/Claude/mcp*.log
```

Запустите Claude. Как показано на рис. 3.28, в логе отображается следующее: клиент отправил запрос `resources/list` на MCP-сервер HeyBeauty и получил возвращенный им список ресурсов.

Книги для программистов: https://t.me/free_books_for_it

```
~/code/all-in-aigc/chatmcp/heybeauty-mcp
tail -f ~/Library/Logs/Claude/mcp*.log | grep resource
```

```
2025-04-26T02:32:16.251Z [info] [heybeauty-mcp] Message from client: {"method":
resources/list","params":{},"jsonrpc":"2.0","id":12}
2025-04-26T02:32:17.394Z [info] [heybeauty-mcp] Message from server: {"jsonrpc":
"2.0","id":12,"result":{"resources":[{"uri":"cloth:///438","mimeType":"text/plai
n","name":"ever_1","description":"green dress"},{"uri":"cloth:///439","mimeType"
:"text/plain","name":"ever_4","description":"red dress"},{"uri":"cloth:///440","
mimeType":"text/plain","name":"ever_3","description":"pink dress"},{"uri":"cloth
:///442","mimeType":"text/plain","name":"ever_5","description":"green dress"}]}}
2025-04-26T02:32:17.530Z [info] [heybeauty-mcp] Message from server: {"jsonrpc":
"2.0","id":11,"result":{"resources":[{"uri":"cloth:///438","mimeType":"text/plai
n","name":"ever_1","description":"green dress"},{"uri":"cloth:///439","mimeType"
:"text/plain","name":"ever_4","description":"red dress"},{"uri":"cloth:///440","
mimeType":"text/plain","name":"ever_3","description":"pink dress"},{"uri":"cloth
:///442","mimeType":"text/plain","name":"ever_5","description":"green dress"}]}}
```

Рис. 3.28. Лог чтения списка ресурсов с MCP-сервера HeyBeauty в Claude

Если пользователь нажмет на иконку в крайнем правом углу поля ввода в Claude, он увидит список ресурсов, возвращенный MCP-сервером HeyBeauty (список одежды, доступной для примерки), как показано на рис. 3.29.

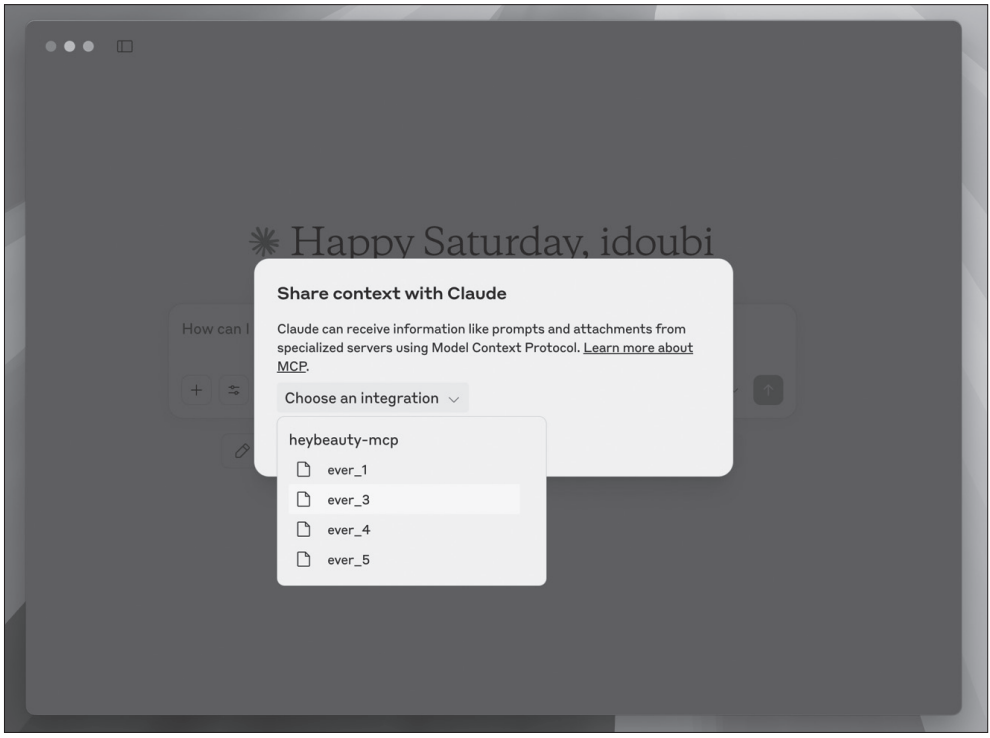


Рис. 3.29. Список ресурсов, возвращенный MCP-сервером HeyBeauty (одежда, доступная для примерки)

Пользователь выбирает один из ресурсов, например *ever_3*. В логе, выводимом Claude, видно, что клиент отправил на MCP-сервер HeyBeauty запрос `re-`

Книги для программистов: https://t.me/free_books_for_it

sources/read с параметром ID ресурса (cloth_id), а MCP-сервер HeyBeauty вернул содержимое ресурса (адрес изображения одежды, доступной для примерки), как показано на рис. 3.30.

```
~/code/all-in-aigc/chatmcp/heybeauty-mcp
tail -f ~/Library/Logs/Claude/mcp*.log | grep cloth

2025-04-26T02:40:08.181Z [info] [heybeauty-mcp] Message from client: {"method":"
resources/read","params":{"uri":"cloth:///440"},"jsonrpc":"2.0","id":76}
2025-04-26T02:40:09.104Z [heybeauty-mcp] [info] Message from server: {"jsonrpc":
"2.0","id":76,"result":{"contents":[{"uri":"cloth:///440","mimeType":"text/plain
","text":"https://selfit-deploy-1256039085.cos.accelerate.myqcloud.com/ClothData
/Clothes/440/show.jpg"}]}}
```

Рис. 3.30. Содержание ресурса, возвращенное MCP-сервером HeyBeauty

Claude отображает выбранный пользователем ресурс под полем ввода; пользователь может нажать на ресурс для просмотра его содержания. В этом сценарии содержанием ресурса является адрес изображения одежды, возвращенный MCP-сервером HeyBeauty, как показано на рис. 3.31.



Рис. 3.31. Адрес изображения одежды, возвращенный MCP-сервером HeyBeauty

Из приведенного выше примера становится понятен процесс получения клиентом большой языковой модели списка ресурсов MCP, а также процесс выбора и просмотра ресурсов пользователем.

Далее продолжим реализацию бизнес-логики MCP-сервера HeyBeauty.

Книги для программистов: https://t.me/free_books_for_it

3. Доработка клиентского класса для взаимодействия с API HeyBeauty

Продолжим совершенствовать логику в src/heybeauty.ts, добавив в класс `HeyBeautyClient` два метода: `submitTask` и `queryTask`, которые вызывают API HeyBeauty для отправки задания виртуальной примерки и запроса результатов примерки соответственно.

`submitTask` – отправка задания виртуальной примерки. Этот метод вызывает асинхронный интерфейс HeyBeauty виртуальной примерки */api/mcp-vton* для отправки задания виртуальной примерки. Код его реализации приведен ниже.

```
async function submitTask({
  user_img_url,
  cloth_img_url,
  cloth_id,
  cloth_description,
}: {
  user_img_url: string;
  cloth_img_url: string;
  cloth_id?: string;
  cloth_description?: string;
}) {
  try {
    if (!user_img_url || !cloth_img_url) {
      throw new Error("необходимы user_img_url и cloth_img_url");
    }

    const req = {
      user_img_url,
      cloth_img_url,
      cloth_id,
      caption: cloth_description || "",
      category: "1",
      is_sync: "0",
    };
    const resp = await fetch(`${this.apiBaseUrl}/mcp-vton`, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${this.apiKey}`,
      },
      body: JSON.stringify(req),
    });

    if (!resp.ok) {
      throw new Error("ошибка запроса со статусом " + resp.status);
    }

    const { code, message, data } = await resp.json();
    if (code !== 0) {
      throw new Error(message);
  }

    return {
```

Книги для программистов: https://t.me/free_books_for_it

```
      task_id: data.uuid,
      created_at: data.created_at,
      updated_at: data.updated_at,
      status: data.status,
      tryon_img_url: data.tryon_img_url || "",
    };
  } catch (error) {
    throw error;
  }
}
```

Этот метод включает следующие параметры запроса.

- **Обязательные поля**: адрес фотографии пользователя и адрес изображения одежды.
- **Опциональные поля**: ID одежды и описание одежды.
- **Прочие параметры**: параметр category имеет фиксированное значение 1; is_sync устанавливается в 0, что означает, что это асинхронная задача.

В ответе интерфейса возвращается `task_id` в качестве идентификатора задачи. В дальнейшем его можно использовать для запроса результатов примерки.

`queryTask` – запрос результатов примерки. Этот метод использует `task_id`, полученный при отправке задачи примерки, в качестве параметра запроса и вызывает */api/get-task-info* в HeyBeauty для запроса результатов задачи примерки. Логика его реализации приведена ниже.

```
async function queryTask({ task_id }: { task_id: string }) {
  try {
    const uri = `${this.apiBaseUrl}/get-task-info`;
    const req = {
      task_uuid: task_id,
    };

    const resp = await fetch(uri, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${this.apiKey}`,
    },
    body: JSON.stringify(req),
  });

  if (!resp.ok) {
    throw new Error("ошибка запроса со статусом: " + resp.status);
  }

  const { code, message, data } = await resp.json();
    if (code !== 0) {
      throw new Error(message);
    }

    return {
      task_id: task_id,
```

Книги для программистов: https://t.me/free_books_for_it

```
        created_at: data.created_at,
        updated_at: data.updated_at,
        status: data.status,
        tryon_img_url: data.tryon_img_url || "",
      };
    } catch (err) {
      throw err;
    }
  }
```

Интерфейс возвращает статус примерки `status`, значения которого могут быть следующими:

- `successed`: примерка прошла успешно;
- `failed`: примерка завершилась неудачно;
- `processing`: задача находится в процессе обработки.

Если примерка прошла успешно, возвращаемое интерфейсом поле `tryon_img_url` будет содержать адрес изображения, представляющего собой результат примерки пользователя.

4. Реализация функций инструментов (Tools)

Определение списка инструментов. В файле *src/index.ts* мы определяем два инструмента: `submit_tryon_task` и `query_tryon_task`. Они соответствуют методам `submitTask` и `queryTask`, реализованным в `HeyBeautyClient`. Логика их реализации приведена ниже.

```
/**
 * Перечисляет список инструментов, реализованных на сервере
 * Предоставляет инструмент submit_tryon_task для отправки задания на примерку
 * Предоставляет инструмент query_tryon_task для запроса результатов примерки
 */
server.setRequestHandler(ListToolsRequestSchema, async () => {
  return {
    tools: [
      {
        name: "submit_tryon_task",
        description:
          "Отправить задачу примерки (tryon) с URL изображения пользователя " +
          "и URL изображения одежды",
        inputSchema: {
          type: "object",
          properties: {
            user_img_url: {
              type: "string",
              description: "URL изображения пользователя " +
                           "(должен быть URL изображения)",
            },
            cloth_img_url: {
              type: "string",
              description:
              "URL изображения одежды, должен быть URL изображения, введенного "+
              "пользователем или полученного из выбранного ресурса одежды",
            },
```

Книги для программистов: https://t.me/free_books_for_it

```
            cloth_id: {
              type: "string",
              description: "id одежды, полученный из выбранного ресурса одежды",
            },
            cloth_description: {
              type: "string",
              description:
                "Описание одежды, данные пользователя или данные " +
                "из выбранного ресурса одежды",
            },
          },
          required: ["user_img_url", "cloth_img_url"],
        },
      },
      {
        name: "query_tryon_task",
        description: "Запрос задачи примерки (tryon) с ID задачи",
        inputSchema: {
          type: "object",
          properties: {
            task_id: {
              type: "string",
              description: "id задачи, полученный из инструмента " +
                           "submit_tryon_task",
            },
          },
          required: ["task_id"],
        },
      },
    ],
  };
});
```

Обратите внимание, что при определении инструментов в поле `description` следует как можно подробнее описывать их назначение, чтобы помочь большой языковой модели правильнее их распознавать и повысить эффективность планирования. Кроме того, необходимо правильно задать типы и описания каждого поля, чтобы модель могла возвращать правильные параметры вызова после распознавания инструмента.

Список инструментов, определенных на MCP-сервере HeyBeauty, можно получить с помощью MCP Inspector, как показано на рис. 3.32.

Книги для программистов: https://t.me/free_books_for_it

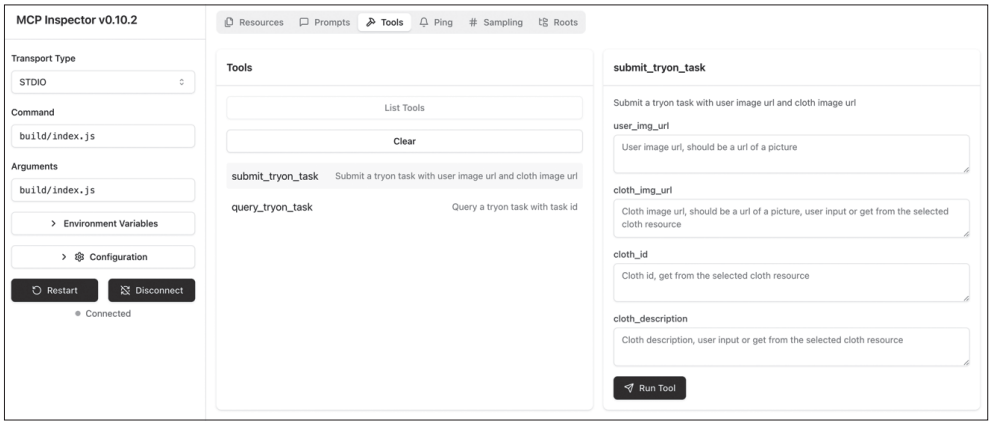


Рис. 3.32. Получение списка инструментов с помощью MCP Inspector

Реализация логики вызова инструментов. В файле *src/index.ts* осуществляется реализация основной логики вызова инструментов для примерки одежды (основная часть включает обработку двух инструментов).

```
/**
 * Реализация логики двух инструментов: submit_tryon_task и query_tryon_task
 */
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  try {
    const apiKey = "xxxxxx";
    const client = new HeyBeautyClient({ apiKey });

    switch (request.params.name) {
      case "submit_tryon_task": {
        const user_img_url = String(request.params.arguments?.user_img_url);
        const cloth_img_url = String(request.params.arguments?.cloth_img_url);
        const cloth_id = String(request.params.arguments?.cloth_id);
        const cloth_description = String(
          request.params.arguments?.cloth_description
        );

        if (!user_img_url) {
          throw new Error("необходимо загрузить изображение пользователя");
        }

        if (!cloth_img_url) {
          throw new Error("необходимо загрузить изображение одежды");
        }

        const res = await client.submitTask({
          user_img_url,
          cloth_img_url,
          cloth_id,
          cloth_description,
        });
```

Книги для программистов: https://t.me/free_books_for_it

```
        return {
          content: [
            {
              type: "text",
              text: JSON.stringify(res),
            },
          ],
        };
      }

      case "query_tryon_task": {
      const task_id = String(request.params.arguments?.task_id);
        if (!task_id) {
          throw new Error("нужен id задачи");
        }

        const res = await client.queryTask({ task_id });
        return {
          content: [
            {
              type: "text",
              text: JSON.stringify(res),
            },
          ],
        };
      }

      default:
        throw new Error("Неизвестный инструмент");
    }
  } catch (error: any) {
    throw new Error("call tool failed: " + error.message);
   }
});
```

- `submit_tryon_task`: при получении запроса на вызов инструмента `submit_tryon_task` из параметров запроса считываются адрес фотографии пользователя, адрес изображения одежды, ID одежды, описание одежды и другая информация. С помощью `HeyBeautyClient` вызывается метод `submitTask`, отправляется задача примерки и получается `task_id` задачи примерки.
- `query_tryon_task`: при получении запроса на вызов инструмента `query_tryon_task` из параметров запроса считывается `task_id`, вызывается метод `queryTask` в `HeyBeautyClient` и запрашиваются результаты примерки.

Инструменты отладки

- **Отладка интерфейса** `submit_tryon_task`. В MCP Inspector необходимо ввести параметры запроса и выполнить отладку инструмента `submit_tryon_task`. В случае успешного ответа инструмент вернет `task_id` задачи примерки, как показано на рис. 3.33.

Книги для программистов: https://t.me/free_books_for_it

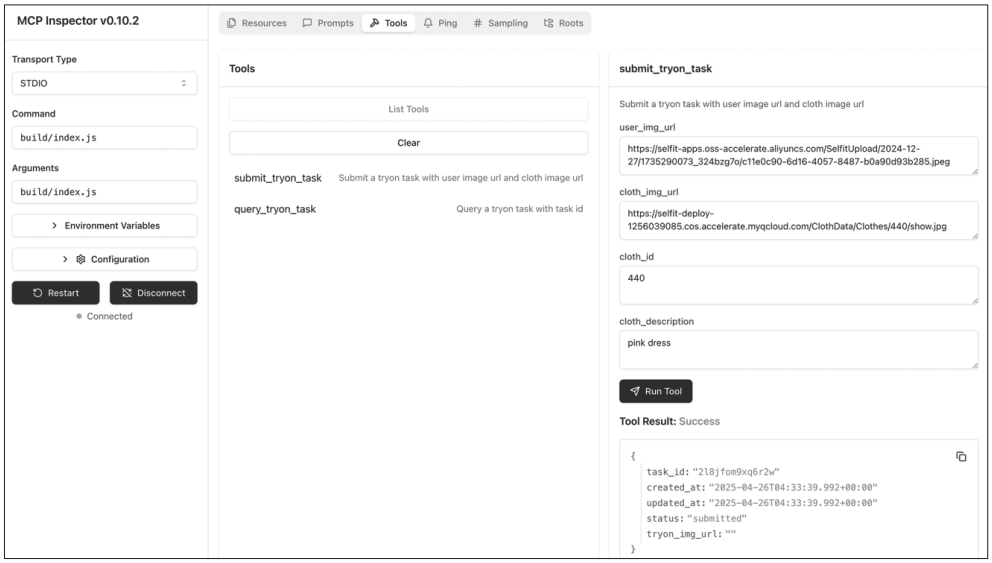


Рис. 3.33. Отладка инструмента submit_tryon_task

- **Отладка интерфейса** `query_tryon_task`. В MCP Inspector необходимо ввести параметры запроса и выполнить отладку инструмента `query_tryon_task`. В случае успешного ответа инструмент вернет информацию о результатах примерки, как показано на рис. 3.34.

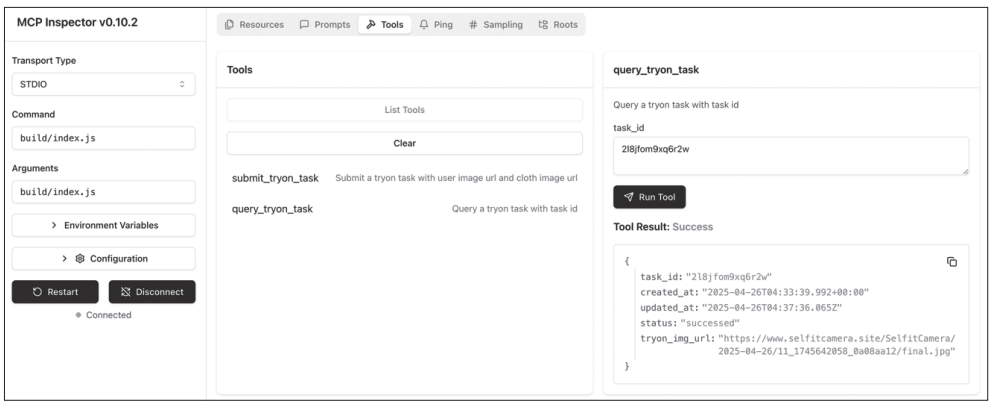


Рис. 3.34. Отладка инструмента query_tryon_task

Когда статус задачи получает значение `successed`, это означает, что процесс примерки успешно завершен, а в поле `tryon_img_url` возвращается адрес фотографии пользователя после примерки одежды.

Книги для программистов: https://t.me/free_books_for_it

5. Тестирование на клиентской стороне большой языковой модели

Перезапустите Claude, выберите изображение одежды, введите адрес изображения и опишите в поле ввода требования к примерке, как показано на рис. 3.35.

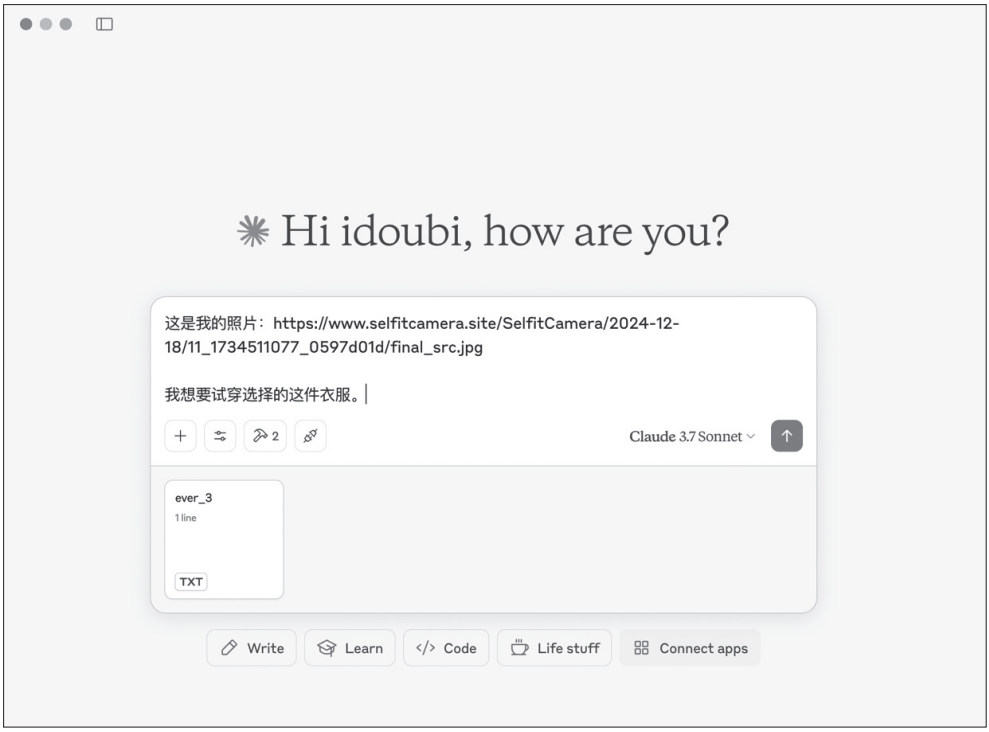


Рис. 3.35. Виртуальная примерка с помощью Claude

Claude анализирует выбранный пользователем адрес изображения и введенный текст, запрашивает у большой языковой модели планирование, а та возвращает название вызываемого инструмента и параметры запроса. Клиент вызывает инструмент `submit_tryon_task` на MCP-сервере HeyBeauty и получает в ответ `task_id`, что означает успешное создание задачи примерки, как показано на рис. 3.36.

Книги для программистов: https://t.me/free_books_for_it

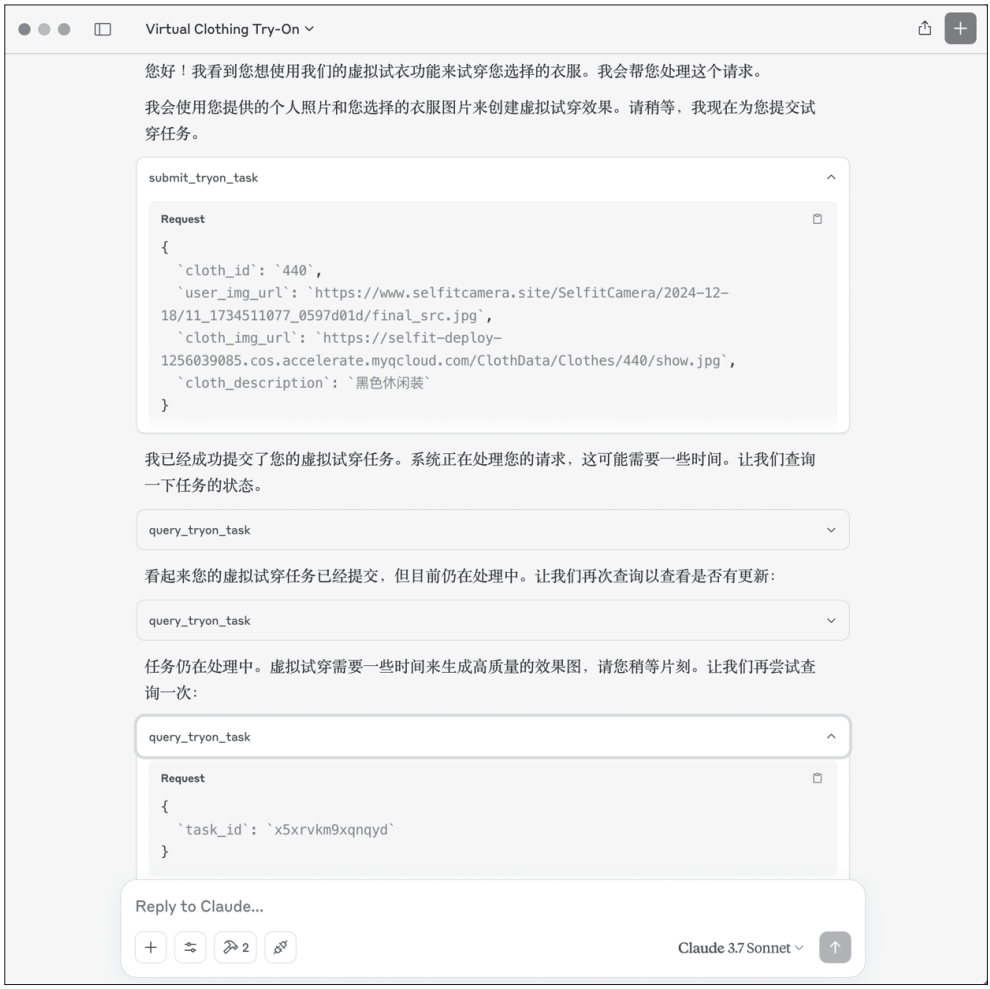


Рис. 3.36. Задача примерки успешно создана

Далее Claude продолжает запрашивать у большой языковой модели планирование и в соответствии с указаниями модели вызывает инструмент `query_tryon_task` на MCP-сервере HeyBeauty для запроса результатов задачи примерки. Когда состояние задачи, найденной при запросе, имеет значение `successed`, запрос к инструменту запроса задачи больше не отправляется, а большая языковая модель по результатам запроса задачи примерки отправляет пользователю окончательное изображение результата примерки. Пользователь открывает ссылку и видит итоговое изображение. Таким образом, пользователь может решить задачу примерки одежды за один диалог, как показано на рис. 3.37.

Книги для программистов: https://t.me/free_books_for_it

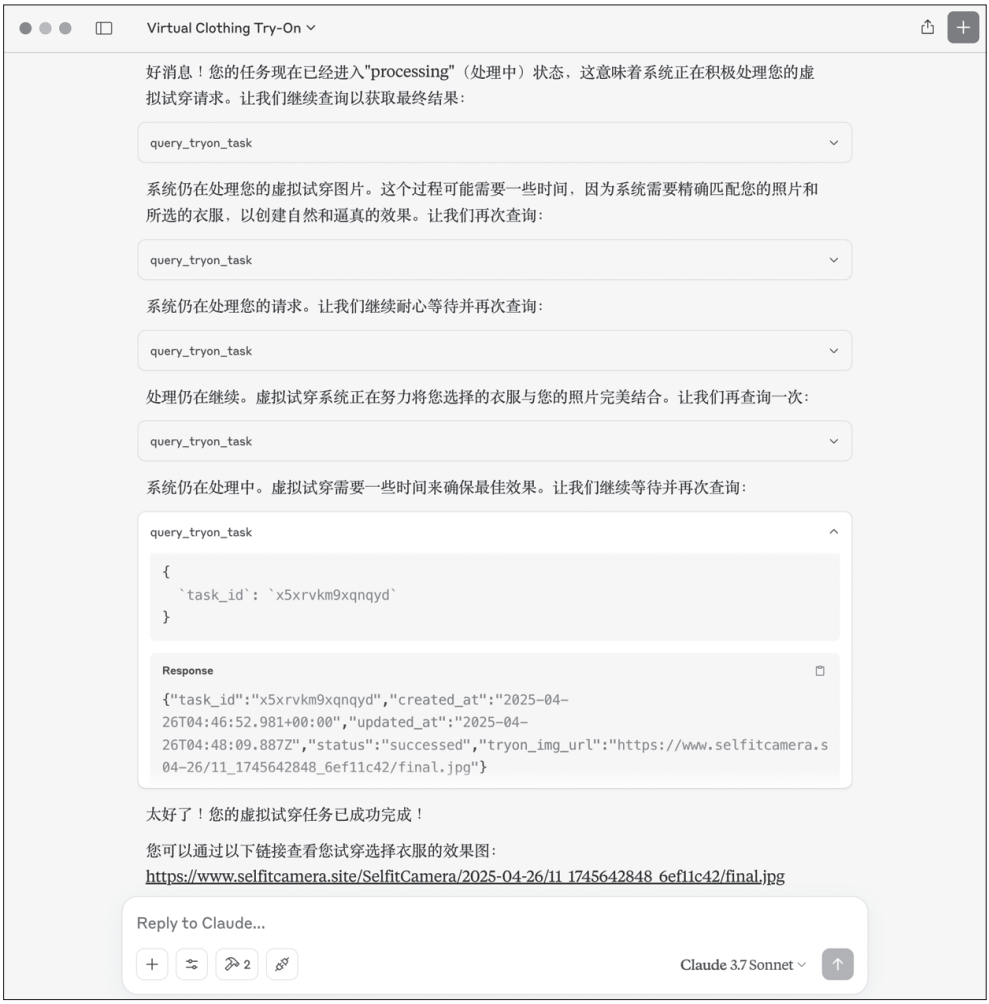


Рис. 3.37. Пользователь решил задачу примерки одежды за один диалог

Я поместил все изображения, связанные с этой задачей, – введенную пользователем фотографию, изображение выбранной одежды и окончательный визуальный результат примерки, – на одном рисунке, чтобы более наглядно продемонстрировать эффект примерки (рис. 3.38).

Книги для программистов: https://t.me/free_books_for_it



Фотография пользователя | Изображение одежды | Изображение результата

Рис. 3.38. Демонстрация результата примерки в HeyBeauty

На этом разработка основных функций MCP-сервера HeyBeauty завершена.

Пользователь может выбрать изображение одежды в любом клиенте, поддерживающем функции `Resources`, а затем отправить свою фотографию для примерки.

Если клиент не поддерживает `Resources`, пользователю необходимо ввести адреса двух изображений. Например, при использовании MCP-сервера HeyBeauty в Cursor необходимо отправить подсказку, как показано на рис. 3.39 (правая область).

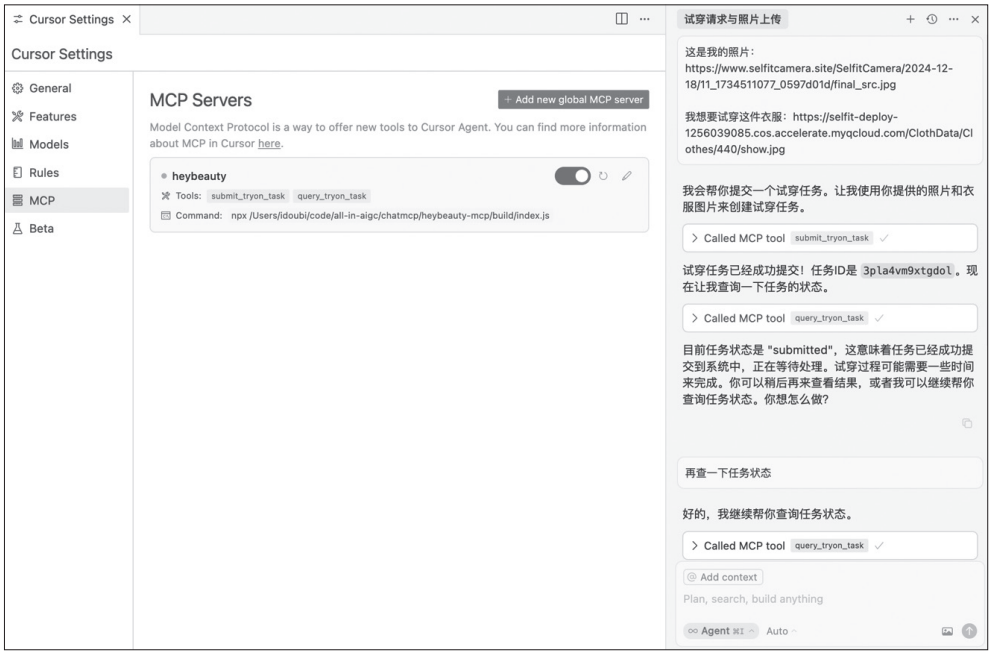


Рис. 3.39. Использование MCP-сервера HeyBeauty в Cursor

Книги для программистов: https://t.me/free_books_for_it

6. Реализация функций подсказок (Prompts)

Встроенные подсказки для MCP-сервера HeyBeauty. После выполнения предыдущих шагов мы разработали MCP-сервер HeyBeauty, который поддерживает запросы на примерку одежды. Однако для этого пользователю необходимо явно указать в клиенте большой языковой модели, что ему требуется примерка. Если отправить только URL-адрес изображения, большая языковая модель может не запустить MCP-сервер HeyBeauty. Кроме того, после успешной отправки задания на примерку необходимо определить, как и с какой периодичностью проверять его статус, поскольку результаты планирования могут значительно различаться в зависимости от модели.

Чтобы повысить точность планирования MCP-сервера HeyBeauty, необходимо настроить подсказки, четко указав большой языковой модели частоту запросов о статусе задачи примерки и сократив тем самым ненужное потребление ресурсов.

Изменим методы возврата списка подсказок и получения их содержания в файле *src/index.ts*, определив подсказку с именем `tryon_cloth`, которая укажет большой языковой модели выполнять автоматический запуск инструмента примерки на MCP-сервере HeyBeauty после получения адреса изображения от пользователя.

```
/**
 * Вывести список промптов, реализованных сервером
 * Добавить промпт tryon_cloth (примерка одежды с использованием фотографии
 * пользователя и изображения одежды)
 */
server.setRequestHandler(ListPromptsRequestSchema, async () => {
  return {
    prompts: [
      {
       name: "tryon_cloth",
        description: "Примерка (tryon) с изображением пользователя " +
                     "и изображением одежды",
      },
    ],
  };
});

/**
 * Вернуть содержание промпта tryon_cloth
 */
server.setRequestHandler(GetPromptRequestSchema, async (request) => {
  if (request.params.name !== "tryon_cloth") {
    throw new Error("Неизвестный промпт");
  }

  return {
    messages: [
      {
        role: "user",
        content: {
```

Книги для программистов: https://t.me/free_books_for_it

```
            type: "text",
            text: `Ты – полезный ассистент, который помогает пользователям примерить
одежду в виртуальном режиме. Когда пользователь указывает URL своей фотографии либо:
  1. URL изображения одежды, которую он хочет примерить, либо
  2. Выбирает предмет одежды из доступных ресурсов

  Вот как действовать в каждом из этих случаев:
  1. Если пользователь указывает URL своего изображения одежды:
   - использовать инструмент submit_tryon_task со следующими параметрами:
   - user_img_url: URL-адрес фотографии пользователя
   - cloth_img_url: URL-адрес изображения одежды, предоставленного пользователем
   - поля cloth_id и cloth_description можно оставить пустыми
  2. Если пользователь выбирает предмет одежды из ресурсов:
   - использовать инструмент submit_tryon_task со следующими параметрами:
   - user_img_url: URL фотографии пользователя
   - cloth_img_url: URL выбранного ресурса одежды
   - cloth_id: ID выбранного ресурса одежды
   - cloth_description: описание выбранного ресурса одежды
  После отправки задачи:
  - извлечь task_id из ответа
  - использовать инструмент query_tryon_task каждые 5 секунд для проверки статуса задачи
  - продолжать проверку, пока статус задачи не станет "succeeded" (успешно) или
"failed" (неудачно)
  - в случае успеха отобразить tryon_img_url в формате Markdown: ![Результат при-
мерки]  (tryon_img_url)
  - В случае неудачи сообщить пользователю о сбое
  На протяжении всего процесса:
  - информировать пользователя о текущем статусе
  - быть терпеливым и дружелюбным
  - корректно обрабатывать любые ошибки
  Вот URL фотографии пользователя и либо URL изображения одежды, либо выбранного
предмета одежды:`,
          },
        },
      ],
    };
  });
```

В MCP Inspector вызовите список промптов и просмотрите содержание этих подсказок, как показано на рис. 3.40.

Использование подсказки в Claude. Перезапустите Claude, нажмите на значок подключения в правом нижнем углу, чтобы увидеть список подсказок и список ресурсов, предоставляемых MCP-сервером HeyBeauty, как показано на рис. 3.41.

Книги для программистов: https://t.me/free_books_for_it

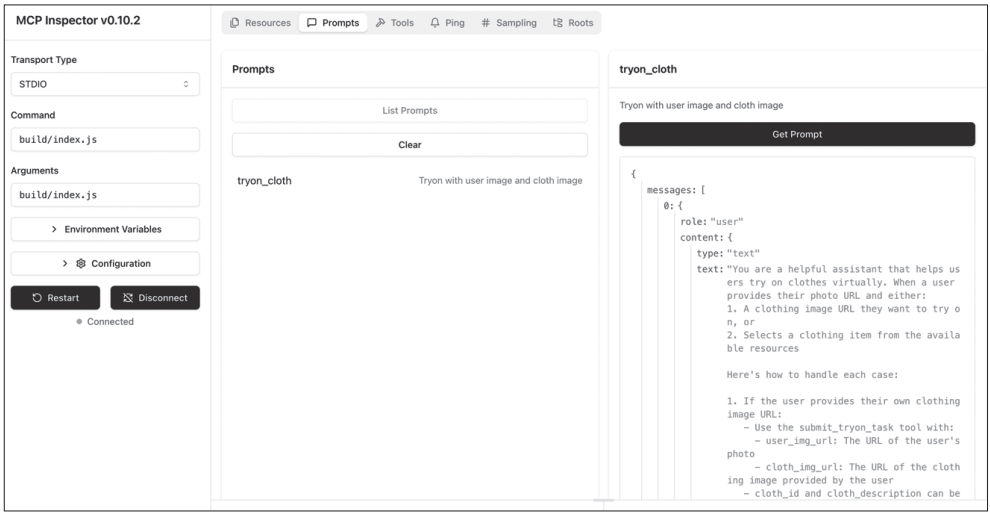


Рис. 3.40. Просмотр содержимого подсказки в MCP Inspector

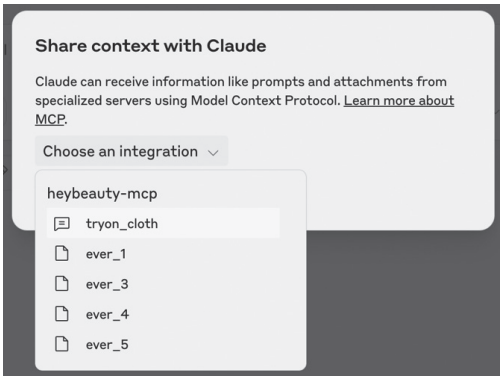


Рис. 3.41. Список подсказок и список ресурсов, предоставляемых MCP-сервером HeyBeauty



В MCP инструменты обычно выбираются большой языковой моделью в автоматическом режиме, а ресурсы и подсказки могут выбираться как хостом, так и пользователем. В реализации Claude ресурсы и подсказки, полученные с MCP-сервера, выводятся пользователю в виде списка, и он может выбрать нужные элементы, нажав на них. Такой подход передает инициативу пользователю, обеспечивает интуитивное взаимодействие и высокий уровень удобства. Пользователь может выбрать сразу несколько ресурсов или подсказок. После завершения выбора Claude запрашивает соответствующий контент у MCP-сервера и отправляет его вместе с введенным пользователем запросом в большую языковую модель для обработки.

Книги для программистов: https://t.me/free_books_for_it

После выбора подсказки `tryon_cloth` и изображения одежды пользователю достаточно ввести адрес фотографии в поле ввода без каких-либо текстовых пояснений, и большая языковая модель сможет распознать инструмент примерки одежды MCP-сервера HeyBeauty. Затем она запустит его, как показано на рис. 3.42.

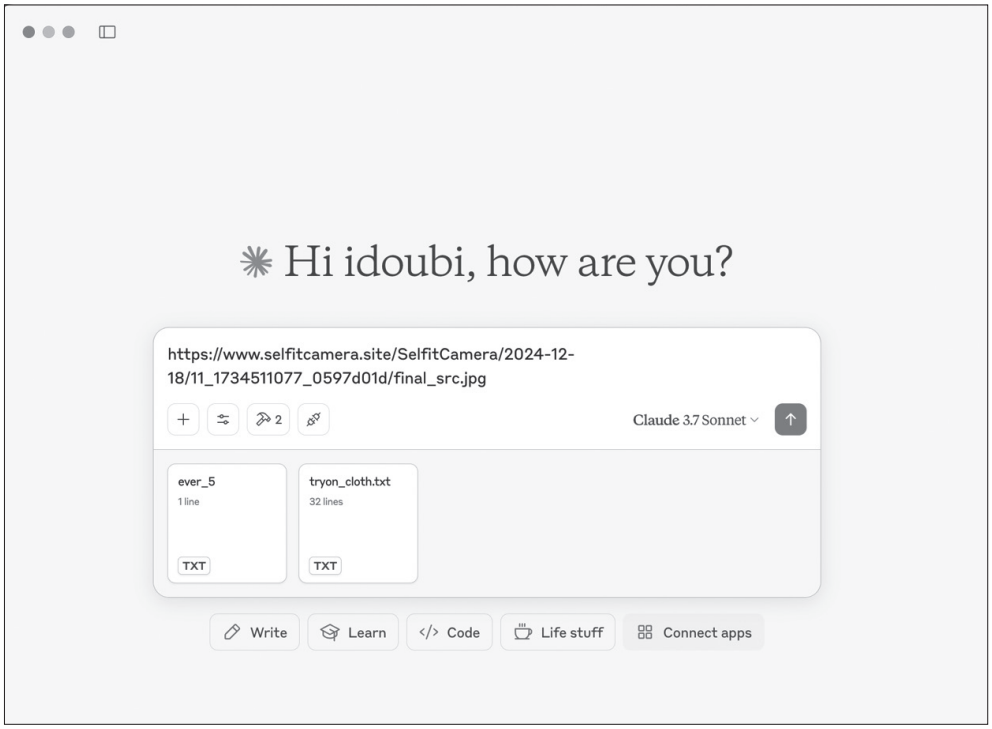


Рис. 3.42. Использование встроенных подсказок и ресурсов MCP-сервера



При таком взаимодействии роль подсказки является особенно важной: она не только определяет условия запуска задачи (например, определение адреса изображения, введенного пользователем), но и подробно описывает процесс последующего вызова инструмента, включая формат параметров, способ отправки задачи и логику запроса статуса. Такой дизайн, управляемый шаблонами подсказок, значительно уменьшает сложность операций для пользователя и повышает точность понимания целей задачи большой языковой моделью. Для разработчиков правильное составление текста подсказки, особенно четкое описание назначения инструмента и ограничений параметров, является основой для правильного планирования инструментов модели, а также ключевым способом для интеграции с минимальным или нулевым объемом кода.

После выполнения клиентом вызова инструмента `submit_tryon_task` на MCP-сервере HeyBeauty для отправки задачи примерки он получает `task_id` и переходит к запросу статуса задачи, как показано на рис. 3.43.

Книги для программистов: https://t.me/free_books_for_it

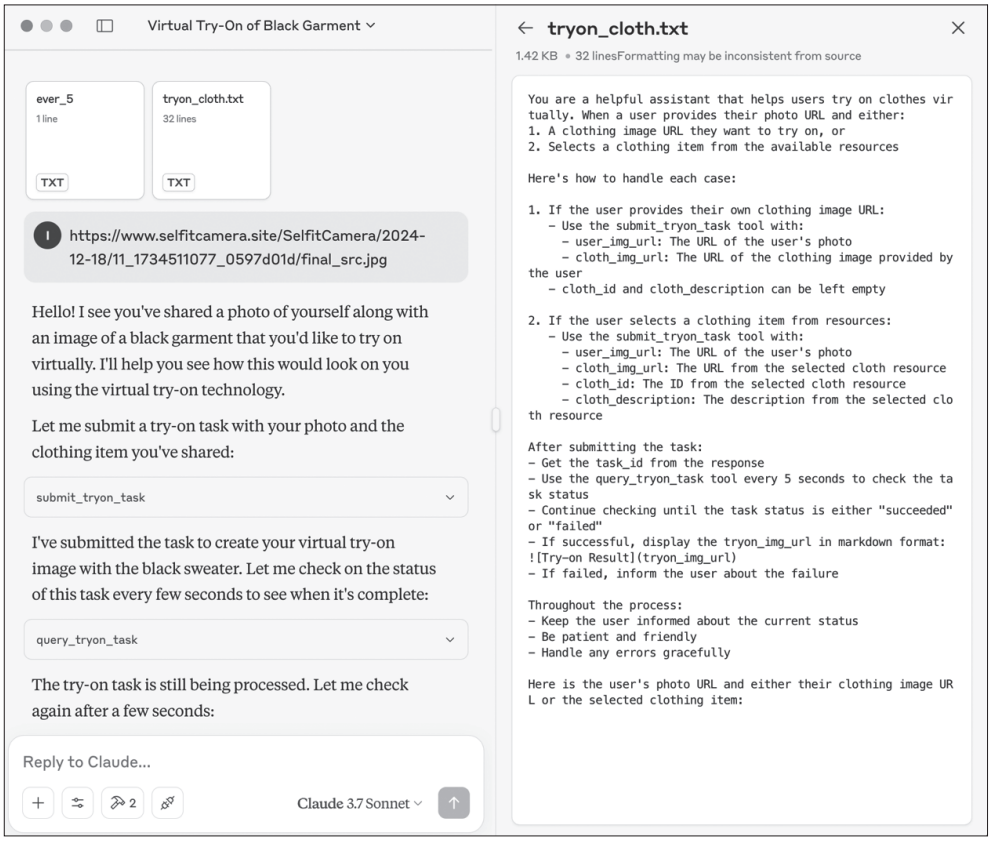


Рис. 3.43. Клиент переходит к запросу статуса задачи



После успешной отправки задачи примерки начинается процесс запроса статуса. Хотя этот процесс определяется ключевыми словами, его логика выполнения полностью контролируется клиентом большой языковой модели. Такая разделенная структура «определение через подсказки + выполнение на стороне клиента» обеспечивает системе высокую гибкость, но в то же время предъявляет более высокие требования к точности подсказок.

Стоит отметить, что стратегия опроса, определенная в подсказках (например, запрос каждые 5 с), не является жестким ограничением, и ее значение может различаться для разных клиентов и моделей. Поэтому при фактическом развертывании разработчикам необходимо уделять внимание балансу между временной задержкой задачи, загрузкой ресурсов и удобством пользователя, а также соответствующим образом корректировать стратегию подсказок, чтобы обеспечить стабильность и эффективность процесса выполнения.

Вызов инструмента `query_tryon_task` для проверки состояния задачи осуществляется каждые 5 с; примерно после 10 запросов клиент получает результаты примерки, как показано на рис 3.44.

Книги для программистов: https://t.me/free_books_for_it

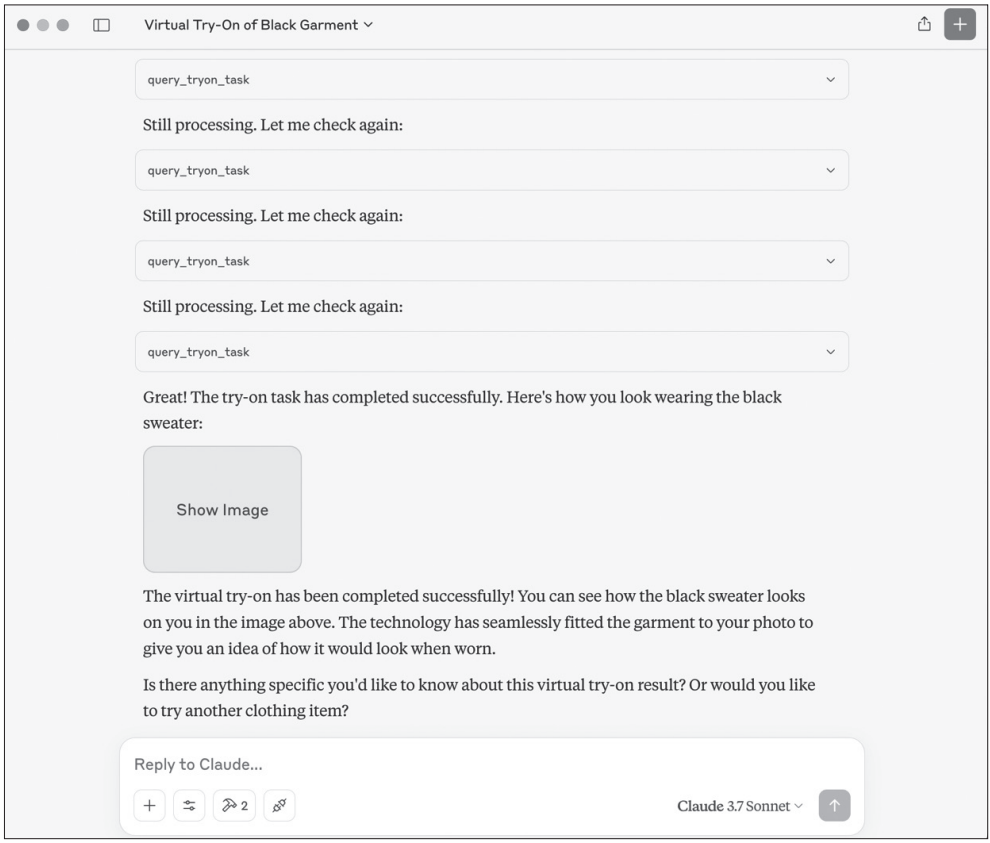


Рис. 3.44. Клиент получил результаты примерки



Приведенный выше процесс управляется инструментом автономного циклического планирования Claude, который в соответствии с указанным в подсказке интервалом повторения вызывает инструмент `query_tryon_task` для проверки статуса задачи примерки. Разработчик может настроить интервал повторения запроса о статусе задачи примерки в соответствии с фактическими условиями.

Следует обратить внимание, что настройка интервала повторения контролируется подсказкой, а не программной логикой, поэтому при вызове разных больших языковых моделей в разных клиентах фактическое поведение может отличаться и, скорее всего, не будет соответствовать вашим ожиданиям.

Сложив воедино введенный пользователем URL фотографии, выбранное изображение одежды и результат примерки, можно увидеть, что задача примерки выполнена очень хорошо, как показано на рис. 3.45. При этом пользователю достаточно ввести в клиенте большой языковой модели адрес фотографии, выбрать изображение одежды и выбрать один из встроенных подсказок: взаимодействие здесь очень простое.

Книги для программистов: https://t.me/free_books_for_it



Фотография пользователя | Изображение одежды | Изображение результата

Рис. 3.45. Изображение результата примерки

3.3.5. Публикация MCP-сервера HeyBeauty

Выполнив предыдущие шаги, мы уже создали полнофункциональный MCP-сервер HeyBeauty, который помогает пользователям проводить виртуальные примерки одежды. Перед тем как официально опубликовать MCP-сервер, нам необходимо проверить код реализации и оптимизировать бизнес-логику.

Для простоты здесь мы не будем вдаваться в подробности: идеи по оптимизации можно почерпнуть из примера реализации MCP-сервера flomo, описанного в разделе 3.2.

1. Публикация на платформе GitHub

Войдите в GitHub, создайте репозиторий и загрузите код MCP-сервера HeyBeauty.

```
git init
git remote add origin git@github.com:chatmcp/heybeauty-mcp.git
git add .
git commit -m "first version"
git push origin main
```

2. Публикация на платформе npm

Измените файл *package.json*, указав, что это общедоступный пакет npm.

```
{
  "name": "heybeauty-mcp",
  "version": "0.1.0",
  "description": "HeyBeauty Virtual TryOn",
  "private": false,
  "type": "module",
```

Книги для программистов: https://t.me/free_books_for_it

```
  "bin": {
    "heybeauty-mcp": "./build/index.js"
  },
  "files": ["build"],
  "scripts": {
    "build": "tsc && node -e \"require('fs').chmodSync('build/index.js',
'755')\"",
    "prepare": "npm run build",
    "watch": "tsc --watch",
    "inspector": "npx @modelcontextprotocol/inspector build/index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "0.6.0"
  },
  "devDependencies": {
    "@types/node": "^20.11.24",
    "typescript": "^5.3.3"
  }
}
```

Выполните команду для публикации на платформе npm:

```
npm publish --access=public
```

После успешной публикации этот пакет будет доступен на официальном сайте npm. Теперь другие пользователи смогут использовать MCP-сервер HeyBeauty после собственной настройки.

3. Публикация в сторонних магазинах MCP-приложений

Можно отправить MCP-сервер HeyBeauty на сторонние маркетплейсы приложений, такие как MCP.so, чтобы его могли найти и использовать как можно больше пользователей, как показано на рис. 3.46.

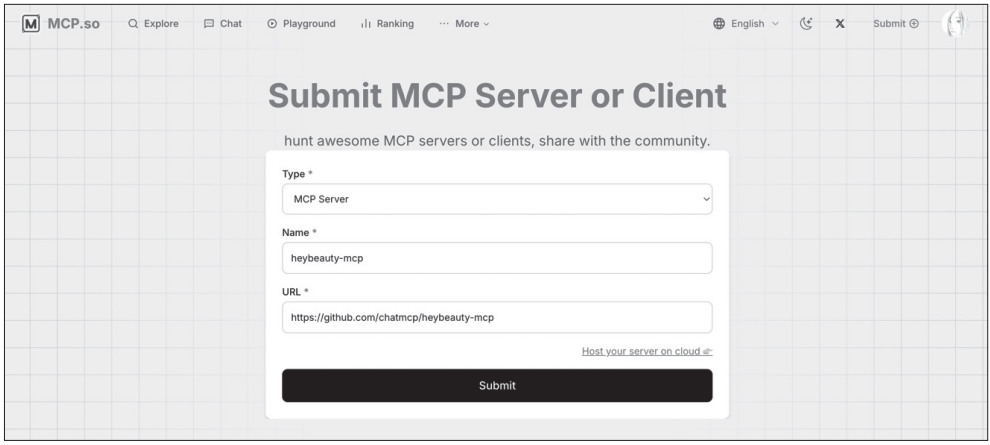


Рис. 3.46. Публикация MCP-сервера HeyBeauty на стороннем маркетплейсе приложений MCP.so

Подробная информация о MCP-сервере HeyBeauty, представленная на маркетплейсе MCP.so, показана на рис. 3.47.

Книги для программистов: https://t.me/free_books_for_it

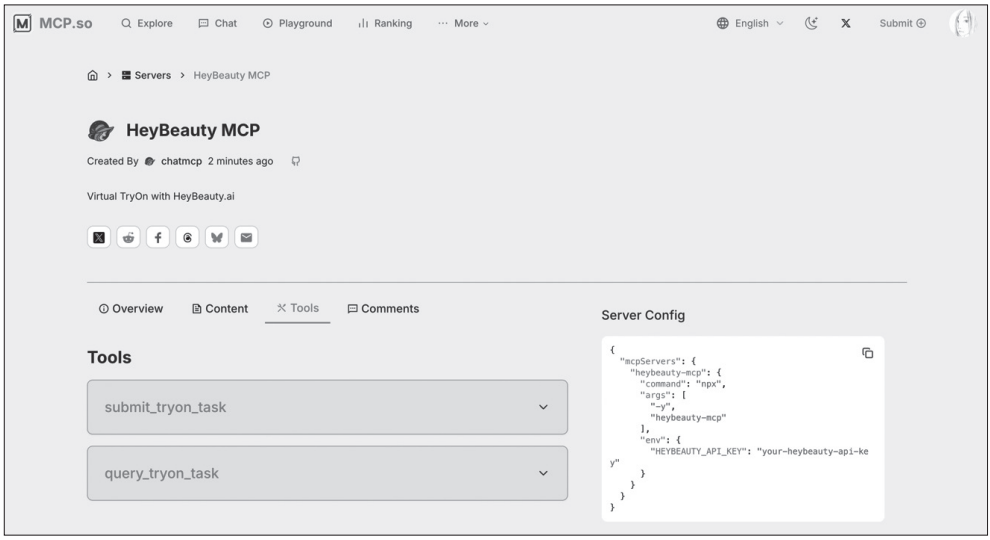


Рис. 3.47. Сервер HeyBeauty MCP, представленный в магазине приложений MCP.so

3.3.6. Итоги примера 2

В этом разделе на примере MCP-сервера HeyBeauty показан полный процесс разработки MCP-сервера с поддержкой обработки изображений, где реализованы следующие три основные функции.

- **Ресурсы** (Resources). Посредством интерфейсов `ListResources` и `ReadResource` клиенту предоставляются ресурсы одежды с платформы HeyBeauty, что дает пользователям возможность выбирать изображения для виртуальной примерки.
- **Инструменты** (Tools). Реализованы два инструмента: `submit_tryon_task` и `query_tryon_task`. Они подключены к API HeyBeauty и обеспечивают отправку и запрос задач виртуальной примерки.
- **Prompts** (подсказки). Встроенная подсказка `tryon_cloth` помогает большой языковой модели корректно вызывать инструмент виртуальной примерки, одновременно контролируя частоту запросов и оптимизируя использование ресурсов.

На примере этого кейса вы узнали, как разработать MCP-сервер с поддержкой обработки изображений, а также как интегрировать в сервер три основные функции – ресурсы, инструменты и подсказки, чтобы улучшить взаимодействие с пользователем.

В то же время этот пример демонстрирует практическое применение MCP в сценариях обработки изображений. Это может послужить справочным материалом для разработки более функциональных MCP-серверов для работы с изображениями, например для редактирования изображений или переноса стилей.

Книги для программистов: https://t.me/free_books_for_it

3.4. Заключение

В этой главе на двух примерах был продемонстрирован полный процесс разработки MCP-сервера. Это поможет читателям освоить основные методы создания, отладки и публикации MCP-серверов и быстро приступить к практической разработке.

В этой главе особое внимание уделяется реализации ключевой функции MCP-сервера – инструментальной, а также подробно описаны методы разработки ресурсов и подсказок. Это дает читателям более полное представление о системе возможностей MCP-сервера и позволяет создавать функциональные и масштабируемые MCP-серверы для практического применения.

Книги для программистов: https://t.me/free_books_for_it

Глава 4

Разработка MCP-клиента

В главе 2, где была представлена архитектура MCP, рассказывалось о том, что в MCP определены три роли: хост, клиент и сервер. MCP-клиент – это дочерний процесс, расположенный на хосте и используемый для взаимодействия с MCP-сервером. Хост, или, как обычно говорят, клиент большой языковой модели, в основном используется для взаимодействия с этой моделью.

В главе 3 на двух практических примерах был рассмотрен полный процесс разработки MCP-сервера. Пользователь может приступить к использованию разработанного нами MCP-сервера сразу после настройки клиента большой языковой модели.

По аналогии с разработкой MCP-сервера тема этой главы называется «Разработка MCP-клиента», но на самом деле речь о разработке клиента большой языковой модели. В этой главе для удобства будет использоваться термин «MCP-клиент», и далее он будет употребляться без дополнительных пояснений.

В этой главе будет представлен полный процесс разработки MCP-клиента на двух реальных примерах. Первый пример посвящен разработке ИИ-ассистента (помощника) для ведения диалога: он считывает локально настроенный пользователем список MCP-серверов и использует инструменты планирования больших языковых моделей для генерации ответных сообщений с их помощью. Второй пример посвящен разработке поискового ИИ-агента: он формирует рабочий поток с помощью определенного набора инструментов на основе предустановленных в программе настроек MCP-серверов с использованием RAG (поисково-усиленная генерация).

4.1. Процесс разработки MCP-клиента

MCP-клиент обычно представляет собой достаточно крупный проект, и для приложений разных типов используются различные технологические стеки. Например, при разработке кросс-платформенных диалоговых приложений для ПК (таких как Claude, Cursor и т. д.) широко используются React Native, Flutter, Electron или Tauri. В то же время проекты типа ИИ-агентов (такие как Manus, Lovart и т. д.) в основном создаются в веб-формате, и в них часто используются такие технологические стеки, как Next.js, Nuxt и т. д. В связи с ограниченным объемом материала в двух примерах этой главы основное внимание будет уделено ключевой логике реализации бэкэнд-функционала, а часть, касающаяся фронтенда / пользовательского интерфейса, рассматриваться не будет.

Книги для программистов: https://t.me/free_books_for_it

Прежде чем перейти к конкретным примерам, давайте сначала разберемся в основном процессе разработки MCP-клиента, сосредоточившись на ключевых шагах и подходах к реализации. Подробности будут рассмотрены в последующих примерах.

4.1.1. Ключевые шаги и подходы к реализации

1. Установка SDK MCP-клиента

После создания проекта MCP-клиента необходимо установить SDK соответствующего языка программирования, который в основном используется для создания экземпляра MCP-клиента, подключения к MCP-серверу, а также для выполнения таких операций, как получение списка инструментов и вызов инструментов.

В качестве примера рассмотрим разработку на TypeScript. Команда для установки соответствующего SDK MCP-клиента выглядит следующим образом:

```
npm install @modelcontextprotocol/sdk
```

2. Чтение списка MCP-серверов

MCP-клиент должен считывать настроенный пользователем список MCP-серверов для доступа к поддерживаемым инструментам и предоставления их для планирования работы больших языковых моделей. Поэтому при разработке MCP-клиента необходимо в первую очередь создать логику чтения списка MCP-серверов. Для разных типов MCP-клиентов способы настройки могут отличаться.

- Если это MCP-клиент диалогового типа для настольных ПК, список MCP-серверов обычно настраивается локально, и пользователь вводит конфигурацию MCP-серверов в соответствии со своими потребностями. Например, в операционной системе macOS конфигурационный файл MCP-сервера Claude находится в *~/Library/Application Support/Claude/claude_desktop_config.json*, а конфигурационный файл MCP-сервера редактора Cursor расположен в *~/.cursor/mcp.json*.
- Если это MCP-клиент типа ИИ-агента, список MCP-серверов обычно настраивается в файлах проекта, и разработчик заранее задает конфигурацию MCP-серверов в соответствии с бизнес-требованиями.

3. Получение списка инструментов с MCP-сервера

После получения списка MCP-серверов MCP-клиент должен подключиться к одному из них для получения списка инструментов, определенных на этом сервере. Здесь необходимо использовать SDK MCP-клиента из шага 1: создать экземпляр MCP-клиента, подключиться к MCP-серверу и отправить запрос `listTools` для получения списка определенных на этом сервере инструментов.

4. Отправка списка инструментов в большую языковую модель и выбор инструмента для вызова

MCP-клиент может выполнять выборку и агрегирование всех инструментов, полученных из настроенного пользователем списка MCP-серверов, а также

Книги для программистов: https://t.me/free_books_for_it

задавать запрос с подсказкой, чтобы большая языковая модель выбрала наиболее подходящий инструмент, после чего модель возвращает информацию о вызове инструмента (название инструмента, параметры).

5. Анализ информации о вызове инструмента из ответа большой языковой модели

Ответ большой языковой модели представляет собой текстовый файл, в котором информация о вызове инструмента заключена в специальные теги. MCP-клиент должен извлечь из него актуальные данные вызова для последующего вызова инструмента.

6. Вызов инструмента

На основании информации о вызове инструмента, проанализированной на предыдущем этапе, MCP-клиент отправляет запрос `callTool` на целевой MCP-сервер и получает результат вызова инструмента.

7. Вывод итоговых результатов или переход к следующему циклу вызова инструмента

MCP-клиент может повторно отправить результаты вызова инструмента в большую языковую модель, перейдя к следующему циклу вызова инструмента (шаг 4). Если большая языковая модель больше не возвращает ответ с информацией о вызове инструмента или количество вызовов достигло установленного предела, MCP-клиент завершает цикл и выводит окончательные результаты.

Процесс взаимодействия MCP-клиента можно обобщить на одной диаграмме (обратите внимание, что она не соответствует описанным выше основным шагам полностью), как показано на рис. 4.1.

Далее мы рассмотрим разработку MCP-клиента на конкретном примере.

4.2. Пример 1. Разработка диалогового ИИ-ассистента

В этом разделе планируется разработка диалогового ИИ-ассистента под названием ChatMCP. На этом примере будет продемонстрирована последовательность разработки MCP-клиента подобного типа.

4.2.1. Цели разработки

Цель данного примера заключается в создании универсального диалогового ИИ-ассистента, расширяющего возможности большой языковой модели с помощью MCP-сервера, при этом расширение внешних функций осуществляется пользователем через настройку MCP-сервера.

Книги для программистов: https://t.me/free_books_for_it

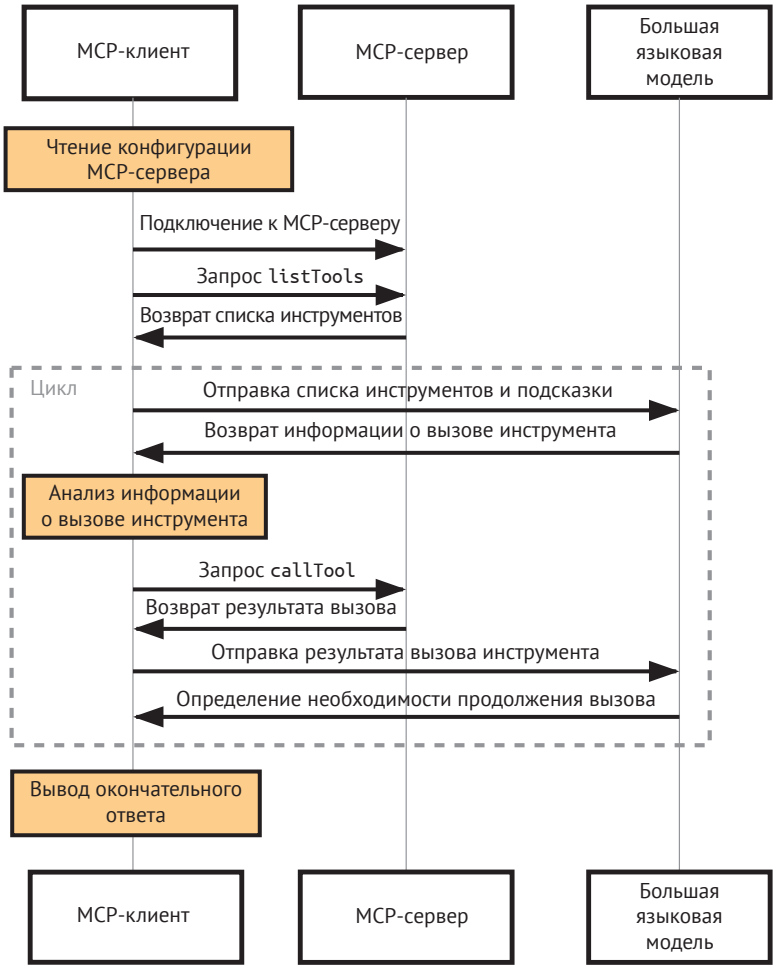


Рис. 4.1. Процесс взаимодействия MCP-клиента

Следуя подходу «разработка на основе тестовых случаев», мы сначала создадим тестовый случай, чтобы проанализировать процесс взаимодействия пользователя с большой языковой моделью через ChatMCP.

1. Пользователь открывает конфигурационный файл ChatMCP и вводит настройки MCP-сервера.
2. Пользователь запускает ChatMCP. Программа считывает список MCP-серверов из конфигурационного файла и получает все инструменты, предоставляемые этими серверами.
3. Пользователь вводит вопрос в диалоговом окне, ChatMCP прилагает список поддерживаемых инструментов и запрашивает у большой языковой модели планирование действий.
4. Большая языковая модель выбирает инструмент, возвращает название инструмента, параметры вызова и название сервера, на котором находится инструмент.

Книги для программистов: https://t.me/free_books_for_it

5. ChatMCP вызывает инструмент и получает результат.
6. ChatMCP прилагает список инструментов и результаты вызова инструментов, после чего повторно запрашивает у большой языковой модели планирование действий.
7. ChatMCP повторяет шаги 4–6 до тех пор, пока ответ модели не будет содержать данных о вызове инструментов либо будет достигнуто установленное программой максимальное количества циклов, после чего цикл завершается.
8. ChatMCP выводит пользователю окончательный ответ.

С помощью веб-версии ChatMCP, доступной по адресу *mcp.so/chat*, можно проиллюстрировать настройку MCP-сервера пользователем, диалог с моделью и получение ответа, как показано на рис. 4.2.



Рис. 4.2. Демонстрация диалога пользователя в веб-версии ChatMCP

Книги для программистов: https://t.me/free_books_for_it

В этом демонстрационном примере пользователь настроил MCP-сервер Amap Maps (amap-maps). После ввода вопроса пользователем ChatMCP передает инструменты `maps_geo`, `maps_direction_driving` и т. д. большой языковой модели.

На основании информации о вызовах инструментов, возвращенной моделью, ChatMCP выполнил три вызова инструментов, что в итоге позволило выполнить запрос пользователя на планирование маршрута.

Этот демонстрационный пример предназначен для наглядного представления процесса взаимодействия с веб-версией ChatMCP с позиции пользователя. Опираясь на этот процесс, мы создадим логику взаимодействия для клиентской части ChatMCP.

4.2.2. Предварительная подготовка

Прежде чем приступить к реализации логики взаимодействия ChatMCP, необходимо выполнить некоторые предварительные действия: установить необходимые внешние зависимости и создать общие вспомогательные функции.

1. Установка SDK

Для разработки ChatMCP мы выбрали TypeScript, поэтому сначала необходимо установить SDK MCP-клиента для TypeScript.

```
npm install @modelcontextprotocol/sdk
```

2. Внедрение функции для получения списка инструментов

Внедрим функцию `listTools`, которая использует SDK для создания экземпляра MCP-клиента и получения списка инструментов, определенных на MCP-сервере.

```
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";

async function listTools({
  command,
  args,
  env = {},
}: {
  command: string;
  args: string[];
  env?: Record<string, string>;
}) {
  const transport = new StdioClientTransport({
    command,
    args,
    env: {
      ...(process.env as Record<string, string>),
      ...env,
    },
  });

  const client = new Client({
    name: "chatmcp",
```

Книги для программистов: https://t.me/free_books_for_it

```
    version: "1.0.0",
  });

  await client.connect(transport);

  const tools = await client.listTools();

  return tools;
}
```

Настроим MCP-сервер, который предоставляет сервис справочной информации о погоде, протестируем функцию `listTools` и получим список инструментов, определенных на этом сервере, как показано на рис. 4.3.



Рис. 4.3. Тестирование функции получения списка инструментов MCP-сервера

3. Внедрение функции вызова инструмента

Ниже представлена функция `callTool`, которая использует SDK для создания экземпляра MCP-клиента и вызова инструмента, реализованного на MCP-сервере.

```
async function callTool({
  command,
  args,
  env = {},
  name,
  params,
}: {
  command: string;
  args: string[];
  env?: Record<string, string>;
  name: string;
  params?: Record<string, unknown>;
```

Книги для программистов: https://t.me/free_books_for_it

```
}) {
  const transport = new StdioClientTransport({
    command,
    args,
    env: {
      ...(process.env as Record<string, string>),
      ...env,
    },
  })

  const client = new Client({
    name: "chatmcp",
    version: "1.0.0",
  });

  await client.connect(transport);

  const result = await client.callTool({
    name,
    arguments: params,
  });

  return result;
}
```

Используя настройки MCP-сервера для запроса информации о погоде, названия инструментов и параметры запроса из предыдущего шага, протестируйте функцию `callTool` и получите результат вызова целевого инструмента, как показано на рис. 4.4.



Рис. 4.4. Тестирование функции вызова инструментов MCP-сервера

После подготовки этих двух служебных функций можно приступать к реализации логики взаимодействия ChatMCP.

Книги для программистов: https://t.me/free_books_for_it

4.2.3. Чтение списка MCP-серверов, настроенных пользователем

ChatMCP требует настройки локального файла для сохранения списка MCP-серверов, настроенных пользователем. При запуске ChatMCP список MCP-серверов считывается из этого файла.

Для удобства разработки в качестве конфигурационного файла ChatMCP мы используем конфигурационный файл MCP приложения Cursor.

В операционной системе macOS путь к конфигурационному файлу Cursor выглядит следующим образом: */Users/$USER/.cursor/mcp.json*.

В этом файле пользователь настраивает список используемых MCP-серверов. Например, мы настроим два MCP-сервера, которые предоставляют услуги по запросу информации о погоде и ведению заметок. Настройки выглядят следующим образом.

```
{
  "mcpServers": {
    "weather-mcp": {
      "command": "npx",
      "args": ["-y", "@chatmcp/weather-mcp"],
      "env": {
        "WEATHER_API_KEY": "xxx"
      }
    },
    "flomo-mcp": {
      "command": "npx",
      "args": ["-y", "@chatmcp/flomo-mcp"],
      "env": {
        "FLOMO_API_URL": "https://flomoapp.com/iwh/xxx/xxxxxx/"
      }
    }
  }
}
```

Здесь значения `WEATHER_API_KEY` и `FLOMO_API_URL` необходимо заполнить в соответствии с фактическими данными. В данном случае для защиты конфиденциальности они указаны с использованием заполнителей.

При разработке ChatMCP следуйте следующим шагам для реализации логики чтения списка MCP-серверов, настроенных пользователем.

Определите тип данных MCP-сервера.

```
interface McpServer {
  name: string;
  command: string;
  args?: string[];
  env?: Record<string, string>;
}
```

Введите функцию, которая анализирует список MCP-серверов из заданного конфигурационного файла.

Книги для программистов: https://t.me/free_books_for_it

```
async function getMcpServers(
  config: string
): Promise<Record<string, McpServer>> {
  const mcpConfig = JSON.parse(config);

  const mcpServers = Object.entries(mcpConfig.mcpServers).reduce(
    (acc, [key, value]) => {
      acc[key] = {
        name: key,
        command: (value as any).command,
        args: (value as any).args,
        env: (value as any).env,
      };
      return acc;
    },
    {} as Record<string, McpServer>
  );

  return mcpServers;
}
```

Эта функция возвращает объект `key-value`, где `key` означает имя MCP-сервера, а `value` содержит информацию о MCP-сервере, определенную на шаге 1.

Считайте конфигурационный файл и извлеките из него список MCP-серверов.

```
const configFile = `/Users/idoubi/.cursor/mcp.json`;
const config = await fs.readFile(configFile, "utf-8");
const mcpServers = await getMcpServers(config);
```

Здесь разработчику необходимо указать адрес локального конфигурационного файла ChatMCP в переменной `configFile`.

Запустите отладочный интерфейс и выведите список настроенных MCP-серверов, как показано на рис. 4.5.

4.2.4. Получение списка инструментов с MCP-сервера

При запуске ChatMCP считывает из конфигурационного файла настроенный пользователем список MCP-серверов, устанавливает соединение с каждым из них через SDK и запрашивает список инструментов, определенных на каждом MCP-сервере.

При разработке ChatMCP необходимо выполнить следующие шаги для реализации логики получения списка инструментов с MCP-сервера.

1. Определите тип данных инструмента.

```
interface McpTool {
  server_name: string;
  name: string;
  description: string;
  inputSchema: Record<string, unknown>;
}
```

Книги для программистов: https://t.me/free_books_for_it



Рис. 4.5. Отладочный интерфейс с выводом списка настроенных MCP-серверов

2. Внедрите функцию для получения списка инструментов из списка серверов MCP.

```
async function getMcpTools(
  mcpServers: Record<string, McpServer>
): Promise<McpTool[]> {
  const allTools = await Promise.all(
    Object.entries(mcpServers).map(async ([name, server]) => {
      const tools = await listTools({
        command: server.command,
        args: server.args || [],
        env: server.env || {},
      });

      return tools.tools.map((tool) => ({
        server_name: name,
        name: tool.name,
        description: tool.description || "",
        inputSchema: tool.inputSchema,
      }));
    })
  );

  return allTools.flat();
}
```

Эта функция проходит по списку MCP-серверов и параллельно вызывает реализованную в предыдущем шаге функцию `listTools`, получая список инстру-

Книги для программистов: https://t.me/free_books_for_it

ментов, определенный на каждом сервере MCP. В конце она объединяет все списки инструментов и возвращает их.

3. Чтение конфигурационного файла, анализ списка MCP-серверов и получение всех инструментов.

```
const configFile = `/Users/idoubi/.cursor/mcp.json`;
const config = await fs.readFile(configFile, "utf-8");
const mcpServers = await getMcpServers(config);
const mcpTools = await getMcpTools(mcpServers);
```

Здесь `configFile` должен быть настроен разработчиком в соответствии с фактической ситуацией как адрес локального конфигурационного файла ChatMCP.

4. Отладочный интерфейс, вывод списка всех полученных инструментов, как показано на рис. 4.6.

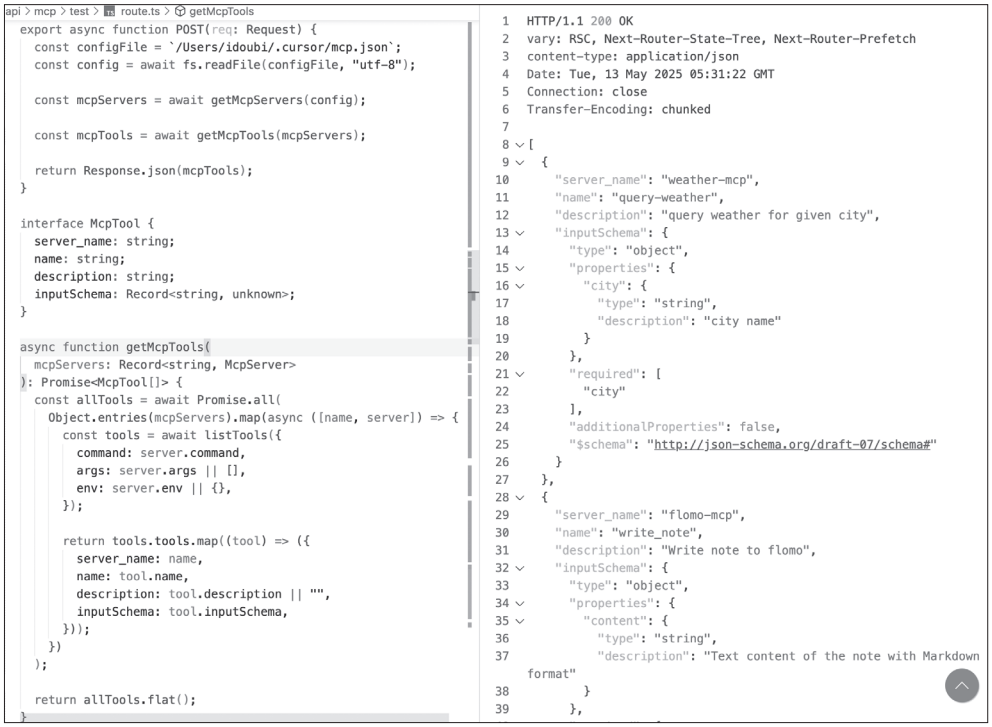


Рис. 4.6. Отладочный интерфейс, вывод списка всех полученных инструментов

4.2.5. Запрос на выбор инструментов большой языковой моделью

При запуске ChatMCP получает настроенный пользователем список MCP-серверов и, подключаясь к каждому из них, получает все доступные инструменты. После того как пользователь вводит вопрос в диалоговом окне ChatMCP, программа отфильтровывает часть инструментов из общего списка в соответ-

Книги для программистов: https://t.me/free_books_for_it

ствии с выбранными пользователем MCP-серверами, отправляет их вместе с запросом пользователя в большую языковую модель, которая выбирает наиболее подходящий инструмент для вызова.

При разработке ChatMCP необходимо выполнить следующие шаги для реализации логики запроса выбора инструмента большой языковой моделью.

1. Настройка системного промпта

Чтобы большая языковая модель смогла лучше разобраться в намерениях пользователя, при запросе на выбор инструмента необходимо задать системный промпт. В данном примере системная подсказка выглядит следующим образом.

```
Ты – ChatMCP, диалоговый ИИ-ассистент, разработанный ThinkAny AI.
# Общие инструкции
На основании доступной контекстной информации (CONTEXT_MESSAGES) и результатов предыдущего вызова инструмента (PREVIOUS_TOOL_RESULTS) составить точный, подробный и исчерпывающий ответ на запрос пользователя (USER_QUERY).
- Ответ должен быть точным, качественным, профессиональным и объективным.
- Язык ответа должен соответствовать языку запроса.
- Запрещается использовать формулировки, содержащие субъективные суждения или двусмысленности; избегать следующих выражений:
  - "Важно, что…"
  - "Неуместно, что…"
  - "…это субъективно"
# Требования к форматированию
- Использовать Markdown для форматирования абзацев, списков, таблиц и цитат.
- Разделять контент с помощью заголовков второго и третьего уровня, например, "## Заголовок", но **не** начинать абзац с заголовка.
- Для разделения пунктов списка использовать один символ переноса строки, для разделения абзацев использовать два символа переноса строки.
- При наличии изображений использовать рендеринг Markdown.
- Не указывать URL-адреса или ссылки.
# Вызов инструментов
Если необходимо вызвать инструмент, следует соблюдать следующий порядок действий.
1. Определить, нужен ли инструмент, основываясь на: текущем запросе пользователя, результатах уже имеющихся инструментов и списке доступных инструментов (AVAILABLE_TOOLS).
2. Возвращаемое tool_name может использовать только инструменты, явно указанные в списке доступных инструментов, а параметры в tool_params должны полностью совпадать с inputSchema.
3. Возвращаемое server_name должно полностью совпадать с tool_name.
4. Формат вызова инструмента следующий, и он должен быть размещен в последней строке ответа:
<<tool-start>>
{
 "server_name": "имя сервера, предоставляющего инструмент",
 "tool_name": "название инструмента",
 "tool_params": {
 "параметр1": "значение параметра 1",
 "параметр2": "значение параметра 2"
 }
}
```

Книги для программистов: https://t.me/free_books_for_it

```
  <<tool-end>>
  - За один раз можно вызвать только один инструмент, нельзя давать ответ заранее.
  - После вызова инструмента необходимо дождаться результата, прежде чем продолжать
    ответ.
  - Если вызов инструмента дважды завершился неудачей, повторные попытки не предпри-
    нимать; ответ следует сформулировать на основе имеющейся информации с указанием
    причины.
  Если инструмент не требуется, следует просто ответить, не включая блок вызова
инструмента.
  ----
  Ниже приведены переменные данного запроса:
  Запрос пользователя: USER_QUERY={USER_QUERY}
  Контекст: CONTEXT_MESSAGES={CONTEXT_MESSAGES}
  Результаты предыдущего вызова инструмента: PREVIOUS_TOOL_RESULTS={PREVIOUS_TOOL_
RESULTS}
  Список доступных инструментов: AVAILABLE_TOOLS={AVAILABLE_TOOLS}
  Используй свои логические способности для анализа информации и создания наиболее
полезного ответа.
```

Основная логика этого запроса заключается в передаче списка доступных инструментов в большую языковую модель через параметр `AVAILABLE_TOOLS`, чтобы она могла выбрать из него инструмент для дополнения контекста. Передача истории сообщений и результатов предыдущего вызова инструмента в большую языковую модель через `CONTEXT_MESSAGES` и `PREVIOUS_TOOL_RESULTS` дает ей достаточно информации для принятия решений при нескольких раундах диалога или последовательном вызове нескольких инструментов.

Если большая языковая модель решает, что для дополнения контекста необходимо вызвать инструмент, она возвращает информацию о нужном инструменте. В приведенных выше подсказках информация о вызываемом инструменте заключена в специальные теги `<<tool-start>>` и `<<tool-end>>`.

2. Реализация функции диалога с большой языковой моделью

При разработке ChatMCP логика диалога с большой языковой моделью осуществляется с помощью библиотеки AI SDK, а в качестве поставщика интерфейса большой языковой модели используется OpenRouter. Команды для установки необходимых зависимостей приведены ниже.

```
npm install ai
npm install @openrouter/ai-sdk-provider
```

Затем мы создадим функцию `chatWithLLM`, с помощью которой можно делать запросы к большой языковой модели и получать ее ответы. Логика реализации функции `chatWithLLM` приведена ниже.

```
import { openrouter } from "@openrouter/ai-sdk-provider";
import { streamText } from "ai";

async function chatWithLLM({
  query,
  contextMessages,
```

Книги для программистов: https://t.me/free_books_for_it

```
    tools,
    toolResults,
  }: {
    query: string;
    contextMessages?: string;
    tools?: string;
    toolResults?: string;
  }) {
    const prompt = mcpPrompt
      .replace("{USER_QUERY}", query)
      .replace("{CONTEXT_MESSAGES}", contextMessages || "")
      .replace("{AVAILABLE_TOOLS}", tools || "")
      .replace("{PREVIOUS_TOOL_RESULTS}", toolResults || "");

    const result = await streamText({
      model: openrouter("anthropic/claude-3.5-sonnet"),
      prompt,
    });

    return result;
  }
```

В этой функции `mcpPrompt` является системным подсказкой, определенной в предыдущих шагах, `query` представляет собой введенный пользователем запрос, `contextMessages` является историей сообщений диалога, `tools` является списком доступных инструментов, а `toolResults` представляет собой результаты предыдущего вызова инструмента, которые в качестве части контекста вставляются вместе с другим контентом в соответствующие места подсказки.

В конечном итоге функция вызывает `streamText` из библиотеки AI SDK, отправляет сформированный промпт в большую языковую модель и получает из нее потоковый ответ.



Реализация `chatWithLLM` довольно проста: в основном используется замена строк, при которой заполнители в системном промпте заменяются фактическим содержанием. Так формируется промпт для запроса к большой языковой модели. Функция `streamText` инкапсулирует параметры, необходимые для вызова модели. Нам нужно только передать название модели и подсказку. В данном случае через поставщика интерфейса модели OpenRouter вызывается модель anthropic/claude-3.5-sonnet, которая демонстрирует более высокую точность распознавания переданного списка инструментов (tools).

Список инструментов, полученный при запуске ChatMCP, передается в качестве параметра tools вместе с запросом пользователя (query) в функцию `chatWithLLM` для отладки результатов вывода большой языковой модели, как показано на рис. 4.7.

Книги для программистов: https://t.me/free_books_for_it



Рис. 4.7. Отладка результатов вывода большой языковой модели

На рисунке видно, что большая языковая модель вернула информацию о необходимых инструментах, заключив ее в специальные теги.

4.2.6. Анализ информации об инструментах в ответе большой языковой модели

В разделе 4.2.5 ChatMCP запрашивает у модели выбор инструмента, и модель отвечает информацией о необходимом инструменте, заключив ее в специальные теги. Далее ChatMCP необходимо проанализировать информацию о необходимом инструменте из ответа модели. Логику анализа информации об инструменте выполняем по следующим шагам.

1. Определение типа данных смешанного контента

Ответы больших языковых моделей содержат специальные теги, вложенные в текст, поэтому мы можем определить тип данных смешанного содержания, чтобы разделить текстовое содержание и информацию об инструментах, заключенную в специальные теги.

```
interface MixContent {
  type: "text" | "tool";
  text?: string;
  tool?: {
    server_name: string;
```

Книги для программистов: https://t.me/free_books_for_it

```
      tool_name: string;
      tool_params?: Record<string, unknown>;
    };
}
```

2. Внедрение функции анализа смешанного контента

Эта функция использует регулярные выражения для извлечения заключенной в специальные теги информации об инструментах из текстового содержания и возвращает массив смешанного содержания.

```
function parseMixContents(input: string): MixContent[] {
  const result: MixContent[] = [];
  // Регулярное выражение для извлечения информации об инструментах
  const regex = /<<tool-start>>\s*([\s\S]*?)\s*<<tool-end>>/g;
  let lastIndex = 0;
  let match: RegExpExecArray | null;

  while ((match = regex.exec(input)) !== null) {
    // Обработка обычного текста перед текущим блоком tool
    if (match.index > lastIndex) {
      const text = input.slice(lastIndex, match.index).trim();
      if (text) {
        result.push({ type: "text", text });
      }
    }
    // Разбор и обработка JSON-содержимого в блоке tool
    const toolJson = match[1].trim();
    try {
      const tool = JSON.parse(toolJson);
      result.push({ type: "tool", tool });
    } catch (e) {
      // Если разбор завершился неудачей, пропустить эту ошибку
      // и продолжить выполнение
    }
    lastIndex = regex.lastIndex;
  }
  // Обработка обычного текста, оставшегося после последнего блока tool
  if (lastIndex < input.length) {
    const text = input.slice(lastIndex).trim();
    if (text) {
      result.push({ type: "text", text });
    }
  }

  // Возврат разбора смешанного содержимого
  return result;
}
```

3. Отладка разбора информации об инструментах

В качестве входных данных используем контент, возвращаемый большой языковой моделью и содержащий информацию о вызове инструментов. Вызываем функцию `parseMixContents` для разбора и получаем массив смешанного

Книги для программистов: https://t.me/free_books_for_it

контента, состоящий из фрагментов текста и заключенной в специальные теги информации об инструментах.

Просматриваем результаты вывода через отладочный интерфейс, как показано на рис. 4.8.



Рис. 4.8. Отладка разбора информации об инструментах

Как видно из рисунка, текстовое содержимое и информация об инструментах в массиве смешанного контента были успешно разделены. В последующих шагах можно обрабатывать эти два типа контента по отдельности.



Массив, возвращаемый функцией `parseMixContents`, может содержать несколько элементов. В последующем коде следует пройти по этому массиву и выполнить соответствующие операции в зависимости от поля `type` каждого элемента. В частности, если `type` имеет значение `tool`, извлекаются параметры и вызывается соответствующий инструмент; если `type` имеет значение `text`, то текстовое содержимое отображается напрямую.

4.2.7. Вызов инструмента

В массиве смешанного контента, возвращаемом большой языковой моделью и проанализированном ChatMCP, содержится информация об инструментах, которые необходимо вызвать.

Книги для программистов: https://t.me/free_books_for_it

```
{
  "type": "tool",
  "tool": {
    "server name": "weather-mcp",
    "tool name": "query-weather",
    "tool_params": {
    "city": "Гуанчжоу"
    }
  }
}
```

Сначала передаем проанализированную информацию об инструментах в качестве фиксированного параметра функции `callTool` и после ее объединения с логикой чтения списка MCP-серверов реализуем логику вызова инструмента, как показано ниже.

```
const configFile = `/Users/idoubi/.cursor/mcp.json`;
const config = await fs.readFile(configFile, "utf-8");

const mcpServers = await getMcpServers(config);

const mcpTools = await getMcpTools(mcpServers);

const mixContents = parseMixContents(pickToolResults);

const callToolResult = await callTool({
  command: mcpServers["weather-mcp"].command,
  args: mcpServers["weather-mcp"].args || [],
  env: mcpServers["weather-mcp"].env || {},
  name: "query-weather",
  params: {
    "city": "Гуанчжоу" },
});
```

Затем передаем полученный массив смешанного контента (содержащий информацию о вызове инструментов, возвращенную моделью в прошлый раз) вместе с результатом вызова инструмента и повторно запрашиваем у большой языковой модели ответ на первоначальный вопрос пользователя. Логика реализации приведена ниже.

```
const result = await chatWithLLM({
  query: "Идет ли дождь в Гуанчжоу?",
  contextMessages: JSON.stringify(mixContents),
  tools: JSON.stringify(mcpTools),
  toolResults: JSON.stringify(callToolResult),
});
```

Производим отладку интерфейса. Результаты вывода показаны на рис. 4.9.

Как видно из рисунка, благодаря наличию результатов вызова инструментов в качестве контекста ответ модели включил в себя информацию в режиме реального времени.

Книги для программистов: https://t.me/free_books_for_it



```
api > mcp > test > route.ts > chatWithLLM
export async function POST(req: Request) {
  const configFile = `/Users/idoubi/.cursor/mcp.json`;
  const config = await fs.readFile(configFile, "utf-8");

  const mcpServers = await getMcpServers(config);

  const mcpTools = await getMcpTools(mcpServers);

  const mixContents = parseMixContents(pickToolResults);

  const callToolResult = await callTool({
    command: mcpServers["weather-mcp"].command,
    args: mcpServers["weather-mcp"].args || [],
    env: mcpServers["weather-mcp"].env || {},
    name: "query-weather",
    params: {
      city: "广州",
    },
  });

  const result = await chatWithLLM({
    query: "广州在下雨吗？",
    contextMessages: JSON.stringify(mixContents),
    tools: JSON.stringify(mcpTools),
    toolResults: JSON.stringify(callToolResult),
  });

  return result.toTextStreamResponse();
}
```

```
HTTP/1.1 200 OK
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
content-type: text/plain; charset=utf-8
Date: Tue, 13 May 2025 06:31:05 GMT
Connection: close
Transfer-Encoding: chunked

根据天气查询结果，广州目前天气状况为多云，没有下雨。具体天气情况如下：

## 实时天气
- 温度：28°C
- 湿度：55%
- 天气：多云
- 风向：南风
- 风力：3级
- 空气质量指数：62

## 未来天气预报
- 5月14日：多云转阵雨，气温21-32°C
- 5月15日：阵雨转雷阵雨，气温23-31°C
- 5月16日：雷阵雨，气温23-30°C
- 5月17日：雷阵雨转中雨，气温24-31°C

虽然现在没有下雨，但未来几天有降雨趋势，建议出门携带雨具。
```

Рис. 4.9. Результаты отладки ответов большой языковой модели

На этом разработка основной логики ChatMCP в основном завершена. В зависимости от введенного пользователем запроса и настроек MCP-сервера ChatMCP может запрашивать у большой языковой модели запуск инструмента, который затем выполняется на стороне клиента. Это позволяет дополнить контекст большой языковой модели, что в конечном итоге делает ее ответы более точными и оперативными.

4.2.8. Оптимизация логики взаимодействия

В предыдущих разделах мы разбили процесс на несколько шагов и создали базовый алгоритм, позволяющий ChatMCP считывать список инструментов с MCP-сервера и взаимодействовать с большой языковой моделью. На этой основе продолжаем оптимизировать логику взаимодействия и дорабатываем логику интерфейсного уровня.

1. Внедрение интерфейса для обработки запросов пользователей

В ChatMCP мы внедряем интерфейс POST, включающий следующие логические части:

- получение введенного пользователем параметра `query`, извлечение списка инструментов, предоставляемых настроенным пользователем MCP-сервером, и запрос к большой языковой модели о выборе одного из инструментов;
- анализ информации об инструменте, вызов инструмента, получение результата вызова инструмента и повторный запрос к большой языковой модели для ответа на запрос пользователя;
- вывод ответа интерфейса.

Логика реализации этого интерфейса следующая.

Книги для программистов: https://t.me/free_books_for_it

```
export async function POST(req: Request) {
  let { query } = await req.json();
  let contextMessages: MixContent[] = [];
 // Загрузка конфигурационного файла MCP, получение списка MCP-серверов
 // и предоставляемых ими инструментов
  const configFile = `/Users/idoubi/.cursor/mcp.json`;
  const config = await fs.readFile(configFile, "utf-8");
  const mcpServers = await getMcpServers(config);
  const mcpTools = await getMcpTools(mcpServers);

  // Отправка пользовательского запроса и списка инструментов в модель,
  // запрос на выбор инструмента для вызова
  const pickToolResult = await chatWithLLM({
    query,
    contextMessages: JSON.stringify(contextMessages),
    tools: JSON.stringify(mcpTools),
    toolResults: "",
  });

  let content = "";
  for await (const chunk of pickToolResult.textStream) {
    content += chunk;
  }

  // Анализ смешанного контента, возвращенного моделью
  const mixContents = parseMixContents(content);

  // Анализ информации о вызываемом инструменте
  let callToolParams = null;
  for (const mixContent of mixContents) {
    if (mixContent.type === "tool") {
      const tool = mixContent.tool;
      if (
        tool &&
        tool.tool_name &&
        tool.server_name &&
        mcpServers[tool.server_name] &&
        mcpServers[tool.server_name].command
      ) {
        callToolParams = {
          command: mcpServers[tool.server_name].command,
          args: mcpServers[tool.server_name].args || [],
          env: mcpServers[tool.server_name].env || {},
          name: tool.tool_name,
          params: tool.tool_params,
        };
        break;
      }
    }

  // Требуется вызов инструмента
  if (callToolParams) {
```

Книги для программистов: https://t.me/free_books_for_it

```
    // Вызов инструмента
    const callToolResult = await callTool(callToolParams);
    // Отправить запрос большой языковой модели с результатами вызова инструмента
    const result = await chatWithLLM({
      query,
      contextMessages: JSON.stringify(mixContents),
      tools: JSON.stringify(mcpTools),
      toolResults: JSON.stringify(callToolResult),
    });

    return result.toTextStreamResponse();
  }

  // Вызов инструмента не требуется
  return pickToolResult.toTextStreamResponse();
}
```

Выполните симуляцию ввода вопроса пользователем и передайте его в качестве параметра `query` в отладочный интерфейс для просмотра ответа большой языковой модели. Результат вывода показан на рис. 4.10.

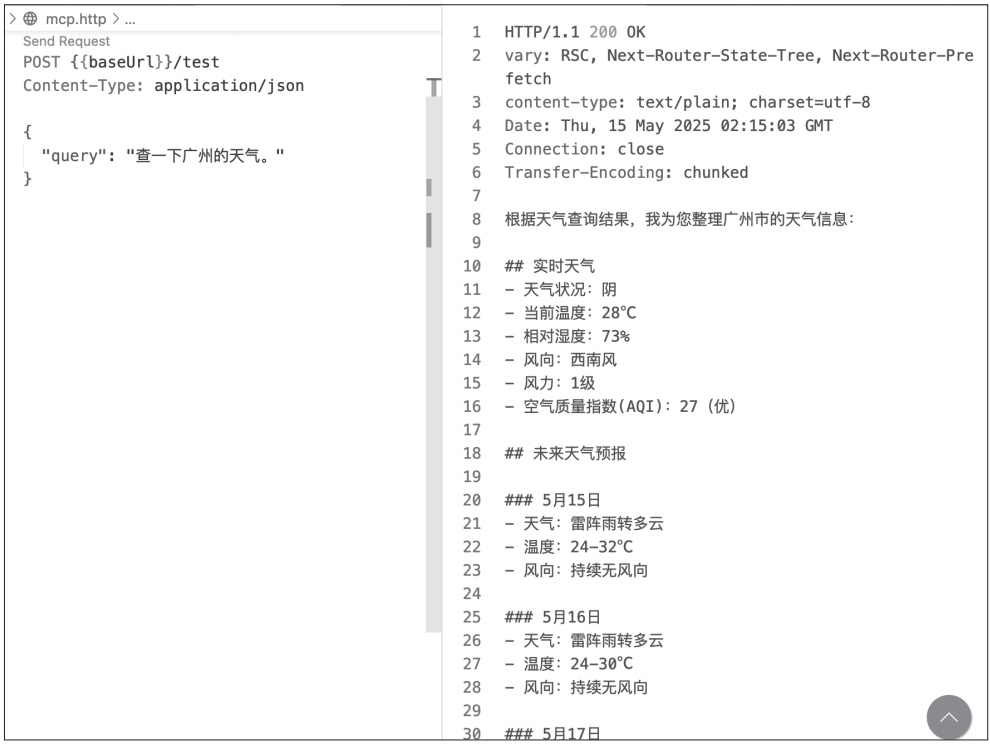


Рис. 4.10. Ответ отладочного интерфейса

Как видно из рисунка, ответ интерфейса соответствует ожидаемому. ChatMCP вызывает внутренний инструмент MCP-сервера для запроса погоды, дополняет данные большой языковой модели информацией о погоде в реальном времени и возвращает пользователю правильный ответ.

Книги для программистов: https://t.me/free_books_for_it

2. Циклический вызов нескольких инструментов для автоматизации рабочего процесса

Реализованная выше логика интерфейса позволяет вызывать инструмент не более одного раза. После передачи результатов первого вызова инструмента в большую языковую модель, даже если она продолжает возвращать информацию о вызове инструментов, ChatMCP больше не вызывает инструменты, а напрямую отвечает пользователю.

Однако в реальной ситуации запросы пользователей иногда бывают сложными и для получения исчерпывающей информации требуют вызова нескольких инструментов. Поэтому можно продолжить оптимизацию интерфейса, добавив логику циклического вызова инструментов, чтобы модели могли автоматически организовывать рабочий процесс на основе переданного списка инструментов и введенного пользователем запроса.

Установим максимальное количество циклов равным 10. Цикл завершается, когда ответ модели не содержит информации о вызове инструментов или достигнуто максимальное количество циклов.

Оптимизированная логика интерфейса приведена ниже.

```
export async function POST(req: Request) {
  let { query } = await req.json();

  // Загрузить конфигурационный файл MCP, получить список MCP-серверов
  // и доступные инструменты
  const configFile = `/Users/idoubi/.cursor/mcp.json`;
  const config = await fs.readFile(configFile, "utf-8");
  const mcpServers = await getMcpServers(config);
  const mcpTools = await getMcpTools(mcpServers);

  let contextMessages: MixContent[] = [];
  let toolResults = "";
  let reply = "";

  // Максимум 10 циклов вывода для нескольких раундов с последовательным
  // вызовом инструментов
  for (let i = 0; i < 10; i++) {
    // Отправить запрос в модель, чтобы она определила, требуется ли
    // вызов инструментов
    const pickToolResult = await chatWithLLM({
      query,
      contextMessages: JSON.stringify(contextMessages),
      tools: JSON.stringify(mcpTools),
      toolResults: toolResults,
    });

    // Разобрать возвращаемый контент модели (текст + информация об инструментах)
    let content = "";
    for await (const chunk of pickToolResult.textStream) {
      content += chunk;
    }
    const mixContents = parseMixContents(content);
```

Книги для программистов: https://t.me/free_books_for_it

```
      contextMessages.push(...mixContents);
      reply += content;

      // Извлечение параметров запроса на вызов инструмента из возвращенного
      // содержимого
      let callToolParams = null;
      for (const mixContent of mixContents) {
        if (mixContent.type === "tool") {
          const tool = mixContent.tool;
          if (
            tool &&
            tool.tool_name &&
            tool.server_name &&
            mcpServers[tool.server_name] &&
            mcpServers[tool.server_name].command
          ) {
            callToolParams = {
              command: mcpServers[tool.server_name].command,
              args: mcpServers[tool.server_name].args || [],
              env: mcpServers[tool.server_name].env || {},
              name: tool.tool_name,
              params: tool.tool_params,
            };
            break;
          }
        }
      }

      // Если требуется вызов инструмента, его выполняют и обновляют
      // результат в контексте следующего раунда
      if (callToolParams) {
        const callToolResult = await callTool(callToolParams);
        toolResults = JSON.stringify(callToolResult);

        reply += `\n\n${toolResults}\n\n`;

        continue;
      }

      // Если вызов инструмента не требуется, цикл прерывается
      break;
    }

    return new Response(reply);
  }
```

Мы используем новый запрос в виде параметра `query` и просматриваем содержимое ответа большой языковой модели через отладочный интерфейс. Результат вывода показан на рис. 4.11.

Как видно из примера, если запрос пользователя предполагает вызов нескольких инструментов, модель автоматически организует рабочий процесс и поочередно возвращает информацию о вызове нескольких инструментов.

Книги для программистов: https://t.me/free_books_for_it

Каждый раз клиент выполняет вызов инструмента и добавляет результат вызова в контекст большой языковой модели, после чего она возвращает информацию о вызове следующего инструмента.



Рис. 4.11. Отладка интерфейса циклического вызова нескольких инструментов

4.2.9. Подведение итогов примера 1

В этом разделе на примере диалогового ИИ-ассистента были рассмотрены основной процесс и ключевая логика разработки клиента MCP. На этом примере была продемонстрирована процедура чтения списка MCP-серверов по настройкам пользователя, получения инструментов, предоставляемых MCP-серверами, настройки подсказок для выбора подходящего инструмента моделью, а также вызова инструментов для дополнения контекста модели.

Книги для программистов: https://t.me/free_books_for_it

В реальной инженерной практике разработка пользовательского MCP-клиента также включает в себя взаимодействие с интерфейсом пользователя, анализ потоковых данных, обработку ошибок при вызове инструментов, повторные попытки в случае сбоев, оптимизацию процессов планирования и т. д. Ввиду ограниченного объема книги эти вопросы здесь не обсуждаются.

После изучения материала этого раздела вы сможете приступить к разработке собственного MCP-клиента.

4.3. Пример 2. Разработка поискового ИИ-агента

В предыдущем разделе мы ознакомились с процессом разработки MCP-клиента типа «диалоговый ассистент», который взаимодействует с большими языковыми моделями посредством настроенного пользователем MCP-сервера. В этом разделе мы перейдем к изучению разработки MCP-клиента агентского типа. Прежде чем приступить к рассмотрению конкретного примера, хочу представить вам разработанный мной в 2024 году продукт – поисковую систему ThinkAny на базе искусственного интеллекта.

В марте 2024 года я выпустил ThinkAny, основная функция которого заключается в следующем: на основе введенного пользователем вопроса сначала осуществляется поиск информации в интернете, а затем отправляется запрос к большой языковой модели для ответа на вопрос пользователя. Основной принцип работы заключается в использовании технологии RAG, которая предоставляет большой языковой модели результаты традиционного поиска в качестве контекста, с целью уменьшить количество «галлюцинаций» модели и предоставить пользователю более точные ответы. Пример вопроса и ответа в ThinkAny представлен на рис. 4.12.

В этом разделе нам предстоит разработать поисковый ИИ-агент с тем же названием, что и ThinkAny, и на этом примере будет продемонстрирован весь процесс разработки MCP-клиента агентского класса.

4.3.1. Цели разработки

Цель этого примера состоит в разработке интеллектуального агента с поддержкой функции ИИ-поиска, способного переформулировать вопросы, выполнять поиск в интернете и считывать содержимое веб-страниц. Все три этапа выполняются с помощью инструмента MCP, а процесс запускается инструментом автоматической компоновки подсказок, что позволяет создать базовые возможности поиска и ответов на вопросы.

Я систематизировал основные процессы продуктов типа поисковых ИИ-систем, как показано на рис. 4.13.

Как упоминалось выше, в основе таких продуктов лежит технология RAG (Retrieval-Augmented Generation, генерация с расширением результатов поиска), где этап поиска обеспечивается за счет вызова интерфейса традиционных поисковых систем. Для получения более подробной информации обычно требуется переформулировать исходный запрос пользователя и прочитать подробное содержание некоторых найденных веб-страниц.

Книги для программистов: https://t.me/free_books_for_it

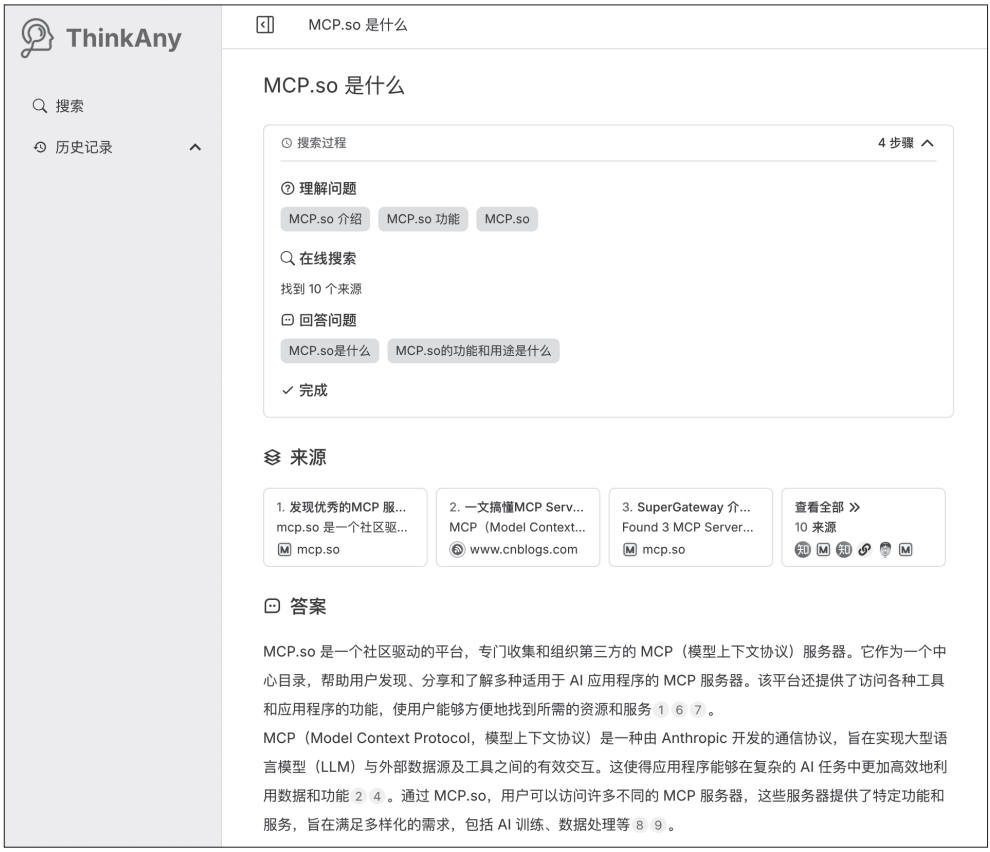


Рис. 4.12. Пример вопроса и ответа в ThinkAny

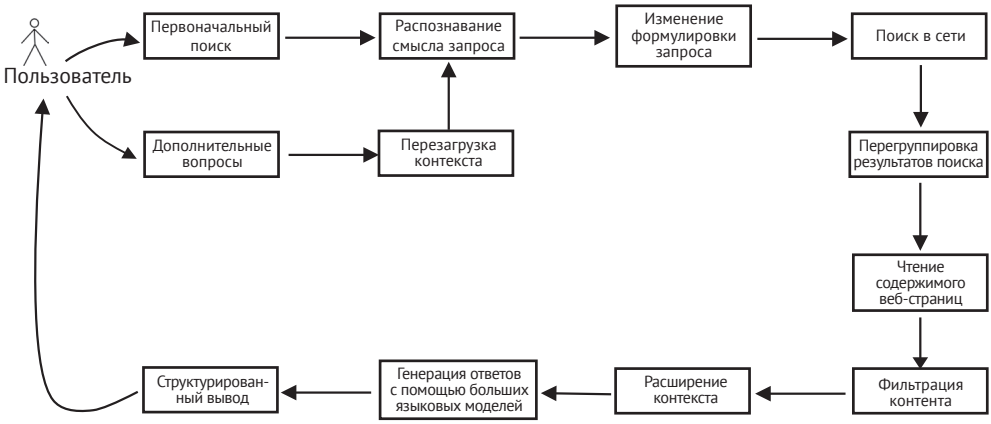


Рис. 4.13. Основной алгоритм работы поисковых ИИ-систем

Книги для программистов: https://t.me/free_books_for_it

Веб-версия ThinkAny полностью поддерживает описанный выше процесс. Ввиду ограниченности объема при рассмотрении разработки поискового ИИ-агента в данном разделе речь пойдет только о реализации трех основных этапов:

- изменение формулировки запроса (для генерации ключевых слов, более соответствующих поисковым традициям, что позволяет получить более точные результаты, соответствующие пожеланиям пользователя);
- сетевой поиск (основа RAG, позволяет получить ссылки и аннотации на ряд релевантных веб-страниц);
- чтение содержимого веб-страниц (дальнейшее извлечение подробной информации из целевых веб-страниц для обогащения последующих сгенерированных ответов).

4.3.2. Подготовительные работы

Сначала выберем подходящие MCP-серверы для перечисленных выше трех этапов и проведем их настройку.

1. Получение списка инструментов MCP-сервера для изменения формулировок вопросов

Вопрос, введенный пользователем, может представлять собой как отдельное ключевое слово, так и целое предложение. Для получения более качественных результатов поиска зачастую необходимо изменить формулировку исходного вопроса пользователя, а затем использовать измененный текст в качестве ключевого слова для поиска в интернете.

В данном случае выбран MCP-сервер под названием `query-rewrite-mcp`, который предоставляет инструмент `rewrite_query` для изменения формулировок введенных пользователем вопросов.

Настройте рабочие параметры этого MCP-сервера, получите список инструментов, определенных в нем, с помощью функции `listTools` и проведите отладку результатов вывода, как показано на рис. 4.14.



Функция `listTools`, используемая в логике отладки, была подробно описана в разделе 4.2, поэтому здесь нет необходимости повторяться. Для отладки этого MCP-сервера необходимо предоставить `OPENROUTER_API_KEY`, который можно получить на платформе OpenRouter. `REWRITE_MODEL` – это модель, используемая для изменения формулировки запросов. Разработчики могут выбрать ее самостоятельно, но рекомендуется использовать модели серии Claude, которые обеспечивают лучший результат изменения формулировок запросов.

2. Вызов инструмента изменения формулировки запросов

Исходя из списка инструментов, выведенного MCP-сервером для изменения формулировки запросов, с помощью функции `callTool` вызывается инструмент `rewrite_query`, а результаты отладки выводятся на экран, как показано на рис. 4.15.

Книги для программистов: https://t.me/free_books_for_it

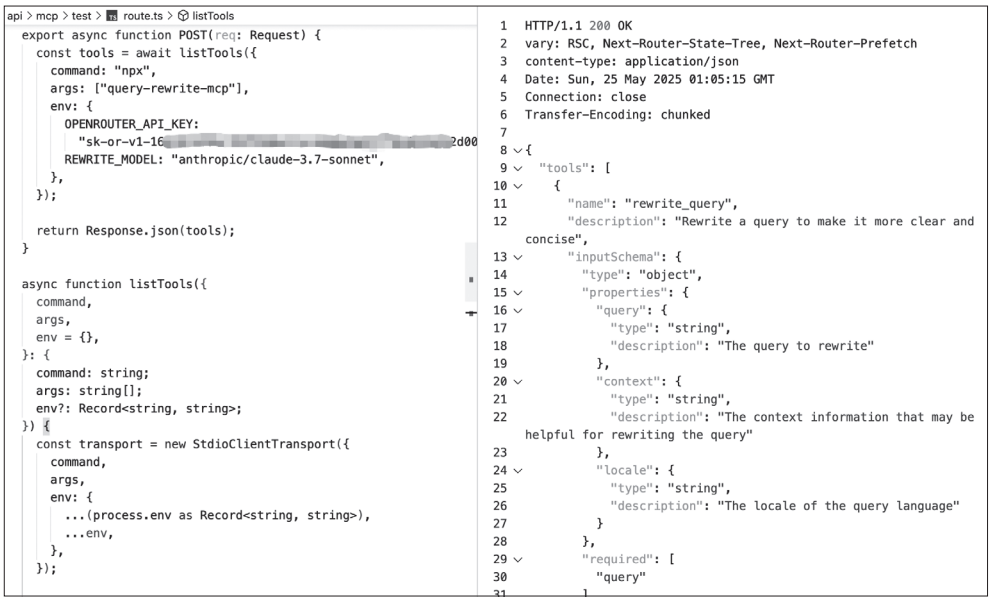


Рис. 4.14. Отладка списка инструментов query-rewrite-mcp

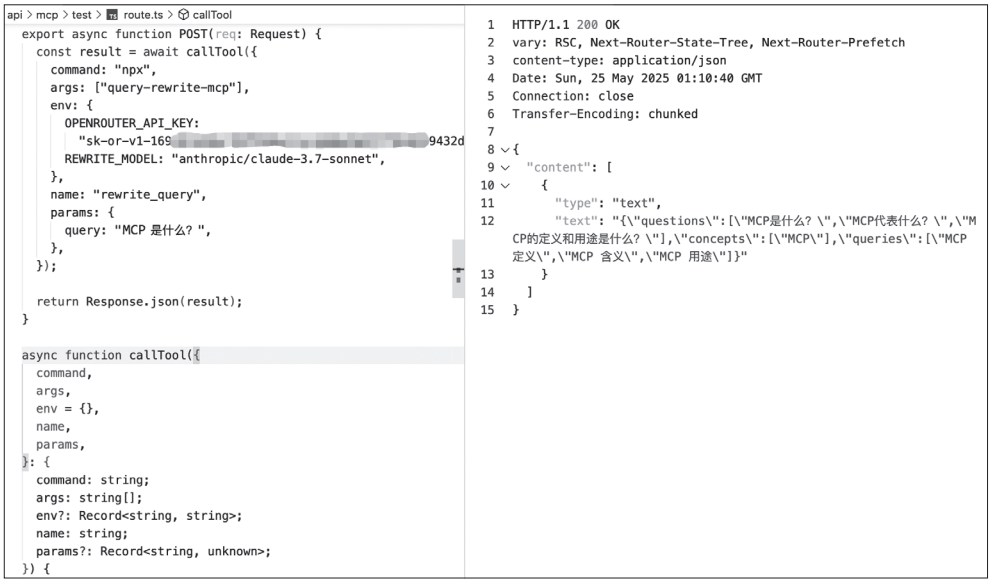


Рис. 4.15. Отладка инструмента rewrite_query

Из приведенного выше отладочного вывода видно, что инструмент `rewrite_query` может преобразовать запрос в более насыщенный контент. Например, так выглядит исходный запрос:

Книги для программистов: https://t.me/free_books_for_it

```
{
  "query": "Что такое MCP? "
}
```

А так выглядит преобразованный контент:

```
{
   "questions": ["Что такое MCP? ", "Что означает MCP? ", "Каковы определение и назначение MCP? "],
   "concepts»: ["MCP"],
   "queries": ["определение MCP", "значение MCP", "назначение MCP"]
}
```

Значение трех полей в приведенном выше коде объясняется следующим образом:

- `questions`: расширенные вопросы, сгенерированные на основе исходного запроса, помогающие большой языковой модели построить ответ по разным критериям;
- `concepts`: соответствующие концепции или термины, извлеченные из исходного запроса, предназначенные для поиска в энциклопедических статьях;
- `queries`: запросы, сгенерированные на основе исходного запроса, предназначенные для поиска в поисковых системах.

3. Получение списка инструментов MCP-сервера для поиска в интернете

В данном случае выбран MCP-сервер под названием `serper-mcp-server`. Он предоставляет инструмент `google_search`, позволяющий имитировать запрос в поисковой системе Google и получать результаты поиска.

Настройте рабочие параметры этого MCP-сервера, получите список определенных внутри него инструментов с помощью функции `listTools`, а затем выполните отладку результатов вывода, как показано на рис. 4.16.



Рис. 4.16. Отладка списка инструментов serper-mcp-server

Книги для программистов: https://t.me/free_books_for_it



Для отладки этого MCP-сервера необходимо ввести ключ SERPER_API_KEY, который можно получить на платформе Serper.

4. Вызов инструмента сетевого поиска

На основании списка инструментов, выведенного MCP-сервером сетевого поиска, вызовите инструмент google_search с помощью функции callTool и выполните отладку результатов вывода, как показано на рис. 4.17.



Рис. 4.17. Отладка инструмента google_search

Из приведенного выше отладочного вывода следует, что инструмент google_search может имитировать запрос в поисковой системе Google и получать результаты поиска. Например, так выглядят введенные параметры запроса:

```
{
  "q": "mcp"
}
```

Для них этот инструмент возвращает список результатов поиска, каждый из которых содержит заголовок, ссылку, описание, источник и другую информацию. Пример одного из результатов поиска приведен ниже.

```
{
  "title": "Model Context Protocol: Introduction",
  "link": "https://modelcontextprotocol.io/introduction",
   "snippet": "MCP - это открытый протокол, стандартизирующий способ предоставления приложениями
  контекста для LLM. Представьте себе MCP как порт USB-C для ИИ-приложений
  }
```

Книги для программистов: https://t.me/free_books_for_it

5. Получение списка инструментов MCP-сервера для чтения содержимого веб-страниц

В данном случае выбран MCP-сервер под названием `jina-mcp-tools`. Он предоставляет инструмент `jina_reader`, способный считывать содержимое веб-страниц. Настройте параметры работы этого сервера MCP, получите список определенных внутри него инструментов с помощью функции `listTools` и проведите отладку результатов вывода, как показано на рис. 4.18.



Рис. 4.18. Список инструментов jina-mcp-tools

Для отладки этого MCP-сервера необходимо ввести ключ `JINA_API_KEY`, который можно получить на платформе Jina.

6. Вызов инструмента для чтения контента веб-страниц

Исходя из выведенного MCP-сервером списка инструментов для чтения контента веб-страниц вызовите инструмент `jina_reader` с помощью функции `callTool` и выполните отладку результатов вывода, как показано на рис. 4.19.
Из результатов отладки следует, что этот инструмент принимает URL-адрес веб-страницы и по умолчанию возвращает прочитанное содержимое в формате Markdown.

На этом подготовка всех MCP-серверов и определенных в них инструментов, необходимых для реализации поискового ИИ-агента, завершена. Теперь можно сосредоточиться на создании логики взаимодействия поискового ИИ-агента с большой языковой моделью.

Книги для программистов: https://t.me/free_books_for_it



Рис. 4.19. Отладка инструмента jina_reader

4.3.3. Определение конфигурации MCP-сервера

В отличие от предыдущего примера, где настраиваемый пользователем MCP-сервер считывался из конфигурационного файла, в данном случае MCP-серверы и инструменты, необходимые для поискового ИИ-агента, имеют фиксированные свойства, поэтому их можно определить непосредственно в коде программы. Конфигурация MCP-сервера выглядит следующим образом:

```
{
  "mcpServers": {
    "query-rewrite-mcp": {
      "command": "npx",
      "args": ["query-rewrite-mcp"],
      "env": {
        "OPENROUTER_API_KEY": "xxx",
        "REWRITE_MODEL": "anthropic/claude-3.7-sonnet"
      }
    },
    "serper-search-mcp": {
      "command": "uvx",
      "args": ["serper-mcp-server"],
      "env": {
        "SERPER_API_KEY": "xxx"
      }
    },
    "jina-reader-mcp": {
      "command": "npx",
      "args": ["jina-mcp-tools"],
```

Книги для программистов: https://t.me/free_books_for_it

```
        "env": {
          "JINA_API_KEY": "xxx"
        }
      }
    }
  }
```



Конфигурационные данные в приведенном выше коде были обезличены. Для параметров, используемых каждым MCP-сервером, использованы заполнители, которые можно изменить в соответствии с конкретными условиями.

4.3.4. Фильтрация списка инструментов MCP-сервера

В отличие от предыдущего примера, где все инструменты, заданные в конфигурации MCP-сервера пользователем, передавались в модель, в данном случае инструменты поискового ИИ-агента фиксированы. Поэтому можно определить список инструментов, отфильтровать полученный список инструментов MCP-сервера и выбрать только те инструменты, которые необходимы для реализации поискового ИИ-агента.

Логика фильтрации списка инструментов MCP-сервера приведена ниже.

```
const mcpServers = await getMcpServers(config);
const mcpTools = await getMcpTools(mcpServers);

const filterTools = ["rewrite_query", "google_search", "jina_reader"];
const filteredTools = mcpTools.filter((tool) =>
  filterTools.includes(tool.name)
);
```



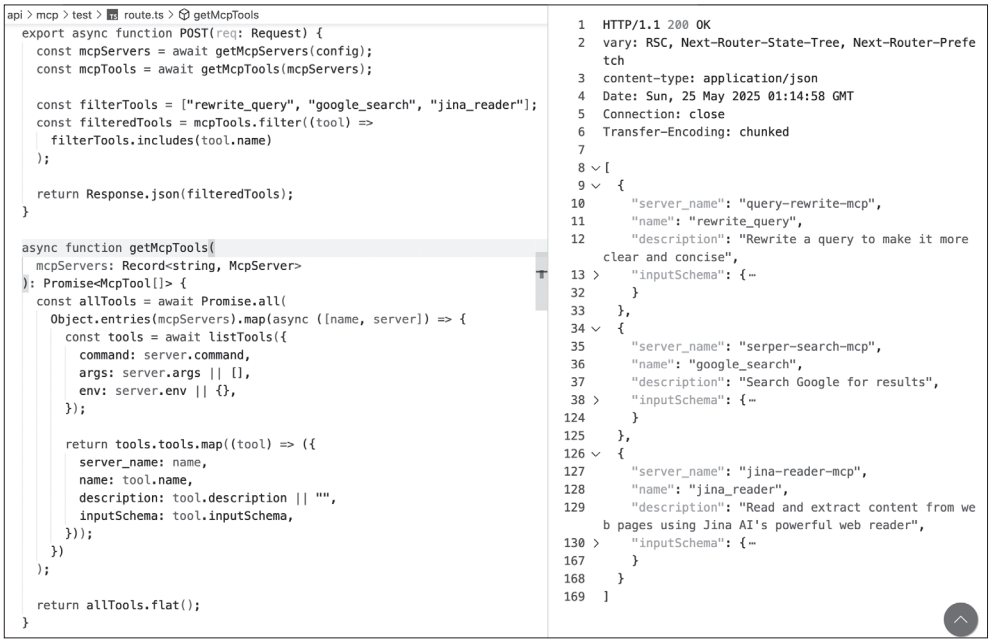
В этой логике `config` обозначает определенную в программе конфигурацию MCP-сервера. Что касается функций `getMcpServers` и `getMcpTools`, их алгоритм выполнения рассматривался в подробностях в разделе 4.2, поэтому здесь повторяться нет необходимости.

Вызываем интерфейс для получения отфильтрованного списка инструментов и выводим результаты отладки, как показано на рис. 4.20.

Из результатов отладки следует, что интерфейс получил три инструмента, необходимых для поискового ИИ-агента, и соответствующие им MCP-серверы:

- инструмент `rewrite_query`, соответствующий MCP-серверу `query-rewrite-mcp`;
- инструмент `google_search`, соответствующий MCP-серверу `serper-search-mcp`;
- инструмент `jina_reader`, соответствующий серверу MCP `jina-reader-mcp`.

Книги для программистов: https://t.me/free_books_for_it



```
api > mcp > test > route.ts > getMcpTools
export async function POST(req: Request) {
  const mcpServers = await getMcpServers(config);
  const mcpTools = await getMcpTools(mcpServers);

  const filterTools = ["rewrite_query", "google_search", "jina_reader"];
  const filteredTools = mcpTools.filter((tool) =>
    filterTools.includes(tool.name)
  );

  return Response.json(filteredTools);
}

async function getMcpTools(
  mcpServers: Record<string, McpServer>
): Promise<McpTool[]> {
  const allTools = await Promise.all(
    Object.entries(mcpServers).map(async ([name, server]) => {
      const tools = await listTools({
        command: server.command,
        args: server.args || [],
        env: server.env || {},
      });

      return tools.tools.map((tool) => ({
        server_name: name,
        name: tool.name,
        description: tool.description || "",
        inputSchema: tool.inputSchema,
      }));
    })
  );

  return allTools.flat();
}
```

```
HTTP/1.1 200 OK
vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
content-type: application/json
Date: Sun, 25 May 2025 01:14:58 GMT
Connection: close
Transfer-Encoding: chunked

[
  {
    "server_name": "query-rewrite-mcp",
    "name": "rewrite_query",
    "description": "Rewrite a query to make it more clear and concise",
    "inputSchema": {…
    }
  },
  {
    "server_name": "serper-search-mcp",
    "name": "google_search",
    "description": "Search Google for results",
    "inputSchema": {…
    }
  },
  {
    "server_name": "jina-reader-mcp",
    "name": "jina_reader",
    "description": "Read and extract content from web pages using Jina AI's powerful web reader",
    "inputSchema": {…
    }
  }
]
```

Рис. 4.20. Получение отфильтрованного списка инструментов MCP-сервера

4.3.5. Организация рабочего процесса с помощью подсказок

Чтобы большая языковая модель могла распределять инструменты в соответствии со спецификациями поисковой ИИ-системы, необходимо настроить системные подсказки и организовать рабочий процесс.

Для этого примера была разработана следующая системная подсказка (промпт).

```
Ты - ThinkAny, интеллектуальный поисковый агент, разработанный ThinkAny AI.

# Основной рабочий процесс

## Принципы выполнения

- Проверять результаты работы уже используемых инструментов в контексте, чтобы избежать повторного выбора инструментов
- Выполнять в следующем порядке: изменение формулировки вопроса → поиск в интернете → чтение содержимого веб-страницы → генерация ответа
- Выбирать только один инструмент за одно обращение, возвращать информацию о вызове инструмента

## Этап 1: Изменение формулировки вопроса

**Условие**: выполняется, только если в контексте отсутствуют результаты rewrite_query
**Инструмент**: rewrite_query
**Цель**: преобразовать вопрос пользователя в структурированные параметры
```

Книги для программистов: https://t.me/free_books_for_it

```
- Сформировать конкретный список вопросов (questions)
- Извлечь ключевые концепции (concepts)
- Оптимизировать поисковые запросы (queries)

## Этап 2: Поиск в интернете

**Инструмент**: google_search (можно вызывать несколько раз, максимум 3 раза)
**Стратегия**:

- В первую очередь использовать queries и concepts из результатов rewrite_query
- При каждом поиске извлекать 10 записей, всего не более 30
- Сортировка по релевантности, предпочтение авторитетным сайтам

## Этап 3: Чтение содержимого веб-страниц

**Инструмент**: jina_reader (можно вызывать несколько раз)
**Требования**:

- Использование формата Markdown для получения содержимого
- Приоритетное чтение наиболее релевантных URL
- Фильтрация некачественного содержимого (страницы проверки, страницы ошибок и т. д.)
- Обеспечить получение подробного содержания 2-3 веб-страниц

## Формирование окончательного ответа

**Требования**:

- Дать прямой ответ на исходный вопрос пользователя
- Использовать нумерованные ссылки [1], [2] и т. д.
- В конце предоставить полный список ссылок
- Использовать понятный формат Markdown

# Формат вызова инструмента

<<tool-start>>
{
"server_name": "Имя сервера инструмента",
"tool_name": "Название инструмента",
"tool_params": {
"имя_параметра": "значение_параметра"
}
}
<<tool-end>>

# Параметры выполнения

- Запрос пользователя: USER_QUERY={USER_QUERY}
- Контекст диалога: CONTEXT_MESSAGES={CONTEXT_MESSAGES}
- Результаты работы инструмента: PREVIOUS_TOOL_RESULTS={PREVIOUS_TOOL_RESULTS}
- Список доступных инструментов: AVAILABLE_TOOLS={AVAILABLE_TOOLS}
```

Книги для программистов: https://t.me/free_books_for_it

Основная идея этого промпта заключается в определении трех шагов, каждый из которых содержит инструмент для вызова. Это дает возможность большой языковой модели организовать рабочий процесс в соответствии с определением и вернуть информацию о вызове инструмента для каждого шага.

Дизайн этого промпта представляет собой типичный пример проектирования подсказок для интеллектуальных агентов. Он определяет четкие правила выполнения, поэтапный процесс задачи, формат вызова и требования к входным параметрам для ИИ-агента, способного вызывать инструменты (например, ThinkAny). Хорошая подсказка для ИИ-агента – это не просто описание функций, а скорее что-то вроде упрощенной «инструкции по эксплуатации»: с помощью правил + состояний + определений входов/выходов + формата исполняемого файла она превращает поведение ИИ-агента в понятный и контролируемый процесс.

4.3.6. Создание кода для взаимодействия с большой языковой моделью

Подобно предыдущему примеру, процесс взаимодействия поискового ИИ-агента с большой языковой моделью включает следующие шаги.

1. Запрос у большой языковой модели на выбор инструмента.
2. Анализ информации об инструменте, полученной в ответе от модели.
3. Вызов инструмента.
4. Повторение шагов 1–3 до тех пор, пока модель не перестанет возвращать информацию о вызове инструмента или пока не будет достигнуто максимально допустимое количество циклов.

В предыдущем примере уже была подробно описана конкретная логика реализации этих шагов, поэтому здесь нет необходимости повторяться. Вместо этого приведем код, реализующий логику взаимодействия поискового ИИ-агента с большой языковой моделью, как показано ниже.

```
export async function POST(req: Request) {
  const { query } = await req.json();

  // Чтение системного запроса
  const mcpPrompt = await fs.readFile(
    path.join(process.cwd(), "app/api/mcp/test/search.md"),
    "utf-8"
  );

  // Чтение данных с MCP-сервера и информации об инструментах
  const mcpServers = await getMcpServers(config);
  const mcpTools = await getMcpTools(mcpServers);

  const filterTools = ["rewrite_query", "google_search", "jina_reader"];
  const filteredTools = mcpTools.filter((tool) =>
    filterTools.includes(tool.name)
  );

  let contextMessages: MixContent[] = [];
  let toolResults = "";
  let reply = "";
```

Книги для программистов: https://t.me/free_books_for_it

```
// Максимум 10 циклов для последовательного вызова инструментов
for (let i = 0; i < 10; i++) {
  // Запрос к большой языковой модели на выбор инструмента
  const pickToolResult = await chatWithLLM({
    mcpPrompt,
    query,
    contextMessages: JSON.stringify(contextMessages),
    tools: JSON.stringify(filteredTools),
    toolResults: toolResults,
  });

  // Анализ возвращаемого моделью контента (текст + информация об инструменте)
  let content = "";
  for await (const chunk of pickToolResult.textStream) {
    content += chunk;
  }
  const mixContents = parseMixContents(content);
  contextMessages.push(...mixContents);
  reply += content;

  // Извлечение информации об инструменте из смешанного контента
  let callToolParams = null;
  for (const mixContent of mixContents) {
    if (mixContent.type === "tool") {
      const tool = mixContent.tool;
      if (
        tool &&
        tool.tool_name &&
        tool.server_name &&
        mcpServers[tool.server_name] &&
        mcpServers[tool.server_name].command
      ) {
        callToolParams = {
          command: mcpServers[tool.server_name].command,
          args: mcpServers[tool.server_name].args || [],
          env: mcpServers[tool.server_name].env || {},
          name: tool.tool_name,
          params: tool.tool_params,
        };
        break;
      }
    }
  }

  // Если требуется вызвать инструмент, выполняется вызов, а результат обнов-
     ляется в контексте следующего раунда
  if (callToolParams) {
    const callToolResult = await callTool(callToolParams);
    toolResults = JSON.stringify(callToolResult);
    reply += `\n\n${toolResults}\n\n`;

    continue;
  }
```

Книги для программистов: https://t.me/free_books_for_it

```
      // Если вызов инструмента не требуется, цикл прерывается
      break;
    }

    return new Response(reply);
  }
```

По сравнению с логикой взаимодействия ИИ-ассистента с большой языковой моделью логика взаимодействия поискового ИИ-агента с моделью отличается главным образом содержанием передаваемого списка инструментов и системных подсказок.

4.3.7. Отладка логики интерфейса

Проведите отладку описанного выше интерфейса поискового ИИ-агента: введите запрос и выведите результаты отладки, как показано на рис. 4.21.

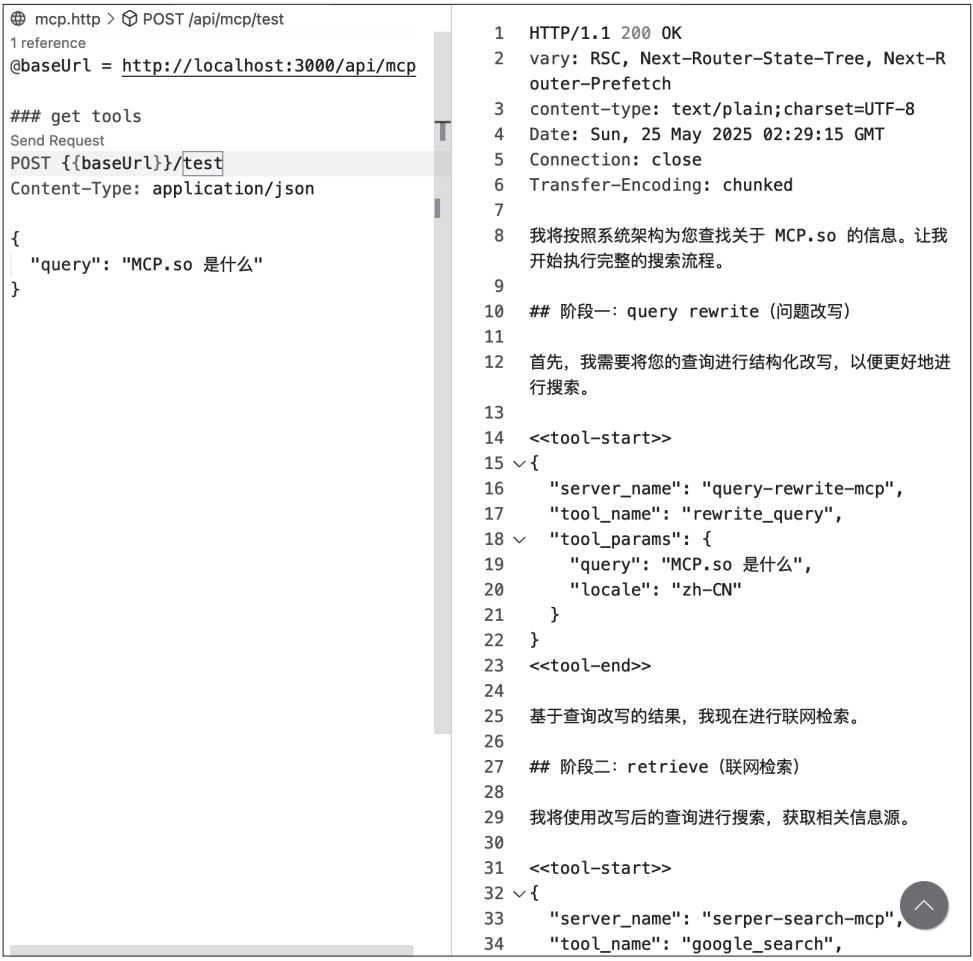


Рис. 4.21. Отладка интерфейса поискового ИИ-агента, просмотр информации о вызове инструментов

Книги для программистов: https://t.me/free_books_for_it

Как видно из рисунка, модель в соответствии с рабочим процессом, заданным подсказкой, последовательно вызвала три инструмента, и окончательный ответ показан на рис. 4.22.



Рис. 4.22. Отладка интерфейса поискового ИИ-агента, просмотр результатов ответа

По окончательному результату ответа видно, что он практически совпадает с ответом, полученным в веб-версии ThinkAny.

На этом разработка бизнес-логики поискового ИИ-агента завершена, а основные функции продукта типа поисковой системы на базе ИИ реализованы.

4.3.8. Тестирование в Cursor

В предыдущих разделах мы рассмотрели основную логику внедрения поискового ИИ-агента и настроили интерфейс отладки для получения окончательного ответа. Однако из-за отсутствия пользовательского интерфейса мы не можем показать интерактивные возможности поискового ИИ-агента в наглядной форме. Мы можем продолжить тестирование в Cursor и оценить эффект работы поискового ИИ-агента с помощью его интерфейса.

Книги для программистов: https://t.me/free_books_for_it

В конфигурационном файле MCP для Cursor необходимо указать настройки встроенного MCP-сервера поискового ИИ-агента. Используя встроенный системный триггер search.md, отправьте тот же самый запрос и просмотрите информацию о вызове инструмента, как показано на рис. 4.23.

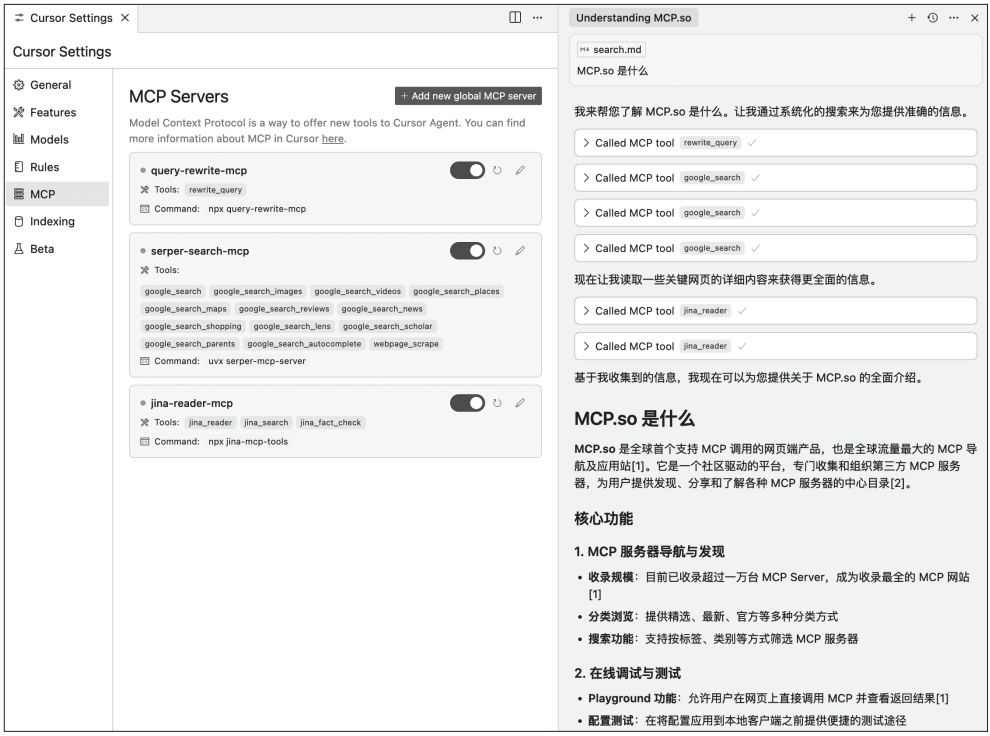


Рис. 4.23. Тестирование вызова инструментов в Cursor

Как видно из рисунка, Cursor, выступая в качестве MCP-клиента, взаимодействует с большой языковой моделью в соответствии с рабочим процессом, определенным промптом: сначала выполняется одна операция изменения формулировки вопроса, затем три операции поиска в интернете, в конце считывается содержимое двух веб-страниц, и в итоге выдается точный ответ на заданный вопрос.

Удалив системный промпт и протестировав тот же самый вопрос, можно увидеть, что Cursor не вызвал MCP-сервер для изменения формулировки вопроса и чтения содержимого веб-страниц, в результате чего точность окончательного ответа оказалась невысокой, как показано на рис. 4.24.

Благодаря двум приведенным выше тестам мы получили представление о процессе взаимодействия пользователя с поисковым ИИ-агентом, оснащенным пользовательским интерфейсом Cursor, проверили влияние подсказок и инструментов на результаты ответов больших языковых моделей. В результате тестирование и проверка поискового ИИ-агента, разработанного в рамках этого раздела, успешно завершены.

Книги для программистов: https://t.me/free_books_for_it

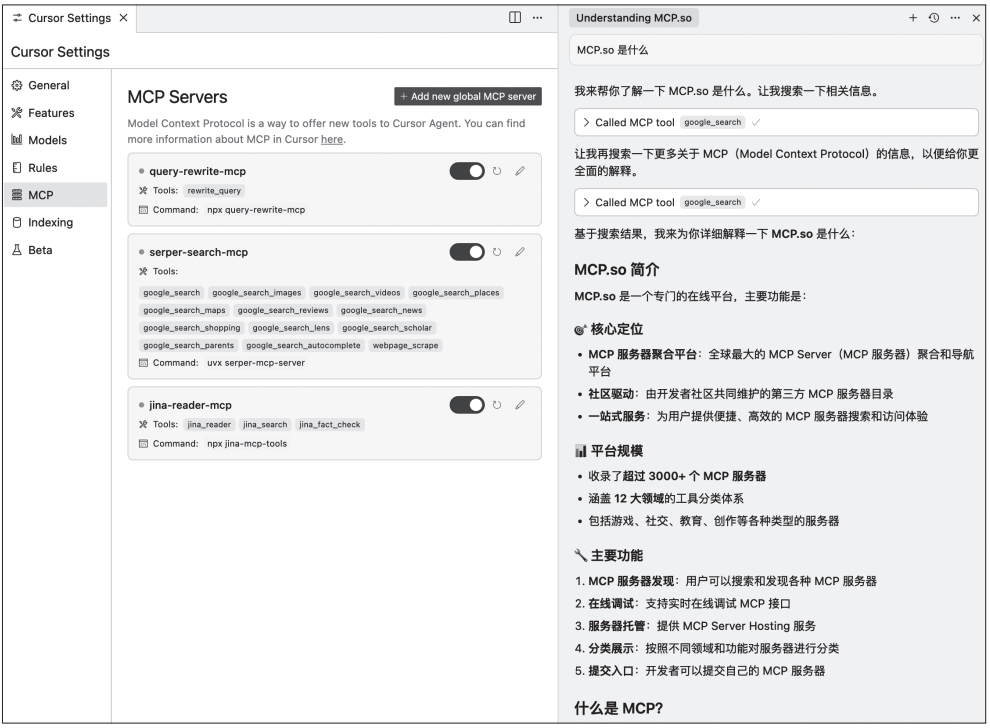


Рис. 4.24. Тестирование вызова инструментов в Cursor в сценарии без системного промпта

4.3.9. Подведение итогов примера 2

В этом разделе на примере поискового ИИ-агента были рассмотрены основные процессы и ключевые этапы разработки MCP-клиента агентского типа.

После изучения материала этого раздела вы сможете приступить к созданию собственного агентского продукта. В процессе разработки агента можно найти подходящие MCP-серверы и инструменты в магазине MCP-приложений, сократить цикл разработки агента за счет повторного использования инструментов и сосредоточить усилия на реализации основной бизнес-логики.

Однако типичные формы взаимодействия с продуктами класса интеллектуальных агентов обычно зависят от использования виртуальных машин для отображения процесса выполнения каждого шага. Это включает в себя кластеры контейнеров, виртуальные машины, взаимодействие с интерфейсом и т. д. Подобные темы выходят за рамки данной книги, поэтому читателям рекомендуется изучить их самостоятельно.

4.4. Заключение

В этой главе в основном рассмотрены процессы и основные принципы разработки MCP-клиентов, а также на примере двух реальных кейсов описаны два типа основной логики реализации MCP-клиентов. Изучив эту главу, читатель сможет понять, как при разработке универсальных диалоговых помощников

Книги для программистов: https://t.me/free_books_for_it

на базе ИИ и продуктов-агентов использовать MCP-сервер и предоставляемые им инструменты для получения необходимых функциональных возможностей. Это помогает повысить эффективность разработки и быстро завершить создание основных функций продукта по принципу сборки конструктора.

Изучив эту главу, читатель также может оценить огромную значимость MCP в плане стандартов интеграции. При разработке различных типов MCP-клиентов мы не тратили много усилий на функциональную интеграцию, а благодаря управлению большими языковыми моделями и в соответствии с коммуникационными стандартами MCP быстро завершили интеграцию всех необходимых функций.

Книги для программистов: https://t.me/free_books_for_it

Глава 5

Классические сценарии применения MCP

В главе 1, где мы рассматривали принцип работы MCP, и в главе 3, где мы разобрали примеры разработки MCP-сервера, мы использовали Claude и Cursor в качестве клиентов больших языковых моделей, чтобы продемонстрировать процесс настройки и использования MCP-сервера.

В этой главе будет рассказано о том, как настроить MCP-сервер для клиентов больших языковых моделей, отличных от Claude и Cursor. На примере двух практических примеров будет показано, как с помощью комбинации нескольких MCP-серверов можно решать типичные задачи в классических сценариях.

5.1. Использование MCP-сервера с популярными клиентами

В этом разделе будут представлены некоторые популярные клиенты больших языковых моделей, получившие высокие оценки пользователей. Также будет рассказано об использовании MCP-сервера с этими клиентами.

5.1.1. Использование MCP-сервера с Cline

Cline – это расширение для VS Code (Visual Studio Code) с открытым исходным кодом, созданное специально для разработчиков с целью бесшовной интеграции больших языковых моделей в локальную среду разработки. С помощью Cline пользователи могут напрямую взаимодействовать с большими языковыми моделями прямо в редакторе VS Code, выполняя такие операции, как генерация и интерпретация кода, без необходимости переключаться в браузер или другие клиентские приложения. Это значительно повышает эффективность разработки и удобство рабочего процесса.

Установите расширение Cline в редакторе VS Code, перейдите на страницу настроек MCP в Cline, добавьте конфигурацию MCP-сервера, после чего отобразится список установленных MCP-серверов и содержащихся в них инструментов, как показано на рис. 5.1.

Введите вопрос в диалоговом окне Cline, например:

```
https://mcp.so - какова основная тематика этого сайта?
```

Как видно из рис. 5.2, Cline подключился к настроенному MCP-серверу, запросил инструмент выбора модели, затем вызвал этот инструмент и выполнил указанную пользователем задачу.

Книги для программистов: https://t.me/free_books_for_it

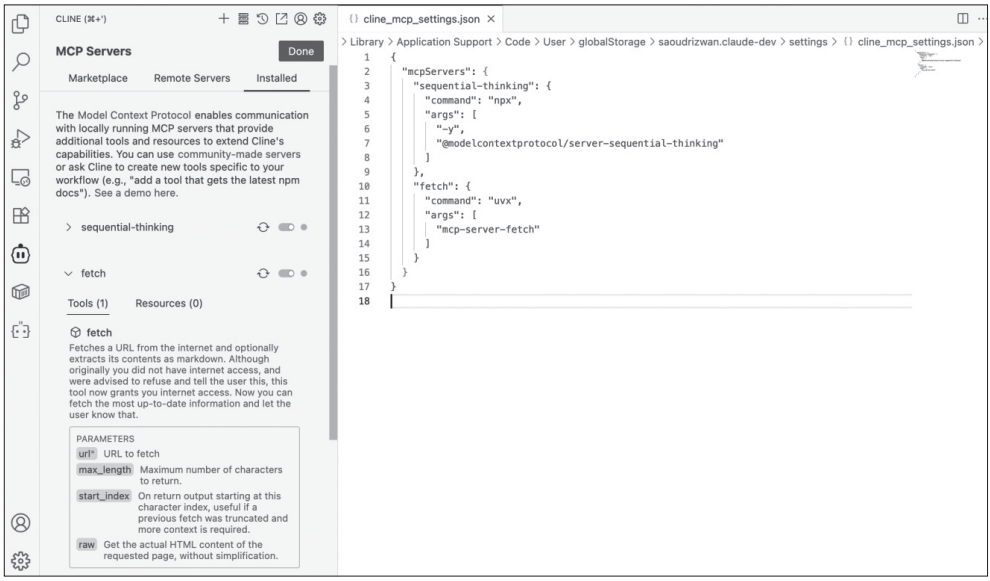


Рис. 5.1. Настройка MCP-сервера в Cline

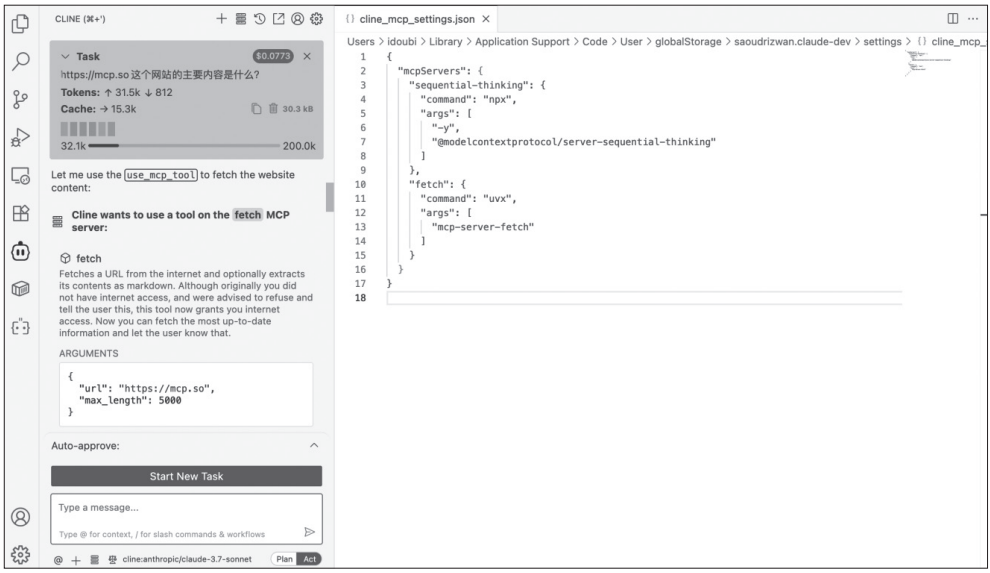


Рис. 5.2. Использование MCP-сервера в Cline

5.1.2. Использование MCP-сервера с ChatWise

ChatWise – это клиент для больших языковых моделей, созданный независимыми разработчиками из Китая. Он отличается широкими функциональными возможностями и комфортным интерфейсом, благодаря чему пользуется большой популярностью среди разработчиков. Он поддерживает множество функций, включая сетевой поиск, рисование, Artifacts и MCP, а также предоставляет

Книги для программистов: https://t.me/free_books_for_it

различные способы интеграции и гибкие настройки управления, позволяющие разработчикам адаптировать функции инструмента под свои нужды. Благодаря простому и понятному интерфейсу с интуитивно понятным управлением с этим инструментом сможет быстро освоиться даже новичок. Следует отметить, что ChatWise не является полностью бесплатным: для использования его расширенных функций или MCP необходимо приобрести соответствующую лицензию. Тем не менее благодаря мощным функциям и отличному соотношению цены и качества ChatWise по-прежнему считается выгодным вложением средств.

Загрузите и установите ПО ChatWise с официального сайта, перейдите в центр настроек, где можно быстро настроить и управлять MCP-сервером, а также просматривать список инструментов на нем, как показано на рис. 5.3.

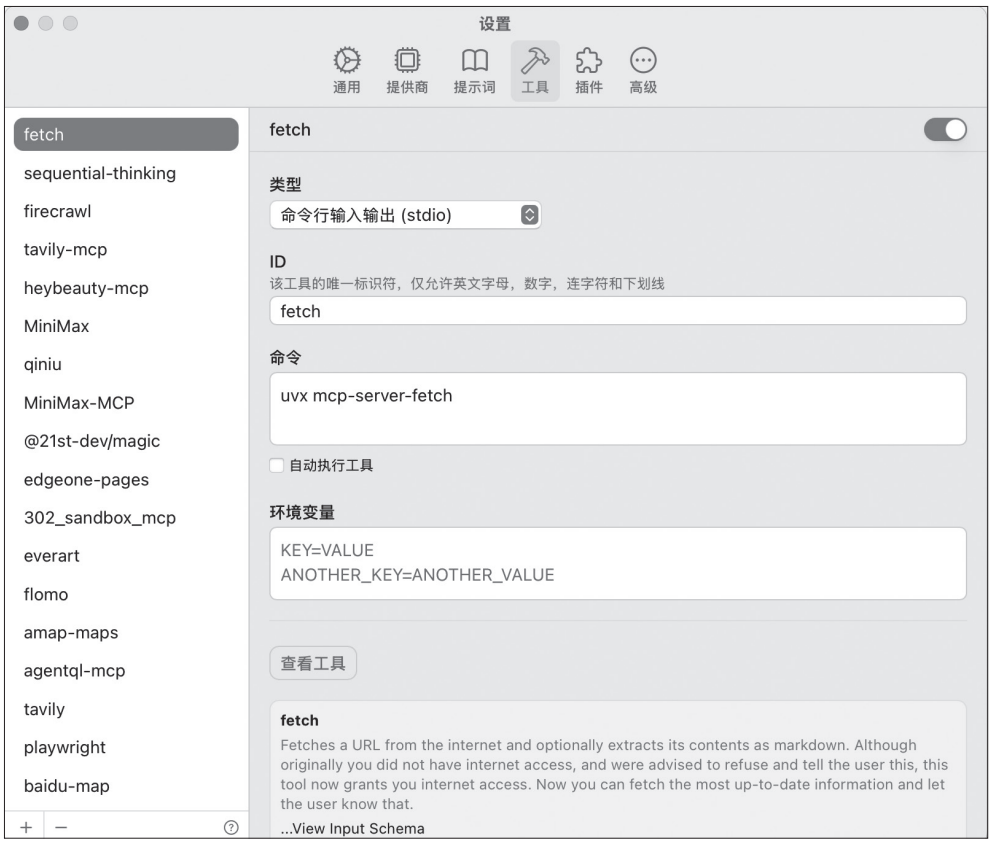


Рис. 5.3. Настройка сервера MCP в ChatWise

Введите запрос в панели диалога ChatWise, например:

```
Проверить маршрут поездки на автомобиле из Гуанчжоу в Шанхай, визуализировать план поездки
```

Как видно из рисунка, ChatWise загрузил настроенный MCP-сервер, запросил инструмент выбора большой модели, затем несколько раз вызвал инструмент и в конечном итоге выполнил задачу, отправленную пользователем, как показано на рис. 5.4.

Книги для программистов: https://t.me/free_books_for_it



Рис. 5.4. Использование MCP-сервера в ChatWise

5.1.3. Использование MCP-сервера с Cherry Studio

Cherry Studio – это клиент для больших языковых моделей, разработанный китайской командой. Он поддерживает такие функции, как переключение моделей, создание графиков, применение подсказок и MCP. Он отличается привлекательным интерфейсом, широкими функциональными возможностями, поддержкой нескольких платформ, а также открытым исходным кодом. Cherry Studio пользуется отличной репутацией среди пользователей.

Скачав и установив Cherry Studio с официального сайта, перейдите в центр настроек, где можно легко настроить и управлять MCP-сервером, как показано на рис. 5.5.

Книги для программистов: https://t.me/free_books_for_it

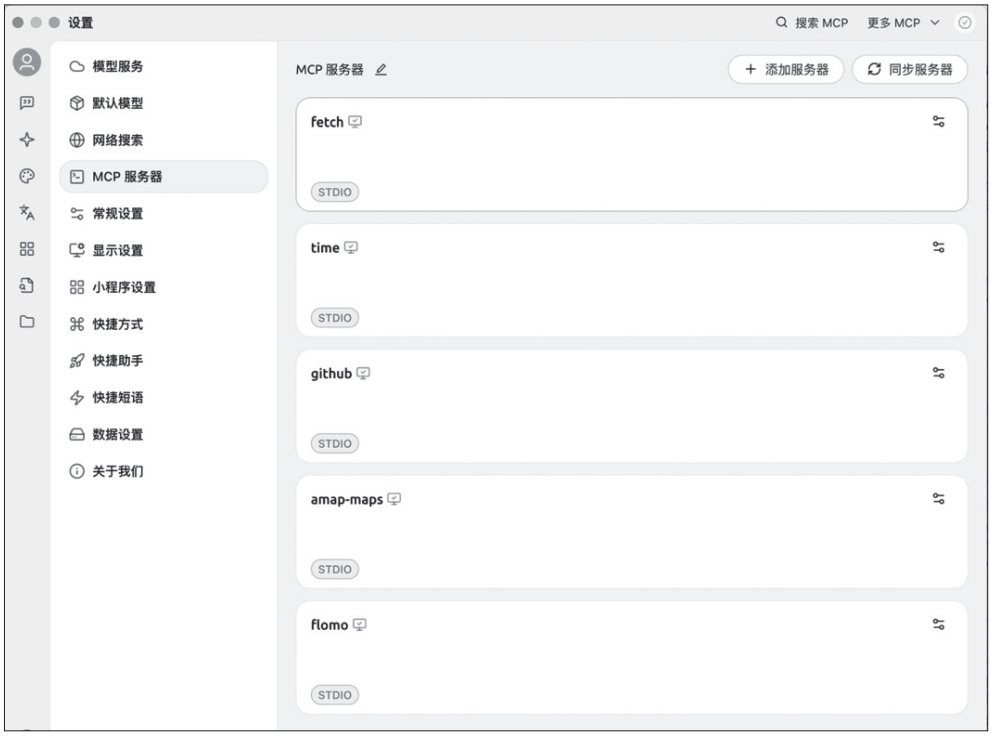


Рис. 5.5. Настройка MCP-сервера в Cherry Studio

Введите вопрос в панели диалога Cherry Studio, например:

```
Расскажи об основной идее этой статьи:
https://www.anthropic.com/news/model-context-protocol
```

Как видно из рис. 5.6, программа Cherry Studio загрузила настроенный MCP-сервер, запросила инструмент выбора большой языковой модели, затем запустила этот инструмент и выполнила указанную пользователем задачу.

5.1.4. Использование MCP-сервера с DeepChat

DeepChat – это открытый клиент для больших языковых моделей, разработанный сообществом ThinkInAI (ранее GoCN). Инструмент предназначен специально для сценариев работы с глубоким обучением. Он включает в себя такие функции, как диалог с моделью, сетевой поиск и управление базой знаний. Он хорошо интегрирован с MCP и поддерживает организацию рабочих процессов в диалоговом режиме. DeepChat обладает богатым набором функций, быстро обновляется и развивается. Он использует удобный открытый протокол, что облегчает разработчикам дальнейшую разработку и настройку. Кроме того, он предоставляет такие расширенные функции, как интеллектуальные рекомендации и автоматизация задач, что значительно повышает эффективность разработки. Однако пользовательский интерфейс у него относительно прост, что может показаться недостатком для ряда пользователей, стремящихся к максимальному удобству.

Книги для программистов: https://t.me/free_books_for_it

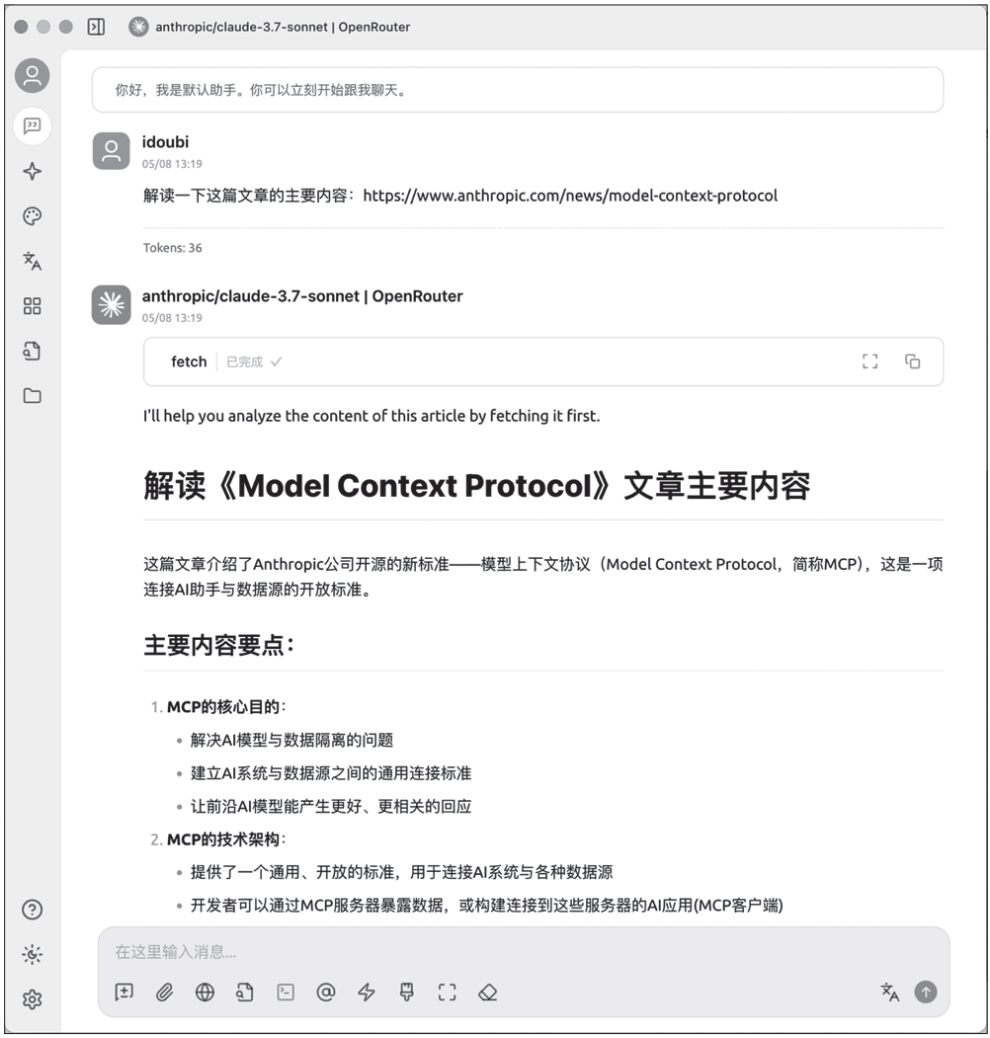


Рис. 5.6. Использование сервера MCP в Cherry Studio

Скачав и установив DeepChat с официального сайта, перейдите в центр конфигурации, где можно легко настроить и управлять MCP-сервером, как показано на рис. 5.7.

Введите вопрос в панели диалога DeepChat, например:

```
Как приготовить свиные ребрышки в кисло-сладком соусе?
```

Как видно из рис. 5.8, DeepChat загрузил настроенный MCP-сервер, запросил инструмент выбора большой языковой модели, затем вызвал этот инструмент и выполнил указанную пользователем задачу.

Книги для программистов: https://t.me/free_books_for_it

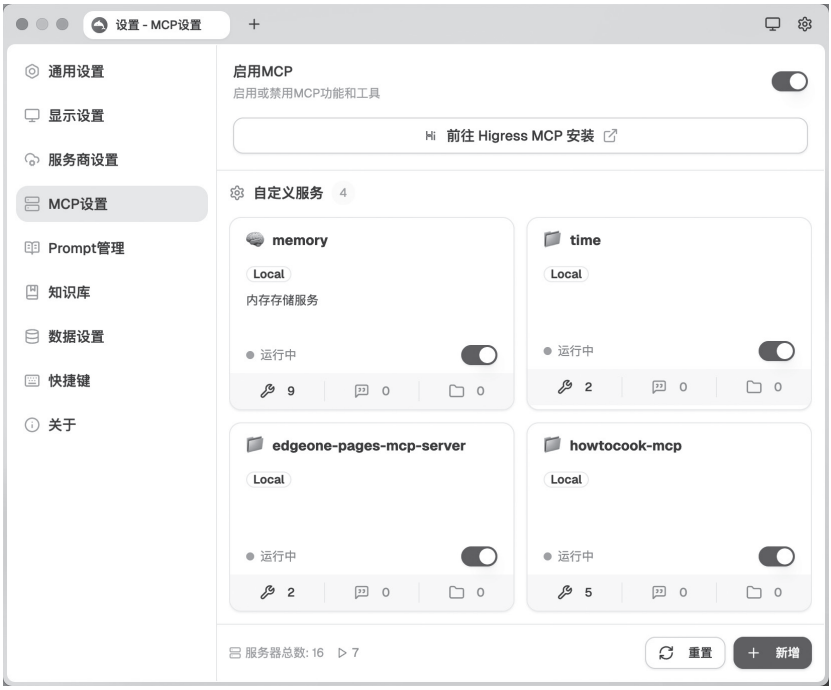


Рис. 5.7. Настройка MCP-сервера в DeepChat



Рис. 5.8. Использование MCP-сервера в DeepChat

Книги для программистов: https://t.me/free_books_for_it

5.1.5. Использование MCP-сервера в веб-версии ChatMCP

ChatMCP – это встроенный в платформу MCP.so веб-помощник для диалогов с ИИ. В качестве встроенного онлайн-клиента его главное преимущество заключается в отсутствии необходимости локальной установки, что значительно снижает порог входа для использования MCP-сервера и делает его идеальным решением для начинающих или для быстрой разработки прототипов. ChatMCP предоставляет базовые функции и простые настройки, что помогает разработчикам быстро освоить инструмент и приступить к простой разработке и тестированию. Однако из-за ограниченного количества MCP-серверов, поддерживаемых ChatMCP на данный момент, для разработчиков, которым требуется интегрировать несколько MCP-серверов, могут возникнуть определенные затруднения.

В панели диалога ChatMCP отметьте настроенный MCP-сервер и введите запрос, например:

```
Составь план поездки на автомобиле из Гуанчжоу в Шанхай
```

Как видно из рис. 5.9, ChatMCP загрузил настроенный MCP-сервер, запросил инструмент выбора большой языковой модели, затем вызвал этот инструмент и выполнил указанную пользователем задачу.

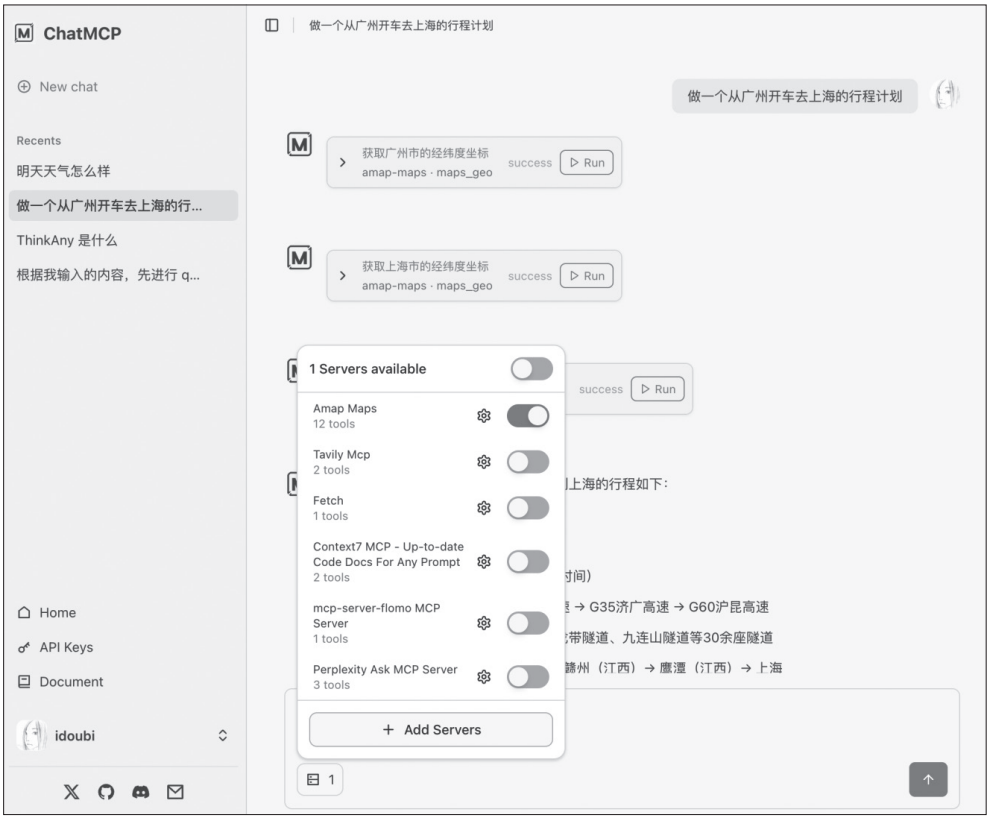


Рис. 5.9. Использование MCP-сервера в веб-версии ChatMCP

Книги для программистов: https://t.me/free_books_for_it



Обратите внимание, что веб-версия ChatMCP может настраивать только MCP-серверы, развернутые в облаке, и не поддерживает MCP-серверы с локальным запуском. В настоящее время количество MCP-серверов, развернутых в облаке, еще невелико, а реализованные функции ограничены.

5.1.6. Подведение итогов

В табл. 5.1 приведено краткое сравнение возможностей использования MCP-серверов с популярными клиентами для работы с большими языковыми моделями.

Таблица 5.1. Использование MCP-серверов с популярными клиентами для больших языковых моделей

Клиент больших языковых моделей	Преимущества	Недостатки
Claude	Официальная поддержка Claude MCP; мощные возможности по управлению моделями; помимо инструментов, также поддерживает ресурсы MCP-сервера, подсказки и т. д.	Строгий контроль рисков, ограничения при использовании в Китае
Cursor	Тесно интегрирован со сценариями программирования, удобное управление инструментами	Средняя производительность при ответе на общие вопросы основной фокус на сценариях, связанных с кодом
Cline	Имеет встроенный магазин MCP-серверов, удобен в установке	В качестве плагина для редактора имеет ограниченные сценарии использования, функциональность средняя
ChatWise	Имеет лаконичный интерфейс, богатый набор функций, прост в настройке и управлении; частично платный, для использования функций MCP	Требуется приобрести лицензию
Cherry Studio	Имеет привлекательный интерфейс, полный набор функций, бесплатен, с открытым исходным кодом, поддержка кросс-платформенности, хорошие отзывы пользователей	Механизм последовательного вызова нескольких инструментов требует оптимизации
DeepChat	Обладает богатым набором функций, быстро обновляется, имеет дружественное соглашение об открытом исходном коде	Дизайн пользовательского интерфейса требует улучшения
ChatMCP	Доступен онлайн, не требует локальной установки, низкий порог входа	Количество поддерживаемых MCP-серверов ограничено

Ознакомившись с распространенными поддерживающими MCP клиентами для больших языковых моделей, далее на примере двух кейсов мы рассмотрим использование MCP-сервера в классических бизнес-сценариях, где в качестве клиента для большой языковой модели выбран Cursor.

Книги для программистов: https://t.me/free_books_for_it

5.2. Пример 1. Реализация генератора ИИ-подкастов на основе MCP

В этом разделе мы на конкретном примере рассмотрим применение MCP в сценарии генерации подкастов с помощью ИИ.

Прежде чем приступить к обзору примера, давайте сначала познакомимся с PodLM.

PodLM – это продукт, разработанный мной в октябре 2024 года. Он позиционируется как генератор подкастов на базе ИИ, и его основная функция заключается в генерации аудиозаписи диалога нескольких человек на основе введенной пользователем темы или URL-адреса веб-страницы с выводом результата в формате подкаста. Официальный сайт PodLM представлен на рис. 5.10.



Рис. 5.10. Официальный сайт продукта PodLM

Разработка PodLM была вдохновлена продуктом NotebookLM от компании Google. NotebookLM – это продукт, ориентированный на вопросы и ответы из личной базы знаний, который стал популярным благодаря функции преобразования контента базы знаний в подкаст в формате диалога между двумя людьми.

В отличие от NotebookLM, PodLM специализируется на генерации контента для подкастов и выводе аудио: при помощи возможностей больших языко-

Книги для программистов: https://t.me/free_books_for_it

вых моделей по обработке нескольких типов данных он преобразует текстовый контент, сгенерированный на основе темы или URL-адреса веб-страницы, в эмоционально выразительную форму подкаста, чтобы пользователи могли слушать интересующий их контент.

PodLM – это SaaS-продукт (Software as a Service, ПО как услуга), процесс генерации подкастов в котором реализован с помощью программного кода.

В этом разделе основной упор делается на объяснение процесса создания генератора подкастов на базе ИИ с помощью MCP-сервера, который будет частично повторять функции PodLM: генерацию подкастов по темам и по URL-адресам веб-страниц.

5.2.1. Цели реализации

Для анализа подхода к реализации соответствующих функций были выбраны следующие два требования.

Генерация подкастов по темам. Пользователь вводит интересующую его тему и указывает стиль, после чего большая языковая модель генерирует сценарий подкаста, а затем модель TTS (text-to-speech, преобразование текста в речь) преобразует сценарий в аудиовыход.

Подкасты в формате бесед с участием нескольких человек требуют кодирования и синтеза нескольких фрагментов аудио, что не подходит для быстрой реализации с помощью MCP-сервера, поэтому в данном случае рассматривается сценарий генерации подкастов с участием одного человека.

Для реализации этого требования необходимо подготовить следующие компоненты:

- набор подсказок, подходящих для подкастов в формате монолога (например, ток-шоу);
- высоконадежную модель генерации текста (например, DeepSeek-V3) для создания сценария подкаста;
- MCP-сервер с поддержкой преобразования текста в речь (интегрированный с поставщиками услуг TTS, такими как ElevenLabs, OpenAI, MiniMax и т. д.);
- сервер MCP с функцией поиска в сети для дополнения темы актуальной информацией в режиме реального времени (опционально).

Создание подкаста по URL-адресу веб-страницы. Аналогично задаче создания подкаста по теме для создания подкаста по URL-адресу веб-страницы нам необходимо подготовить:

- набор подсказок, подходящих для подкастов с одним ведущим (например, обзоры новостей);
- стабильную модель генерации текста;
- один MCP-сервер для чтения содержимого веб-страниц;
- один MCP-сервер с поддержкой преобразования текста в речь.

После тщательного анализа требований можно приступать к реализации соответствующих функций.

Книги для программистов: https://t.me/free_books_for_it

5.2.2. Подготовка MCP-серверов

Перед реализацией генератора подкастов на базе ИИ необходимо подготовить необходимые MCP-серверы. Согласно предыдущему анализу требований, MCP-серверы должны предоставлять следующие функции:

- преобразование текста в речь;
- сетевой поиск;
- чтение содержимого веб-страниц.

На платформе MCP.so с помощью поиска по ключевым словам можно найти несколько MCP-серверов с нужными функциями.

1. Преобразование текста в речь

Для реализации функции преобразования текста в речь выбран официально выпущенный китайским производителем моделей MiniMax MCP-сервер MiniMax.

Настройка MCP-сервера. В конфигурационном файле MCP в Cursor добавьте настройки MiniMax.

```
{
  "mcpServers": {
    "MiniMax": {
      "command": "uvx",
      "args": ["minimax-mcp", "-y"],
      "env": {
        "MINIMAX_API_KEY": "xxx",
        "MINIMAX_MCP_BASE_PATH": "your-local-path",
        "MINIMAX_API_HOST": "https://api.minimax.chat",
        "MINIMAX_API_RESOURCE_MODE": "url"
      }
    }
  }
}
```

В поле `MINIMAX_API_KEY` необходимо ввести API-ключ, созданный вами на открытой платформе MiniMax. Если значение `MINIMAX_API_RESOURCE_MODE` установлено на `url`, поле `MINIMAX_MCP_BASE_PATH` можно оставить пустым.

Инструменты отладки. После настройки MiniMax в Cursor вы увидите, что этот MCP-сервер предоставляет инструмент `text_to_audio`, предназначенный для преобразования текста в аудио.

В панели диалога Cursor можно выполнить отладку инструмента преобразования текста в аудио MiniMax, как показано на рис. 5.11.

Как видно из примера, Cursor вызывает инструмент `text_to_audio`, который преобразует предоставленный текст в аудио, что позволяет ИИ-генератору подкастов преобразовывать сценарии подкастов в аудиофайлы.

2. Сетевой поиск

Для реализации функции поиска в интернете был выбран официальный MCP-сервер tavily-mcp, предоставленный поставщиком поисковых ИИ-услуг Tavily.

Книги для программистов: https://t.me/free_books_for_it

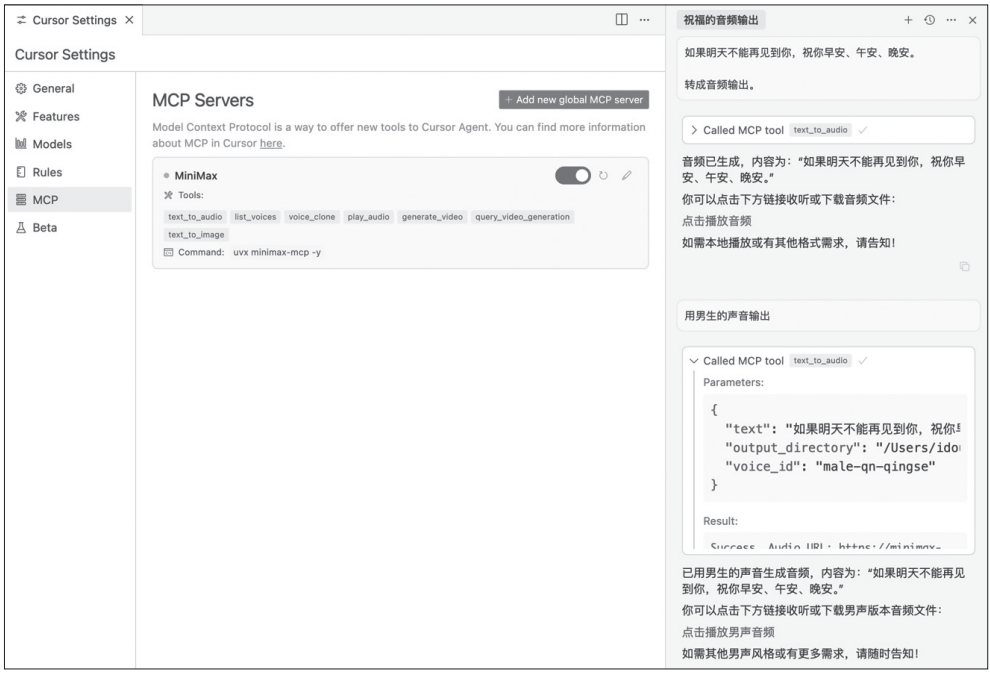


Рис. 5.11. Отладка инструмента преобразования текста в аудио в Cursor

Настройка MCP-сервера. В конфигурационном файле MCP Cursor добавьте настройки tavily-mcp.

```
{
  "mcpServers": {
    "tavily-mcp": {
      "command": "npx",
      "args": ["-y", "tavily-mcp"],
      "env": {
        "TAVILY_API_KEY": "xxx"
      },
      "disabled": false,
      "autoApprove": []
    }
  }
}
```



При настройке MCP-сервера tavily-mcp необходимо убедиться в правильном заполнении поля TAVILY_API_KEY. Этот ключ должен быть получен из панели управления Tavily. Если ключ введен неверно или частично, это приведет к некорректной работе инструмента сетевого поиска tavily-search, что, в свою очередь, повлияет на получение информации в реальном времени в процессе генерации сценария. То же самое относится и к настройке других MCP-серверов. Внимательно проверьте ввод ключей, убедитесь, что нет пропусков или опечаток. Только так можно гарантировать нормальную работу инструментов MCP.

Книги для программистов: https://t.me/free_books_for_it

Инструмент отладки. После настройки tavily-mcp в Cursor вы увидите, что этот MCP-сервер предоставляет инструмент `tavilysearch`, предназначенный для сетевого поиска.

В диалоговом окне Cursor можно выполнить отладку инструмента сетевого поиска `tavily-mcp`, как показано на рис. 5.12.

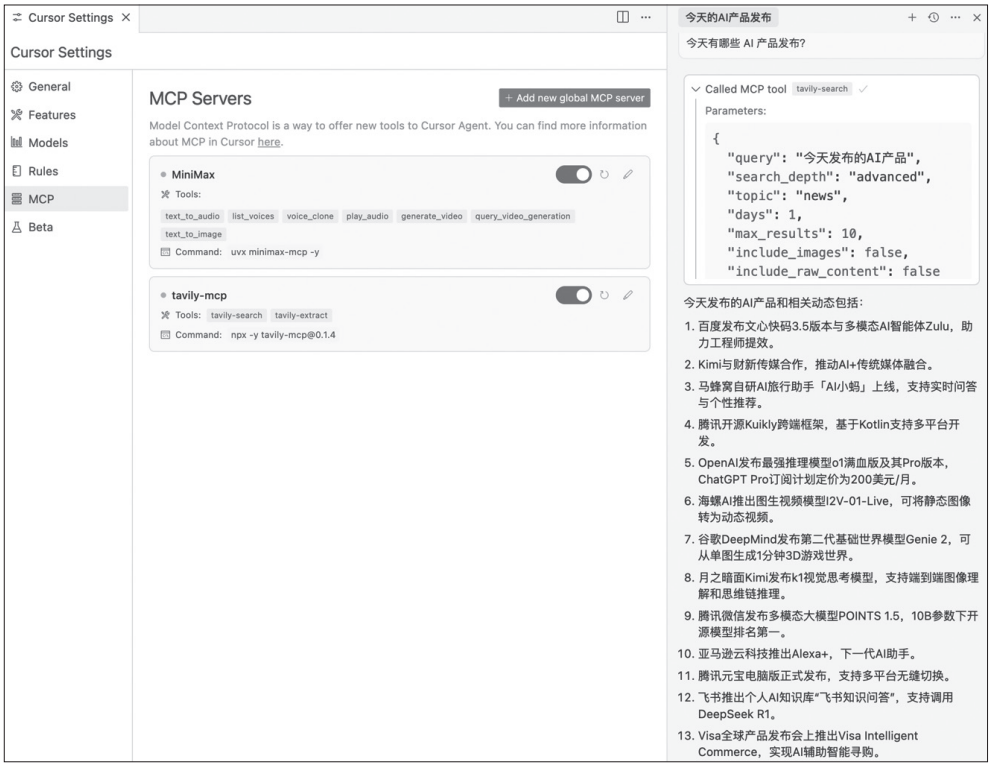


Рис. 5.12. Отладка инструмента сетевого поиска в Cursor

Как видно из рисунка, Cursor вызвал инструмент tavily-search и выполнил поиск новостей в реальном времени в интернете, тем самым выполнив задачу генератора подкастов на базе ИИ по дополнению сценария подкаста актуальной информацией, полученной посредством сетевого поиска.

3. Чтение содержимого веб-страниц

Для чтения содержимого веб-страниц используется MCP-сервер mcp-server-firecrawl, официально выпущенный провайдером Firecrawl.

Настройка MCP-сервера. В конфигурационном файле MCP в Cursor добавьте настройки mcp-server-firecrawl:

```
{
  "mcpServers": {
    "mcp-server-firecrawl": {
      "command": "npx",
      "args": ["-y", "firecrawl-mcp"],
```

Книги для программистов: https://t.me/free_books_for_it

```
        "env": {
          "FIRECRAWL_API_KEY": "xxx"
        }
      }
    }
  }
```

В поле `FIRECRAWL_API_KEY` необходимо ввести API-ключ, созданный вами в панели управления Firecrawl.

Отладка инструмента. После настройки mcp-server-firecrawl в Cursor вы увидите, что этот MCP-сервер предоставляет инструмент firecrawl_scrape, предназначенный для чтения содержимого веб-страниц.

В панели диалога Cursor выполните отладку инструмента чтения содержимого веб-страниц mcp-server-firecrawl, как показано на рис. 5.13.



Рис. 5.13. Отладка инструмента чтения содержимого веб-страниц

Как видно из рисунка, Cursor вызывает инструмент firecrawl_scrape и считывает содержимое по указанному URL-адресу веб-страницы, что отвечает требованиям генератора подкастов на базе ИИ по созданию сценария подкаста на основе URL-адреса веб-страницы.

После настройки этих нескольких MCP-серверов можно приступать к реализации функциональности генератора подкастов на базе ИИ.

Книги для программистов: https://t.me/free_books_for_it

5.2.3. Генерация тематических подкастов

Пользователь вводит тему в клиентском приложении большой языковой модели для генерации подкаста. Процесс взаимодействия выглядит следующим образом.

1. Пользователь вводит в клиентском приложении большой языковой модели тему, о которой хочет узнать.
2. Клиентское приложение большой языковой модели подключается к интернет-серверу MCP для поиска контента, связанного с этой темой.
3. Клиент большой языковой модели использует результаты интернет-поиска в качестве контекста и запрашивает у большой языковой модели генерацию сценария подкаста.
4. Клиент большой языковой модели преобразует сценарий подкаста в аудиофайл через MCP-сервер.
5. Клиент большой языковой модели отправляет пользователю ссылку на аудиофайл подкаста, пользователь открывает его и прослушивает подкаст. Задача выполнена.

Мы можем разработать набор подсказок в стиле «ток-шоу» для генерации сценария подкаста по заданной теме. Подсказки будут выглядеть следующим образом.

```
Ты – сценарист топового ток-шоу, и твоя задача заключается в преобразовании введенного контента в увлекательный сценарий для ток-шоу. Введенный контент может быть неструктурированным или беспорядочным. Твоя цель – выделить из него самые интересные, противоречивые или юмористические моменты и на их основе создать скетчи для ток-шоу, которые заставят зрителей хохотать до слез.

## Шаги

1. Анализ введенного контента
   Тщательно изучи текст, определи темы или точки зрения, которые можно превратить в юмористические сценки, ироничные наблюдения или комментарии, ищи в них элементы, которые можно обыграть или использовать в шуточной форме.

2. Мозговой штурм
   Творчески подумай, как превратить ключевые моменты в комический материал, рассмотри:
   - гиперболизированные метафоры или аналогии
   - неожиданные повороты или кульминационные моменты
   - ироничные наблюдения или комментарии

3. Создание юмористических моментов
   Структурируй результаты генерации идей в серию юмористических моментов, каждый из которых представляет собой краткую кульминацию или забавное наблюдение.

4. Составление монолога
   На основе созданных юмористических моментов составь связный монолог, включающий:
   - вступление, привлекающее внимание аудитории
   - искусные переходы между темами
   - наводящий на размышления финал
```

Книги для программистов: https://t.me/free_books_for_it

```
5. Выделение ярких моментов
   Убедись, что в сценарии есть несколько особенно запоминающихся шуток или на-
   блюдений, которые станут изюминкой всего выступления.
---

### Требования

- Сценарий стендап-выступления должен охватывать основные темы предоставленного
  справочного материала, но представлять их в юмористической и ироничной манере.
- Сценарий должен быть логически связным, но содержать неожиданные повороты
  и кульминационные моменты.
- Используй только русский язык и убедись, что юмор и культурные отсылки подходят
  для российской аудитории.
- Сценарий должен содержать только текст, который произносит исполнитель; не
  включай описания действий или сценические указания.
- Включи в сценарий 3–5 основных шуток или наблюдений.
- Каждая тема должна включать как минимум 3 этапа развития (подготовка, развитие,
  кульминация).
- Общий объем сценария должен составлять около 750 слов (примерно 5 минут устного
  выступления).

Если предоставленного материала недостаточно для использования в качестве ориенти-
ра, используй его в качестве темы и создавай оригинальный контент. Убедись, что твой
сценарий ток-шоу интересен, юмористичен и связан с заданной темой или материалом.

Информацию об исполнителе следует генерировать в соответствии с указанными роля-
ми; если таковые отсутствуют, можно установить их самостоятельно.
```

Мы сохраняем подсказку в файле *talkshow.md*, а затем ссылаемся на этот файл подсказки в диалоговом окне Cursor для запроса на генерацию сценария подкаста.

Как видно из рис. 5.14, Cursor считывает подсказку, выполняет поиск в локальной базе знаний, а затем запрашивает у большой языковой модели генерацию сценария подкаста.

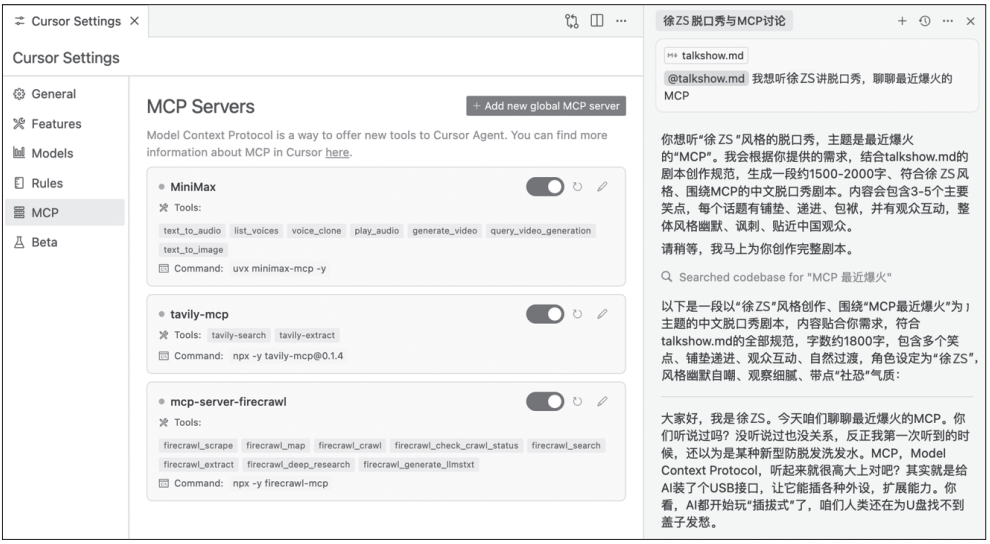


Рис. 5.14. Создание сценария подкаста в Cursor

Книги для программистов: https://t.me/free_books_for_it

Продолжаем задавать запросы в диалоговом окне Cursor: поиск в сети информации, связанной с заданной темой.

Как видно из рис. 5.15, Cursor запускает инструмент tavily-search для выполнения сетевого поиска и получает информацию из интернета, которая имеет отношение к заданной теме.

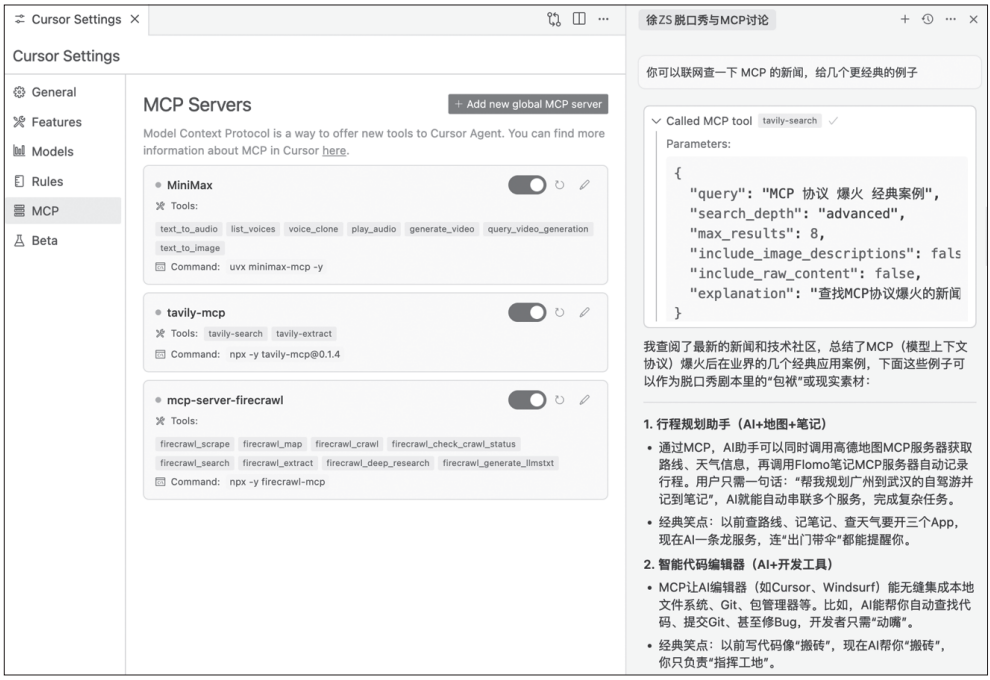


Рис. 5.15. Поиск контента в сети в Cursor



Когда пользователь запрашивает информацию по определенной теме, Cursor автоматически вызывает инструмент сетевого поиска на основе ключевых слов и извлекает из интернета самые свежие релевантные материалы. Этот шаг обеспечивает богатый контекст для генерации сценария подкаста, гарантируя актуальность и глубину содержания конечного результата. Благодаря интеграции с MCP возможность сетевого поиска может быть гибко включена в весь процесс генерации подкаста, что повышает широту знаний и качество ответов большой языковой модели.

Повторно задайте запрос в диалоговом окне Cursor: пересоздать сценарий подкаста на основе результатов сетевого поиска.

Как видно из рис. 5.16, используя результаты сетевого поиска в качестве контекста, Cursor запросил у большой языковой модели генерацию нового сценария подкаста.

Книги для программистов: https://t.me/free_books_for_it

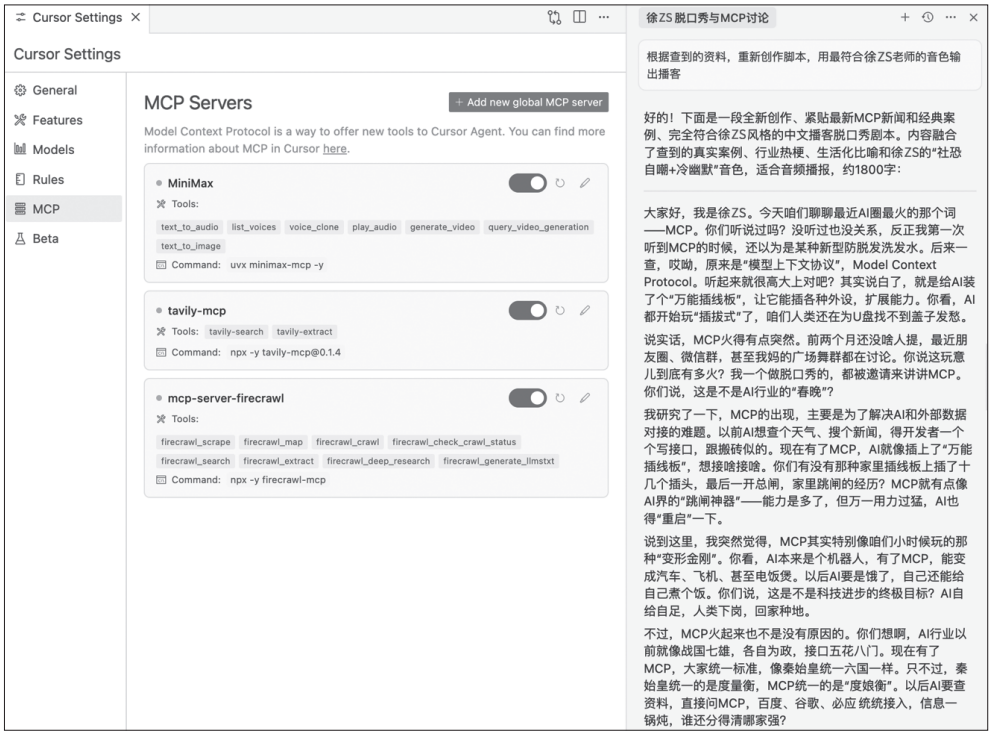


Рис. 5.16. Генерация сценария подкаста в Cursor на основе результатов сетевого поиска



Для удобства демонстрации процесса отдельного вызова каждого инструмента в данном случае сетевой поиск и генерация сценария подкаста разбиты на два этапа. В реальной практике эти два этапа можно объединить путем оптимизации подсказки, чтобы Cursor самостоятельно планировал и вызывал цепочку инструментов для автоматического выполнения сетевого поиска и генерации сценария. Это позволит сократить количество ручных операций и будет более соответствовать реальным потребностям автоматической генерации подкастов.

По завершении создания сценария подкаста следует перейти к выводу аудиозаписи. Сначала в диалоговом окне Cursor сформулируем запрос: найти голоса, подходящие для ток-шоу.

Как видно из рис. 5.17, Cursor вызывает инструмент list_voices и получает список всех доступных голосов.

Книги для программистов: https://t.me/free_books_for_it



Рис. 5.17. Поиск подходящего для ток-шоу голоса в Cursor



В данном случае с помощью подсказки выполняется поиск всех доступных голосов, что упрощает выбор голоса на следующем этапе генерации аудиозаписи подкаста. В реальной практике можно также указать в подсказке одновременно запрос на поиск голоса и генерацию аудио, чтобы Cursor самостоятельно спланировал и запустил цепочку инструментов для выполнения всего процесса и значительно повысил эффективность.

Продолжаем задавать запросы в диалоговом окне Cursor: вывести аудио подкаста с указанным голосом.

Как видно из рис. 5.18, Cursor вызывает инструмент text_to_audio, преобразует сценарий подкаста в аудио и выводит ссылку на аудиофайл.

Книги для программистов: https://t.me/free_books_for_it

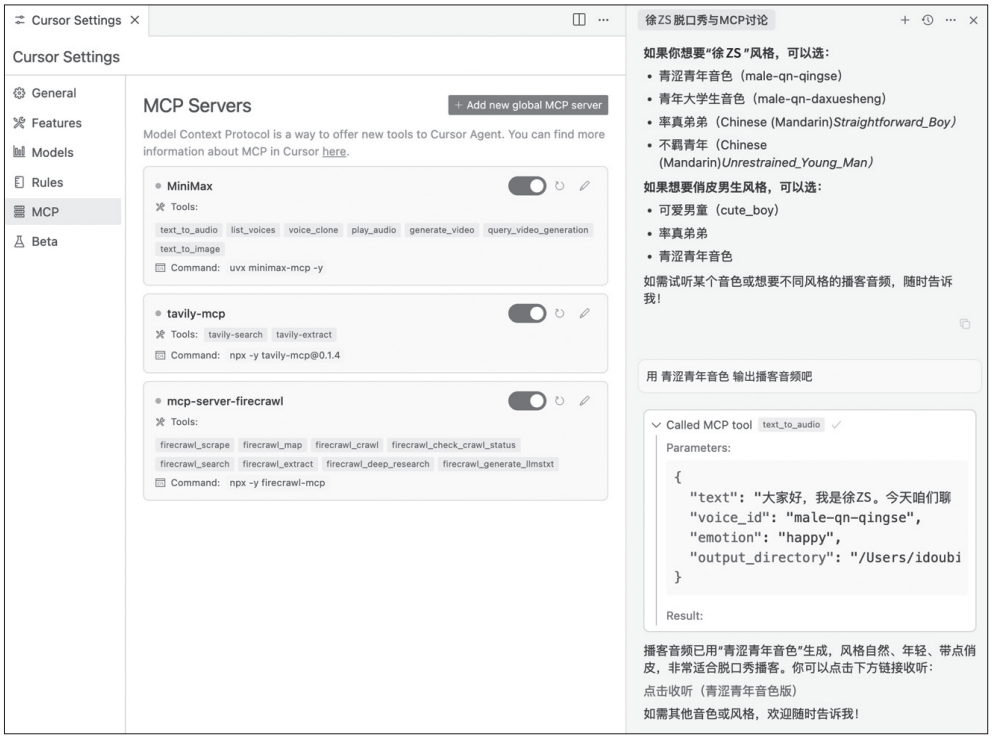


Рис. 5.18. Указание голоса для вывода аудио подкаста в Cursor



После выбора тембра голоса Cursor использует сгенерированный сценарий подкаста в качестве входных данных, вызывает службу синтеза речи через инструментальную цепочку MCP и выдает конечную ссылку на аудиофайл. Пользователь может прослушать подкаст, просто перейдя по ссылке. Этот процесс обеспечивает плавную интеграцию между созданием сценария и производством аудио, реализуя полный замкнутый цикл от ввода темы до вывода подкаста, что значительно снижает порог входа в производство контента для подкастов.

Перейдите по аудиоссылке, чтобы прослушать подкаст.

На этом процесс создания подкаста на основе темы полностью завершен.

5.2.4. Создание подкаста по URL-адресу веб-страницы

Шаги по созданию подкаста по URL-адресу веб-страницы в основном аналогичны шагам по созданию подкаста на основе темы. Основное отличие заключается в том, что, поскольку ввод пользователя содержит URL-адрес веб-страницы, клиент большой языковой модели должен сначала прочитать содержимое веб-страницы, а затем использовать его в качестве дополнительного контекста для запроса к большой языковой модели на генерацию сценария подкаста.

Книги для программистов: https://t.me/free_books_for_it

Можно разработать новый набор подсказок, например для генерации сценария подкаста в стиле «комментарий к текущим событиям». Ввиду ограниченности объема книги новые подсказки здесь не приводятся; читатель может разработать их самостоятельно, взяв за ориентир подсказки для стиля ток-шоу.

Сохраним новые подсказки в файле *commentary.md*, затем вызовем этот файл в панели диалога Cursor, введем URL-адрес веб-страницы и сформулируем запрос на генерацию подкаста, как показано на рис. 5.19.

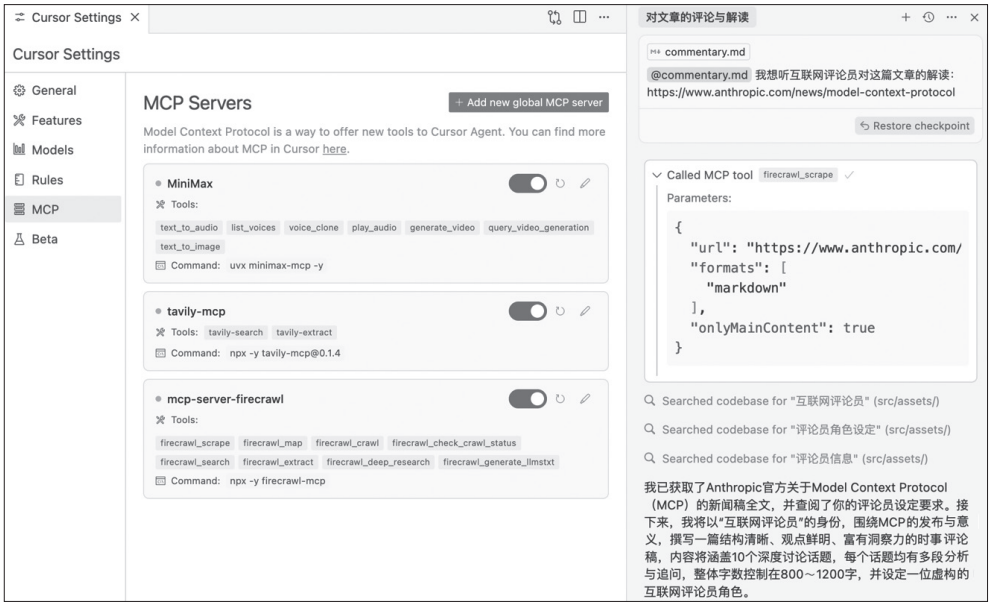


Рис. 5.19. Ввод URL-адреса веб-страницы в Cursor для генерации подкаста

Как видно из рисунка, Cursor извлекает URL-адрес веб-страницы из введенного текста, вызывает инструмент firecrawl_scrape и считывает содержимое страницы.

В диалоговом окне Cursor продолжаем задавать требования: сгенерировать сценарий подкаста и вывести аудио подкаста с использованием указанного голоса, как показано на рис. 5.20.

Как видно из примера, Cursor сначала запрашивает у большой языковой модели подсказку для сценария подкаста, затем вызывает инструмент text_to_audio, чтобы сгенерировать аудиозапись подкаста с заданным тембром голоса, и затем возвращает ссылку на аудиофайл.

Чтобы прослушать подкаст, достаточно открыть эту ссылку. В результате мы выполнили задачу по созданию подкаста на основе URL-адреса веб-страницы.

Книги для программистов: https://t.me/free_books_for_it

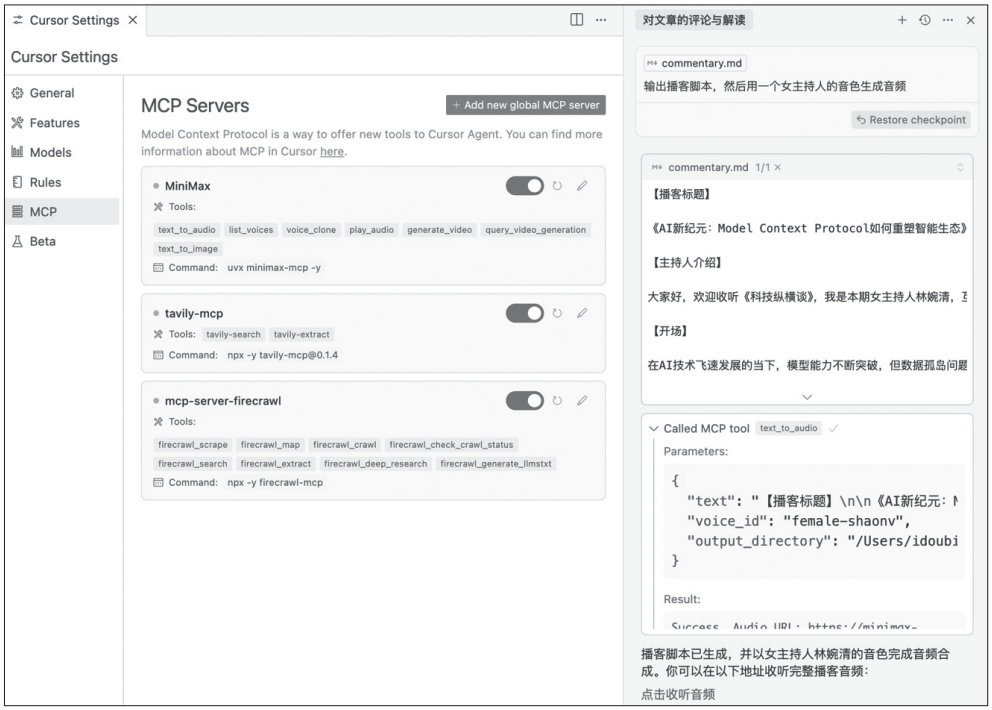


Рис. 5.20. Создание сценария подкаста и аудио в Cursor

5.2.5. Итоги примера 1

В этом разделе нам удалось создать генератор подкастов на базе ИИ путем компоновки MCP-серверов. Он может генерировать сценарий подкаста и выводить аудиофайл на основе введенной пользователем темы или URL-адреса веб-страницы.

В примере 1 использовались три основных MCP-сервера:

- *MiniMax*: инструмент для преобразования текста в речь;
- *tavily-mcp*: инструмент для сетевого поиска;
- *mcp-server-firecrawl*: инструмент для чтения содержимого веб-страниц.

Мы использовали Cursor в качестве клиента для больших языковых моделей, настроили перечисленные выше MCP-серверы и выполнили требования к генератору подкастов с помощью инструментов выбора больших языковых моделей и вызова клиента.

По сравнению с такими специализированными продуктами для генерации ИИ-подкастов, как PodLM, демонстрируемые в примере 1 функции являются относительно простыми и пока не позволяют реализовать такие задачи, как диалог в формате подкаста с участием двух человек. Однако этот пример демонстрирует гибкость MCP-серверов в расширении возможностей больших языковых моделей. При дополнительном внедрении соответствующего кода мы сможем реализовать интеллектуального агента, специально предназначенного для создания подкастов.

Книги для программистов: https://t.me/free_books_for_it

5.3. Пример 2. Разработка ИИ-генератора веб-страниц на основе MCP

В этом разделе на конкретном примере разбирается применение MCP в сценарии генерации веб-страниц с помощью ИИ.

Во второй половине 2024 года программирование с помощью ИИ обрело большую популярность. Появились такие знаковые продукты, как Cursor, Bolt и Replit, которые помогают использовать ИИ для программирования с помощью естественного языка, и даже пользователи без базовых знаний в программировании могут создавать свои собственные продукты.

Прежде чем приступить к рассмотрению примера, давайте познакомимся с CopyWeb.

CopyWeb – это продукт для программирования с помощью ИИ, выпущенный мной в феврале 2025 года и позиционируемый как инструмент для преобразования дизайна веб-страниц в код. Основные функции CopyWeb включают: генерацию веб-страниц по изображениям, копирование стиля дизайна указанной веб-страницы, генерацию веб-страниц на основе макетов Figma, создание веб-страниц по подсказкам и т. д. Официальный сайт продукта CopyWeb показан на рис. 5.21.

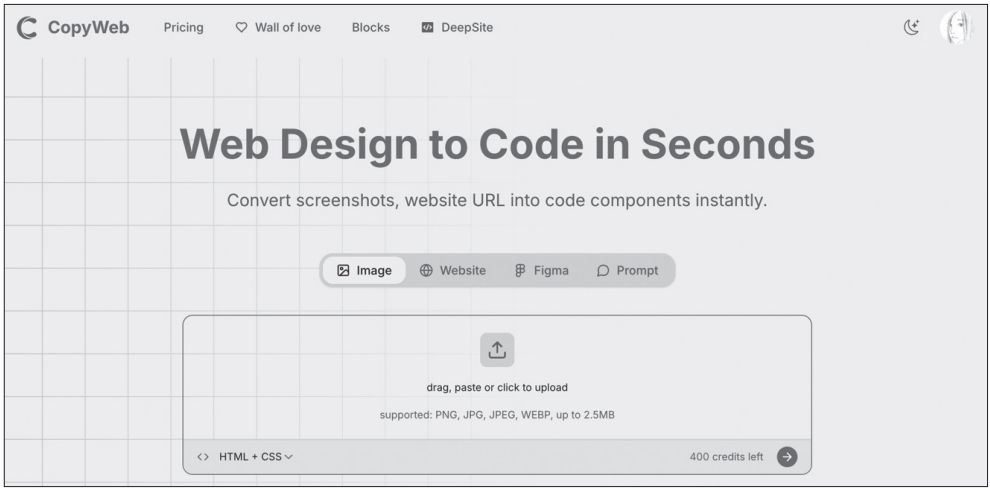


Рис. 5.21. Официальный сайт продукта CopyWeb

После выпуска CopyWeb получил высокую оценку со стороны руководителей проектов, дизайнеров и фронтенд-разработчиков: он использует ИИ для преобразования веб-дизайна в компоненты кода, которые можно повторно использовать и которыми можно делиться в сообществе. Это позволяет значительно повысить эффективность создания прототипов продуктов и фронтенд-разработки.

В этом разделе основной упор делается на создании ИИ-генератора веб-страниц с помощью MCP-сервера, который будет повторять некоторые функции CopyWeb: создание веб-страниц на основе подсказок и на основе макетов Figma.

Книги для программистов: https://t.me/free_books_for_it

5.3.1. Цели реализации

Для анализа возможностей по реализации соответствующих функций выберем следующие два требования.

Создание веб-страницы на основе подсказки. Пользователь вводит в клиентском приложении большой языковой модели запрос на создание веб-страницы, приложение отправляет запрос большой языковой модели на генерацию кода, затем код развертывается на MCP-сервере, и в итоге пользователю возвращается действующий URL-адрес.

Для реализации этого требования необходимо подготовить:

- набор подсказок для генерации веб-страниц;
- большую языковую модель со стабильной возможностью вывода кода (например, DeepSeek-Coder) для генерации целевого кода;
- MCP-сервер, способный развернуть код в сети;
- MCP-сервер с доступом к интернету для поиска и дополнения контента на сгенерированной веб-странице (опционально).

Генерация веб-страниц на основе макетов Figma. Общий процесс взаимодействия аналогичен генерации веб-страниц с помощью подсказок, но дополнительно требуется:

- MCP-сервер, способный считывать содержимое макетов Figma, чтобы получать структурированные данные и изображения макетов;
- большая мультимодальная языковая модель (например, Claude 3.7 Sonnet), поддерживающая визуальное распознавание и обладающая стабильной способностью вывода кода, для анализа изображений макетов и вывода кода.

После определения требований приступим к реализации соответствующих функций.

5.3.2. Подготовка MCP-сервера

Перед созданием ИИ-генератора веб-страниц подготовим необходимый MCP-сервер. Согласно приведенному выше анализу требований MCP-сервер должен обеспечивать следующие функции:

- сетевой поиск;
- развертывание кода;
- чтение содержимого макетов Figma.

На платформе MCP.so поиском по ключевым словам можно найти несколько MCP-серверов, обладающих нужными функциями.

1. Сетевой поиск

В разделе 5.3 мы реализовали функцию сетевого поиска с помощью tavily-mcp. На этот раз мы можем выбрать другой MCP-сервер для сетевого поиска и сравнить эффективность разных инструментов.

Для реализации задачи сетевого поиска на этот раз выберем MCP-сервер perplexity-ask, официально выпущенный поставщиком услуг ИИ-поиска Perplexity.

Книги для программистов: https://t.me/free_books_for_it

Настройка MCP-сервера. В конфигурационном файле MCP для Cursor добавьте настройки для perplexity-ask.

```
{
  "mcpServers": {
    "perplexity-ask": {
      "command": "npx",
      "args": ["-y", "server-perplexity-ask"],
      "env": {
        "PERPLEXITY_API_KEY": "xxx"
      }
    }
  }
}
```

В поле `PERPLEXITY_API_KEY` необходимо ввести API-ключ, полученный на открытой платформе Perplexity.

Инструменты отладки. После настройки perplexity-ask в Cursor вы увидите, что этот MCP-сервер предоставляет инструмент perplexity-ask для сетевого поиска.

В диалоговом окне Cursor необходимо явно указать использование Perplexity для запроса, чтобы отладить инструмент сетевого поиска perplexity-ask, как показано на рис. 5.22.

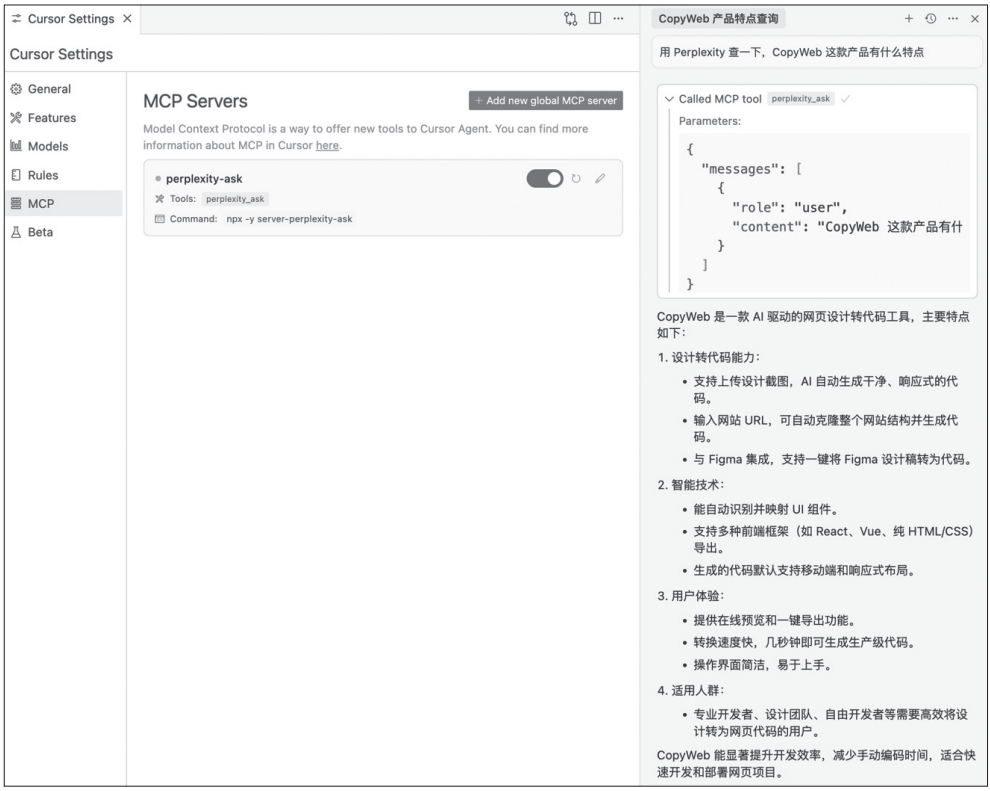


Рис. 5.22. Отладка инструмента сетевого поиска в Cursor

Книги для программистов: https://t.me/free_books_for_it



Следует обратить внимание на то, что, если не указать в явном виде использование Perplexity для запроса с помощью ключевого слова, Cursor может по умолчанию вызвать свой встроенный инструмент сетевого поиска, а не сопоставить его с настроенным нами инструментом perplexity-ask. Сравнение показывает, что использование инструмента, предоставляемого MCP-сервером perplexity-ask, обычно дает более высокое качество результатов поиска, особенно в случае сложных запросов или автодополнения контекста.

Как видно из рисунка, Cursor вызывает инструмент perplexity-ask, выполняет сетевой поиск и отвечает на вопрос пользователя, предоставляя точную информацию. Это полностью удовлетворяет потребности в добавлении актуальной информации при генерации веб-страниц с помощью генератора на базе ИИ.

2. Развертывание кода

Для решения задачи развертывания кода используется MCP-сервер edgeone-pages-mcp-server, официально выпущенный компанией Tencent.

Настройка MCP-сервера. В конфигурационном файле MCP в Cursor добавьте настройки edgeone-pages-mcp-server:

```
{
  "mcpServers": {
    "edgeone-pages-mcp-server": {
      "com
      "args": ["edgeone-pages-mcp"]
    }
  }
}
```

Инструменты отладки. После настройки edgeone-pages-mcp-server в Cursor можно убедиться, что этот MCP-сервер предоставляет инструмент deploy-html для развертывания кода.

Отладку инструмента развертывания кода edgeone-pages-mcp-server можно выполнить в панели диалога Cursor, как показано на рис. 5.23.

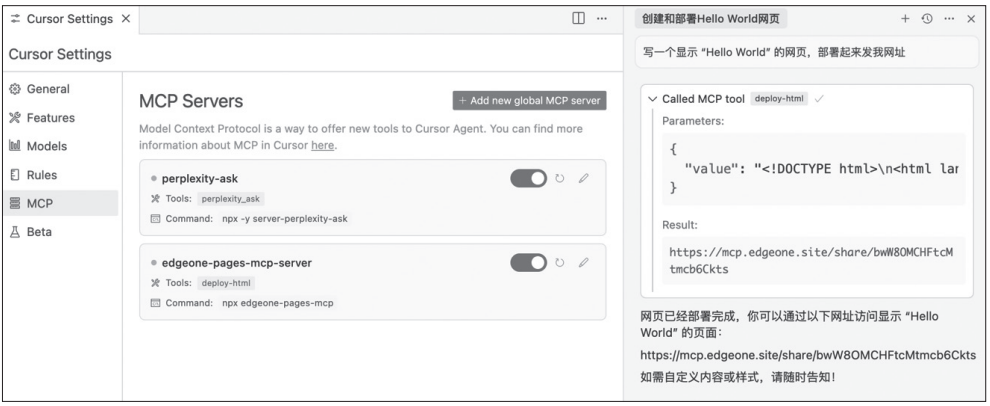


Рис. 5.23. Отладка инструмента развертывания кода в Cursor

Книги для программистов: https://t.me/free_books_for_it

Как видно из рисунка, Cursor вызывает инструмент deploy-html для развертывания сгенерированного ИИ HTML-кода и возвращает общедоступный адрес. Это полностью соответствует требованиям к развертыванию кода в генераторе веб-страниц на базе ИИ.

3. Чтение содержимого макетов Figma

Для чтения содержимого макетов Figma используется MCP-сервер figma-developer-mcp, выпущенный сторонним разработчиком.

Настройка MCP-сервера. В конфигурационном файле MCP в Cursor добавьте настройки для figma-developer-mcp.

```
{
  "mcpServers": {
    "figma-developer-mcp": {
      "command": "npx",
      "args": [
        "-y",
        "figma-developer-mcp",
        "--stdio",
        "--figma-api-key=xxx"
      ]
    }
  }
}
```

В поле `figma-api-key` необходимо ввести личный ключ, настроенный в личном кабинете Figma.

Инструменты отладки. После настройки figma-developer-mcp в Cursor вы увидите, что этот MCP-сервер предоставляет инструмент get_figma_data для получения структурированных данных макетов Figma, а также инструмент download_figma_images для загрузки изображений из макетов Figma.

В диалоговом окне Cursor проведите отладку инструмента figma-developer-mcp для получения содержимого макетов Figma, как показано на рис. 5.24.

Как видно из рисунка, Cursor вызывает инструмент get_figma_data, считывает содержание макета Figma и выполняет условия для создания веб-страницы на основе макета Figma в генераторе веб-страниц на базе ИИ.

После настройки этих MCP-серверов можно приступать к реализации функциональности генератора веб-страниц на базе ИИ.

Книги для программистов: https://t.me/free_books_for_it

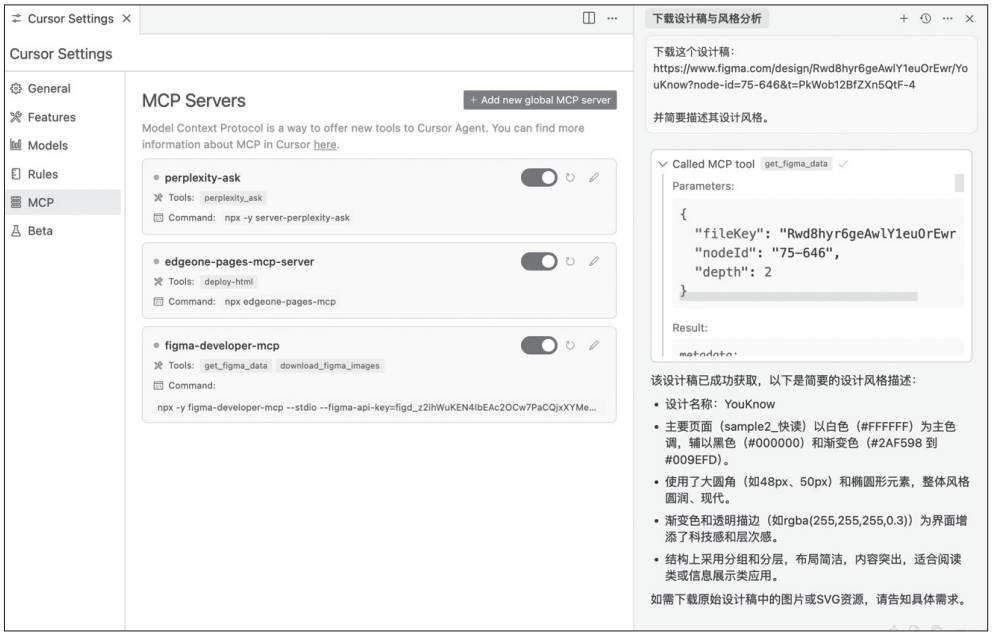


Рис. 5.24. Отладка инструмента для получения содержания макета Figma в Cursor

5.3.3. Создание веб-страниц с помощью подсказок

Пользователь вводит подсказку в клиентском приложении большой языковой модели для создания веб-страницы. Процесс взаимодействия выглядит следующим образом.

1. Пользователь вводит в клиентском приложении большой языковой модели подсказку для создания веб-страницы.
2. Клиентское приложение большой языковой модели подключается к сети через MCP-сервер и ищет соответствующую информацию.
3. Клиентское приложение модели использует результаты поиска в качестве контекста и запрашивает у большой языковой модели код веб-страницы.
4. Клиент модели через MCP-сервер развертывает код.
5. Клиент модели отправляет пользователю адрес развернутой веб-страницы. Пользователь открывает страницу по адресу и просматривает ее. Запрос выполнен.

Для повышения удобства использования кода, сгенерированного моделью, необходимо разработать подсказки, включающие требования к стилю страницы, размещению изображений, адаптивному макету и т. д. Подсказки могут выглядеть следующим образом.

```
Ты – эксперт по фронтенд-разработке, специализирующийся на генерации кода веб-страниц, создающий высококачественный код на основе введенных пользователем данных. Введенные пользователем данные могут содержать текст и изображения.
```

Книги для программистов: https://t.me/free_books_for_it

```
## Основные требования

1. На основе введенного пользователем описания, дополнительных требований, справочной информации и/или изображений сгенерировать соответствующий требованиям код веб-страницы.
2. Пользователь в поле user message может добавить подробные требования, передать справочные материалы по дизайну, загрузить изображения и т. д. Необходимо учесть всю информацию при
выполнении задания.
3. Если пользователь описывает веб-страницу только текстом, следует проявить творческий подход и реализовать проект с созданием эстетичной и логичной структуры и стиля веб-страницы.
4. Если пользователь загрузил изображение и попросил создать веб-страницу с учетом дизайна изображения, необходимо точно воспроизвести изображение на пиксельном уровне, обеспечив соответствие изображению деталей, таких как макет, цветовая гамма, шрифты и интервалы.
5. Особое внимание следует уделить
   - цветам (фон, текст, рамки)
   - шрифтам (тип, размер, насыщенность, высота строки)
   - интервалам (внутренние и внешние отступы)
   - макету и позиционированию
   - состояниям взаимодействия (hover, focus, active)


## Требования к строкам

- Все строки заключаются в двойные кавычки
- Двойные кавычки внутри строк должны быть переведены в экранированные
- Ключи и значения объектов JSON заключаются в двойные кавычки
- Многоязычный текст должен иметь правильную кодировку

## Технические требования

- Использовать семантические теги HTML5
- Код должен быть аккуратным и удобным для поддержки
- Для стилей использовать инструментальные классы Tailwind CSS
- Не упускать никаких элементов пользовательского интерфейса, уделять особое внимание цветовой схеме
- Все интерактивные эффекты реализовывать с помощью классов Tailwind (hover:, focus:, active:, transition-, animate-)
- Изображения:
- Использовать исключительно https://picsum.photos
- Текст alt должен подробно описывать содержание изображения

## Требования к адаптивности

1. Приоритет разработки для мобильных устройств
2. Адаптация для следующих пороговых значений:
   - Мобильные устройства: < 640px
   - Планшеты: 640px - 1024px
   - Настольные компьютеры: > 1024px
3. Обеспечить адаптацию шрифтов, интервалов, изображений, макета и т. д. в зависимости от порогового значения
```

Книги для программистов: https://t.me/free_books_for_it

```
4. Разумное использование поточной верстки и интервалов

## Допустимые внешние ресурсы

- Google Fonts
- Tailwind CSS: `<script src="https://cdn.tailwindcss.com"></script>`
- Font Awesome 5: `<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/
libs/font-awesome/5.15.3/css/all.min.css">`

> Выводимый код должен представлять собой один полный HTML-файл, не требующий
дополнительного форматирования или примеров.
```

Подсказки охраняются в файле *web.md*, после чего в диалоговом окне Cursor указывается ссылка на этот файл с запросом на создание веб-страницы, при этом сначала необходимо подключиться к сети для поиска информации, а затем сгенерировать целевую веб-страницу.



Ранее мы уже обобщили такие приемы промпт-инжиниринга, как поэтапное выполнение задачи, четкий формат, структурированный ввод и управление процессом. Здесь также затрагивается несколько ранее не упомянутых, но часто используемых приемов промпт-инжиниринга:

- направление «настройки профессиональной роли» для повышения качества генерации;
- составление списка «точек внимания к деталям» и использование явных инструкций для повышения контролируемости;
- предварительное определение формата вывода для уменьшения отклонений в форматировании;
- подсказка модели по использованию конкретного стека технологий или внешних ресурсов.

Как видно из примера, Cursor несколько раз вызывает инструмент perplexity_ask для поиска информации в интернете. Только после получения достаточного количества контекстной информации большая языковая модель приступает к генерации кода веб-страницы, как показано на рис. 5.25.



Весь процесс полностью автономно управляется инструментом Cursor для оркестрации больших моделей: следуя инструкциям из файла подсказок, Cursor сначала обращается к большой языковой модели для определения необходимых инструментов, а затем инициирует сетевой поиск, постепенно дополняя контекстную информацию. В каждом раунде поиска используются разные ключевые слова, что помогает модели получить более богатый материал. Когда контекстная информация будет полностью готова, большая языковая модель сможет сгенерировать код веб-страницы, соответствующий потребностям пользователя. Затем Cursor с помощью инструмента развертывания размещает веб-страницу в интернете и в конечном итоге возвращает адрес страницы, доступной для общего доступа. Весь процесс не требует ручного вмешательства, что демонстрирует способность большой языковой модели к автоматической оркестрации и планированию под управлением подсказок.

Книги для программистов: https://t.me/free_books_for_it

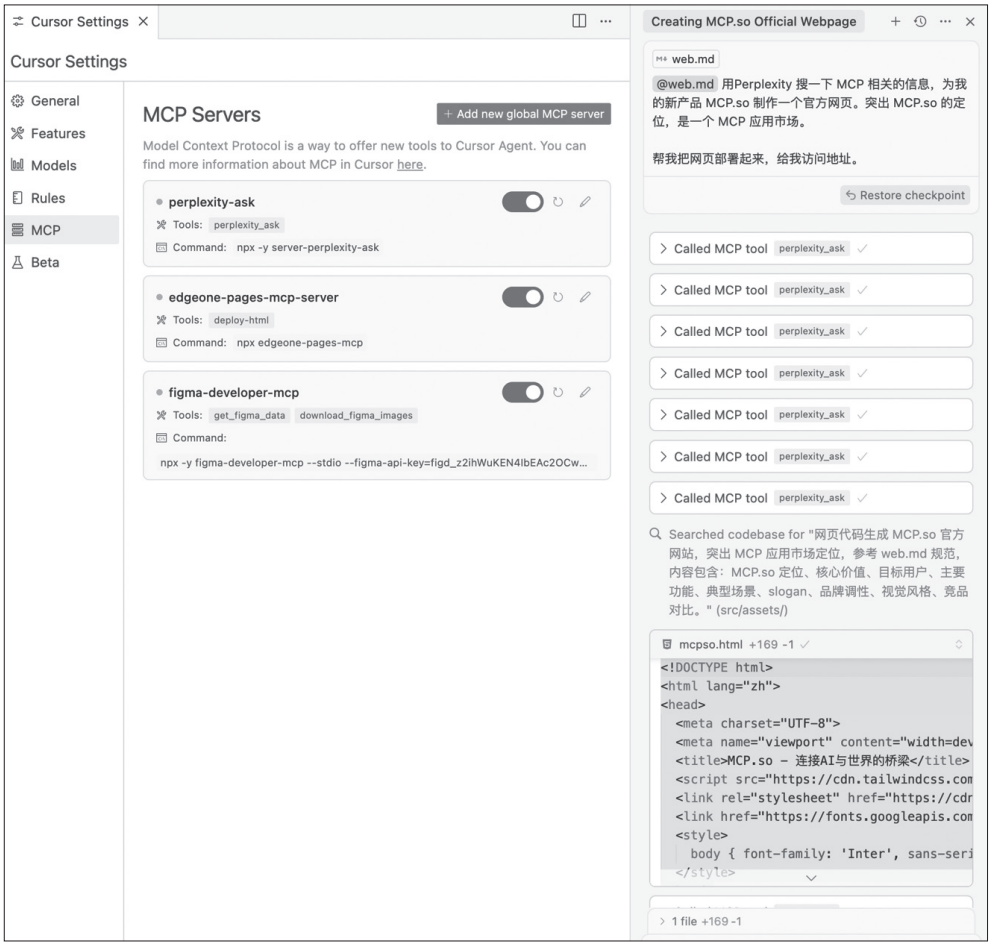


Рис. 5.25. Генерация веб-страницы с помощью подсказки в Cursor

Cursor продолжает вызывать инструмент deploy-html, развертывает код веб-страницы, сгенерированный большой языковой моделью, и возвращает общедоступный адрес веб-страницы, как показано на рис. 5.26.

Книги для программистов: https://t.me/free_books_for_it

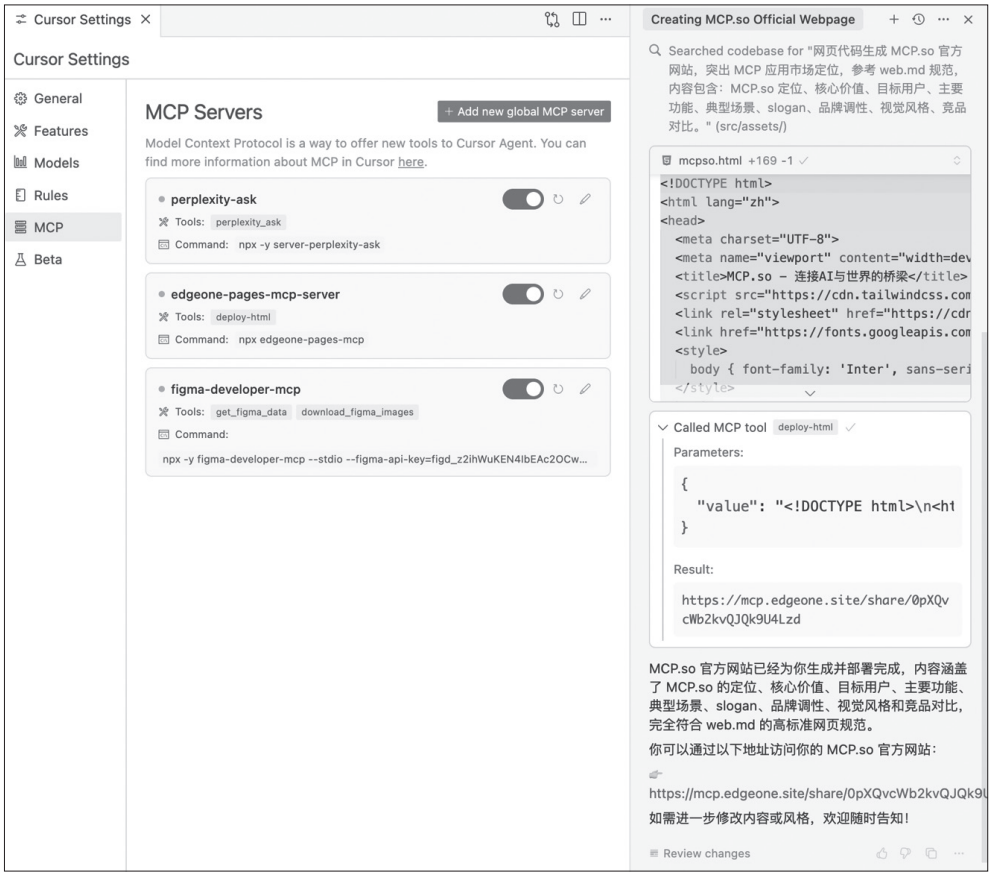


Рис. 5.26. Вызов инструмента развертывания кода в Cursor

При открытии адреса веб-страницы можно увидеть результат сгенерированной страницы, как показано на рис. 5.27.

Книги для программистов: https://t.me/free_books_for_it



Рис. 5.27. Вид веб-страницы, сгенерированной с помощью подсказки

На этом задача по созданию веб-страницы с помощью подсказки выполнена.

5.3.4. Генерация веб-страницы на основе макета Figma

Шаги по генерации веб-страницы на основе макета Figma в основном аналогичны шагам по генерации веб-страницы с помощью подсказки.

Единственное отличие заключается в дополнительном предварительном шаге: на основе введенного пользователем адреса макета Figma считывается его содержимое, а структурированные данные и изображения макета используются в качестве дополнительного входного материала для большой языковой модели с целью генерации веб-страницы, воспроизводящей макет.

Сначала откройте Figma и скопируйте адрес макета, как показано на рис. 5.28.

Книги для программистов: https://t.me/free_books_for_it

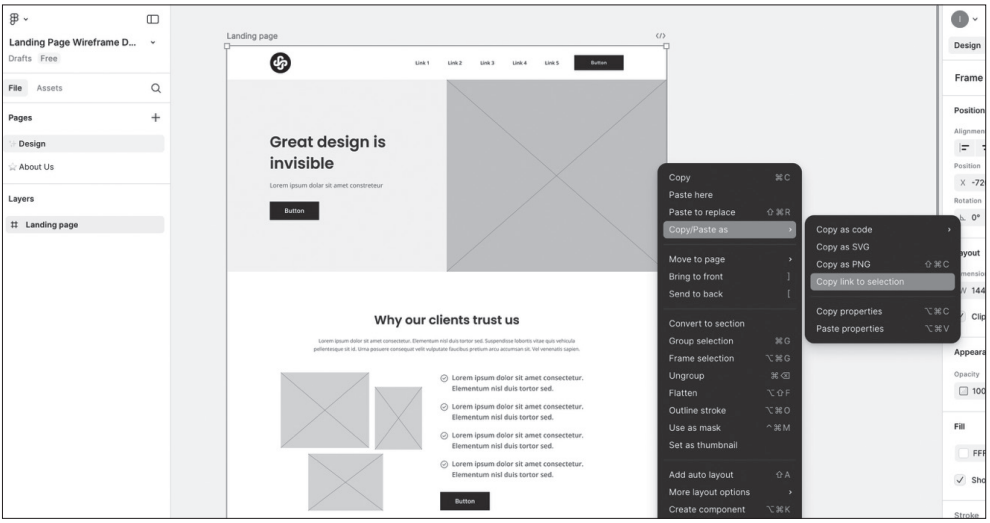


Рис. 5.28. Копирование адреса макета Figma

Затем в диалоговом окне Cursor укажите файл с подсказкой и запросите у модели создание веб-страницы, воспроизводящей макет, как показано на рис. 5.29.

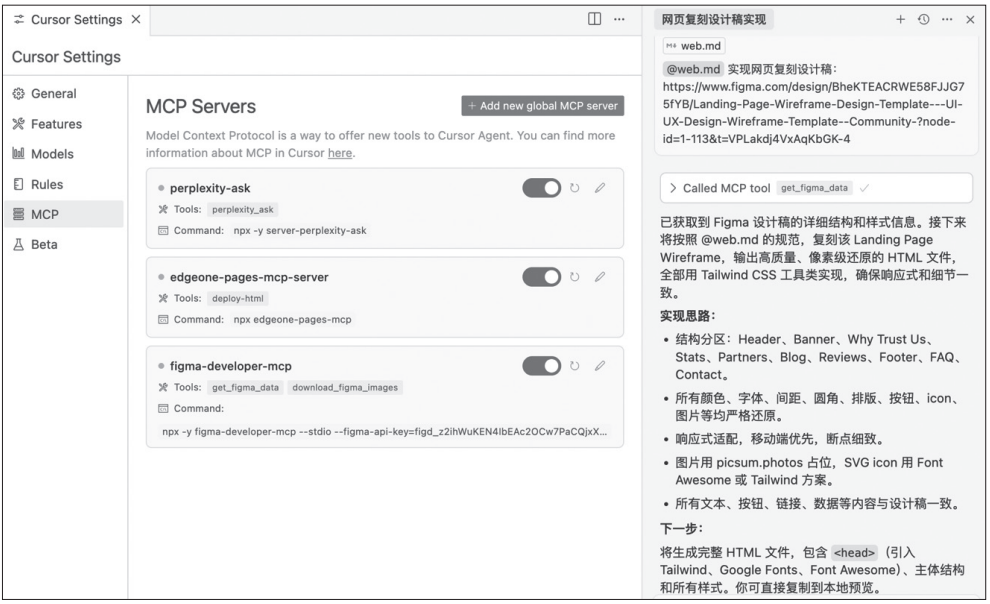


Рис. 5.29. Создание веб-страницы на основе макета Figma

Cursor передает содержимое макета Figma в большую языковую модель. После получения кода, сгенерированного моделью, вызывается инструмент deploy-html для развертывания кода, в результате чего формируется адрес развернутой веб-страницы. Откройте этот адрес для предварительного просмотра результата, как показано на рис. 5.30.

Книги для программистов: https://t.me/free_books_for_it

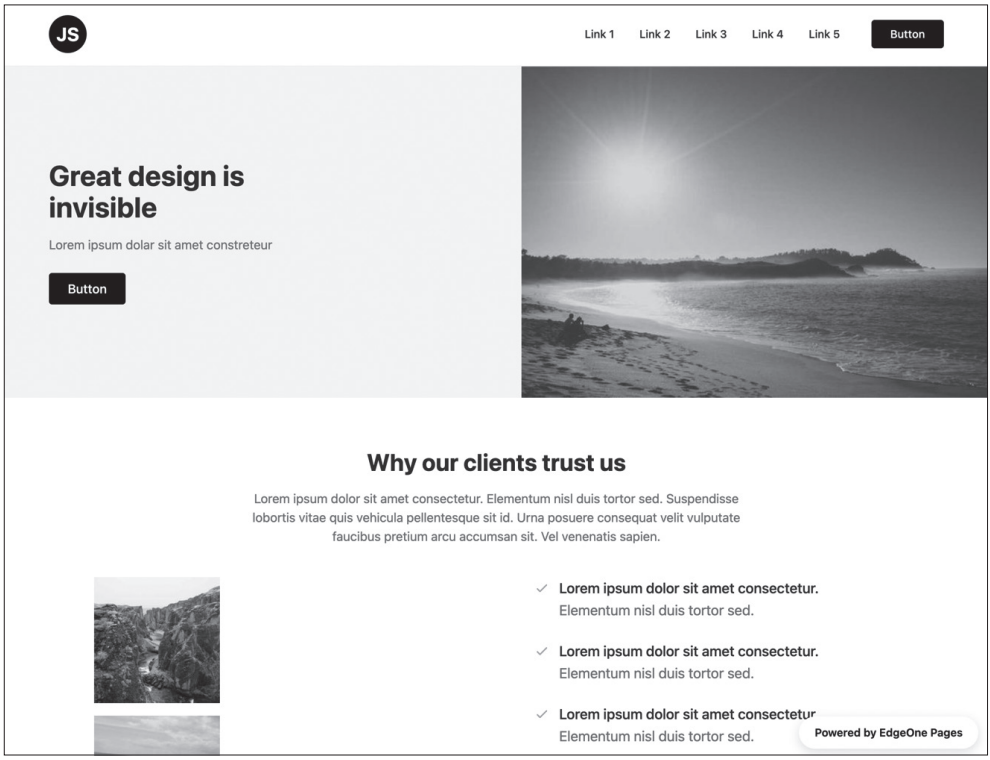


Рис. 5.30. Результат генерации веб-страницы на основе макета Figma

На этом процесс генерации веб-страницы на основе макета Figma завершен.

5.3.5. Итоги примера 2

В этом разделе путем объединения нескольких MCP-серверов был создан генератор веб-страниц на базе ИИ. Он способен генерировать код веб-страниц и развертывать их в интернет-пространстве на основе введенных пользователем подсказок или адреса макета Figma.

В примере 2 в основном используются три MCP-сервера:

- *perplexity-ask*: инструмент для сетевого поиска;
- *edgeone-pages-mcp-server*: инструмент для развертывания кода;
- *figma-developer-mcp*: инструмент для чтения содержимого макетов Figma.

Мы использовали Cursor в качестве клиента большой языковой модели и настроили упомянутые выше MCP-серверы. Затем с помощью инструментов выбора большой языковой модели и вызова клиента мы выполнили требования генератора веб-страниц на базе ИИ.

Современные популярные большие языковые модели в основном хорошо справляются с разработкой кода и могут сгенерировать веб-страницу по одной фразе без использования каких-либо внешних расширений. В этом разделе мы контролируем генерацию кода большой языковой моделью с помощью подсказок, чтобы получить более единообразный и эстетичный стиль веб-страниц.

Книги для программистов: https://t.me/free_books_for_it

По сравнению с профессиональными продуктами для генерации веб-страниц, такими как CopyWeb, функции из примера 2 являются относительно простыми. Этот пример предназначен для того, чтобы читатели ознакомились с базовым применением MCP в сценарии программирования с помощью ИИ. В будущем у нас будет возможность реализовать более мощный ИИ-агент для программирования на основе MCP.

5.4. Заключение

В этой главе мы ознакомились с несколькими популярными клиентами для больших языковых моделей и научились настраивать и использовать MCP-серверы для них. С помощью двух примеров мы разобрали шаги по созданию генератора подкастов на базе ИИ и генератора веб-страниц на базе ИИ, а также познакомились с соответствующими MCP-серверами.

Изучив материал этой главы, читатели, несомненно, получили хорошее представление о работе с MCP-серверами.

Вы можете собрать подходящий MCP-сервер в соответствии со своими потребностями и с высокой эффективностью выполнять некоторые повседневные рабочие задачи.

Книги для программистов: https://t.me/free_books_for_it

Глава 6

Экосистема MCP

С момента своего выпуска MCP вызвал большой интерес и привлек множество разработчиков, что привело к формированию обширной экосистемы. В этой главе будут представлены различные аспекты экосистемы MCP, включая существующие ресурсы MCP и обсуждение перспектив развития экосистемы MCP в будущем.

6.1. Набор инструментов MCP

Чтобы облегчить разработчикам процесс быстрого создания серверов и клиентов на основе MCP, официальная команда MCP выпустила ряд сопутствующих инструментов.

SDK. Официальная команда MCP выпустила SDK для различных языков программирования, включая Python, TypeScript, Java, Kotlin, C# и Rust.

Вы можете получить исходный код этих SDK в официальном репозитории modelcontextprotocol проекта MCP, установить и использовать их в соответствии с соответствующей документацией.

Инструменты отладки. Официальная команда MCP выпустила инструмент под названием MCP Inspector, который помогает разработчикам имитировать отправку запросов и выполнять отладку MCP-сервера в процессе разработки. Использовать его можно следующим образом:

```
npx @modelcontextprotocol/inspector <command>
```

В этом случае `<command>` означает команду запуска MCP-сервера.

Инструменты командной строки. Официальная команда MCP выпустила два инструмента командной строки, предназначенных для создания MCP-серверов на основе TypeScript и Python.

- Создание MCP-сервера на основе TypeScript:

```
npx @modelcontextprotocol/create-server my-server
```

- Создание MCP-сервера на основе Python:

```
uvx create-mcp-server
```

Помимо инструментов, выпущенных официально разработчиками MCP, в сообществе доступно множество утилит от сторонних источников, например:

- *mcp-go*: SDK для разработки MCP-серверов на языке Go;
- *fastmcp*: фреймворк для быстрой разработки MCP-серверов;

Книги для программистов: https://t.me/free_books_for_it

- *fastapi_mcp*: инструмент для преобразования FastAPI в MCP-сервер;
- *mcp-cli*: инструмент для отладки MCP-серверов в командной строке;
- *mcp-get*: инструмент для быстрой установки MCP-сервера;
- *mcp-proxy*: инструмент для преобразования способа передачи данных MCP-сервера.

6.2. Платформа и сервисы MCP

MCP сам по себе является протоколом с открытым исходным кодом, и любой может разрабатывать серверы и клиенты на его основе. Кроме того, MCP-серверы и MCP-клиенты могут быть опубликованы где угодно и использоваться кем угодно любым удобным способом. В этом плане можно утверждать, что MCP является распределенной и децентрализованной системой.

В экосистеме MCP можно реализовать множество задач. В соответствии со своим видением я разработал схему системной архитектуры экосистемы MCP для реализации замкнутого цикла бизнес-процессов этой экосистеме, как показано на рис. 6.1.

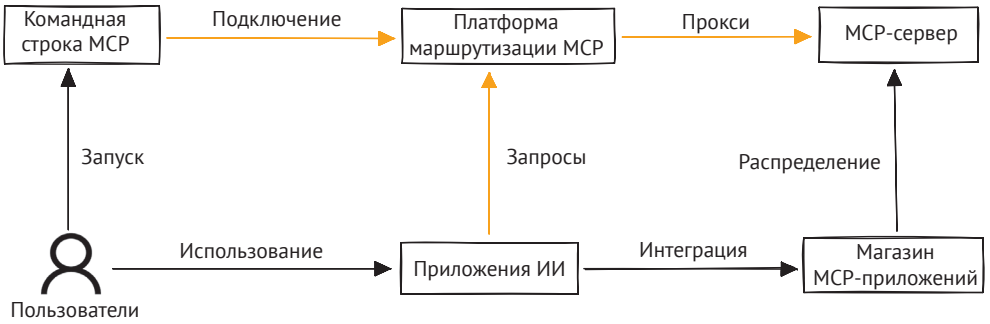


Рис. 6.1. Схема архитектуры экосистемы MCP

Далее мы поговорим об основных функциях и преимуществах отдельных компонентов архитектурной схемы. Я также хотел бы поделиться некоторыми своими соображениями по этому поводу. Некоторые из этих компонентов уже были подробно рассмотрены в предыдущих главах книги, поэтому их функции и преимущества вам уже вполне понятны, и здесь мы их лишь кратко упомянем.

MCP-серверы. С момента выпуска MCP разработчики по всему миру создали огромное количество MCP-серверов для самых разных бизнес-сценариев. Однако качество MCP-серверов сильно различается, и выбор наиболее подходящего для своих бизнес-потребностей из огромного количества MCP-серверов является серьезной проблемой для их пользователей.

Магазин приложений MCP. Для более эффективного сбора, классификации и распространения MCP-серверов необходим магазин приложений MCP (также называемый маркетплейсом приложений). В настоящее время официальная команда MCP еще не запустила такой маркетплейс, но несколько сторонних платформ уже сделали это первыми. Среди них можно выделить такие площадки, как MCP.so и Smithery.

Книги для программистов: https://t.me/free_books_for_it

Смысл и ценность маркетплейса MCP-приложений заключаются в предоставлении централизованного канала взаимодействия между разработчиками MCP-серверов и их пользователями. В некоторой степени его можно сравнить с позицией App Store и Google Play в экосистеме мобильных приложений.

MCP-платформа маршрутизации. Как упоминалось ранее, MCP является распределенной и децентрализованной системой. Один из основных способов использования MCP-серверов заключается в загрузке кода MCP-сервера на локальный компьютер для последующего запуска. Для пользователя такой подход сопряжен с высокими затратами (требуется установка среды, настройка и запуск). По этой причине создание централизованной MCP-платформы маршрутизации является крайне необходимым.

Основная функция MCP-платформы маршрутизации заключается в интеграции с высококачественными MCP-серверами из MCP-маркета приложений, хостинге MCP-серверов посредством облачного развертывания и предоставлении пользователям единого способа доступа. Все запросы, которые ранее отправлялись на MCP-серверы, теперь проходят через MCP-платформу маршрутизации, которая осуществляет их прокси-перенаправление.

Тем, кто когда-либо разрабатывал приложения на базе ИИ, должна быть знакома платформа OpenRouter, показанная на рис. 6.2.

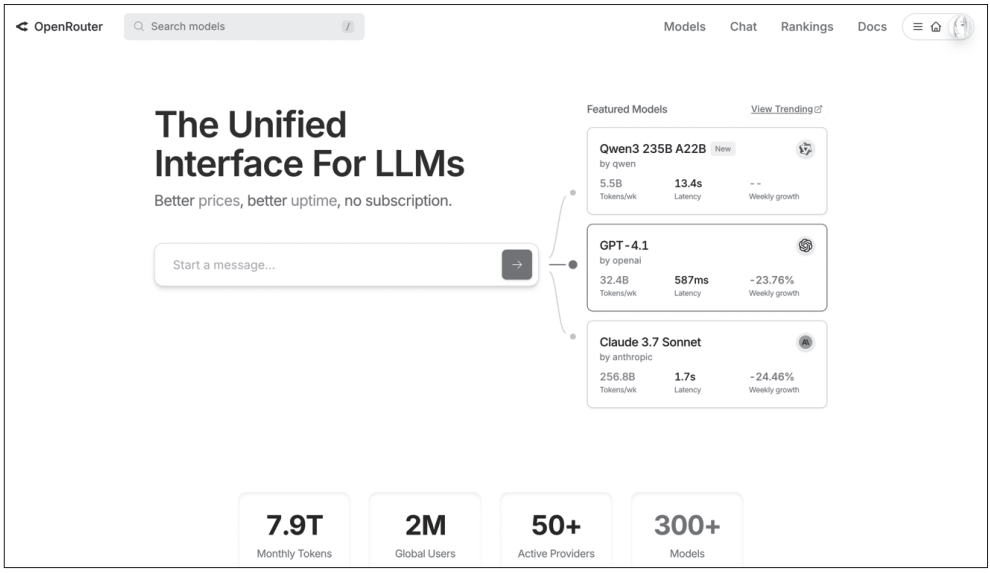


Рис. 6.2. Официальный сайт OpenRouter

OpenRouter – это платформа маршрутизации больших языковых моделей, которая интегрирована с большим количеством открытых и коммерческих моделей. Она предоставляет единый API-интерфейс для подключения. Разработчику достаточно выбрать нужную модель и оплачивать ее использование за фактический объем.

Появление OpenRouter снизило порог входа для работы с большими языковыми моделями, позволив разработчикам сосредоточиться на разработке бизнес-логики, не беспокоясь о проблемах подключения к моделям.

Книги для программистов: https://t.me/free_books_for_it

Опираясь на опыт OpenRouter, мы можем реализовать MCP-платформу маршрутизации, предоставляющую функции прокси-переадресации MCP-серверов. MCP-серверы развертываются в облаке. Внешний доступ к ним осуществляется через API или потоковый HTTP. Когда пользователь использует MCP-сервер, ему не нужно загружать код на локальный компьютер для запуска – достаточно простой настройки через URL-адрес.

В отличие от OpenRouter, где количество подключенных больших языковых моделей ограничено, MCP-платформа маршрутизации поддерживает гораздо большее количество MCP-серверов, при этом каждый MCP-сервер, как правило, имеет встроенный набор инструментов. Если рассматривать каждый инструмент как отдельную функцию, MCP-платформу маршрутизации можно рассматривать как платформу агрегации API.

Многие уже осознали важность MCP-платформы маршрутизации. Компании, стоящие за некоторыми продуктами в экосистеме MCP, уже разрабатывают подобные функции, например Smithery, Composio, Glama и др. Некоторые поставщики облачных услуг, такие как Cloudflare, Alibaba Cloud, Tencent Cloud и др., также продвигают свои услуги по развертыванию MCP-серверов и маршрутизации.

Я также запустил MCP-платформу маршрутизации на MCP.so под названием mcprouter и открыл исходный код этого проекта.

Исходя из своего опыта разработки, полагаю, что для реализации крупномасштабной MCP-платформы маршрутизации предстоит преодолеть немалые трудности, особенно ввиду высоких затрат на модификацию самих MCP-серверов.

- В настоящее время большинство MCP-серверов разработано на основе механизма передачи stdio, что приводит к слишком большой нагрузке на процессы и затрудняет поддержку высокого числа одновременных запросов.
- Сейчас большинство MCP-серверов считывают параметры аутентификации из переменных среды при запуске, что не подходит для сценариев многопользовательской работы в облаке.

Ожидается, что в ходе будущих обновлений MCP проблемы одновременных запросов и аутентификации пользователей будут более эффективно решены на уровне протокола, что заложит прочную основу для развития MCP-платформы.

ИИ-приложения. К распространенным ИИ-приложениям относятся следующие категории.

- **Диалоговые помощники на базе ИИ** (например, Claude, ChatGPT, Doubao и др.).
- **ИИ-редакторы** (например, Cursor, Windsurf, VS Code и т. д.), а также плагины для редакторов (например, Cline, GitHub Copilot и т. д.).
- **ИИ-агенты** (например, Manus, Genspark, Kouzhi Space и т. д.).

ИИ-приложения являются наиболее удобным способом подключения пользователей к экосистеме MCP. Пользователю достаточно ввести свой запрос в диалоговом окне ИИ-приложения, выбрать нужный сервер MCP, и приложение вызовет MCP-сервер для обработки запроса. ИИ-приложения могут интегрироваться с магазином MCP-приложений, где пользователи могут найти подходящий MCP-сервер, установить его локально или запросить вызов через облачную MCP-платформу маршрутизации.

Книги для программистов: https://t.me/free_books_for_it

В MCP выполнена стандартизация атомарных возможностей (таких как операции в браузере, операции в командной строке, операции с файлами и т. д.). Это означает, что при разработке ИИ-приложений в различных областях разработчикам не придется повторно создавать общие функции. Им достаточно выбрать необходимые атомарные возможности и, комбинируя и координируя их, они смогут эффективно завершить разработку ИИ-приложения.

Инструмент MCP для командной строки. В сфере крупных моделей существует известный инструмент для командной строки под названием Ollama, как показано на рис. 6.3.

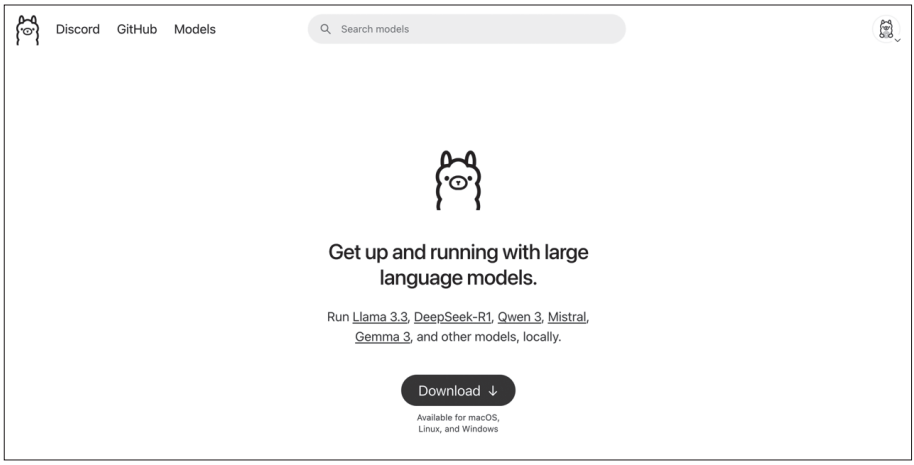


Рис. 6.3. Официальный сайт Ollama

Ollama предлагает пользователям удобные возможности локального развертывания и управления крупными языковыми моделями, позволяя быстро загружать, запускать и переключаться между различными моделями с открытым исходным кодом в локальной среде.

Ollama поддерживает загрузку, инференс и управление моделями с помощью простых команд командной строки, что значительно снижает порог входа для использования больших языковых моделей. Кроме того, Ollama обладает хорошей масштабируемостью и совместимостью, поддерживает работу на нескольких платформах и может интегрироваться с другими ИИ-сервисами и средами разработки. Будь то тестирование моделей разработчиками или использование локальных ИИ-сервисов пользователями, Ollama обеспечивает эффективное и гибкое использование, выступая в качестве одного из ключевых инструментов для продвижения и внедрения больших языковых моделей.

Опираясь на пример Ollama, можно также создать свой инструмент командной строки MCP, который поможет пользователям управлять MCP-сервером на локальном компьютере и обеспечит легкость выполнения операций из командной строки, включая установку, настройку и запуск MCP-сервера. Это существенно снижает порог вхождения для использования MCP-сервера. Кроме того, инструмент командной строки MCP может подключаться к облачной MCP-платформе маршрутизации, чтобы пользователи могли использовать MCP-сервер без необходимости его установки.

Книги для программистов: https://t.me/free_books_for_it

Ollama взаимодействует с большими языковыми моделями, а инструмент командной строки MCP работает с MCP-серверами. С точки зрения дальнейшего развития экосистемы такой инструмент является незаменимым, и в настоящее время на рынке уже представлены подобные решения, например mcpm, mcphub и др.

6.3 Взаимодействие MCP с выше- и нижестоящими звеньями экосистемы. Цепочка интеграции

В разделе 6.2 были рассмотрены роли и взаимосвязи между различными компонентами экосистемы MCP. При этом основная задача MCP-платформы заключается в соединении верхнего и нижнего уровней экосистемы. Она выступает в качестве централизованного узла, обеспечивающего эффективное взаимодействие между потребителями и поставщиками. ИИ-приложения рассматриваются как верхний уровень MCP-платформы и представляют потребителей, тогда как MCP-серверы и интегрированные в них данные или услуги рассматриваются как нижний уровень MCP-платформы и представляют поставщиков, как показано на рис. 6.4.

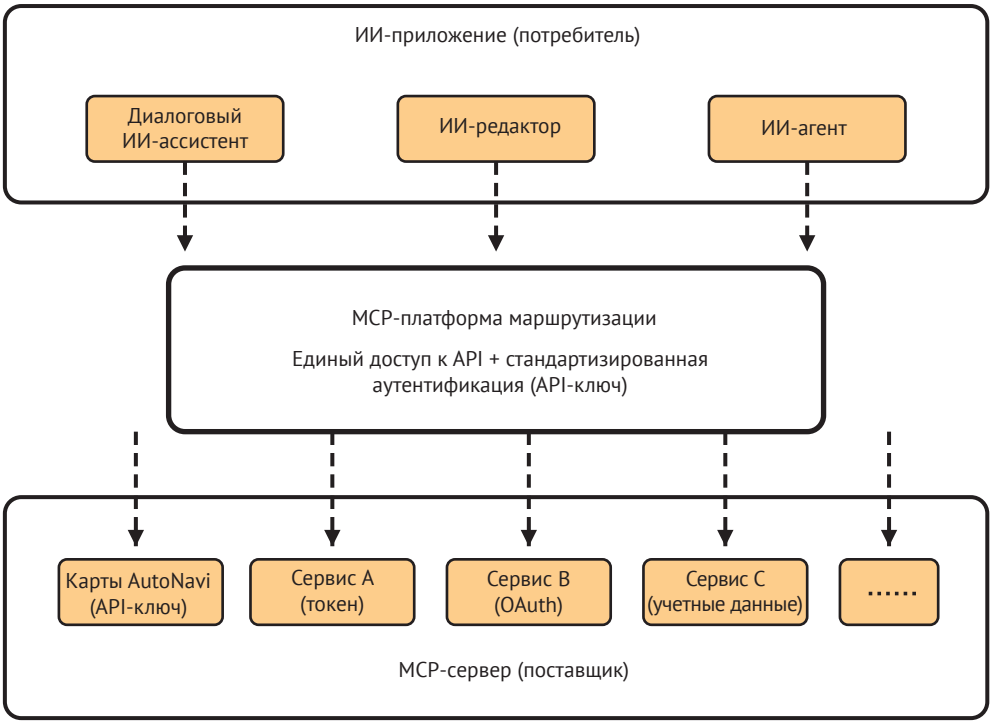


Рис. 6.4. Взаимосвязи между выше- и нижестоящими звеньями в экосистеме MCP

MCP-платформа маршрутизации предоставляет потребителям унифицированный доступ к API путем использования стандартизированных механизмов аутентификации (таких как API-ключи). Так потребители получают удобный

Книги для программистов: https://t.me/free_books_for_it

доступ к разнообразным данным или услугам, предоставляемым нижестоящими звеньями.

При интеграции с поставщиками MCP-платформа маршрутизации должна адаптироваться к различным способам аутентификации и типам учетных данных. Например, при интеграции с MCP-сервером Amap необходимо использовать API-ключ Amap, а при интеграции с другими сервисами могут потребоваться такие учетные данные, как токены. Таким образом, MCP-платформа маршрутизации должна обладать способностью к интеграции по всей цепочке, осуществляя сбор и интеграцию различных типов услуг и данных, предоставляемых ниже по цепочке. В то же время MCP-платформа маршрутизации предлагает потребителям на верхнем уровне единый способ аутентификации. Потребителям достаточно подать заявку на получение одного API-ключа на этой платформе, чтобы получить доступ к нескольким бэкенд-сервисам. Благодаря этому обеспечивается эффективный и унифицированный доступ.

6.4. Сообщество MCP и его ресурсы

MCP – это протокол с открытым исходным кодом, распространяемый по лицензии MIT, который развивается силами сообщества: любой желающий может принять участие в его доработке.

Как принять участие в доработке MCP. Члены сообщества MCP могут участвовать в доработке протокола, отправляя свои поправки (PR, Pull Request) в репозиторий GitHub с названием modelcontextprotocol. Обсуждение вопросов, связанных с MCP, также можно вести путем создания задач (issue) в этом репозитории.

Ресурсы MCP. Официальная команда MCP собрала обширную коллекцию ресурсов MCP, включая MCP-серверы, MCP-клиенты, MCP-инструменты и т. д., которые объединены в репозитории GitHub по адресу *modelcontextprotocol/servers* и представлены в виде README-документа.

Сторонние сообщества. Сторонние сообщества MCP, созданные третьими сторонами, ежедневно обсуждают соответствующие темы. К наиболее известным сторонним сообществам относятся группа Model Context Protocol на Discord и сообщество r/mcp на Reddit.

6.5. Ограничения MCP

Поскольку MCP – это протокол, выпущенный совсем недавно, он пока не является особо зрелым и совершенным. С точки зрения экосистемы MCP у него есть некоторые ограничения.

Недостатки в проектировании протокола. На момент написания книги на уровне протокола MCP было выпущено только три версии: версия от 05.11.2024, версия от 26.03.2025 и версия от 18.06.2025.

Целостность протокола по-прежнему оставляет желать лучшего. Так, например, в протоколе отсутствуют положения относительно взаимодействия хоста с большими языковыми моделями. Также отсутствуют четкие указания о порядке обработки хостом вложений, загружаемых пользователями. Форматы запросов и ответов MCP-сервера, определенные в протоколе, являются слиш-

Книги для программистов: https://t.me/free_books_for_it

ком однообразными, что затрудняет поддержку различных типов контента. Хотя протокол и определяет основные возможности клиента и его способности к выборке данных, в нем отсутствуют рекомендации по их реализации, что значительно затрудняет его использование.

Высокий порог входа для пользователей. Большинство существующих на рынке MCP-серверов требуют от пользователей загрузки кода для локального запуска, что для обычных пользователей (не разработчиков) создает высокий порог входа в плане установки среды, настройки и использования MCP-сервера. Кроме того, в настоящее время не встречается особо хорошо поддерживающих MCP ИИ-приложений, поэтому взаимодействие пользователей с MCP-серверами оставляет желать лучшего.

Слишком разное качество серверов. Хотя на рынке уже существует более десяти тысяч MCP-серверов и их количество растет очень быстрыми темпами, их качество сильно различается, и доля действительно работоспособных, удобных и эффективных MCP-серверов невысока.

Недостаточно надежный механизм аутентификации. В настоящее время большинство MCP-серверов считывают учетные данные пользователя через переменные среды, и пользователю приходится вручную настраивать учетные данные для каждого MCP-сервера, что довольно неудобно. В сценарии облачного многопользовательского (multitenant) доступа для поддержки динамической передачи учетных данных пользователя необходимо модифицировать механизм аутентификации MCP-сервера, однако действующая версия протокола этого не поддерживает, а затраты на модификацию высоки. Хотя в версии протокола MCP от 26.03.2025 был введен способ аутентификации OAuth, в настоящее время количество MCP-серверов, действительно поддерживающих этот способ, по-прежнему очень ограничено.

Слабые возможности защиты. Поскольку MCP является проектом с открытым исходным кодом, существует риск его некорректного использования. Злоумышленники могут создавать вредоносные MCP-серверы и вводить пользователей в заблуждение для их установки, чтобы затем осуществлять такие действия, как внедрение вредоносных инструментов, кража конфиденциальной информации, выполнение вредоносных команд и несанкционированный доступ к файлам. Для повышения безопасности MCP-серверов необходимо разработать более строгие стандарты на уровне протокола, а также ужесточить контроль качества MCP-серверов со стороны разработчиков. Пользователи должны проявлять бдительность, тщательно проверять источник MCP-серверов и устанавливать разумные настройки прав доступа.

6.6. MCP и A2A

9 апреля 2025 года компания Google представила протокол Agent2Agent (сокращенно A2A). Этот протокол предлагает стандартизированный подход к обеспечению взаимодействия и коммуникации между агентами. После выпуска A2A развернулась оживленная дискуссия о его взаимосвязи с MCP. По личному мнению автора этой книги, A2A разработан не для замены MCP, а, напротив, как дополнение к нему, чтобы они вместе способствовали созданию более эффективных агентов. Взаимосвязь между A2A и MCP представлена на рис. 6.5.

Книги для программистов: https://t.me/free_books_for_it

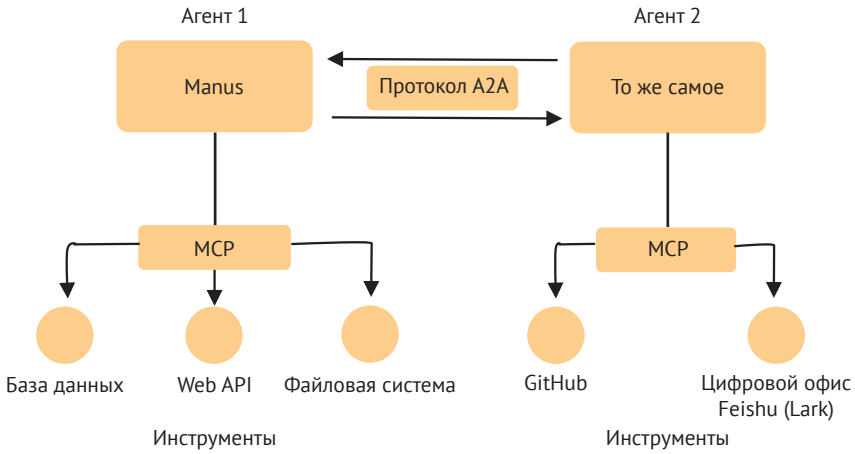


Рис. 6.5. Взаимосвязь A2A и MCP

Как видно из рисунка, MCP предоставляет агентам возможности для работы с внешними инструментами, а A2A позволяет агентам взаимодействовать друг с другом, тем самым обеспечивая совместную работу нескольких агентов.

Если рассматривать агента в роли ИИ-приложения экосистемы MCP, можно сравнить MCP и A2A. Взаимосвязь между ними показана в табл. 6.1.

Таблица 6.1. MCP в сравнении с A2A

Протокол	MCP	A2A
Основная цель	Стандартизация подключения и взаимодействия агентов с внешними данными или сервисами, такими как API и базы данных	Стандартизация коммуникации и совместной работы между агентами как равноправными участниками
Объекты взаимодействия	Внешние данные или сервисы	Автономные агенты
Особенности взаимодействия	• Структурированные входные и выходные данные • Предсказуемое поведение • Обычно это однократный цикл «запрос–ответ»	• Способность к логическому анализу, планированию и использованию различных инструментов • Поддержка состояния в ходе длительного взаимодействия • Участие в сложных многораундовых диалогах
Основные механизмы	• Определение структурированного описания возможностей инструмента • Передача входных параметров инструменту • Прием структурированного вывода	• Выявление продвинутых навыков и способностей других агентов • Согласование режима взаимодействия (текст, файлы, структурированные данные) • Управление общими, сохраняющими состояние и долгосрочными задачами

Книги для программистов: https://t.me/free_books_for_it

Окончание табл. 6.1

Протокол	MCP	A2A
Типичные сценарии использования	• Вызов внешних API (например, для получения котировок акций, погоды в реальном времени) • Запрос к базе данных • Подключение к корпоративным сервисам	• Агент службы поддержки вызывает агента по биллингу • Агент по планированию путешествий взаимодействует с агентами по бронированию авиабилетов, отелей и мероприятий • Агент по маркетингу в режиме реального времени передает пользовательские намерения агенту по подбору рекомендаций
Сложность взаимодействия	Обычно представляет собой вызов инструмента; низкая сложность	Динамическое, сохраняющее состояние и даже мультимодальное взаимодействие; высокая сложность
Управление состоянием	Обычно без состояния	Поддержка длительного взаимодействия с сохранением состояния
Роль в экосистеме	Агент поставщика данных или услуг, помогающий ему открыть доступ к своим возможностям	Участник сети агентов, поддерживающий сложное взаимодействие

MCP и A2A не являются взаимоисключающими, они в значительной степени дополняют друг друга, удовлетворяя потребности ИИ-агентов во взаимодействии на разных уровнях. Полноценный ИИ-агент должен использовать MCP для взаимодействия с конкретными инструментами и ресурсами внутри системы, а A2A – для совместной работы с другими ИИ-агентами. Объединяя MCP и A2A, разработчики могут создавать более мощные, гибкие и функционально совместимые системы совместной работы нескольких ИИ-агентов.

6.7. Заключение

В этой главе была рассмотрена экосистема MCP: от цепочки инструментов, платформ и сервисов до верхних и нижних звеньев цепочки поставок, а также сообщества и ресурсов, что позволило наметить контуры все более совершенной ИИ-инфраструктуры. В этой главе были рассмотрены инструменты, платформы и сервисы, а также экосистема MCP, включая сообщества и ресурсы. Мы рассказали о том, как MCP упрощает разработку и способствует формированию замкнутого цикла бизнеса, а также о роли сообщества в развитии экосистемы. В то же время были отмечены существующие ограничения MCP, такие как несовершенство протоколов, высокий порог входа, разные уровни качества серверов, несовершенство механизмов аутентификации и слабые возможности защиты. Надеюсь, что после изучения этой главы у вас сложится более четкое и целостное представление о MCP, и вы обратите внимание на потенциал его будущего развития (например, создание систем совместной работы множества агентов с использованием A2A). Этот динамичный и быстро развивающийся сектор требует участия и вклада большего числа участников.

Книги для программистов: https://t.me/free_books_for_it

Предметный указатель

А

Авторизация через третью сторону 102
Архитектура MCP 38
Архитектура клиент–сервер 21, 49
Архитектура хост–клиент–сервер 21, 38, 49

Б

Базовый протокол MCP 52
Безопасность коммуникации 53
Большая языковая модель 25

В

Внедрение функции анализа смешанного контента 179
Возможности MCP 43
Выборка 94
Вызов инструмента для чтения контента веб-страниц 194
Вызов инструмента сетевого поиска 193
Вызов функции 29
Вызов функции, определение 28
Вызов функций в сравнении с MCP 31
Вызов функций в сравнении с RAG 28
Выпуск MCP-сервера 107

Г

Генератор ИИ-подкастов на основе MCP 215
Генерация веб-страницы на основе макета Figma 230, 239
Генерация тематического подкаста 221

Д

Динамическая передача параметра 119
Динамическая регистрация клиента 100
Дополнительный инструмент MCP 52
Доступ большой языковой модели к информации 25

Ж

Жизненный цикл соединения 59

З

Запрос модели на выбор инструмента 174

И

Инструмент MCP-сервера для изменения формулировки вопроса 190
Инструмент командной строки MCP 243
Инструмент MCP для командной строки 247
Инструмент отладки MCP 243
Инструменты в MCP 88
Использование MCP-сервера в веб-версии ChatMCP 213
Использование MCP-сервера с ChatWise 207
Использование MCP-сервера с Cherry Studio 209
Использование MCP-сервера с Cline 206
Использование MCP-сервера с DeepChat 210
Использование токена доступа 102

К

Как принять участие в доработке MCP 249
Ключевые компоненты MCP 51
Комбинация хоста и клиента в MCP 37
Корень 91

М

Магазин приложений MCP 244
Механизм авторизации MCP 96
Механизм передачи MCP 64
Механизм плагина 33
Механизм согласования MCP 52
Механизм тайм-аута 63
Модуль функций MCP 52

Н

Набор инструментов MCP 243
Настройка системного промпта 175
Настройка среды разработки 105
Недостатки в проектировании MCP 249
Нейросетевая языковая модель (GPT) 32

О

Обработка ошибки 64
Ограничения MCP 249

Книги для программистов: https://t.me/free_books_for_it

Определение метаданных сервера авторизации 99
Определение списка инструментов 144
Определение типа данных смешанного контента 178
Оптимизация логики взаимодействия 182
Организация рабочего процесса с помощью подсказки 197
Основной процесс авторизации MCP 98
Отладка MCP-сервера 107

П

Поддержка MCP в Cursor 40
Поддержка MCP в Windsurf 40
Получение списка инструментов MCP-сервера для интернет-поиска 192
Получение списка инструментов MCP-сервера для чтения веб-страниц 194
Потоковая HTTP-передача 74
Потоковая HTTP-передача, преимущества и недостатки 79
Потоковый HTTP 64
Преобразование текста в речь 216, 217
Пример подсказки (промпта) 81
Принцип один раз открыть, везде использовать 47
Принципы разработки на основе MCP 51
Проблема MЧN 19
Протокол Agent2Agent (A2A) 250
Процесс 50
Процесс разработки MCP-клиента 163
Процесс с кодом авторизации + PKCE 98

Р

Развертывание кода 232
Разработка MCP-сервера 104
Разработка MCP-сервера flomo 108
Разработка MCP-сервера HeyBeauty 128
Разработка диалогового ИИ-ассистента 165
Разработка ИИ-генератора веб-страниц 229
Разработка поискового ИИ-агента 188
Реализация логики вызова инструментов 146
Реализация логики получения списка ресурсов 137
Реализация функции диалога с моделью 176
Реализация функции подсказок 153
Рекомендация по внедрению MCP 54
Ресурсы MCP 249
Решение проблемы экосистемы ИИ с помощью MCP 34

С

Сетевой поиск 217
Создание веб-страницы на основе подсказки 230, 234
Создание кода для взаимодействия с моделью 199
Создание подкаста по URL-адресу веб-страницы 226
Создание проекта MCP-сервера 106
Сообщество MCP 249
Сторонние сообщества MCP 249
Схема авторизации MCP 52
Сценарий персонального ИИ-ассистента 44
Сэм Альтман 42

Т

Тестирование ресурса в клиенте большой языковой модели 139
Технология RAG 188
Тим Бернерс-Ли 38

У

Установка fnm 105
Установка Node.js 105
Установка npm 105
Установка SDK MCP-клиента 164
Установка клиентов больших языковых моделей 106

Ф

Фонд a16z 40

Ч

Хост MCP 163
Хранение памяти 26

Ц

Цель разработки диалогового ИИ-ассистента 165
Цепочка интеграции MCP 248

Ч

Чтение макета Figma 233
Чтение содержимого веб-страниц 219

Э

Экосистема MCP 243

Книги для программистов: https://t.me/free_books_for_it

A

Agents SDK 41
Alibaba Cloud Bailian 42

B

Baidu 42

C

ChatGPT 25
ChatMCP 213
ChatWise 207
Cherry Studio 209
Cline 206
Cloudflare 42
CopyWeb 229

D

DeepChat 210
Docker 42

F

Firecrawl 219

G

Google 42

H

HTTP, определение 38

J

JSON-RPC 2.0, пакетная обработка 59
JSON-RPC 2.0, спецификация 57
JSON-RPC, определение 56
JSON, определение 54
JSON, особенности 54

L

LSP, определение 36
LSP, преимущества 37

M

Manus 40, 47
MCP 30
MCP Inspector 134
MCP (Model Context Protocol), определение 19
mcprouter 246
MCP в сравнении с A2A 251
MCP в сравнении с HTTP 38
MCP-клиент 50
MCP-клиент, определение 21, 163
MCP, первоначальная цель создания 26
MCP-платформа маршрутизации 245
MCP, принцип работы 20
MCP, процесс взаимодействия 30
MCP-сервер 51, 244
MCP-сервер, определение 21
MCP, сравнение с док-станцией 24
MCP-хост 22, 49
MCP-хост, определение 20
MCP-хост, свойства 21
Monica AI 40

N

NotebookLM 215

O

OAuth 2.0 96
OAuth 2.1 96, 97
Ollama 247
OpenAI 25, 32, 41
OpenRouter 176, 190, 245

P

PodLM 215

R

RAG 27
RPC, определение 55
RPC, особеннсти 56

S

SaaS 216
SDK MCP 104, 243
SSE 64, 68
SSE, преимущества и недостатки 73
stdio 64, 65
stdio, преимущества и недостатки 67

T

ThinkInAI 210
ToB (B2B) 47

U

URI 84

Z

Zapier 42

Книги для программистов: https://t.me/free_books_for_it

Книги издательства «ДМК Пресс» можно купить оптом и в розницу
на складе издательства по адресу:
Москва, ул. Электродная, д. 2, стр. 12, офис 7,
тел. +7 (499) 322–19–38,
а также заказать на сайте **www.dmkpress.com**
с доставкой в любой регион РФ

Айдоуби (@idoubi)

Протокол MCP для систем искусственного интеллекта

Главный редактор *Яценков В. С.*
editor@dmkpress.com

Перевод *Бахур В. И.*
Корректор *Абросимова Л. А.*
Верстка *Луценко С. В.*
Дизайн обложки *Трофимова С. В.*

Формат 70×100 1/16.
Гарнитура «PT Serif». Печать цифровая.
Усл. печ. л. 20,8. Тираж 200 экз.

Веб-сайт издательства: *www.dmkpress.com*

Книги для программистов: https://t.me/free_books_for_it

Протокол контекста модели (Model Context Protocol, MCP) служит связующим звеном между ИИ-приложениями и внешними расширениями. MCP стремительно меняет экосистему ИИ и стал общепринятым отраслевым стандартом. Основная цель этой книги – помочь читателю получить всестороннее и системное представление о MCP, включая историю его появления, принципы работы, архитектуру протокола, сценарии применения и экосистему. На примере реальных кейсов в книге рассказывается об использовании и разработке MCP-серверов, а также о том, как создавать на основе MCP востребованные ИИ-приложения – диалоговых помощников и интеллектуальных агентов.

Основные темы книги:

- предпосылки появления, принципы работы и возможности MCP;
- модель «хост-клиент-сервер», JSON-RPC, транспорт и авторизация;
- разработка MCP-серверов на практических примерах;
- разработка MCP-клиентов и приложений искусственного интеллекта;
- классические сценарии применения MCP;
- экосистема MCP, ограничения и связь с A2A.

Издание адресовано всем, кто хочет детально изучить MCP и применять его в работе.

ДМК ПРЕСС
ИЗДАТЕЛЬСТВО
www.дмк.рф

ISBN 978-5-93700-439-0

Книги для программистов: https://t.me/free_books_for_it